

GALAXY XR · UNITY 6 · HAND TRACKING

콕핏 속으로

건담 콕핏 VR 게임을 함께 만든 11일의 기록
— 요청 한 줄에서 시작해 우주 콜로니 전장까지

기획·테스트 · 이건호 (NC AI 대전)

구현·기록 · Claude

작업 기간 · 2026년 9월 21일 - 10월 1일

프로젝트 · gundam game (Unity 6000.6.0f1 / URP 17.6 / OpenXR)

CONTENTS

차례

머리말

1부 · 프로젝트 한눈에 보기

1장 무엇을 만들었나

2장 작업 방식과 지켜온 원칙

2부 · 날짜별 개발 일지 — 처음부터 끝까지

한 장으로 보는 일정

9월 21일~22일 (1) · 빈 프로젝트에서 구형 콕핏까지

9월 22일 (2) · 손으로 조종하기, 첫 무기와 첫 적

9월 23일 · 조종간 다듬기, 캘리브레이션, 디스플레이 3개, 헤드 발칸

9월 28일 · 시점 회전, 손가락 버튼, 락온 링, 적 체력

9월 29일 · T자 레버, 자쿠 AI, 빔 샷벨, 빔 라이플, 머신건

9월 30일 · 우주 콜로니 전장, 출격 위치, 최적화

10월 1일 · 사운드, 자쿠 플레이, 바주카, 부스터

3부 · 시스템별 해설

3장 콕핏 — 구형 돔과 360도 머리 카메라

4장 손으로 잡는 조종간

5장 이동과 시점

6장 락온 시스템

7장 건담의 무기

8장 적 — 자쿠와 적 건담

9장 전장 — 우주와 콜로니

10장 자쿠 플레이 모드

11장 소리와 부스터 불꽃

12장 최적화와 화질

4부 · 문제 해결 노트

13장 막혔던 문제와 해결 방법

부록

A. 기체·무기 수치표

B. 스크립트 지도

C. 에셋과 사운드 목록

머리말

이 책은 한 사람이 머릿속에 그린 장면 — “건담 콕핏에 앉아, 손으로 조종간을 잡고, 건담의 눈으로 세상을 본다” — 을 Galaxy XR 헤드셋 안에서 실제로 움직이게 만든 과정을 기록한 것이다.

작업은 대화로 진행되었다. 이건호가 헤드셋을 쓰고 직접 해 보고, 마음에 들지 않거나 고장 난 부분을 짧은 한국어 문장으로 알려 주면, Claude가 코드를 읽고 고치고 다시 넘겼다. “조종관을 잡는 순간 바닥 쪽으로 내려가서 안 올라옴”, “엄지를 펴도 발칸이 나갈 때가 있어”, “락온이 거리에 따라서 작아졌으면 좋겠어” 같은 문장 하나하나가 이 게임의 설계도가 되었다.

흥미로운 점은 그 문장들이 사라지지 않았다는 것이다. 모든 스크립트의 주석에는 그 기능을 만들게 한 원래 요청이 따옴표로 남아 있다(예: `per "건담이 상대를 통과하지않게 충돌을 넣어"`). 이 책은 그 334개의 인용문과 Git 커밋 19개, 파일 생성 기록, 작업 중에 찍은 렌더 이미지를 모아 날짜별·시스템별로 다시 엮은 것이다.

2부는 시간 순서대로 “그날 무엇을 요청했고 무엇이 생겼는지”를 보여 주고, 3부는 같은 내용을 콕핏·조종간·무기·적·전장처럼 시스템 단위로 다시 풀어 “어떻게 동작하는지”를 설명한다. 4부는 막혔던 문제와 그 원인을 따로 모았다. 같은 기능이 여러 번 나오는 것은 의도한 것이다 — 한 번은 이야기로, 한 번은 구조로 읽을 수 있게 했다.

이 책은 두 번에 걸쳐 쓰였다. 처음 판은 요약에 가까웠고, 이건의 요청에 따라 “진짜 처음부터 만드는 과정 전부”를 담도록 2부를 완전히 새로 썼다. 새 2부는 Git에 남은 19개 커밋의 코드를 한 줄 한 줄 비교해, 빈 Unity 프로젝트를 만들고 패키지를 설치하던 순간부터 부스터 불꽃의 색을 고르던 마지막 날까지 무엇이 어떤 순서로, 왜 바뀌었는지를 추적한다. 처음 시도가 실패하고 다른 방법으로 바뀐 과정도 지우지 않고 그대로 실었다. 같은 문제를 다시 만나는 사람에게는 정답보다 그 과정이 더 쓸모 있기 때문이다.

읽는 법

요청 파란 상자는 이건호가 실제로 남긴 요청 문장(맞춤법 그대로)이다. **문제 보고** 붉은 상자는 테스트 중 발견된 문제다. 코드 조각은 실제 프로젝트 코드에서 핵심 부분만 짧게 옮겼다.

PART 1

프로젝트 한눈에 보기

무엇을 만들었고, 어떤 도구와 규칙으로 만들었는지.

CHAPTER 1

무엇을 만들었나

Galaxy XR 헤드셋용 1인칭 건담 콕핏 VR 게임. 플레이어는 구형 콕핏 안에 앉아 맨손으로 두 개의 구형 조종간과 T자 레버를 잡고 18m 높이의 모빌슈트를 조종한다. 컨트롤러는 쓰지 않는다 — 모든 입력은 손 추적이다.

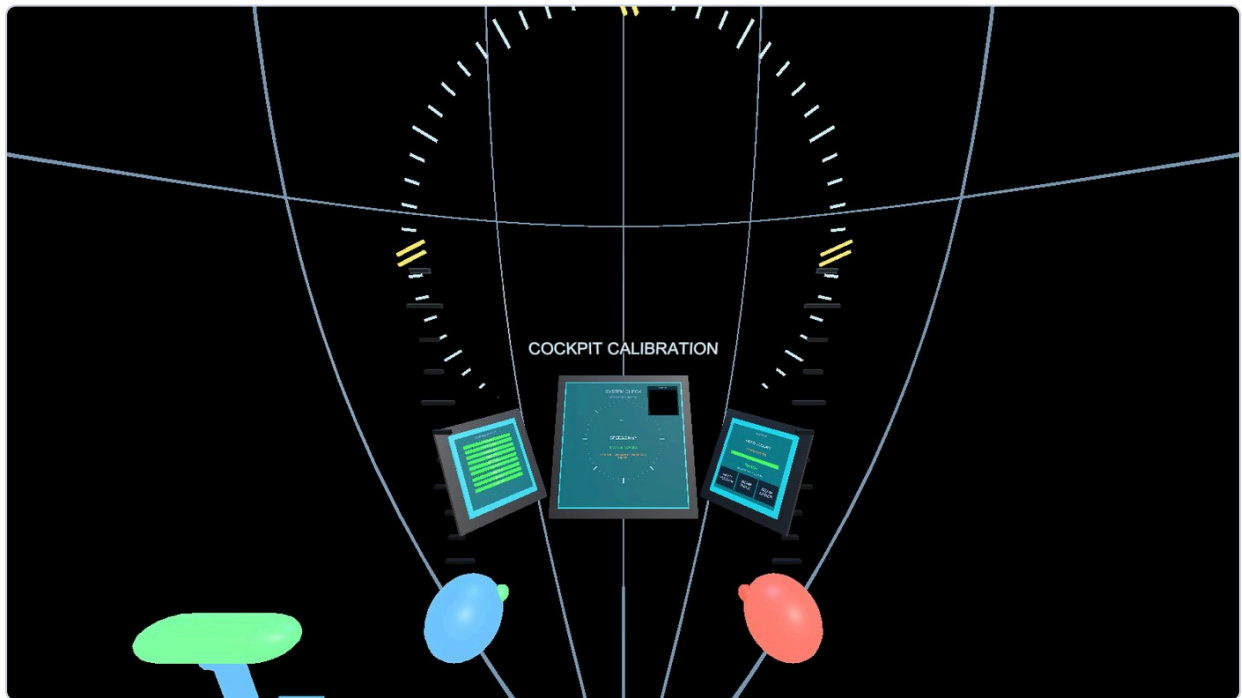


그림 1-1 · 조종석 시점. 원형으로 둘러싼 구형 돔(바깥 세상이 360도로 비치는 화면), 위쪽의 궤도 표시 링(락온 링), 가운데 디스플레이 3개, 아래쪽의 왼손·오른손 구형 조종간과 T자 레버.

11일

9월 21일 ~ 10월 1일, 실제 작업한 날은 7일

46개

런타임 스크립트
(Assets/Scripts/Cockpit)

6,489줄

썬 생성기 GundamCockpitSetup.cs

334개

코드 주석에 남은 원래 요청 문장

12 km

우주 콜로니 길이 (반지름 3.2 km)

2기체

RX-78-2 건담 / MS-06 자쿠 II 선택 플레이

게임의 흐름

1. 조종간 위치 설정(캘리브레이션) — 앱을 켜면 왼쪽, 오른쪽 조종간을 손으로 잡아 내 팔 길이에 맞는 자리로 옮기고, 손을 쥐어 확인한다.
2. 기체 선택 — 손가락으로 *RX-78-2 GUNDAM* 또는 *MS-06 ZAKU II*를 누른다. 자쿠를 고르면 콕핏이 지오폭 초록색으로 바뀌고 적은 건담 6기가 된다.
3. 출격 위치 선택 — 우주 / 콜로니 외관 / 콜로니 입구 / 도심 중 하나를 고른다.
4. 전투 — 왼손 조종간으로 이동, T자 레버로 상승·하강, 오른손 조종간으로 시점 회전, 무기 디스플레이를 손가락으로 눌러 무기 교체, 락온 후 오른손 엄지로 발사, 빔 샤벨·히트 호크는 오른손 조종간을 밀고 당겨 찌르고 휘두른다.

기술 스택

구분	내용
엔진	Unity 6000.6.0f1, Universal Render Pipeline 17.6
XR	OpenXR, XR Hands(손 관절 추적), XR Interaction Toolkit 3.3(Near-Far Interactor 잡기 판정). AR Foundation·ARCore는 쓰지 않음
기기	Samsung Galaxy XR (Android, arm64)
모델	건담·자쿠 FBX(본 리깅, 애니메이션 클립 없음), 빔 라이플·빔 샤벨 손잡이·자쿠 머신건·히트 호크·자쿠 바주카 FBX, VARCO 3D로 만든 콜로니 건물과 우주 잔해
이펙트	Asset Store “2D Booster Flame Shader Controller”(RockingProjects)
개발 환경	사용자 PC의 Unity 에디터 + Claude(클라우드 작업 공간에서 코드 편집, 기기 브리지로 파일 반영, Unity MCP로 컴파일 확인·렌더 테스트)

프로젝트 구조

씬은 손으로 배치하지 않는다. `Assets/Editor/GundamCockpitSetup.cs` 한 파일이 메뉴 **Gundam > Build Cockpit Scene**을 누를 때마다 콕핏, 조종간, 디스플레이, 건담, 자쿠 10기, 적 건담 6기, 콜로니, 우주 배경까지 전부 새로 짓는다. 매 프레임 동작하는 로직은 `Assets/Scripts/Cockpit/`의 `MonoBehaviour` 46개가 맡는다. “구조는 생성기, 동작은 런타임 스크립트”라는 이 분리가 프로젝트 전체를 관통하는 규칙이다.

```
// GundamCockpitSetup.cs - 씬 생성기의 입구 (요약)
[MenuItem("Gundam/Build Cockpit Scene")]
public static void BuildCockpitScene()
{
    // 콕핏(구형 돔·디스플레이 3개·좌석) → 조종간 2개 + T자 레버 → XR Origin
    // → 건담(ExternalGundam, 머리 카메라) → 콜로니 → 자쿠 10기 → 무기·HUD 연결
    // → 자쿠 모드(플레이어 자쿠, 적 건담 6기, 기체 선택 패널) → 최적화
}
```


작업 방식과 지켜온 원칙

한 번의 요청이 반영되기까지

1. 이건호가 헤드셋으로 테스트하고 요청이나 문제를 한두 문장으로 보낸다. 필요하면 사진이나 다운로드한 에셋(모델·사운드)을 함께 준다.
2. Claude가 관련 스크립트를 읽고 원인을 찾는다. 선택지가 갈리면(예: 자쿠 모드의 적을 무엇으로 할지) 질문으로 먼저 확인한다.
3. 코드를 고치고 사용자 PC의 프로젝트 폴더에 반영한다. 반영 직후 파일을 다시 읽어 와 바이트 단위로 비교(cmp)해 실제로 저장되었는지 확인한다 — 기기 브리지는 가끔 이전 버전으로 덮어쓰는 일이 있었기 때문이다.
4. Unity에서 컴파일을 확인하고, 필요하면 임시 씬을 만들어 렌더 이미지로 결과를 검증한다(사용자가 열어둔 씬은 건드리지 않는다).
5. 이건호가 **Gundam > Build Cockpit Scene**으로 씬을 다시 빌드하고 헤드셋에서 확인한다.

지켜온 원칙

원칙	이유 / 출처
기존 기능을 지우지 않는다	“삭제하지 말고 기존 코드는 보존한다”. 옛 기능은 끄거나 분리할 뿐 코드는 남긴다. 예: 처음 만든 콧빛 포탑 조준 시스템(WeaponAimFireController)은 지금도 남아 있다.
구조는 생성기, 동작은 런타임 스크립트	씬을 언제든지 처음부터 다시 지을 수 있게. 생성기에 매 프레임 로직을 넣지 않는다.
손 추적 시스템을 새로 만들지 않는다	“새로운 MediaPipe나 별도 손 추적 시스템을 추가하지 않는다”. 모든 기능이 같은 XR Hands 이벤트를 구독한다.
건드리지 않는 파일	AndroidManifest, XR Origin·OpenXR 설정, HandJointTracker.cs, Packages 폴더, 이동 로직(ShipMovementController·조종간 이동·T레버). AR Foundation/ARCore는 다시 넣지 않는다.
PC 시뮬레이션 입력 없음	키보드·마우스 대체 입력을 만들지 않고 실제 손으로만 테스트한다.
저장 후 반드시 재확인	파일을 반영하면 다시 읽어 와 비교한다. 다른 버전이면 다시 반영한다.
사용자의 열린 씬을 건드리지 않는다	테스트는 추가로 연 임시 씬에서 하고, 끝나면 저장하지 않고 닫는다.

숫자로 보는 커밋 기록

Git 저장소는 9월 23일 오전에 만들어졌고 10월 1일까지 19번 커밋되었다. 커밋 메시지는 대부분 “gundam game”이라 내용을 알 수 없지만, 각 커밋 시점의 스크립트를 꺼내 주석 속 요청 문장을 비교하면 어떤 요청이

언제 반영되었는지 거의 정확히 알 수 있다. 2부의 날짜는 이 방법과 파일 생성 시각으로 재구성한 것이다.

날짜	커밋 수	새로 생긴 스크립트
9/21-9/22	(저장소 이전)	WeaponAimFireController, HitTarget, TargetingHUD, EnemyMarker, HandJointTracker, 손 추적 진단 스크립트 2개
9/23	4	CalibrationManager, HandExclusiveGrabAdapter, CockpitHUDManager, CockpitStatusHUD, CockpitWeaponHUD, HeadVulcanController
9/28	7	CockpitViewController, HeadCamManualLook, JoystickFingerButtons, OrbitHUDTargetLock, EnemyHealth
9/29	5	VerticalTLever, VerticalThrustController, ZakuCombatAI, HeadVulcanBullet, SpaceDustField, SpaceDebrisField, BeamSaber 3종, WeaponModeController, WeaponTouchPanel, XRHandSpace, SaberLookAssist, BeamRifleController, BeamRifleShot, ZakuMachineGun, ZakuBullet, PlayerHealth
9/30	2	ColonyStructure, ColonyAtmosphere, SpawnSelectPanel, SuitCollision, HeadCamInsetCube 셰이더(최적화)
10/1	1	LoopingFireSound, GundamMoveSound, MeleeWeaponAI, MechWalkAnimator, CockpitTheme, MechSelectPanel, PilotMechSwitcher, MechBoosterFlames

PART 2

날짜별 개발 일지

빈 프로젝트를 만든 9월 21일부터 10월 1일까지, 실제로 작업한 7일 동안 이견호가 무엇을 요청했고 무엇이 어떤 시행착오를 거쳐 만들어졌는지 시간 순서대로 아주 자세히 정리했다. 코드 조각은 그날 커밋에 실제로 들어 있던 코드에서 옮겼다. 날짜는 Git 커밋 시점의 코드에 남은 요청 문장과 파일 생성 시각을 맞춰 재구성했으며, 커밋 사이에 걸친 작업은 범위로 표시했다.

TIMELINE

한 장으로 보는 일정

날짜	주제	주요 결과물
9/21 (월)	프로젝트 시작, 콕핏 뼈대	씬 생성기, 첫 조종간·포탑 사격 프로토타입, 구형 콕핏으로 전면 재 설계
9/22 (화)	건담·자쿠, 손 추적	건담 FBX 배치, 머리 카메라 360도 돔, 자쿠 적 배치, 보이는 손, 손으로 잡는 구형 조종간
9/23 (수)	조종간 안정화, 캘리브레이션, HUD	잡을 때 끌려오는 문제 해결, 양손 분리, 조종간 위치 설정 단계, 디스플레이 3개(상태·전술·무기), 헤드 발칸
9/28 (월)	시점·손가락 버튼·락온	오른손 조종간 시점 회전, 시선 기준 이동, 손가락별 버튼 5개, 엄지 발칸, OrbitHUD 락온 링, 적 체력
9/29 (화)	전투의 틀	T자 수직 레버, 자쿠 전투 AI, 별·잔해 배경, 빔 샷벨, 무기 터치 디스플레이, 빔 라이플, 자쿠 머신건, 플레이어 체력
9/30 (수)	콜로니 전장	출격 위치 선택, 길이 12km 콜로니와 도시, 중력·충돌, 시야 가림, 자쿠 10기, 화질·최적화
10/1 (목)	완성도와 두 번째 기체	사운드 5종, km/h 속도, 히트 호크, 자쿠 플레이 모드, 적 건담, 건기 애니메이션, 바주카, 부스터 불꽃

9월 24일-27일에는 커밋이나 파일 변경 기록이 없다.

이 장을 읽기 전에 — 남아 있는 자료

이 프로젝트의 Git 저장소는 9월 23일 오전 9시 50분에 빈 커밋(00)으로 처음 만들어졌다. 그리고 16분 뒤인 10시 06분, 커밋 01(8a233a1)이 찍히면서 그때까지의 모든 것이 한꺼번에 들어왔다. 다시 말해 9월 21일 월요일 오후부터 22일 화요일까지 이견호와 Claude가 주고받은 수십 번의 요청과 수정은 “이전 버전 대비 차이(diff)”의 형태로는 하나도 남아 있지 않다. 남아 있는 것은 그 이들의 결과물 전체, 즉 커밋 01의 스냅샷뿐이다.

그런데도 이 이들을 꽤 자세히 되짚을 수 있는 이유는 Claude가 코드를 고칠 때마다 주석에 “왜 이렇게 바꿨는지”를 길게 남겼기 때문이다. 주석에는 이견호의 요청이 따옴표 안에 한국어 원문 그대로 들어 있고 (“per request”), 문제 보고는 “per report”로, 이전 방식을 걷어낸 흔적은 “was …”, “used to …”, “REVERSED”, “UPDATE”, “Bug fix” 같은 말로 남아 있다. 특히 장면(씬)을 만드는 편집기 스크립트 `Assets/Editor/GundamCockpitSetup.cs` 는 이 시점에 2017줄이나 되는데, 그 가운데 절반 가까이가 이런 설명 주석이다. 이 장은 그 주석들과 실제 수치를 근거로 “빈 프로젝트에서 구형(球形) 콕핏까지”를 처음부터 다시 따라간다.

시간의 단서는 많지 않다. 이견호의 PC에 남은 파일 생성 시각으로 보면, 9월 21일 오후 4시 58분 무렵 `Assets/Scripts` 와 `Assets/Editor` 폴더, 그리고 사격 관련 스크립트 `HitTarget · WeaponAimFireController` 가 생겼고, 1분 뒤인 4시 59분 무렵 XR Hands와 XR Interaction Toolkit(XRI)의 샘플 폴더가 생겼다. 9월 22일에는 오전 9시 55분 무렵 `TargetingHUD` , 10시 30분 무렵 건담 모델 폴더(`Assets/Models/Gundam`), 12시 51분 무렵 `HandTrackingStatusLogger` , 12시 57분 무렵 자쿠 모델 폴더, 12시 59분 무렵 `EnemyMarker` , 오후 2시 17분 무렵 `HandSubsystemBootDiagnostics` , 3시 25분 무렵 `HandJointTracker` 가 보인다. 파일은 다시 쓰일 때 시각이 바뀔 수 있으므로 “이 무렵 있었다” 정도로만 읽어야 한다. 이 장에서는 이 단서와 주석 속 순서 단서를 엮어서, 어떤 결정이 어떤 결정보다 먼저였는지를 말할 수 있을 때만 말한다.

이 장이 다루는 범위

프로젝트 생성과 패키지 구성, “씬을 코드로 짓는” 구조, 콕핏 모양의 변천(평범한 조종석 → 사진을 보고 전면 재설계 → 구형 돔), 건담 FBX와 머리 카메라, 360도 큐브맵 돔, 화질 문제까지다. 손 추적, 조종간, 포탑 사격, 자쿠 배치, 타깃 HUD는 다음 장에서 자세히 다루고, 여기서는 콕핏·카메라와 맞물리는 지점만 짧게 짚는다.

1. 프로젝트를 만든다 — Unity 6, URP, Galaxy XR

1-1. 엔진과 렌더 파이프라인

출발점은 Unity 6000.6.0f1(Unity 6 계열)로 만든 URP(Universal Render Pipeline) 프로젝트다. URP 패키지 버전은 `com.unity.render-pipelines.universal 17.6.0` 이다. 대상 기기는 삼성 Galaxy XR, 운영체제로 보면 Android XR 헤드셋이다. 모바일 칩에서 양쪽 눈을 매 프레임 그려야 하는 독립형 헤드셋이므로, 가벼운 URP를 고른 것은 자연스럽다. 이 선택은 뒤에서 여러 번 설계 판단의 근거가 된다. 예를 들어 Claude는 돔에 영상을 입힐 재질을 고를 때마다 “URP에서 확실히 그려지는 셰이더인가”를 따졌다. 레거시 내장 셰이더(Skybox/Cubemap 같은)를 일반 메시에 바로 씌우면 URP에서 “지원되지 않는 셰이더”를 뜻하는 분홍색으로 깨질 가능성이 높다는 판단이 주석에 거듭 나온다.

URP 템플릿으로 만든 새 프로젝트에는 기본 장면 `Assets/Scenes/SampleScene.unity` 가 들어 있고, 빌드 설정(Build Settings)의 0번 자리에 이 장면이 올라가 있다. 이 사소한 기본값이 나중에 실제 문제를 일으킨다(1-5에서 다룬다).

1-2. 패키지 — 무엇을 넣었고 각각 무슨 일을 하나

장면 생성기 맨 위의 설명 주석은 “*Packages/manifest.json에 추가한 XR 패키지들이 다 풀린 다음에 실행하라(프로젝트를 한 번 열고 Package Manager가 가져오기를 끝내게 둔 뒤 아래 메뉴를 쓰라)*”고 적고 있다. 즉 XR 패키지는 Package Manager 창에서 하나씩 누른 것이 아니라 `manifest.json` 의 `dependencies` 에 직접 적어 넣는 방식으로 추가됐다. 커밋 01의 manifest에서 이 프로젝트에 의미 있는 패키지는 다음과 같다.

패키지	버전	이 프로젝트에서의 역할
<code>com.unity.xr.openxr</code>	1.18.0	OpenXR 플러그인. 헤드셋 런타임과 표준 OpenXR로 대화하는 바탕. 머리 추적·렌더링·입력이 모두 이 위에서 돈다.
<code>com.unity.xr.androidxr-openxr</code>	1.4.1	Android XR용 OpenXR 확장 기능 묶음. Galaxy XR이 Android XR 기기이므로 이 패키지의 기능들을 OpenXR 설정에서 켜다.
<code>com.unity.xr.hands</code>	1.6.3	손 관절 데이터를 주는 <code>XRHandSubsystem</code> . 손 모양을 실제로 보여 주는 공식 샘플 <code>HandVisualizer</code> 도 이 패키지에 딸려 있다.
<code>com.unity.xr.interaction.toolkit</code>	3.3.2	XRI. 손이나 컨트롤러로 물체를 잡고 누르는 상호작용(Interactor/Interactable)과, 카메라·손이 갖춰진 XR Origin 리그 샘플을 제공한다.
<code>com.unity.xr.management</code>	4.6.0	XR Plug-in Management. 프로젝트 설정에서 어떤 XR 로더(여기서는 OpenXR)를 쓸지 고르는 관리 계층.
<code>com.unity.inputsystem</code>	1.20.0	새 Input System. XRI와 XR 입력 액션이 이것을 바탕으로 한다.
<code>com.unity.ugui</code>	2.6.0	uGUI. 콧 화면의 월드 공간 Canvas, Text, Image, 머리 카메라 영상을 띄우는 <code>RawImage</code> 가 모두 이것이다.
<code>com.unity.render-pipelines.universal</code>	17.6.0	URP.

이 밖에 `ai.navigation`, `collab-proxy`, `ide.rider`, `ide.visualstudio`, `test-framework`, `timeline`, `visualscripting` 은 템플릿이 기본으로 넣는 패키지이고, 이 이를 동안의 코드에서는 쓰이지 않는다.

1-3. OpenXR 설정과 Android 설정

패키지를 넣는 것만으로는 아무것도 켜지지 않는다. 장면 생성기는 일을 다 마친 뒤 콘솔에 “아직 안 했다면 *Project Settings > XR Plug-in Management*에서 *OpenXR + Hand Tracking*을 켜라”라는 안내를 찍는다. 이 단계는 코드로 하지 않고 편집기 설정 화면에서 손으로 했다. 프로젝트 설정에 남은 상태를 보면 Android 대상 OpenXR 기능 가운데 다음이 켜져 있다.

OpenXR 기능(Android)	왜 필요한가
Hand Tracking Subsystem	XR Hands의 XRHandSubsystem 에 실제 손 관절 데이터를 공급한다. 손을 보이게 하고 조종간을 잡는 모든 일이 여기서 시작한다.
Microsoft Hand Interaction Profile	손의 집기(pinch)·쥐기(grip) 동작을 입력 액션으로 노출하는 상호작용 프로필. XRI의 손 Interactor가 이 입력을 읽는다.
Meta Hand Tracking Aim	손의 조준 방향과 집기 세기 같은 보조 정보를 준다.
Android XR Display Utilities 등	Android XR 디스플레이 관련 유틸리티.

Android 플레이어 설정은 최소 SDK 29, ARM64 아키텍처, 선형(Linear) 색공간이다. XR 헤드셋은 64비트 ARM 빌드를 요구하고, URP에서 조명과 발광 재질이 의도대로 보이려면 선형 색공간이 맞다. 빌드에 들어가는 장면은 `Assets/Scenes/GundamCockpit.unity` 하나다.

1-4. 샘플 — 손 모양과 XR 리그를 “공식 부품”으로

프로젝트에는 패키지 샘플 세 가지가 들어 있다. XR Hands의 **HandVisualizer**, XRI의 **Starter Assets**와 **Hands Interaction Demo**다. 이 가운데 XRI 샘플 들은 장면 생성기가 스스로 가져오기(import)까지 한다. `TryImportSampleAndFindXR0riginPrefab()` 은 XRI 패키지 정보를 찾아 그 샘플 목록을 돌면서, 이름이 정확히 일치하는 두 개만 가져온다.

```
bool isNeededSample =
    sample.displayName.Equals("Hands Interaction Demo", StringComparison.OrdinalIgnoreCase) ||
    sample.displayName.Equals("Starter Assets", StringComparison.OrdinalIgnoreCase);

if (!isNeededSample) continue;
if (sample.isImported) { importedAny = true; continue; }

sample.Import();
importedAny = true;
```

“Hands Interaction Demo”에는 손 추적용 XR Origin 리그 프리팹(`XR Origin Hands (XR Rig)`)이 들어 있고, “Starter Assets”는 그 샘플이 의존하는 입력 액션과 프리셋 묶음이다. 가져온 뒤에는 `Assets/Samples` 아래에서 경로에 “XR Origin”이 들어간 프리팹을 찾되, “Hand”가 들어간 것을 우선해서 고른다.

AR Foundation은 쓰지 않는다

XRI에는 “AR Starter Assets”라는 샘플도 있다. 이것도 “Starter Assets”에 의존하지만, 추가로 `com.unity.xr.arfoundation` 을 끌어온다. 주석은 이것을 “이 프로젝트가 원하지 않는 것”이라고 분명히 적고, 실제로 이 패키지가 “XRSimulation-Session” 중복 ID 경고의 원인이었다고 기록한다. 그래서 샘플 이름을 고를 때 `Contains("Starter Assets")` 같은 부분 문자열 검사를 쓰지 않고 `Equals` 로 정확히 비교한다. 부분 문자열로 찾으면 “AR Starter Assets”도 함께 걸리기 때문이다. manifest에도 AR Foundation이나 ARCore 관련 패키지는 없다. 이 게임은 현실 공간을 인식하는 AR이 아니라, 완전히 가상의 콕핏 안에 들어가는 VR이라는 원칙이 처음부터 코드에 박혀 있는 셈이다.

XR Hands의 HandVisualizer 샘플은 `Assets/Samples/XR Hands/1.6.3/HandVisualizer/` 에 한 번 가져와 둔 상태이고, 생성기는 그 안의 Android XR용 손 메시(`LeftHandAndroidXR.fbx`, `RightHandAndroidXR.fbx`)와 재질, 관절 프리팹 경로를 상수로 들고 있다. 이것은 손을 보이게 하는 일이라 다음 장의 주제다.

1-5. “그냥 유니티 공간만 보임” — 빌드 설정 0번 자리

처음 기기에서 실행했을 때 콕핏이 아니라 아무것도 없는 공간이 나온 일이 있었다. 이견호의 보고는

문제 보고 · 그냥 유니티 공간만 보임

이었다. 원인은 앞에서 말한 템플릿 기본값이었다. 처음 버전의 생성기는 GundamCockpit 장면이 빌드 목록에 “없으면 끝에 덧붙이기”만 했다. 그러면 새 프로젝트에서는 템플릿이 넣어 둔 `SampleScene` 이 0번, GundamCockpit이 1번이 되고, 빌드는 0번 장면(카메라 하나와 조명 하나뿐, XR 세션도 콕핏도 없는 장면)으로 시작한다. 오류도 없이 그냥 Unity 기본 배경만 보이는 증상이 정확히 이것이다.

```
// 이 장면을 "목록 어딘가"가 아니라 반드시 빌드 인덱스 0으로
static void AddSceneToBuildSettings(string path)
{
    var scenes = EditorBuildSettings.scenes.ToList();
    scenes.RemoveAll(s => s.path == path);
    scenes.Insert(0, new EditorBuildSettingsScene(path, true));
    EditorBuildSettings.scenes = scenes.ToArray();
}
```

기존 항목을 지우고 0번에 다시 넣기 때문에, 빌드 목록에 무엇이 들어 있든 이 장면이 실제로 실행된다. 이 함수는 장면을 저장한 직후 매번 호출된다.

2. 씬을 손으로 만들지 않고 코드로 짓는다

2-1. 메뉴 한 번이면 장면 전체가 새로 생긴다

이 프로젝트의 가장 큰 특징은 콕핏 장면을 Unity 편집기에서 손으로 끌어다 놓아 만들지 않는다는 것이다. 좌석, 돔, 화면, 조종간, 카메라, 건담 배치까지 전부 편집기 스크립트 하나가 만든다. 편집기 상단 메뉴에

Gundam > Build Cockpit Scene이라는 항목이 있고, 이것을 누르면

GundamCockpitSetup.BuildCockpitScene() 이 실행되어 빈 장면을 새로 만들고, 그 안에 모든 것을 짓고, Assets/Scenes/GundamCockpit.unity 로 저장한 뒤 빌드 목록 0번에 올린다.

```
[MenuItem("Gundam/Build Cockpit Scene")]
public static void BuildCockpitScene()
{
    Scene scene = EditorSceneManager.NewScene(NewSceneSetup.EmptyScene, NewSceneMode.Single);

    CreateLighting();

    GameObject suitRoot = new GameObject("MobileSuitRoot");
    ShipMovementController ship = suitRoot.AddComponent<ShipMovementController>();

    GameObject interior = new GameObject("CockpitInterior");
    interior.transform.SetParent(suitRoot.transform, false);
    // ... 재질, 좌석, 건담, 돔, 화면, 링, 조종간, XR Origin ...
}
```

왜 이렇게 했을까. 이건호는 헤드셋을 쓰고 테스트한 뒤 “이거 지워”, “이거 올려”, “이건 없어야 해” 같은 짧은 요청을 계속 보냈고, Claude는 Unity 편집기를 직접 열어 볼 수 없었다. Claude가 할 수 있는 일은 텍스트 파일, 곧 C# 코드를 고치는 것뿐이다. 장면 파일(.unity)을 직접 편집하는 것은 위험하고 읽기도 어렵다. 반면 “장면을 짓는 코드”를 고치면, 이건호는 메뉴를 한 번 누르는 것으로 늘 같은 결과를 얻는다. 돔 주석의 표현대로 이 장면은 “*항상 이 생성기와 동기화되어 있으므로 Build Cockpit Scene이 이전 상태로 되돌아가는 일이 없다*”. 손으로 고친 장면은 다음 번 생성 때 덮어써지지만, 반대로 생성기에 들어간 수정은 절대 사라지지 않는다. 요청 하나하나가 코드 한 줄과 주석 한 단락으로 남는 이 방식 덕분에, 이 책이 지금 그 이들을 복원할 수 있다.

2-2. 생성기와 런타임 스크립트를 나눈 이유

코드는 두 갈래로 나뉜다. Assets/Editor/GundamCockpitSetup.cs 는 네임스페이스 `Gundam.EditorTools` 에 속하는 편집기 전용 스크립트이고, Assets/Scripts/Cockpit/ 아래의 파일들(`GundamHeadCam360`, `ExternalGundamFollower`, `CockpitHUD`, `ShipMovementController`, `JoystickLever` 등)은 네임스페이스 `Gundam.Cockpit`에 속하는 런타임 컴포넌트다.

Unity에서 `Editor`라는 이름의 폴더에 든 스크립트는 빌드에 포함되지 않는다. 생성기는 편집기 안에서 딱 한 번 돌아 장면의 “구조”를 만들고 끝난다. 매 프레임 일어나야 하는 일, 예를 들어 머리 카메라로 360도 영상을 다시 그리거나 건담을 플레이어 위치에 맞춰 옮기는 일은 생성기가 할 수 없다. 그래서 그런 일은 별도의 `MonoBehaviour` 파일로 만들고, 생성기는 그 컴포넌트를 오브젝트에 붙이고 필드를 연결해 주기만 한다. `GundamHeadCam360`의 설명 주석이 이 원칙을 그대로 적고 있다. “이 스크립트는 프레임마다의 런타임 동작만 담는다(생성기 스크립트는 장면 구조를 한 번 짓기만 하고, 매 프레임 로직을 돌릴 수 없다).” 손 추적 로그를 붙이는 대목에도 “런타임 동작은 여기가 아니라 자기 스크립트 파일에 둔다(이 파일은 장면 구조만 짓는다)”라고 같은 말이 반복된다. 자쿠에 붙이는 `EnemyMarker`도 “나중에 실제로 필요해지면 런타임 동작은 여기가 아니라 자기 스크립트 파일에” 넣겠다는 전제 아래 지금은 꼬리표 역할만 한다.

이 분리는 한 가지 더 깔끔한 습관을 낳았다. 생성기 함수들은 무언가를 찾지 못해도 장면 짓기 전체를 멈추지 않는다. 건담 FBX가 아직 가져와지지 않았으면 경고를 찍고 건담만 건너뛰고, XR 리그 샘플이 없으면

XR_ORIGIN_PLACEHOLDER_REPLACE_ME 라는 임시 리그(카메라 하나만 든)를 만들고 어떻게 바꿔 끼울지 안내를 적는다. 돔에 입힐 셰이더를 못 찾으면 한 단계 덜 좋은 재질로 물러선다. 이걸호가 어떤 순서로 파일을 넣든 메뉴를 누르면 일단 장면은 생긴다.

2-3. 재생성해도 깨지지 않게 — 저장된 에셋

코드로 장면을 매번 새로 짓는 방식에는 함정이 하나 있다. 실행 중에만 존재하는 객체(예: `new RenderTexture(...)`)를 장면에 연결해 두면, 장면을 저장하고 다시 열었을 때 그 참조가 사라진다 (“Missing”). 그래서 머리 카메라 영상용 렌더 텍스처는

`Assets/Models/Gundam/GundamHeadCam.renderTexture`, 360도 큐브맵은

`Assets/Models/Gundam/GundamHeadCam360.renderTexture` 라는 실제 에셋 파일로 저장한다. 그리고

`GetOrCreate...` 함수가 이미 있는 에셋을 다시 쓰도록 했다. 해상도를 바꾸게 되면 에셋을 지우고 새로 만드는 대신, 같은 에셋의 크기만 바꾼다. 그래야 GUID가 유지되어 보조 화면의 `RawImage`, 돔의 재질, 카메라의 `targetTexture` 가 모두 같은 파일을 계속 가리킨다(이 부분은 5절의 화질 이야기에서 다시 나온다).

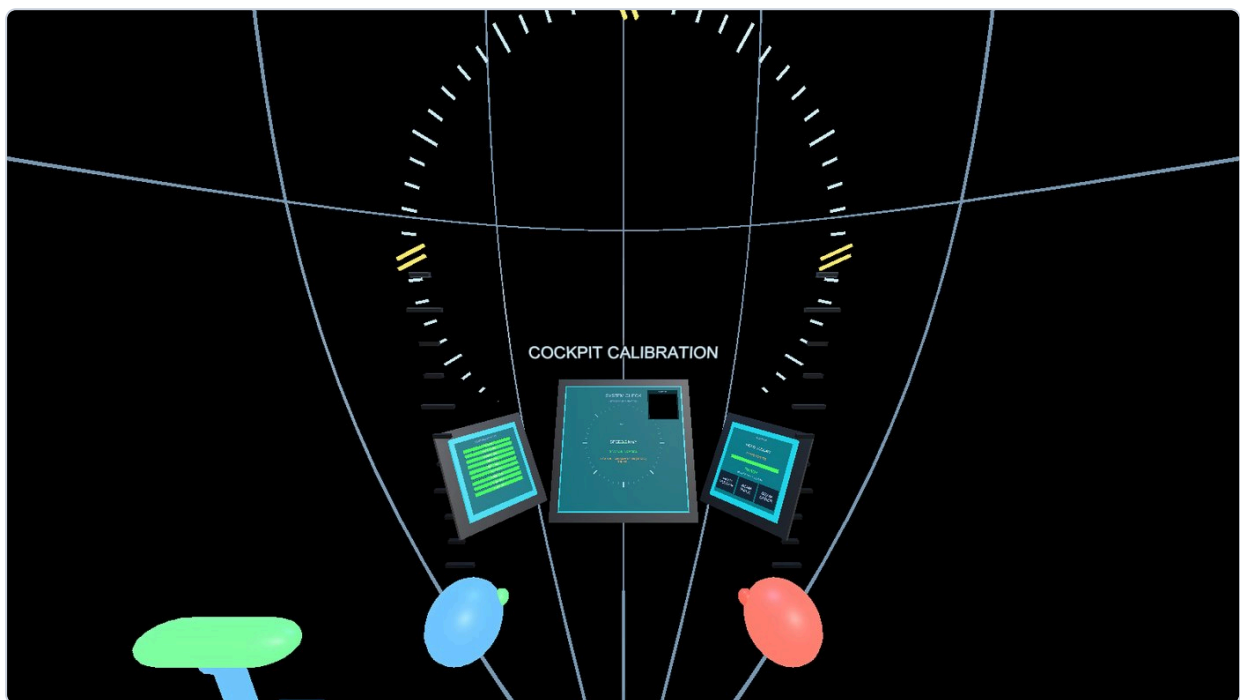
2-4. BuildCockpitScene이 짓는 순서

커밋 01 시점에 `BuildCockpitScene()` 은 아래 순서로 일한다. 순서 자체에도 이유가 있다. 특히 건담 배치가 돔과 화면보다 먼저 오는 것은, 건담 머리 카메라가 만든 렌더 텍스처를 돔과 화면에 곧바로 연결하기 위해서다. 주석에는 이것이 원래 반대 순서였다가 옮겨졌다고 적혀 있다(“*Moved to run AFTER PlaceExternalGundam (was before it)*”).

순 서	호출 / 생성물	하는 일
1	<code>NewScene(EmptyScene)</code>	완전히 빈 장면에서 시작.
2	<code>CreateLighting()</code>	방향광과 콕핏 안 보조광.
3	<code>MobileSuitRoot + ShipMovementController</code> , 그 아래 <code>CockpitInterior</code>	플레이어가 탄 기체의 뿌리. 콕핏 내부와 XR Origin이 모두 이 아래에 들어가 함께 움직인다.
4	재질 14종	<code>MakeMat / MakeEmissiveMat</code> 로 선체·좌석·화면·버튼·조종간·표적·별·궤도 링 재질을 만든다.
5	<code>BuildSeat()</code>	받침대·방석·등받이·머리받침·옆 받침으로 된 버킷 시트.
6	<code>PlaceExternalGundam()</code>	건담 FBX를 월드 공간에 세우고, 머리에 카메라를 달고, 평면 렌더 텍스처와 360도 큐브맵을 준비한다.
7	<code>ExternalGundamFollower</code> 부착	건담이 <code>MobileSuitRoot</code> 의 위치를 따라가게 한다.
8	<code>PlaceZakuEnemy()</code>	적 자쿠 배치(다음 장).
9	<code>BuildCockpitEnclosure()</code>	(숨긴) 바닥, 360도 영상을 보여 주는 구형 돔, 돔 위의 테두리 골조.
10	<code>BuildSystemCheckDisplay()</code>	“SYSTEM CHECK” 화면 3개와 장식. 왼쪽 화면에 머리 카메라 영상.
11	<code>BuildOrbitHUD()</code>	앞쪽 허공에 뜬 큰 눈금 링(궤도 표식).

12	<code>CreateHandTracker()</code> ×2, <code>CreateJoystick()</code> ×2	손 추적기와 좌우 조종간(다음 장).
13	<code>GunTurret</code> , <code>WeaponAimFireController</code> , <code>CockpithUD</code>	포탑과 무기, 화면 글자 갱신 연결.
14	<code>CreateXROrigin()</code>	XR 리그를 세우고 시야각·배경·컬링·추적 기준을 설정.
15	<code>BuildHandVisualizer()</code> , <code>WireJoystickHandInteractors()</code>	손 모양과 XRI 잡기 연결(다음 장).
16	<code>headCam360.playerCamera</code> 연결	플레이어 머리를 돌리면 건담 머리가 돌게.
17	<code>Targets</code> , <code>Starfield</code>	표적 큐브 6개, 별 60개.
18	<code>SaveScene</code> , <code>AddSceneToBuildSettings</code>	저장하고 빌드 0번에 등록.

이제 각 단계를 하나씩 들여다본다.



조종석 시점(완성 후 에디터 렌더). 구형 돔의 경선·위선, 위쪽 궤도 눈금 링, 가운데 SYSTEM CHECK 디스플레이 3개, 아래쪽 구형 조종간과 T 자 레버. 이 장에서 다루는 구형 콕핏 설계가 최종적으로 이런 모습이 되었다.

2-5. 조명 — CreateLighting

방향광(`Directional Light`) 하나는 세기 1.0, 회전 (50°, -30°, 0°)으로 장면 전체를 비춘다. 여기에 `Cockpit_FillLight` 라는 점광원을 하나 더 둔다. 색 (0.55, 0.75, 1)의 차가운 푸른빛, 세기 0.6, 범위 3.5m, 위치 (0, 1.6, 0.6)이다. 범위를 좁게 잡은 이유는 주석에 있다. 어두운 콕핏 안이 새까맣게 보이지 않고 “어두운 조종실 + 빛나는 패널”로 읽히게 하되, 바깥의 표적이나 별까지 밝히지는 않으려는 것이다.

2-6. 재질과 기본 도형 — MakeMat, MakeEmissiveMat, CreatePrimitiveGo

이 시점의 콕핏에는 외부에서 가져온 아트가 하나도 없다(건담과 자쿠 모델을 빼면). 전부 Unity 기본 도형(Cube, Sphere, Cylinder)과 단색 재질로 만든다. `MakeMat(Color)` 는 `Universal Render Pipeline/Lit` 셰이더로 재질을 만들고(없으면 `Standard`), `MakeEmissiveMat(base, emission)` 은 거기에 `_EMISSION` 키워드를 켜고 발광색을 넣는다. 공유 재질로 만들어 두는 것은 “싸고, 나중에 한 곳에서 색을 바꾸기 쉽기” 때문이다.

재질 변수	색(기본 → 발광)	쓰임
<code>hullMat</code>	(0.045, 0.045, 0.05)	어두운 주 선체, 좌석 받침대, 화면 테두리, 포탑 몸통
<code>hullAccentMat</code>	(0.11, 0.11, 0.13)	주석상 “패널 판(대시, 벽)”. 지금은 숨긴 바닥에만 쓰인다
<code>seatMat</code> / <code>seatAccentMat</code>	(0.07, 0.07, 0.085) / (0.035, 0.035, 0.045)	좌석 / 옆 받침
<code>gearMat</code>	(0.35, 0.35, 0.38)	주석상 “금속 칼라/기어”. 지금은 포신에만 쓰인다
<code>displayMat</code>	(0.02, 0.05, 0.07) → (0, 0.65, 0.8)	화면 면
<code>buttonGreenMat</code> / <code>buttonRedMat</code>	초록·빨강 발광	조종간 버튼
<code>leftAccentMat</code> / <code>rightAccentMat</code>	파랑(0.15, 0.55, 1) / 빨강(1, 0.2, 0.15) 발광	왼쪽(조향)·오른쪽(무기) 조종간 손잡이
<code>targetMat</code>	주황 발광	표적 큐브
<code>starMat</code>	검정 → 흰색 발광	별
<code>hudLineMat</code> / <code>hudGateMat</code>	하늘색(0.6, 0.85, 1) / 노랑(1, 0.82, 0.15) 발광	퀘도 링 눈금 / 관문 표식

“대시, 벽”, “칼라/기어” 같은 주석은 지금은 사라진 초기 콕핏의 흔적이다. 이 재질들은 남았지만 원래 칠하던 부품은 3절에서 볼 재설계 과정에서 지워졌다. 마찬가지로 `CreateCapsule()` 도우미 함수는 정의만 남아 있고 이제 어디서도 호출되지 않는다. 조종간 손잡이가 가로로 누운 캡슐 막대였던 시절의 흔적이다.

기본 도형은 모두 `CreatePrimitiveGo()` 를 거친다. 도형을 만들고, 이름을 붙이고, 부모 아래 로컬 위치와 크기를 정하고, 재질을 입힌다. `CreateCube`, `CreateCylinder`, `CreateSphere` 는 이 함수의 한 줄짜리 포장이다. 두 점을 잇는 가는 막대가 필요할 때는 `CreateBeam(a, b, thickness)` 을 쓴다. 두 점의 중간에 길이만큼 늘인 얇은 큐브를 놓고 `Quaternion.LookRotation` 으로 방향을 맞춘다. Unity에는 곡선 도형이 없기 때문에, 돔의 둥근 골조는 이 짧은 막대를 수십 개 이어 붙여 만든다.

2-7. 좌석 — BuildSeat

좌석은 `CockpitInterior` 아래 로컬 좌표로 짓는다. 원기둥 받침대 `Seat_Base` 가 (0, 0.18, -0.25)에 크기 (0.28, 0.18, 0.28)로 서고, 그 위에 방석 `Seat_Pan` 이 높이 0.42m에 0.55×0.12×0.55로 얹힌다. 등받이 `Seat_Back` 는 (0, 0.85, -0.5)에 0.55×0.75×0.12 크기로 뒤로 8° 젖혀 있고, 머리받침 `Seat_Headrest` 는

(0, 1.28, -0.52)에 $0.4 \times 0.22 \times 0.12$ 로 -6° 기울어 있다. 양옆의 `Seat_Bolster_L/R`은 $x = \pm 0.32$, 높이 0.5에 $0.1 \times 0.2 \times 0.55$ 크기로, 각각 $\mp 12^\circ$ 안쪽으로 기울어 몸을 감싼다. 콕핏 공간의 기준점은 이 좌석이다. 플레이어의 눈 위치는 $z = -0.15$ 근처로 잡혀 있고(XR Origin의 위치와 화면 배치의 기준점이 모두 -0.15다), 화면과 조종간과 궤도 링은 모두 그 눈 앞쪽(+z)에 놓인다.

나머지 단계(건담 배치, 돔, 화면, 링, XR Origin)는 이 장의 핵심이라 3·4절에서 콕핏 설계가 바뀌어 간 순서에 맞춰 자세히 다룬다. 마지막 두 가지 장식만 여기서 짚어 둔다.

2-8. 표적과 별

`Targets` 아래에는 사격용 표적 큐브 6개(`Target_0 ~ Target_5`, 한 변 0.8m, `HitTarget` 컴포넌트)가 놓인다. 위치는 고정 시드 1234의 난수로 정한다. 짝수 번은 왼쪽, 홀수 번은 오른쪽이고, $|x|$ 는 3.5~12.5m, 높이는 -1~5m, 거리 z 는 12m부터 6m 간격(12, 18, ... 42m)이다. 주석은 표적을 “앞쪽 화면 바로 뒤에 겹쳐 쌓지 않고, 화면 주변의 트인 공간에” 두려고 옆으로 밀어냈다고 설명한다. 표적은 기체의 자식이 아니라 월드 공간에 고정된다.

`Starfield`는 “움직이는 느낌”을 주기 위한 별 60개다. 단위 구면 위의 무작위 방향으로 25~70m 떨어진 곳에 0.25m짜리 흰 발광 큐브를 하나씩 놓고, 충돌체는 지운다. 이 별들은 플레이어가 기체를 움직일 때 시차로 이동감을 준다. 다만 5절에서 보듯 플레이어가 실제로 보는 바깥 풍경은 결국 “건담 머리 카메라가 본 영상”이 되므로, 별도 그 영상 속에서 보이게 된다.

3. 콕핏 설계의 변천 — 평범한 조종석에서 구형 돔까지

커밋 01의 콕핏은 처음 모습과 완전히 다르다. 처음 모습은 지금 코드에 직접 남아 있지 않지만, “old”, “was”, “has been removed” 같은 주석이 무엇이 있었는지를 꽤 구체적으로 알려 준다. 그 흔적을 모으면 다음과 같다.

3-1. 처음 형태 — 주석이 기억하는 초기 콕핏

처음 콕핏은 흔히 떠올리는 “상자형 조종실”이었다. 앞에는 평범한 곡면 앞유리(curved windshield)와 구석의 HUD가 있었다. 좌석 앞에는 계기와 버튼 격자가 달린 물리적인 대시보드 콘솔이 있었다. 옆으로는 기동·레일·버팀대·벽 패널로 된 구조 골조가 둘러싸고, 위와 옆은 평평한 천장과 벽으로 막혀 있었다.

조종간은 좌석 팔걸이 자리에 있었고, 콘솔 하우징, 연결 팔, 래칭 칼라, 표시등까지 갖춘 기계식 조이스틱처럼 생겼으며, 손잡이는 가로로 누운 캡슐 막대였다. 사격 스크립트 `WeaponAimFireController`와 `HitTarget`이 9월 21일 오후 4시 58분 무렵에 이미 생겨 있었으니, 기체에 고정된 포탑(`GunTurret`, 로컬 (0, 1.0, 2.4))과 주황색 표적 큐브도 초기 구성에 속했다고 볼 수 있다. 화면 글자를 갱신하는 `CockpitHUD`의 “THROTTLE ... TURN ...” 표시는 `ShipMovementController` 주석이 “원래부터 있던(pre-existing)” 표시라고 부르는 것이다.

앞에서 본 남은 재질 이름(“대시, 벽”의 패널 판, “칼라/기어”의 금속)과 호출되지 않는 `CreateCapsule`이 바로 이 초기 형태의 화석이다.

3-2. 사진을 보고 전면 재설계 — “SYSTEM CHECK” 화면

첫 번째 큰 전환은 이건호가 참고 스크린샷을 보여 주며 한 요청이었다.

요청 · 지금 콧속에 있는거 다지우고 이런거 만들어

참고 이미지는 애니메이션 속 메카 콧속의 “SYSTEM CHECK / BOOT CONFIGURATION” 부팅 화면이었다. 주석에 따르면 그 화면은 큰 중앙 화면(원형 눈금 다이얼과 조종사·무기·탄약 표시), 그 양옆의 작은 보조 화면 두 개, 아래의 빨간 강조 막대, 양 끝의 세로 눈금 사다리로 이루어져 있었다. Claude는 앞유리와 구석 HUD를 통째로 걷어내고, 이 구성을 기본 도형과 절차적으로 만든 uGUI로 흉내 냈다. 주석은 “가져온 아트 없이 도형과 uGUI로만 만들었으므로, 참고 이미지를 픽셀 단위로 재현하는 게 아니라 분위기를 근사한 것”이라고 스스로 한계를 밝힌다. 이것이 `BuildSystemCheckDisplay()` 다.

이 화면을 어디에 둘지는 이건호가 직접 짰었다.

요청 · 내가 앉은 자리에 허벅지쯤 위에 있는거야 이게

주석은 이 배치를 일부러 앞유리와 대비시켜 설명한다. 화면들은 “멀리 눈높이에 뜬 앞유리”가 아니라 “앉은 사람의 허벅지 바로 위에 있는, 얼굴 쪽으로 들려 있는 계기판”이 되었다. 높이 `baseY = 0.92f`, 위로 젖힌 각도 `tiltX = 36f` 로, 아래를 내려다보는 조종사에게 화면이 정면으로 보이도록 기울였다.

화면 세 개의 배치 — `CreatePolarScreen`

세 화면은 플레이어 눈 위치를 중심으로 한 원호 위에 놓인다. 각 화면의 각도 `angleDeg` 와 반지름 `radius` 가 주어지면 위치는 $x = r \cdot \sin \theta$, $z = \text{eyeZ} + r \cdot \cos \theta$ 가 되고, 회전은 $(\text{tiltX}, \theta, 0)$ 이다. 그러면 모든 화면이 눈 쪽을 향한다.

```
float angleRad = angleDeg * Mathf.Deg2Rad;
float x = radius * Mathf.Sin(angleRad);
float z = eyeZ + radius * Mathf.Cos(angleRad);

GameObject root = new GameObject(name);
root.transform.SetParent(parent, false);
root.transform.localPosition = new Vector3(x, baseY, z);
root.transform.localRotation = Quaternion.Euler(tiltX, angleDeg, 0f);

CreateCube(name + "_Frame", root.transform, Vector3.zero, new Vector3(width + 0.06f, height + 0.06f, 0.03f), frameMat);
GameObject screen = CreateCube(name + "_Screen", root.transform, new Vector3(0, 0, -0.02f), new Vector3(width, height, 0.02f), screenMat);
```

화면마다 가로세로 6cm 더 큰 테두리 큐브(`hullMat`)가 뒤에 붙고, 그 앞 2cm에 발광 화면 면(`displayMat`) 이 붙는다. 함수는 화면 면의 Transform을 돌려주고, 그 위에 월드 공간 Canvas를 세운다.

부품	각도	크기(가로× 세로, m)	내용
----	----	------------------	----

SysCheck_Center	0°	0.42 × 0.42	“SYSTEM CHECK”, “- BOOT CONFIGURATION -”, 48개 눈금 원형 다이얼, “PILOT: ---”, 무기 이름, “AMMO ---/---”, 원격측정 글자
SysCheck_Left	-40°	0.20 × 0.24	건담 머리 카메라 영상(HEAD CAM). 없으면 “HEAD CAM - NO SIGNAL”
SysCheck_Right	+40°	0.20 × 0.24	“AUX-R” 장식(가는 막대 6줄)
SysCheck_AccentBar	0°, 0.24m 아래	0.36 × 0.015	빨간 발광 막대
SysCheck_TickLadder_L/R	x = ±0.52	눈금 14개, 간격 0.09m	세 번째마다 긴 눈금(0.09m), 나머지 0.05m. 순수 장식

중앙 Canvas는 760×760 단위에 배율 0.00054(약 0.41m 폭)로, 보조 화면 Canvas는 380×440 단위에 배율 0.00047로 화면 면 바로 앞(-0.015m)에 붙는다. 중앙 화면의 원형 다이얼 `BuildTickRing()` 은 반지름 230 단위에 48개의 눈금을 놓고, 6번째마다 길고 밝은 “주 눈금”(5×30)으로, 나머지는 3×16으로 그린다. 글꼴은 Unity 내장 `LegacyRuntime.ttf` 다.

이 화면들은 장식만이 아니다. 무기 이름과 탄약 글자, 원격측정 글자는 `out Text` 로 돌려받아 `CockpitHUD` 에 연결된다. `CockpitHUD.Update()` 는 매 프레임 왼쪽 조종간의 기울기로 “THROTTLE … % TURN … %”, 오른쪽 조종간으로 “AIM X … Y …”, 잡고 있으면 “[L GRIP] [R GRIP]”을 찍고, 무기 컨트롤러에서 현재 무기 이름과 “AMMO 030/030” 형식의 탄약을 읽어 넣는다.

3-3. “360도 전부 둥그런 화면” — 상자에서 돔으로

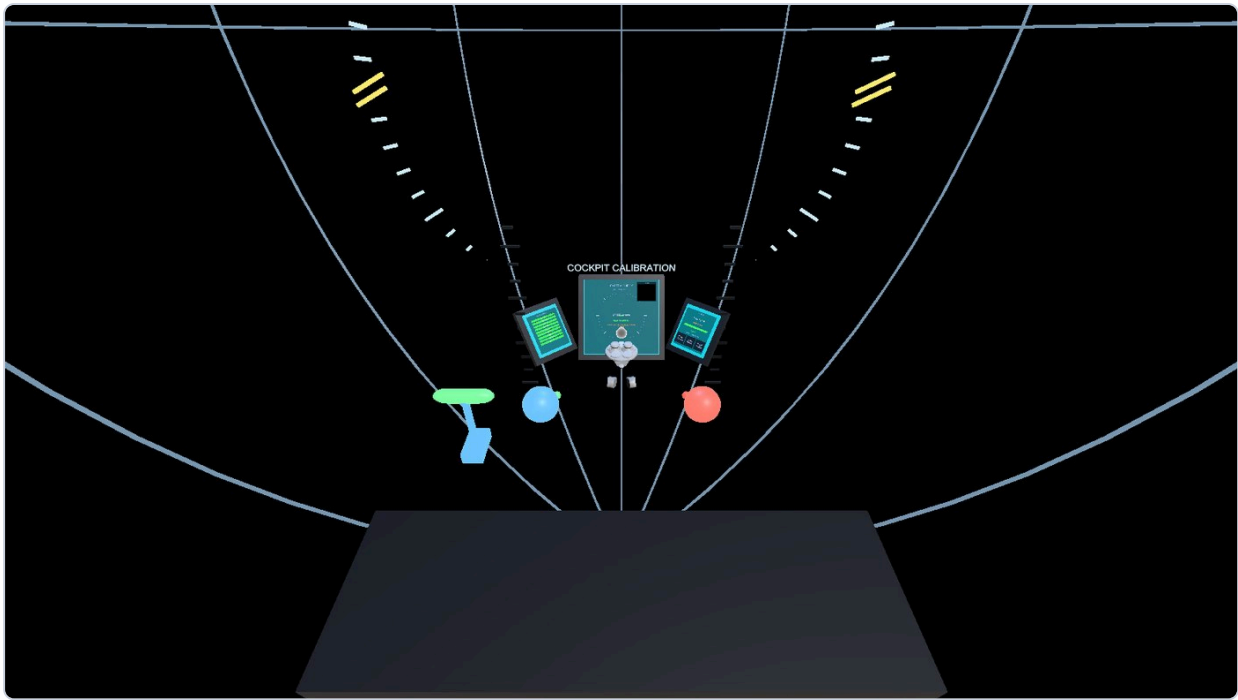
평평한 천장과 벽으로 둘러싼 상자는 다음 요청으로 사라졌다.

요청 · 공간이 360도 전부 둥그런 화면이어야해

Claude는 천장·뒤·양옆·앞 위쪽을 각각의 판으로 막는 대신, 이음매 없는 하나의 곡면으로 모든 방향을 감싸기로 했다. Unity 기본 구(Sphere)를 축마다 다르게 늘여 타원체로 만든 `Cockpit_Dome` 이다. 판을 이어 붙이면 틈이나 이음매로 현실 공간(또는 스카이라이프)이 비칠 수 있지만, 하나의 닫힌 구면은 원리상 틈이 없다.

```
Vector3 domeCenter = new Vector3(0, 1.75f, 0.5f);
Vector3 domeScale = new Vector3(4.6f, 4.2f, 6.2f);
```

기본 구의 지름이 1이므로 반지름은 좌우 2.3m, 위아래 2.1m, 앞뒤 3.1m다. 따라서 돔은 로컬 좌표로 x -2.3~2.3, y -0.35~3.85, z -2.6~3.6 범위를 감싼다. 앞뒤로 길쭉한 이유는 좌석 앞쪽에 화면, 조종간, 궤도 링이 늘어설 공간이 필요하기 때문이다. 이 여유도 일부러 넉넉히 잡았다. 주석은 이유를 이렇게 설명한다. XR 리그 샘플은 추적 기준 모드가 “지정 안 함(Not Specified)”이라 대부분의 OpenXR 런타임에서 바닥 기준(Floor)으로 해석되고, 그러면 실제 카메라 높이는 플레이어의 실제 머리 높이(앉아서도 대략 1.6~2.0m)가 된다. 돔이 빠듯하면 카메라가 돔 표면에 묻히거나 돔 위로 빠져나가 “새까만 화면”이 된다. 그래서 예전 평평한 천장·벽이 쓰던, 이미 충분하다고 확인된 범위를 그대로 돔에 썼다.



좌석 뒤에서 본 콕핏 전체. 조종석을 감싼 불투명한 구 안쪽 면이 곧 바깥 세상을 비추는 스크린이다(에디터 사진이라 바깥 영상은 비치지 않음).

안쪽에서 보이는 구 — Cull Front와 Unlit

보통의 메시는 바깥 면만 그린다. 카메라가 구 안쪽에 있으면 아무것도 보이지 않는다. 그래서 돔 재질은 컬링을 **앞면 제거(Cull Front)**로 바꿔 안쪽 면(뒷면)만 그리게 한다. 또 하나, 조명을 받는 Lit 셰이더를 쓰면 안 된다. 뒤집어 그려지는 면의 법선은 여전히 바깥을 향하므로, Lit 셰이더는 모든 면이 조명 반대쪽을 본다고 계산해 새까맣게 칠한다. 그래서 법선과 조명을 무시하는 **Unlit** 셰이더를 쓴다. `MakeShellMat()` 은 `Universal Render Pipeline/Unlit` 에 색 (0.05, 0.055, 0.07)을 넣고 `_Cull` 을 1(Front)로 설정한 가장 기본적인 돔 재질이다. 이 원칙은 뒤에 나오는 모든 돔 재질과 직접 쓴 셰이더에 그대로 이어진다.

3-4. 구 안에 있다는 느낌 — 테두리 선(캐노피 골조)

돔은 한동안 “보이지 않는” 상태였다. 주석에 따르면 그 전의 한 요청이 진짜 “바깥”을 보고 싶다는 것이어서, 돔의 렌더러를 꺼 뒤쪽의 별과 스카이박스가 비치게 했던 시기가 있다. 그 상태에서는 돔이 둥글다는 것을 플레이어가 알 수 없다. 이건호는 둥근 유리 캐노피가 조종사를 감싸는 사진을 보여 주며 이렇게 요청했다.

요청 · 이런식으로 콕핏이 구형으로 보여야해 그안에 있을때 구형안에 있는 느낌

답은 돔과 똑같은 타원체를 따라가는 가는 골조, 곧 `Canopy_Frame` 이다. 실제 캐노피의 유리판 사이 프레임처럼, 둥근 선이 공간의 곡률을 보여 준다. `BuildCanopyFrame()` 은 두 종류의 선을 긋는다.


```

const int ribCount = 6;           // 경선 6개, 30°씩 (각각 구의 양쪽을 다 지남)
const int meridianSegments = 20;
const int equatorSegments = 24;
const float beamThickness = 0.025f;

for (int r = 0; r < ribCount; r++) {
    float theta = r * (180f / ribCount) * Mathf.Deg2Rad;
    for (int s = 0; s <= meridianSegments; s++) {
        float phi = s * (360f / meridianSegments) * Mathf.Deg2Rad;
        Vector3 unit = new Vector3(Mathf.Sin(phi) * Mathf.Cos(theta), Mathf.Cos(phi),
        Mathf.Sin(phi) * Mathf.Sin(theta));
        Vector3 point = domeCenterLocal + Vector3.Scale(unit, domeRadii);
        // 이전 점과 이 점을 CreateBeam으로 잇는다
    }
}

```

세로로 도는 대원(大圓) 6개가 30°씩 돌아가며 위아래를 잇고, 각 대원은 20개의 짧은 막대로 근사된다. 여기에 돔 중심 높이에서 수평으로 한 바퀴 도는 적도 고리(24개 막대)가 경선들을 묶는다. 막대 굵기는 2.5cm, 재질은 어둡게 은은히 빛나는 발광색 (0.08, 0.1, 0.14)이다. 돔 반지름에 그대로 `domeScale * 0.5f` 를 넘기므로 골조는 돔 표면 위에 정확히 얹힌다. 모든 막대에서 충돌체를 지운다(`StripCollider`).

지워지지 않은 옛 설명

`BuildCanopyFrame` 의 요약 주석은 아직 “돔 메시 자체는 보이지 않는 상태로 남아 있어도 둥근 모양이 보이도록”이라고 말한다. 골조를 만들 때 돔이 투명했다는 증거다. 아래 3-8에서 보듯 돔은 나중에 다시 불투명한 “디스플레이”가 되었지만, 이 요약 문장은 고쳐지지 않고 남았다. 커밋 01의 주석을 읽을 때는 이런 시차를 염두에 두어야 한다.

3-5. 벽과 기둥을 걷어낸다

돔이 공간을 감싸게 되자 초기의 구조 골조는 쓸모가 줄었다. 이견호의 요청은 간단했다.

요청 · 옆에 벽이랑 기둥은 없애도 되고

옆 기둥, 레일, 버팀대, 벽 패널로 된 골조가 지워졌다. 주석은 이것들이 “돔이 공간을 감싸게 된 지금은 대체로 중복이었고, 오히려 돔을 보는 시야를 가리고 있었다”고 정리한다. 지금 `BuildCockpitScene` 에는 그 자리에 이 설명 주석만 남아 있다.

3-6. 대시보드는 화면 세 개가 대신한다

계기와 버튼 격자가 달린 물리 대시보드 콘솔도 같은 운명이었다.

요청 · 대시보드 역할이 아까 만든 디스플레이 3개가 하는거야

“아까 만든 디스플레이 3개”는 3-2의 SYSTEM CHECK 화면이다. 정보 표시 역할을 전부 그 세 화면이 넘겨받았으므로 물리 콘솔은 제거되었다. 좌석 바로 다음 줄에 남은 주석이 이 결정의 기록이다. 이로써 콕핏 안에 “만질 수 있는 물건”은 좌석, 화면 세 개, 그리고 조종간 두 개만 남게 된다.

3-7. 궤도를 알려 주는 링 — OrbitHUD

이건호는 애니메이션 콕핏의 HUD 링 사진(둥근 눈금 고리와 궤도상의 “관문” 표식이 그려진)을 보여 주며 이렇게 요청했다.

요청 · 콕핏에 외관에 구형이 있어서 그 구형에 밖에 보이면서 저런 표식으로 궤도를 알려주는 시스템이 있어야해

Claude는 중앙 화면의 원형 다이얼(`BuildTickRing`)과 같은 “눈금 고리” 개념을, 이번에는 평면 Canvas가 아니라 3D 월드 공간에 큰 크기로 만들었다. 작은 모니터 안에 갇히지 않고 조종사의 앞 시야에 떠서 바깥 풍경 위에 겹쳐 보이게 하려는 것이다. 이것이 `BuildOrbitHUD()` 다.

항목	값
중심 위치(로컬)	(0, 1.45, 2.4) — 낮은 화면 묶음보다 한참 위·앞, 돔 안쪽
링 반지름	1.5m
눈금 수	60개(6°마다)
주 눈금	5번째마다, $0.02 \times 0.16 \times 0.02$ (보통 눈금은 높이 0.09)
관문(gate) 표식	15번째마다 → 0°, 90°, 180°, 270°의 4개. 노란 막대 두 개($0.022 \times 0.22 \times 0.022$)를 $\pm 0.035\text{m}$ 간격으로 세우고 둘 다 25° 기울인 “평행 사선” 표식
재질	눈금 <code>hudLineMat</code> (하늘색 발광), 관문 <code>hudGateMat</code> (노란 발광)

주석은 이 링이 “지금은 정적인 장식”이라는 점도 분명히 한다. 실제 궤도나 경로 데이터는 계산하지 않고, 노란 관문 표식은 참고 이미지의 밝은 사선 쌍 표식을 대신하는 자리표시일 뿐이다. 이 링은 다음 장에서 “적을 보면 타겟팅이 되는” 시스템의 바탕이 된다(`TargetingHUD`, 9월 22일 오전 9시 55분 무렵 파일 생성).

순서를 알려 주는 주석

`BuildOrbitHUD` 위의 설명은 링이 “투명한 *Cockpit_Dome* 너머로 보이는 바깥 위에 겹쳐 뜬다”고 말하고, 링 위치를 정한 주석은 화면 묶음의 반지름을 “radius 0.55”라고 적는다. 링을 만들 때 돔은 아직 투명했고, 화면은 아직 0.55m 거리에 있었다는 뜻이다. 반면 같은 링을 호출하는 `BuildCockpitScene` 쪽 주석은 이미 “지금은 돔 자체에 보이는 실시간 머리 카메라 영상 위에 겹친다”로 고쳐져 있다.

3-8. 바닥을 없애고, 바깥은 전부 디스플레이로

콕핏 아래에는 $4.0 \times 0.1 \times 5.4\text{m}$ 의 바닥 판 `Cockpit_Floor` (중심 (0, -0.05, 0.5))가 있었다. 다음 요청으로 이것이 사라졌다.

요청 · 콕피드 아래 바닥은 없어야하고... 건담에 머리에서 바라보고 있는 모습이 나와야해

주변 풍경 전체가 “건담 머리가 보는 것”이 되어야 하는데, 콕핏에 붙은 작은 바닥 판은 그 그림에 어울리지 않는다는 판단이다. 조종사가 실제로 만지는 좌석·화면·조종간은 그대로 두고 바닥만 숨겼다. 다만 지우지 않고 “렌더러만 끈” 채로 남겼다. 주석의 말로는 “한 줄만 바꾸면 되살릴 수 있게” 게임 오브젝트는 계속

짓는다. 충돌체도 지운다. 이 주석은 바닥을 숨기는 처리가 “아래 Cockpit_Dome과 같은 처리”라고 적고 있는데, 이것 역시 바닥을 숨긴 시점에 돔도 렌더러가 꺼진 투명 상태였다는 단서다.

그리고 결정적인 반전이 왔다.

요청 · 콕피트에서 밖에 보이던 콕피트에 외관은 전부 디스플레이어야해 그래야 로봇안에 있는거지 로봇에 시점으로만 세상을 보는거야

투명한 돔 너머로 별과 배경이 그대로 보이는 것은 “열린 우주에 둥둥 떠 있는” 느낌이지, “밀폐된 로봇 조종실 안에서 화면을 보는” 느낌이 아니라는 것이다. 이견호가 원한 것은 분명했다. 조종사는 로봇 안에 갇혀 있고, 바깥 세상은 오직 로봇의 눈(카메라)을 통해서만 본다. 그래서 돔의 렌더러가 다시 켜졌다. 이번에는 단색 껍데기가 아니라, 건담 머리 카메라의 실시간 영상을 표면 전체에 입힌 “디스플레이”다. 처음에는 왼쪽 보조 화면에 이미 쓰고 있던 평면 렌더 텍스처를 URP Unlit 재질(MakeUnlitScreenMat, `_Cull = Front`)에 입혀 구 전체에 씌웠다. 이 방식이 왜 다시 바뀌어야 했는지는 4절과 5절에서 이어진다.

이 결정과 함께 BuildCockpitEnclosure()는 건담 배치 뒤로 옮겨졌다. 돔에 칠할 렌더 텍스처를 만드는 것이 건담 배치 함수이므로, 순서가 바뀌어야 텍스처를 돔에 바로 넘길 수 있다.

3-9. 화면 거리와 조종간 위치

화면 세 개는 처음 원호 반지름 0.55m에 놓였다. 테스트 후 이견호의 보고는 이랬다.

문제 보고 · 화면이 너무 가까이 붙어 보인다

Claude는 방향(기울기 36°, 화면별 각도 0°/±40°)과 배치 모양은 그대로 두고, 같은 방향으로 0.08m만 뒤로 물렸다. `const float radius = 0.63f; // was 0.55f - still within arm's reach`. 여전히 팔이 닿는 거리라는 점을 확인하는 주석이다. 이 변경으로 중앙 화면은 로컬 (0, 0.92, 0.48), 보조 화면은 약 (±0.405, 0.92, 0.333)으로 옮겨졌다.

조종간은 다음 장의 주제지만, 위치가 화면 배치에 묶여 있어서 여기서 짧게 짚는다. 조종간은 원래 좌석 팔걸이 자리에 있었는데, 이견호가 이렇게 요청했다.

요청 · 조종장치에 위치가 디스플레이 3개 사이에 그쯤에 있어야할거같아

Claude는 세 화면의 좌표를 “중앙 (0, 0.92, 0.4), 왼쪽 (-0.3536, 0.92, 0.2713), 오른쪽 (0.3536, 0.92, 0.2713)”이라고 적고, 그 사이 틈인 (±0.22, 0.82, 0.18)에 조종간을 놓았다. 이 좌표는 반지름 0.55m 시점의 값이다($0.55 \times \sin 40^\circ \approx 0.3536$). 즉 조종간을 화면 사이로 옮긴 것은 화면을 0.63m로 물리기 전이었다. 주석은 Claude 스스로 “편집기에서 손잡이가 화면 테두리를 뚫고 들어가 보이는지 확인할 수 없으니, 그렇게 보이면 말해 달라”고 적고 있다. 이후 보고

문제 보고 · 조종기를 조금 위쪽으로 올린다

에 따라 높이만 0.05m 올려 지금의 (±0.22, 0.87, 0.18)이 되었다. X와 Z는 그대로다.

3-10. 좌석에 앉은 눈높이 — 카메라가 너무 높거나, 아예 돔 밖으로

VR에서 “내 눈이 어디에 있느냐”는 헤드셋 추적과 장면 배치가 함께 정한다. 이 이틀 동안 이 문제가 두 번 불거졌고, 두 번의 수정이 좌석에 겹쳐 남아 있다.

첫 번째는 “카메라가 너무 높다”는 불만이었다(주석은 “camera too high”라고만 적고 있다). 주석에 따르면 그 전에는 XR Origin에 좌석 높이만큼의 오프셋이 따로 들어 있었다. Claude의 설명은 이렇다. 샘플 리그의 추적 기준 모드는 “Not Specified”이고 실제 OpenXR 런타임은 이것을 바닥 기준으로 해석하므로, 런타임 카메라의 Y는 이미 “실제 바닥에서 젠 플레이어의 실제 머리 높이”다. 거기에 좌석 높이를 또 더하면 카메라가 좌석보다 한참 위로 올라간다. 그래서 XR Origin을 Y = 0에 두었다.

`CreateXROrigin(suitRoot.transform, new Vector3(0, 0f, -0.15f))`. 콕핏 바닥(y = 0)과 현실 바닥이 일치하므로, 실제로 앉으면 눈이 대략 머리받침(1.28m) 높이 근처에 오리라는 계산이었다.

두 번째는 더 심각했다.

문제 보고 · 검정화면만 나와... 360도가 다 바뀌어야해

어느 쪽을 봐도 바뀌지 않는 검은 화면. Claude의 진단은 “카메라가 아예 360도 돔 안에 있지 않다”였다. 바닥 기준 추적에서는 카메라 높이가 플레이어의 실제 키와 방 조건에 따라 1.6~2.0m 이상으로 달라지고, 경우에 따라 돔 밖으로 나가 버린다. 이 게임은 방 안을 걸어 다니는 룸 스케일이 아니라 앉아서 하는 고정 콕핏이므로, 현실 바닥 높이가 카메라 높이에 영향을 주어서는 안 된다. 그래서 `CreateXROrigin()`의 끝에서 추적 기준을 **Device**로 강제했다.

```
XROrigin xrOriginComponent = instance.GetComponentInChildren<XROrigin>(true);
if (xrOriginComponent != null)
{
    xrOriginComponent.RequestedTrackingOriginMode = XROrigin.TrackingOriginMode.Device;
}
```

Device 모드는 헤드셋의 상대 자세만 쓰고 실제 바닥 높이를 무시하므로, 카메라는 스크립트가 놓은 자리(돔 안)에 머문다. 흥미로운 것은 `BuildCockpitScene` 안의 XR Origin 주석은 여전히 첫 번째 수정 때의 “바닥 기준이 자연스럽게 눈을 머리받침 높이에 놓는다”는 논리를 설명하고 있다는 점이다. 두 번째 수정이 그 논리 위에 덧씌워졌지만 앞 주석은 갱신되지 않았다. 커밋 01에서 실제로 동작하는 것은 Device 모드 쪽이다.

3-11. 시야를 넓혀라

요청 · 내 시점이 더 넓어야 할 것 같아

같은 `CreateXROrigin()`에서 리그의 카메라를 찾아 `fieldOfView = 100f`로 넓혔다. Unity 기본값 60°는 사람의 자연스러운 시야보다 좁아 “줌 인”된 느낌을 주고, 콕핏을 실제보다 답답하게 만든다는 이유다. 다만 Claude는 한계도 정직하게 적었다. 실제 OpenXR 헤드셋에서 눈마다의 시야각은 결국 렌즈와 런타임이 정하므로 이 값이 기기에서 시야를 바꾼다는 보장은 없고, 편집기 미리보기나 이 값을 존중하는 경로에서만 확실히 효과가 있으며, 어느 쪽이든 해는 없다는 것이다. 근접 클리핑 평면도 0.05m 이하로 당겨, 가까운 조종간과 화면이 잘려 보이지 않게 했다.

4. 건담을 들여오다 — 머리 카메라와 360도 돔

4-1. 건담 FBX — 있는 구조를 그대로

9월 22일 오전 10시 30분 무렵 `Assets/Models/Gundam` 폴더가 생겼다. 이건호가 “방금 만든” 건담 FBX 모델을 프로젝트에 넣은 것이다. 요청은 다음과 같았다.

요청 · 방금 만든 Gundam FBX 모델을 현재 gundam game 프로젝트에 추가해줘... 콕핏 앞쪽 가상 공간에 배치한다

그리고 모델 구조에 대한 분명한 지시가 함께 있었다.

요청 · FBX 내부에 실제 본/메시 구조가 이미 있다면 임의로 다시 만들지 말고 기존 구조를 최대한 유지한다

FBX 안에는 이미 22개 뼈로 된 이족 보행 골격과 스킨드 메시 하나가 들어 있었다(Root → Hips → Spine/Spine1/Spine2 → Neck → Head, 좌우 어깨·팔·손 사슬, Hips에서 갈라지는 좌우 다리 사슬). Claude는 이 내부 계층을 전혀 건드리지 않고, 프리팹으로 인스턴스를 하나 만든 뒤 그 뿌리 오브젝트의 이름 (`ExternalGundam`)과 위치·크기만 정했다. 몸통·머리·팔·다리를 따로 묶는 계층을 새로 만들지 않았다. 모델 경로는 생성기 맨 위 상수 `GundamModelPath` 한 곳에만 적어, 이름이 바뀌면 거기만 고치면 되게 했다(“*Build Cockpit Scene*이 읽는 유일한 기준”).

FBX를 막 복사해 넣은 직후에는 Unity가 아직 가져오기를 끝내지 않아 `LoadAssetAtPath`가 null을 돌려줄 수 있다. 이 경우 `PlaceExternalGundam()`은 장면 짓기를 멈추지 않고, “Assets > Refresh를 하거나 프로젝트를 다시 연 뒤 메뉴를 다시 실행하라”는 경고만 찍고 건담(과 머리 카메라)을 건너뛴다.

크기와 위치

Claude는 편집기를 열어 가져온 뒤의 수치를 볼 수 없었으므로, 원본 FBX의 경계 상자를 assimp 정보 출력으로 읽었다. 최소 (-0.222, -0.156, 0.000), 최대 (0.222, 0.156, 0.985). Z축이 가장 길고(0.985) 최솟값이 정확히 0이므로, Z-위(Z-up)로 내보낸 모델이고 기준점이 발밑이라고 해석했다. Unity의 FBX 가져오기는 파일에 적힌 위쪽 축을 보고 Y-위로 바꿔 주므로 똑바로 서서 들어올 것이라 예상했지만, 주석에는 “누워 있거나 돌아가 있으면 말해 달라, 보정 회전을 넣겠다”는 단서가 붙어 있다.

```
const float rawHeight = 0.985469f;
const float targetHeight = 18f;
float scale = targetHeight / rawHeight;

instance.transform.position = new Vector3(0f, 0f, 20f);
instance.transform.rotation = Quaternion.identity;
instance.transform.localScale = Vector3.one * scale;
```

목표 키는 RX-78-2급 건담의 정석 높이인 약 18m다. $18 \div 0.985469 \approx 18.27$ 배로 키운다. 이 “모빌슈트는 18m로 보인다”는 약속은 상수 `MobileSuitTargetHeight = 18f`로도 정의되어, 적 자쿠의 크기를 맞출 때 다시 쓰인다. 위치는 월드 (0, 0, 20)이다. 돔 앞 끝(로컬 z 약 3.6)보다 확실히 바깥이고, 표적들(z 12~42) 사이에 있어 “조종사 정면에 바로 보이는, 착각할 수 없는 시험 배치”라는 것이 주석의 설명이다. 건담은

MobileSuitRoot 의 자식이 아니라 월드 공간에 둔다(SetParent(null)). 기체의 회전을 물려받지 않게 하려는 것인데, 이 결정은 4-7에서 문제를 낳는다.

색을 입히고, 색이 번진 버그

처음 들어온 건담은 기본 회색 재질 하나뿐이었다. 이건호가 색 텍스처를 따로 준비해 넣었다.

요청 · 건담 색깔 다운해둬

Mobile Suit Gundam-baseColor.png 를 ApplyBaseColorTexture() 로 입힌다. 이 함수는 특정 셰이더 속성 이름을 가정하지 않고 Material.mainTexture 를 쓴다. URP Lit이라면 Unity가 알아서 _BaseMap 에 넣어 준다. 같은 함수로 자쿠에도 색을 입히게 되자 버그가 났다.

문제 보고 · 자쿠 색깔이 건담한테도 들어갔는데

원인은 공유 재질이였다. 두 FBX 모두 자기 재질 없이 같은 기본 재질로 들어왔는데, 처음 함수는 renderer.sharedMaterials[i] 를 그 자리에서 고쳤다. 공유 재질 에셋 하나에 자쿠 텍스처를 넣으면, 같은 재질을 가리키는 건담도 자쿠 색이 된다. 수정은 모델마다 재질을 복사해 자기만의 재질을 갖게 한 뒤 텍스처를 넣는 것이다.

```
foreach (Renderer renderer in root.GetComponentsInChildren<Renderer>(true))
{
    Material[] materials = renderer.sharedMaterials;
    for (int i = 0; i < materials.Length; i++)
    {
        if (materials[i] == null) continue;
        Material uniqueMat = new Material(materials[i]); // 모델별 복사본
        uniqueMat.name = materials[i].name + " (" + label + ")";
        uniqueMat.mainTexture = texture;
        materials[i] = uniqueMat;
    }
    renderer.sharedMaterials = materials;
}
```

4-2. “건담 머리의 시선이 내 콕핏 화면에” — HeadCam

건담을 세운 다음 요청이 이 게임의 방향을 정했다.

요청 · 건담에 머리에 시선이 내 콕핏 화면에 나와야해

건담의 골격에는 이미 Head 뼈가 있다. FindDeepChild() 가 계층 깊이를 가정하지 않고 이름(대소문자 무시)으로 재귀 탐색해 찾는다. 거기에 HeadCam 이라는 카메라를 달고, 그 카메라가 렌더 텍스처 에셋 GundamHeadCam.renderTexture 에 그리게 한다. 카메라 설정은 시야각 60°, 근점 클리핑 0.05m, 배경은 단색 (0.01, 0.01, 0.025, 1)로 플레이어 시점과 같은 “깊은 우주색”이다. 그리고 BuildSystemCheckDisplay() 가 왼쪽 보조 화면에 BuildHeadCamScreenUI() 로 “HEAD CAM” 글자와 340×340 크기의 RawImage 를 만들어 그 텍스처를 띄운다. 이것이 건담 시점이 콕핏에 들어온 첫 모습이다. 작은 모니터 하나에 건담의 눈이 보는 장면이 나오는 것이다.

카메라를 어디에 둘 것인가 — 세 번의 시도

카메라 위치는 쉽지 않았다. 주석에는 세 단계가 기록되어 있다.

첫 번째는 `Head` 뼈의 원점에 카메라를 그대로 두는 것이었다. 결과는 흐릿하고 균일한 회색 화면이었다. 스크린샷에는 깨끗한 별 대신 평평한 안개 속에 궤도 링만 보였다고 한다. 원인은 자동 리깅된 뼈의 기준점이 머리 표면이 아니라 머리 덩어리의 *안쪽 중심*에 있는 경우가 흔하다는 것이었다. 카메라가 머리 속에 묻혀, 회색 기본 재질로 된 머리 내부 면을 코앞에서 찍고 있었던 것이다.

두 번째는 모든 기하를 피해 머리 위로 카메라를 올리는 것이었다. 회색 안개는 사라졌지만, 이번에는 “너무 높다, 조종사의 눈이 아니라 드론 촬영 같다”는 후속 요청이 왔다.

세 번째가 지금의 방식이다. 뼈의 기준점이 아니라 스킨드 메시의 *실제 렌더 경계*를 기준으로, 전체 키의 92% 높이를 눈높이로 잡는다. 그런데 눈높이는 머리의 세로 범위 *안에* 있으므로 그냥 내리기만 하면 다시 머리 속에 묻힐 위험이 있다. 그래서 키의 4%만큼 앞(+z)으로 밀어 머리 덩어리의 중심에서 얼굴 앞면 쪽으로 빼낸다. 앞으로 미는 양을 경계 상자의 깊이가 아니라 키에 비례하게 잡은 것은, 깊이는 뺀 팔이나 든 무기 때문에 부풀 수 있기 때문이다.

```
SkinnedMeshRenderer meshRenderer = instance.GetComponentInChildren<SkinnedMeshRenderer>();
Bounds meshBounds = meshRenderer != null ? meshRenderer.bounds : new Bounds(head.position,
Vector3.one);

float eyeHeight = meshBounds.min.y + meshBounds.size.y * 0.92f;
float forwardClearance = meshBounds.size.y * 0.04f;

GameObject camGo = new GameObject("HeadCam");
camGo.transform.position = new Vector3(meshBounds.center.x, eyeHeight, meshBounds.center.z +
forwardClearance);
camGo.transform.rotation = Quaternion.LookRotation(Vector3.forward, Vector3.up);
camGo.transform.SetParent(head, true); // 월드 위치 유지한 채 Head 뼈에 붙임
```

키가 18m라면 눈높이는 발밑에서 약 16.6m, 앞으로 민 거리는 약 0.72m다. 카메라는 +z(표적과 궤도 링이 늘어선 방향)를 바라본다. 위치를 정한 뒤 `SetParent(head, true)` 로 뼈에 붙이는 이유는, 4-5에서 머리 뼈가 돌아갈 때 카메라가 위치와 회전을 함께 따라가게 하기 위해서다. 92%와 4%라는 비율은 Claude가 눈으로 확인할 수 없는 추정값이라고 주석에 적혀 있다. 다시 회색 안개가 끼면 아직 묻혀 있는 것이고, 어색하면 비율을 조정하겠다는 것이다.

4-3. “내가 저 안에 타고 있는 설정이니까” — 레이어 30과 컬링

건담을 콕핏 정면 20m에 세우자, 플레이어의 눈에 “다른 로봇 한 대가 앞에 서 있는” 모습이 보였다. 이건호는 이 설정의 논리를 짚었다.

요청 · 건담이 콕핏 안에서 보이면 안돼 왜냐면 내가 저 안에 타고 있는 설정이니까

조종사가 곧 이 건담이므로, 건담의 겉모습이 조종사 자신의 1인칭 시야에 떠 있어서는 안 된다. Claude는 이것을 레이어로 풀었다. 상수 `GundamBodyLayer = 30` 을 정하고, `SetLayerRecursively()` 로

`ExternalGundam`의 메시와 모든 뼈를 레이어 30에 넣는다. 그리고 플레이어 카메라의 컬링 마스크에서 그 한 레이어만 뺀다.

```
viewCamera.cullingMask &= ~(1 << GundamBodyLayer);
```

왜 30일까. 주석의 설명은 이렇다. 레이어는 Tag Manager에 이름을 붙이지 않아도 번호만으로 동작하고, 30은 Unity가 기본적으로 아무것에도 배정하지 않는 두 레이어 중 하나라서 이 프로젝트의 다른 무엇과도 겹칠 가능성이 매우 낮다. 별, 표적, 궤도 링, 콧핏 같은 나머지는 플레이어 카메라에 그대로 보인다.

머리는 안 보이고, 몸통은 보이게

처음에는 머리 카메라(`HeadCam`)도 같은 레이어를 제외했다. 같은 영상이 돔 전체로 늘어나자, 화면 아래쪽에 가슴과 어깨가 보이는 것이 “내 로봇의 몸을 바깥에서 보는” 것처럼 느껴졌기 때문이다. 그런데 이견호의 다음 요청은 그 반대였다.

요청 · 건담에 머리는 안보이는데 몸통은 보였으면 좋겠어

주석은 “REVERSED”라는 말로 이 뒤집기를 기록한다. 지금 `HeadCam`은 레이어를 하나도 제외하지 않는 기본 마스크다. 그런데도 머리는 화면에 나오지 않을 것이라고 Claude는 예상했다. 카메라가 사실상 머리 앞면에 붙어서 바깥을 보고 있으므로 머리의 기하는 렌즈 옆이나 뒤에 있고, 더 아래·더 멀리 있는 몸통과 어깨가 정면 시야에 들어온다는 기하학적 가정이다. 주석은 이것도 확인하지 못한 가정이며, 예컨대 바로 아래를 볼 때 머리가 보이면 셰이더 기반의 높이 자르기를 넣겠다고 적고 있다. 정리하면, 플레이어의 직접 시야(XR 카메라)는 레이어 30을 빼서 바깥 건담을 숨기고, 건담 시점 영상(`HeadCam`)은 레이어 30을 포함해 자기 몸통을 보여 준다.

4-4. “360도로” — 스카이박스 큐브맵

왼쪽 작은 화면에 건담 시점이 나오게 되자, 이견호는 그것을 콧핏을 둘러싼 어두운 공간 전체로 넓히길 원했다.

요청 · 콧핏에 검정부분에서 건담에 머리부분에서 보이는거 처럼 해야해 360도로

한 대의 카메라는 한 방향만 본다. 모든 방향을 보려면 큐브맵이 필요하다. 정육면체의 여섯 면에 각각 한 방향씩 그려 놓은 텍스처다. Claude는 런타임 컴포넌트 `GundamHeadCam360`을 새로 만들어 `HeadCam`에 붙였다. 생성기는 Unity 내장 `Skybox/Cubemap` 셰이더로 스카이박스 재질을 만들어

`RenderSettings.skybox`에 넣고, 이 컴포넌트에 카메라·재질·머리 뼈·큐브맵을 연결한다. 그러면 `CreateXROrigin()`은 스카이박스가 설정된 것을 보고 플레이어 카메라의 배경을 단색이 아닌 `CameraClearFlags.Skybox`로 바꾼다. 셰이더를 찾지 못하면 360 배경만 건너뛰고 평면 화면은 계속 동작한다.

`GundamHeadCam360.Start()`는 큐브맵 렌더 텍스처를 준비해 스카이박스 재질의 `_Tex`에 넣는다. 이 시점에는 생성기가 넘겨준 저장 에셋이 있으면 그것을 쓰고, 없으면(예전에 지은 장면) 실행 중에만 존재하는 큐브맵을 `resolution = 256` 크기로 만든다. 그리고 `LateUpdate()`에서 매 프레임

`headCam.RenderToCubemap(_cubemap)` 을 불러 여섯 방향을 모두 다시 그린다. 건담과 그 주변이 움직여도 둘러싼 풍경이 실시간으로 따라온다.

4-5. “내가 머리를 돌리면 건담 머리도” — 머리 따라 돌리기

요청 · 내가 머리를 돌리면 건담 머리도 돌아야해

같은 `GundamHeadCam360` 에 `playerCamera` 필드가 생겼다. 문제는 연결 순서였다. 건담과 머리 카메라는 장면 짓기 6단계에서 만들어지는데, 플레이어 카메라를 가진 XR 리그는 14단계에서야 생긴다. 그래서 생성기는 XR Origin을 만든 직후 두 번째로 돌아와, 리그 안의 `Camera` 를 찾아

`gundamResult.headCam360.playerCamera` 에 넣는다. 이 때문에 `PlaceExternalGundam()` 은 텍스처 하나가 아니라 `GundamPlacementResult` 라는 작은 클래스(평면 텍스처, 360 컴포넌트, 큐브맵, 건담 인스턴스)를 돌려준다. 모델이 없으면 `null`을 돌려줄 수 있도록 구조체가 아닌 클래스로 만들었다.

```
if (!_haveCameraStart)
{
    _startCameraRotation = playerCamera.rotation;
    _haveCameraStart = true;
}
Quaternion cameraDelta = playerCamera.rotation * Quaternion.Inverse(_startCameraRotation);
head.rotation = cameraDelta * _startHeadRotation;
```

플레이어 카메라의 회전을 머리 뼈에 그대로 복사하지 않는다는 점이 중요하다. 리그에 구워진 머리 뼈의 기본 자세가 카메라 축과 맞는다는 보장이 없으므로, 그대로 복사하면 엉뚱한 “정면”으로 머리가 꺾일 수 있다. 대신 시작할 때 두 회전을 기준으로 저장하고, “플레이어가 처음보다 얼마나 돌았는가”(델타)만큼을 머리 뼈의 원래 자세에 곱한다. 머리는 자기 자연스러운 자세에서 출발해 플레이어가 돈 만큼만 돈다. 이 계산은 `LateUpdate` 에서 하므로, 그 프레임의 헤드셋 추적이 반영된 뒤 머리를 돌리고, 바로 이어서 큐브맵을 그린다. 머리 뼈에 붙은 `HeadCam` 은 위치와 방향이 함께 따라 돌므로, 왼쪽 보조 화면의 평면 영상도 플레이어가 보는 쪽을 비춘다.

다시 만들어 볼 독자를 위한 메모

Unity의 `Camera.RenderToCubemap` 은 카메라의 위치에서 여섯 면을 월드 축 방향으로 그린다. 그래서 머리를 돌리는 효과는 큐브맵의 “방향”보다는 평면 영상의 방향, 그리고 머리 앞 0.72m쯤에 달린 카메라 위치가 머리를 따라 원을 그리며 옮겨 가는 것으로 나타난다. 돔은 아래 4-6에서 보듯 월드 방향으로 큐브맵을 읽으므로, 플레이어가 왼쪽을 보면 돔의 왼쪽 면에서 큐브맵의 왼쪽 방향 그림이 보인다. 머리를 돌리지 않아도 “창문”처럼 맞는 방향이 보이는 구조다.

4-6. 평면 영상을 구에 씌우면 생기는 일 — 큐브맵 돔 셰이더

3-8에서 돔은 “바깥은 전부 디스플레이”라는 요청에 따라, 평면 렌더 텍스처(`headCamTex`)를 Unlit 재질로 구 전체에 씌우는 방식이 되었다. 이 방식은 확실히 그러지는 대신 처음부터 약점이 알려져 있었다. 주석의 원래 단서는 “완벽하게 이음매 없는 360 감싸기를 포기하고, 반드시 렌더되는 결과를 택했다”였다. 이 약점이 두 번의 보고로 드러났다. 첫 번째는 화질(5절), 두 번째는 이것이다.

40m 떨어진 곳에 자쿠를 세웠는데 돔에 보이지 않았다. Claude의 분석은 정확했다. 평면 영상은 그 순간 머리 카메라가 보는 60° 원뿔 안의 것만 담는다. 원뿔 밖의 물체는 처음부터 영상에 없으므로, 그것을 구 위에 어떻게 늘려 붙여도 되살아나지 않는다. 자쿠는 대부분의 순간 그 원뿔 밖에 있었다.

해법은 이미 매 프레임 그리고 있던 스카이박스용 큐브맵을 돔에도 쓰는 것이었다. 큐브맵은 모든 방향을 담고 있다. 다만 스카이박스는 카메라 배경이라, 불투명한 돔이 카메라를 완전히 감싸는 지금은 결코 보이지 않는다. 그렇다고 내장 Skybox/Cubemap 셰이더를 돔 메시에 직접 씌우면 URP에서 분홍색으로 깨질 수 있다. 그래서 Claude는 URP용 셰이더를 직접 손으로 썼다. Assets/Shaders/CockpitDomeCubemap.shader, 이름은 Custom/CockpitDomeCubemap 이다. Shader Graph가 아니라 읽을 수 있는 텍스트 파일 하나로 쓴 것도 의도된 선택이다.

```
Varyings Vert(Attributes IN)
{
    Varyings OUT;
    OUT.positionHCS = TransformObjectToHClip(IN.positionOS.xyz);
    // 구의 정점 법선은 중심에서 바깥을 향한다 → 월드로 돌리면 곧 "보는 방향"
    OUT.dirWS = TransformObjectToWorldNormal(IN.normalOS);
    return OUT;
}

half4 Frag(Varyings IN) : SV_Target
{
    half3 col = SAMPLE_TEXTURECUBE(_CubeTex, sampler_CubeTex, normalize(IN.dirWS)).rgb;
    return half4(col * _Exposure, 1.0);
}
```

원리는 단순하다. 기본 구의 정점 법선은 구 중심에서 바깥으로 뻗는다. 이것을 월드 공간으로 돌리면 “돔 중심에서 이 점을 향한 방향”이 되고, 그 방향으로 큐브맵을 읽으면 그 방향에 실제로 있는 풍경이 나온다. 방향 벡터에는 평행이동이 의미가 없으므로 돔이 어디에 있든 상관없고, 돔 중심이 조종사의 머리 가까이에 있으므로 이 방향은 “조종사가 여기서 그쪽을 볼 때의 방향”과 거의 같다. 정상적인 스카이박스와 똑같은 원리를, 카메라 배경이 아닌 보통 메시 위에서 쓰는 것이다. 늘어남도 없고, 머리 카메라가 그 순간 어디를 보고 있었든 빠지는 방향이 없다.

셰이더의 나머지 설정도 앞의 원칙을 따른다. 태그는 "RenderPipeline" = "UniversalPipeline", 패스는 "LightMode" = "UniversalForward" 하나, Cull Front 로 안쪽 면만 그리고, 조명 계산 없이 텍스처 색만 낸다. 노출 조절용 _Exposure (0.1~4, 기본 1.0)가 있고, 문제가 생기면 Universal Render Pipeline/Unlit 으로 물러선다.

돔 재질의 3단 대비책

BuildCockpitEnclosure() 는 돔 재질을 고를 때 세 단계로 물러선다. 첫째, MakeCubemapScreenMat() 이 큐브맵과 셰이더가 모두 있으면 360 재질(Cockpit_Dome_LiveFeed360)을 만든다. 둘째, 셰이더를 못 찾았는데 큐브맵은 있으면 “평면 영상을 늘려 붙이면 자쿠 같은 먼 물체가 안 보일 것”이라는 경고를 찍고 평면 영상 재질(Cockpit_Dome_LiveFeed)로 간다. 셋째, 머리 카메라가 아예 없으면 단색 껍데기

(MakeShellMat)를 쓴다. 어느 경우든 돔에 구멍이 나서 바깥 허공이 보이는 일은 없다. 돔의 충돌체는 지운다.

이 수정을 위해 큐브맵도 실행 중에만 존재하는 것에서 **저장된 에셋**(GundamHeadCam360.renderTexture, 차원 Cube)으로 바뀌었다. 같은 큐브맵을 스카이박스과 돔 재질이 함께 참조하려면, 장면 저장 후에도 살아 있는 실제 에셋이어야 하기 때문이다. GundamHeadCam360 에 cubemap 필드가 추가된 것이 이때다. 커밋 01에 이 셰이더 파일이 이미 들어 있으므로 이 수정은 9월 23일 오전 10시 06분 이전의 일이다(사용자 PC의 셰이더 파일 시각 9월 28일 11시 13분은 이후 수정 시각으로 보인다).

4-7. “LeftJoystick을 움직여도 …” — ExternalGundamFollower

다음 장에서 다룰 조종간 이동이 동작하기 시작하자, 이번에는 바깥 풍경이 꼼짝하지 않는 문제가 드러났다.

문제 보고 · LeftJoystick을 움직여도 실제 Gundam이 움직이는 것이 화면에서 보이지 않는다

왼쪽 조종간은 ShipMovementController를 통해 MobileSuitRoot를 정상적으로 움직이고 있었다(최대 8m/s, 회전 60°/s). 콧핏 내부와 XR Origin은 그 아래에 있으므로 함께 움직인다. 그런데 4-1에서 건담은 일부러 월드 공간에 따로 세웠다. 기체의 회전을 물려받지 않으려던 이 결정 때문에 건담은 이동도 물려받지 않았다. 그리고 돔과 왼쪽 화면에 바깥 풍경을 그리는 카메라는 바로 그 건담의 머리에 붙어 있다. 기체는 움직이는데 풍경을 찍는 카메라는 제자리였으니, 화면에는 아무 변화도 없었던 것이다.

Claude는 최소한의 수정을 택했다. 새 런타임 컴포넌트 ExternalGundamFollower를 건담에 붙이고, target에 MobileSuitRoot를 넣는다.

```
void LateUpdate()
{
    if (target == null) return;

    if (!_haveOffset)
    {
        _offset = transform.position - target.position;    // 처음 한 번, 지금의 상대 위치
        _haveOffset = true;
    }

    transform.position = target.position + _offset;
}
```

처음 실행될 때의 상대 위치(배치한 그대로)를 한 번 기억하고, 그 뒤 매 프레임 기체의 현재 위치에 그 차이를 더한다. 오프셋을 생성기에서 미리 계산하지 않고 첫 실행 때 잡는 것은, 컴포넌트 초기화 순서에 상관없이 “지금 내 위치와 대상의 차이”를 잡으면 되기 때문이다. 재생 전에는 아무것도 움직이지 않으므로 그 값은 원래 배치 그대로다. Update가 아니라 LateUpdate를 쓰는 것은 ShipMovementController가 그 프레임에 기체를 옮긴 뒤에 따라가야 한 프레임 늦는 일이 없기 때문이다.

이 스크립트는 위치만 다룬다. 회전은 건드리지 않고, MobileSuitRoot나 조종간, 카메라에도 쓰지 않는다. 회전은 이미 따로 정리된 문제였다. ShipMovementController의 주석에는 이전 보고 “왼손으로 LeftJoystick을 잡고 돌리면 XR Origin / Main Camera의 머리 시야까지 같이 돌아간다”가 기록되어 있다.

기체 뿌리를 `transform.Rotate` 로 돌리면 그 아래의 XR 카메라까지 돌아가고, 헤드셋 움직임과 무관한 시야 회전은 VR 멀미의 대표적인 원인이다. 그래서 기체의 방향은 `HeadingYaw` 라는 숫자로만 관리하고 실제 Transform 회전은 절대 바꾸지 않는다. 결과적으로 건담 머리의 회전은 오직 `GundamHeadCam360` 이 플레이어의 실제 머리 움직임으로만 정한다.

5. “건담시야로 보는 밖이 화질이 깨짐” — 화질과의 첫 씨름

평면 영상을 돔 전체에 씌운 직후, 이건호가 헤드셋으로 본 바깥 풍경은 거칠었다.

문제 보고 · 건담시야로 보는 밖이 화질이 깨짐

원인은 해상도와 면적의 불일치였다. 머리 카메라의 렌더 텍스처는 원래 512×512 였다. 가로 20cm짜리 왼쪽 보조 화면에서는 그 정도면 충분했다. 그런데 같은 텍스처가 이제 반지름 2~3m짜리 돔 전체로 늘어났다. 512 픽셀이 시야를 둘러싼 면 전체에 퍼지면, 가까이서 볼 때 계단처럼 깨지고 흐려 보인다.

5-1. 첫 대응 — 1024로 올리고, 같은 에셋을 키운다

Claude는 해상도를 1024×1024 로 올렸다. `const int HeadCamResolution = 1024;` 주석은 이것을 “모바일 XR 헤드셋에 합리적인 중간값”이라고 부른다. 이 카메라는 매 프레임 그리므로 해상도가 공짜가 아니다. 그래서 “아직 선명하지 않으면 더 올리겠고, 프레임이 너무 떨어지면 다시 내리겠다”는 말이 함께 붙어 있다.

숫자만 바뀌서는 효과가 없다는 점도 Claude는 짚었다. 렌더 텍스처는 이미 저장된 에셋이므로, `GetOrCreateHeadCamRenderTexture()` 는 그것을 찾으면 그대로 돌려줄 뿐이고 상수를 바꿔도 반영되지 않는다. 그래서 기존 에셋의 크기가 다르면 그 자리에서 크기를 바꾸도록 했다.

```
if (existing.width != HeadCamResolution || existing.height != HeadCamResolution)
{
    existing.Release();
    existing.width = HeadCamResolution;
    existing.height = HeadCamResolution;
    existing.Create();
    EditorUtility.SetDirty(existing);
    AssetDatabase.SaveAssets();
}
return existing;
```

지우고 새로 만들지 않고 같은 에셋을 키우는 이유는 2-3에서 말한 GUID다. 보조 화면의 `RawImage`, 돔의 재질, 카메라의 `targetTexture` 가 모두 같은 파일을 계속 가리키므로, 끊어지는 참조가 생기지 않는다. 새로 만드는 경우에는 깊이 버퍼 16비트로 만든다.

5-2. 근본 원인을 더 파고들다 — 해상도가 아니라 방식

`GundamHeadCam360` 의 `cubemap` 필드 주석은 이 과정을 “‘건담시야로 보는 밖이 화질이 깨짐’을 더 파고든 뒤, 이어서 ‘밖에 자쿠가 보이지 않아’라는 말로 기록한다. 화질 문제는 해상도만의 문제가 아니었다. 60° 시야로

찍은 평면 영상 하나를 구 전체에 늘려 붙이는 한, 해상도를 아무리 올려도 영상은 왜곡되고 시야 밖의 것은 빠진다. 결국 4-6의 큐브맵 돔이 화질 문제와 자쿠 문제를 함께 푸는 근본 해법이 되었다. 큐브맵은 방향마다 제 그림을 갖고 있으므로 늘어남이 없다.

5-3. 큐브맵의 해상도 — 256에서 512로

큐브맵에도 해상도 판단이 따랐다. 처음 `GundamHeadCam360` 이 실행 중에 만들던 큐브맵은 면당 256이었다. 그때는 이 큐브맵이 스카이박스에만 쓰였고, 불투명한 돔이 스카이박스를 늘 완전히 가리고 있었으므로 실제로 보이는 일이 없었다. 하지만 이제 돔이 이 큐브맵을 직접 읽으므로 해상도를 더 줄 가치가 생겼다. 저장 에셋으로 만들면서 면당 512로 정했다. `const int HeadCam360CubemapResolution = 512;` 이 역시 “여섯 면을 매 프레임 다시 그리는 것은 공짜가 아니므로” 고른 중간값이고, 선명도와 프레임의 저울질은 다시 이건호의 테스트에 맡겨졌다. 기존 에셋이 있으면 크기나 차원(Cube)이 다를 때 같은 에셋을 고쳐 쓰는 방식도 평면 텍스처와 같다.

텍스처	처음	커밋 01	쓰이는 곳
GundamHeadCam (평면)	512×512	1024×1024	왼쪽 보조 화면 RawImage, (대비책) 돔의 평면 영상 재질
GundamHeadCam360 (큐브맵)	면당 256, 실행 중에만 존재	면당 512, 저장 에셋	스카이박스 <code>_Tex</code> , 돔 셰이더 <code>_CubeTex</code>

계산해 보면 이 구성은 매 프레임 평면 1024×1024(약 105만 픽셀)에 큐브맵 512×512×6면(약 157만 픽셀)을 추가로 그린다. 그 위에 헤드셋이 양쪽 눈을 그린다. 이 비용과 선명도의 줄다리기는 9월 22일에 끝나지 않았다. 셰이더 파일의 수정 시각(9월 28일)과 9월 30일 무렵의 `HeadCamInsetCube.shader` 가 보여 주듯, 바깥 풍경의 화질은 이후 장에서 계속 다듬어진다.

5-4. 또 하나의 화질 문제 — 회색 안개

화질 문제로 분류할 수 있는 것이 하나 더 있다. 4-2에서 본 “흐릿하고 균일한 회색” 영상이다. 이것은 해상도가 아니라 카메라가 모델 속에 묻혀서 생긴 문제였고, 카메라를 메시 경계 기준으로 눈높이(92%)와 앞쪽(4%)에 옮겨 해결했다. 이건호가 같은 “화면이 이상하다”는 증상을 보고해도 원인은 전혀 다를 수 있다는 것, 그리고 Claude가 매번 “다시 회색이 되면 아직 묻혀 있는 것”처럼 다음 보고를 해석할 단서를 남겨 둔 것이 이 이틀 작업의 특징이다.

6. 커밋 01 시점의 콕핏 — 무엇이 어디에 있다

이들의 결과로 메뉴 **Gundam > Build Cockpit Scene**을 누르면 이어지는 장면은 다음과 같다(손·조종간·자쿠 관련 세부는 다음 장).

```
GundamCockpit.unity
├ Directional Light / Cockpit_FillLight
├ MobileSuitRoot (ShipMovementController, WeaponAimFireController, CockpitHUD)
│ └ CockpitInterior
│   └ Seat_Base / Pan / Back / Headrest / Bolster_L / Bolster_R
│   └ Cockpit_Floor (렌더러 꺼짐)      └ Cockpit_Dome (Custom/CockpitDomeCubemap)
│   └ Canopy_Frame (경선 6 + 적도 1)    └ SystemCheckDisplay (Center / Left=HEAD CAM / Right /
AccentBar)
│   └ SysCheck_TickLadder_L / R      └ OrbitHUD (눈금 56 + 관문 4)
│   └ LeftJoystick_Mount / RightJoystick_Mount
│   └ GunTurret
│   └ XR Origin (Gundam Cockpit) (FOV 100, Skybox 배경, 레이어 30 제외, Device 추적)
│     └ LeftHandTracker / RightHandTracker ...
├ ExternalGundam (레이어 30, ExternalGundamFollower) ... Head └ HeadCam (GundamHeadCam360)
├ ZakuEnemy (EnemyMarker) └ Targets (Target_0 ~ 5)
└ Starfield (Star_0 ~ 59)
```

콕핏의 정체성은 이 이틀 동안 근본적으로 바뀌었다. 시작은 앞유리와 대시보드, 벽과 기둥으로 이루어진 “창문 달린 조종실”이었다. 끝은 바닥도 벽도 없이 좌석 하나가 구형 화면 한가운데 떠 있고, 그 화면에 18m 건담의 눈높이(약 16.6m)에서 본 360도 실시간 풍경이 흐르는 구조다. 그 위에 캐노피 골조가 구의 곡률을 그리고, 앞에는 궤도 링이 뜨고, 허벅지 위에는 SYSTEM CHECK 화면 세 개가 계기판 역할을 한다. 플레이어의 직접 시야에서 건담의 몸은 레이어 30으로 숨겨져 있고, 건담의 눈이 보는 영상에서만 몸통이 보인다. 플레이어가 고개를 돌리면 건담의 고개가 따라 돌고, 왼쪽 조종간을 밀면 건담이 따라 움직인다. 이견호가 말한 “로봇안에 있는거지 로봇에 시점으로만 세상을 보는거야”가 코드로 처음 구현된 상태다.

이 구조는 이후 프로젝트 전체의 뼈대가 된다. 조종간을 잡는 손, 포탑과 무기, 적 자쿠와 타깃 링은 모두 이 콕핏 위에 얹힌다. 다음 장은 같은 이틀 동안 그 손과 조종간이 어떻게 만들어졌는지를 따라간다.

처음부터 따라 해 보려면 — 이 장의 순서 요약

① Unity 6000.6.0f1로 URP 프로젝트를 만든다. ② Packages/manifest.json 에 OpenXR 1.18.0, Android XR OpenXR 1.4.1, XR Hands 1.6.3, XRI 3.3.2, XR Management 4.6.0, Input System 1.20.0을 넣고 가져오기가 끝날 때까지 기다린다. ③ XR Plug-in Management에서 Android용 OpenXR을 켜고 Hand Tracking Subsystem, Microsoft Hand Interaction Profile, Meta Hand Tracking Aim 등을 켜다. 최소 SDK 29, ARM64, Linear. ④ XR Hands의 HandVisualizer 샘플을 가져온다(XRI 샘플 두 개는 생성기가 가져온다. AR Starter Assets는 가져오지 않는다). ⑤ Assets/Editor 에 장면 생성기를, Assets/Scripts/Cockpit 에 런타임 컴포넌트를, Assets/Shaders 에 돔 셰이더를 둔다. ⑥ 건담 FBX와 색 텍스처를 Assets/Models/Gundam 에 넣고 가져오기가 끝나면 Gundam > Build Cockpit Scene을 누른다. 장면은 매번 처음부터 다시 지어지고, 렌더 텍스처 에셋만 재사용된다.

이 장이 다루는 것

앞 장에서는 Unity 프로젝트를 만들고, 조종석 의자와 돔, 건담 머리 카메라로 "로봇 안에 앉아 있는" 느낌을 만드는 과정을 따라갔다. 이 장은 같은 이틀(9월 21일~22일)의 나머지 절반을 다룬다. 이건호가 콧핏 안에서 자기 손을 보고, 그 손으로 조종간을 잡아 기체를 움직이고, 오른손 조종간으로 포를 겨누어 쏘고, 바깥에 첫 적인 자쿠를 세워 두는 데까지다.

이 구간은 Git을 쓰기 전에 한 작업이라 날짜별 diff가 없다. 9월 23일 오전 10시 6분에 찍힌 첫 실질 커밋(커밋 01)에 9/21~9/22 결과물이 한꺼번에 들어 있다. 그래서 이 장은 그 스냅샷의 코드와 주석을 재료로 쓴다. 다행히 Claude는 코드를 고칠 때마다 주석에 이건의 요청 원문을 따옴표로 남기고, 이전 방식이 왜 실패했는지도 적어 두었다. "per request", "per report", "Bug fix", "used to", "REVERSED", "UPDATE" 같은 표시가 붙은 주석을 따라가면, diff가 없어도 시행착오의 순서가 꽤 선명하게 드러난다. 다만 커밋 01은 9/23 오전에 찍혔으므로, 이 장에서 설명하는 최종 상태에는 9/23 아침까지 손본 내용이 섞여 있을 수 있다. 아래에서 "이 시점의 코드"라고 하면 커밋 01 스냅샷을 말한다.

이건의 PC에 남은 파일 생성 시각을 보면 대략적인 흐름이 보인다. 생성 시각은 파일을 다시 쓰면 바뀌기도 하므로 "이 무렵 생겼거나 다시 쓰였다" 정도로만 읽어야 한다.

시각	파일	이 장에서의 의미
9/21 16:58	Assets/Scripts·Editor 폴더, HitTarget.cs , WeaponAimFireController.cs	프로젝트 생성 첫날부터 있던 무기·표적 뼈대
9/21 16:59	XR Hands·XRI 샘플	HandVisualizer, Starter Assets, Hands Interaction Demo
9/22 09:55	TargetingHUD.cs	링 HUD로 적을 조준·락온하는 표시
9/22 10:30	Models/Gundam	건담 FBX(앞 장과 공유)
9/22 12:51	HandTrackingStatusLogger.cs	손 추적 획득/상실 로그
9/22 12:57	Models/Zaku	자쿠 FBX와 텍스처
9/22 12:59	EnemyMarker.cs	"적"이라는 꼬리표
9/22 14:17	HandSubsystemBootDiagnostics.cs	시작 시 한 번만 찍는 손 추적 진단
9/22 15:25	HandJointTracker.cs	쥐기(GripAmount) 판정이 추가된 시점으로 보임
9/23 10:06	커밋 01	여기까지의 결과물 전체

이 장을 읽는 법

Claude는 Unity 에디터 화면을 직접 볼 수 없다. 그래서 씬을 손으로 꾸미지 않고, 메뉴 `Gundam > Build Cockpit Scene` 하나로 씬 전체를 처음부터 다시 만드는 에디터 스크립트 `Assets/Editor/GundamCockpitSetup.cs` 를 키워 가는 방식을 택했다. 이건호가 메뉴를 누르고 APK를 빌드해 Galaxy XR에서 보고 한 줄로 보고하면, Claude가 이 생성기와 런타임 스크립트를 고친다. 이 장에 나오는 조종간·손 추적기·포탑·표적·자쿠는 모두 이 생성기 안의 함수 (`CreateJoystick`, `CreateHandTracker`, `BuildHandVisualizer`, `WireJoystickHandInteractors`, `PlaceZakuEnemy` 등)가 만든다. 따라 하려면 이 함수들을 순서대로 짜면 된다.

생성기가 만드는 씬 구조(이 장과 관련된 부분)

먼저 전체 그림이다. `BuildCockpitScene()` 은 빈 씬을 만들고 조명을 놓은 다음, 기체 전체를 대표하는 루트 오브젝트 `MobileSuitRoot` 를 만든다. 여기에 이동 스크립트 `ShipMovementController` 가 붙는다. 그 아래 `CockpitInterior` 에 의자·디스플레이·HUD 링·조종간이 들어가고, 같은 루트 아래에 포탑과 XR Origin이 들어간다. 건담 본체와 자쿠, 표적, 별은 월드 공간에 따로 놓인다.

```
MobileSuitRoot          // ShipMovementController, WeaponAimFireController, CockpitHUD
├─ CockpitInterior
│   ├─ Seat_*, SystemCheckDisplay, OrbitHUD, Cockpit_Dome ...
│   ├─ LeftJoystick_Mount // JoystickLever
│   │   └─ LeftJoystick_Pivot └─ LeftJoystick_Handle
│   │       └─ _GripBall, _ThumbButton, _Index/Middle/Ring/PinkyButton
│   │       └─ _GripPoint
│   └─ RightJoystick_Mount // 같은 구조 + _Trigger
├─ GunTurret └─ GunPivot └─ GunHousing, Barrel └─ MuzzlePoint
└─ XR Origin (Gundam Cockpit)
    └─ LeftHandTracker / RightHandTracker // XRHandTrackingEvents + HandJointTracker
        └─ (샘플 리그의 Left Hand, Right Hand, Hand Visualizer ...)
ExternalGundam, ZakuEnemy, Targets(Target_0~5), Starfield // 월드 공간
```

1. 손 추적의 뼈대 — XR Origin 아래의 두 손

OpenXR에서 스크립트까지 손 데이터가 흐르는 길

이 게임은 컨트롤러 없이 맨손으로 조종하는 것이 전제였다. 그래서 가장 먼저 필요한 것은 "지금 왼손 손바닥이 어디 있고, 손을 얼마나 쥐었는가"를 매 프레임 알려 주는 부품이다. 데이터는 다음 순서로 흐른다.

Galaxy XR의 OpenXR 런타임이 카메라로 손을 인식한다 → 프로젝트 설정의 OpenXR Android 탭에서 **컨 Hand Tracking Subsystem** 기능이 이것을 Unity의 `XRHandSubsystem` (com.unity.xr.hands 1.6.3 패키지)으로 넘긴다 → 같은 패키지의 `XRHandTrackingEvents` 컴포넌트가 지정한 손(왼손/오른손)의 관절이 갱신될 때마다 `jointsUpdated` 이벤트를 낸다 → 이 프로젝트의 `HandJointTracker` 가 그 이벤트를 받아 손바닥 위치·회전과 집기/쥐기 정도를 계산해 속성으로 내놓는다 → 조종간(`JoystickLever`)과 로그 스크립트가 그 속성을 읽는다.

HandJointTracker의 클래스 주석은 이 스크립트가 XR Interaction Toolkit(XRI)의 상호작용기나 프리팹에 기대지 않는다는 점을 분명히 적는다. 필요한 것은 XR Hands 패키지와, 손 데이터를 보내 주는 OpenXR 런타임, 그리고 "Project Settings > XR Plug-in Management > OpenXR > Android 탭"에서 켜진 Hand Tracking Subsystem뿐이다. 나중에 XRI 쪽 잡기 경로가 따로 붙지만, 이 독립된 경로는 끝까지 대비책으로 남는다.

CreateHandTracker — 손 하나에 오브젝트 하나

생성기의 CreateHandTracker는 아주 짧다. 빈 GameObject를 만들고 XRHandTrackingEvents와 HandJointTracker를 붙인 뒤, 두 컴포넌트에 같은 손 방향(Handedness)을 넣는다. BuildCockpitScene은 이것을 "LeftHandTracker" (Handedness.Left)와 "RightHandTracker" (Handedness.Right) 두 번 호출한다.

```
static HandJointTracker CreateHandTracker(string name, Handedness handedness)
{
    GameObject go = new GameObject(name);
    XRHandTrackingEvents events = go.AddComponent<XRHandTrackingEvents>();
    HandJointTracker tracker = go.AddComponent<HandJointTracker>();
    tracker.handedness = handedness;
    events.handedness = handedness;
    return tracker;
}
```

이 두 오브젝트는 만들어지는 순간에는 부모가 없다. XR Origin이 아직 없기 때문이다. 조종간까지 다 만든 뒤 CreateXROrigin이 리그를 만들고 나서야 leftHand.transform.SetParent(xrOrigin.transform, false)로 XR Origin 바로 아래에 붙는다. 여기에는 이유가 있다. XR Hands가 내주는 관절 자세는 XR Origin의 로컬 추적 공간 기준 좌표다. 그래서 HandJointTracker는 손바닥 관절 자세를 받아 자기 transform.localPosition/localRotation에 그대로 쓴다. 부모가 XR Origin이어야 그 로컬 값이 올바른 월드 위치가 된다. 주석도 "이 오브젝트는 XR Origin 바로 아래에 있어야 한다"고 적어 둔다.

HandJointTracker가 계산하는 값

Awake에서 같은 오브젝트의 XRHandTrackingEvents를 찾아 손 방향을 맞추고 jointsUpdated에 OnJointsUpdated를 등록한다(OnDestroy에서 해제). 밖으로 내주는 값은 다섯 가지다.

속성	뜻	계산
PalmPosition / PalmRotation	손바닥의 월드 위치·회전	XRHandJointID.Palm 자세를 로컬에 쓴 뒤 transform.position/rotation을 읽음
PinchAmount	엄지-검지 집기 정도 0~1	엄지 끝(ThumbTip)과 검지 끝(IndexTip) 거리. 0.08m 이상이면 0, 0.025m 이하면 1
GripAmount	주먹 쥐기 정도 0~1	검지·중지·약지·새끼 끝이 손바닥에서 떨어진 거리 평균. 0.11m면 0, 0.045m면 1
IsTracked	지금 그 손이 추적되는가	XRHandTrackingEvents.handIsTracked

집기 계산은 `Mathf.InverseLerp(openHandDistance, fullPinchDistance, dist)` 다. 인자 순서를 일부러 "벌린 거리 → 집은 거리"로 넣었기 때문에, 거리가 줄어들수록 값이 0에서 1로 올라간다. 기본값은 `fullPinchDistance = 0.025f` (인스펙터 범위 0.01~0.08), `openHandDistance = 0.08f` (범위 0.03~0.15) 다.

`GripAmount` 는 처음부터 있던 값이 아니다. 조종간이 잡히지 않는다는 보고를 받고 나중에 덧붙인 것으로, 그 사연은 4절에서 자세히 다룬다. 여기서는 계산만 보자. 손바닥(Palm) 관절 위치를 기준점으로 잡고, 네 손가락 끝마다 `AccumulateFingerCurl` 을 불러 손바닥까지의 거리를 `InverseLerp(openGripDistance, fullGripDistance, dist)` 로 0~1 값으로 바꾼 뒤 더한다. 그 프레임에 자세가 유효했던 손가락 수로 나눠 평균을 낸다. 손가락 하나가 가려져 자세를 못 받아도 나머지로 계산이 이어지게 한 것이다.

```
// HandJointTracker.cs - 네 손가락 끝이 손바닥 쪽으로 말린 정도의 평균
AccumulateFingerCurl(args.hand, XRHandJointID.IndexTip, palmForGrip.position, ref gripSum, ref gripCount);
AccumulateFingerCurl(args.hand, XRHandJointID.MiddleTip, palmForGrip.position, ref gripSum, ref gripCount);
AccumulateFingerCurl(args.hand, XRHandJointID.RingTip, palmForGrip.position, ref gripSum, ref gripCount);
AccumulateFingerCurl(args.hand, XRHandJointID.LittleTip, palmForGrip.position, ref gripSum, ref gripCount);
if (gripCount > 0) GripAmount = Mathf.Clamp01(gripSum / gripCount);
...
void AccumulateFingerCurl(XRHand hand, XRHandJointID tipId, Vector3 palmPosition, ref float sum, ref int count)
{
    if (!hand.GetJoint(tipId).TryGetPose(out Pose tipPose)) return;
    float dist = Vector3.Distance(tipPose.position, palmPosition);
    sum += Mathf.InverseLerp(openGripDistance, fullGripDistance, dist);
    count++;
}
```

XR Origin — 샘플 리그를 가져와 콕핏에 얹히기

손 추적이가 붙을 XR Origin은 `CreateXROrigin` 이 만든다. 머리 카메라와 시야 처리는 앞 장의 주제이므로 여기서는 손과 관련된 부분만 짚는다.

`TryImportSampleAndFindXROriginPrefab` 은 패키지 관리자 API로 XR Interaction Toolkit(3.3.2)의 샘플 목록을 얻어, 이름이 정확히 "**Hands Interaction Demo**"나 "**Starter Assets**"인 것만 가져온다. 앞의 것에 손 추적용 리그 "XR Origin Hands (XR Rig)"가 있고, 뒤의 것은 그 샘플이 의존하는 입력 액션·프리셋 묶음이다. "AR Starter Assets"는 일부러 뺐다. 이 샘플은 com.unity.xr.arfoundation을 끌고 들어오는데, 프로젝트에 그 패키지를 원하지 않았고 "XRSimulation-Session" 중복 ID 경고의 원인이기도 했다고 주석이 적는다. 이름 비교를 `Contains` 가 아니라 `Equals` 로 한 이유도 적혀 있다. "AR Starter Assets"에도 "Starter Assets"라는 글자가 들어 있어서 부분 일치로 찾으면 함께 걸려 버린다. 가져온 뒤에는 `Assets/Samples` 아래에서 경로에 "XR Origin"이 들어간 프리팹을 찾되, "Hand"가 들어간 것을 우선한다.

찾은 프리팹은 "XR Origin (Gundam Cockpit)" 이라는 이름으로 `MobileSuitRoot` 아래 로컬 위치 (0, 0, -0.15) 에 놓인다. 샘플을 못 찾으면 카메라 하나만 있는 `XR_ORIGIN_PLACEHOLDER_REPLACE_ME` 를 대신 만들고, 샘플을 직접 가져와 바꿔 끼우라고 경고한다. 이 대체 리그에는 손 상호작용기가 없다. 그래서 뒤에

나올 손 메시와 XRI 잡기 연결 함수가 모두 "리그에 이미 있으면 쓰고, 없으면 대비책으로 간다"는 구조를 갖게 된다.

2. 보이는 손 — 디버그 구슬도, 컨트롤러 모델도 없이

헤드셋을 쓰고 콧속에 앉았을 때 자기 손이 보이지 않으면, 조종간을 향해 손을 뻗을 기준이 없다. 이견호의 요청은 분명했다. 생성기의 `BuildHandVisualizer` 위 주석에 원문이 세 문장으로 남아 있다.

요청 · Galaxy XR 실제 손 추적 데이터를 이용한 '보이는 XR 손'

요청 · 21개 관절을 구형 오브젝트로 표시하는 디버그 방식은 사용하지 마

요청 · 컨트롤러 모델이 게임 안에 나타나는 것도 원하지 않는다

세 가지 제약이 겹친다. 실제 손 추적 데이터로 움직여야 하고, 관절마다 작은 구슬을 찍는 흔한 디버그 표시는 안 되며, 컨트롤러 모양이 떠서도 안 된다. Claude는 직접 손 모델을 만들지 않고 XR Hands 패키지의 공식 샘플 `HandVisualizer`를 쓰기로 했다. 이 컴포넌트는 리깅된 손 메시 (`SkinnedMeshRenderer`)를 `XRHandSkeletonDriver`로 뼈마디마다 움직이고, 손이 추적되는 동안에만 보이다가 추적을 놓치면 즉시 숨긴다(샘플 코드의 `OnTrackingAcquired/OnTrackingLost`). 컨트롤러 입력 경로와는 상관없이 `XRHandSubsystem`만 읽는다.

첫 시도: `HandVisualizer`를 하나 더 만들었다

처음 구현은 XR Origin 아래에 `"HandVisualizer"` 오브젝트를 새로 만들고 샘플 컴포넌트를 붙이는 것이었다. 그런데 이견호가 실기에서 손이 여전히 안 보인다고 다시 알렸다. 주석은 이 대목을 "IMPORTANT"로 적는다. Claude가 가져온 프리팹의 원본 파일을 직접 열어 보니, 실제로 쓰고 있던 "XR Origin Hands (XR Rig)"에 이미 `"Hand Visualizer"` 자식이 들어 있었다. 게다가 `drawMeshes=true`였고, `m_AndroidXRLeftHandMesh/m_AndroidXRRightHandMesh`에는 진짜 리깅 메시가 이미 연결돼 있었다. 그러니 하나를 더 만드는 것은 손 렌더러를 두 벌, 서로 다른 위치에 두는 일일 뿐 "손이 안 보인다"를 고칠 수 없었다. 오히려 헛갈리게 하거나, 위치가 다른 손 메시지를 한 벌 더 띄울 위험만 있었다.

그래서 최종 코드는 먼저 리그 안을 뒤진다. `xrOrigin.GetComponentInChildren<HandVisualizer>(true)`로 기존 것이 나오면 그대로 쓰고 로그만 남긴다. 그 로그 문장이 인상적이다. 이래도 손이 실기에서 안 보이면 문제는 이 썬보다 앞단, 즉 손 추적 데이터가 앱까지 오지 않는 데 있다는 것이다. 바로 다음 줄의 진단 로그를 보고, 프로젝트의 OpenXR 설정만이 아니라 헤드셋 자체 시스템 설정의 손/컨트롤러 입력 전환도 확인하라고 적었다.

대비책: 리그에 없을 때만 직접 조립

기존 `HandVisualizer`가 없을 때(예: 샘플을 못 찾아 대체 리그를 쓸 때)만 새로 조립한다. 쓰는 에셋은 `Assets/Samples/XR Hands/1.6.3/HandVisualizer/` 아래에 있다.

상수	경로(샘플 루트 기준)	용도
HandVisAndroidLeft/RightMeshPath	Models/LeftHandAndroidXR.fbx, RightHandAndroidXR.fbx	대상 기기(Galaxy XR)에 맞는 손 메시
HandVisFallbackLeft/RightMeshPath	Models/LeftHand.fbx, RightHand.fbx	다른 런타임용 예비 메시
HandVisMaterialPath	Materials/HandsDefaultMaterial.mat	손 재질
HandVisJointPrefabPath / VelocityPrefabPath	Prefabs/Joint.prefab, VelocityPrefab.prefab	컴포넌트가 요구하는 필수 참조(실제로는 꺼진 채 생성)

HandVisualizer의 메시·프리팸·재질 필드는 공개 setter가 없는 private `[SerializeField]` 다. 그래서 에디터 스크립트에서 `SerializedObject` 로 `m_AndroidXRLeftHandMesh`, `m_MetaQuestLeftHandMesh`, `m_HandMeshMaterial`, `m_DebugDrawPrefab`, `m_VelocityPrefab` 등을 채운다. 다른 회사 컴포넌트의 인스펙터 전용 필드를 설정하는 표준 방법이다. 마지막 세 줄이 요청을 그대로 옮긴 부분이다.

```
handVisualizer.drawMeshes = true;
handVisualizer.debugDrawJoints = false; // 관절 구슬 없음 - 실제 손 메시만
handVisualizer.velocityType = HandVisualizer.VelocityType.None;
```

관절 디버그 프리팸 참조는 비워 둘 수 없지만, `debugDrawJoints=false` 이고 `velocityType=None` 이면 그 오브젝트들은 꺼진 채 만들어지고 끝내 그려지지 않는다. "21개 관절 구슬"은 어떤 경우에도 나타나지 않는다.

컨트롤러 모델 차단

세 번째 요청에 대해서는 방어 코드를 하나 더 넣었다. `DisableAnyControllerVisuals` 는 XR Origin 아래 모든 `Renderer`를 훑어, 오브젝트 이름에 "Controller"가 들어가면(대소문자 무시) 끄고 로그를 남긴다. 주석에 따르면 Claude가 확인해 보니 지금 쓰는 리그에는 컨트롤러 메시가 없었다(손의 집기·찌르기·조준 로직뿐). 그래서 평소에는 아무 일도 하지 않는다. 다만 나중에 리그나 샘플을 다시 가져오거나 바뀌더라도 컨트롤러가 슬며시 나타나지 않게 막는 장치다.

3. 손 추적이 들어오는지 확인하기 — 진단 로그 두 가지

손이 안 보이고 조종간이 안 잡힐 때 가장 먼저 가려야 할 질문은 "데이터가 아예 안 들어오는가, 들어오는데 찢이 잘못 연결됐는가"다. 헤드셋 안에서는 Unity 콘솔을 볼 수 없으므로, Claude는 adb logcat으로 읽을 수 있는 로그를 두 단계로 심었다.

HandTrackingStatusLogger — 상태가 바뀔 때만 한 줄

첫 요청은 이것이었다(파일 생성 시각 9/22 12:51 무렵).

요청 · 손 추적이 정상적으로 들어오는지 확인할 수 있는 최소한의 로그

"최소한"이라는 말에 맞춰, `HandTrackingStatusLogger` 는 매 프레임 로그를 찍지 않는다. 왼손·오른손마다 직전 프레임의 추적 여부를 `bool` 로 기억해 두고, 값이 바뀌는 순간에만 한 줄을 남긴다.

```
static void LogTransition(HandJointTracker tracker, string label, ref bool wasTracked)
{
    if (tracker == null) return;
    bool isTracked = tracker.IsTracked;
    if (isTracked == wasTracked) return; // 변화 없으면 조용히
    wasTracked = isTracked;
    Debug.Log(isTracked
        ? "[Gundam] " + label + " hand tracking ACQUIRED."
        : "[Gundam] " + label + " hand tracking LOST.");
}
```

설계상 판단이 하나 있다. 이 로거는 `XRHandSubsystem` 을 따로 구독하지 않고, 이미 조종간 잡기에 쓰는 `HandJointTracker.IsTracked` 를 읽는다. "이 손이 지금 추적되는가"의 출처를 하나로 두려는 것이다. 덕분에 이 로그의 ACQUIRED/LOST는 손 메시가 보이는지와 별개로, **조종간을 잡을 수 있는 상태인지**를 직접 알려 준다. 생성기는 이 컴포넌트를 `HandVisualizer`가 아니라 `XR Origin` 자체에 붙인다. `HandVisualizer`는 앞 절처럼 새로 만들어지지 않을 수도 있으니, 어느 경우에도 반드시 생기는 곳에 둔 것이다.

HandSubsystemBootDiagnostics — 시작 3초 뒤 딱 한 번

두 번째 요청은 더 구체적이었다(9/22 14:17 무렵). 생성기 주석에 남은 원문이다.

요청 · `XRHandSubsystem` 존재 여부 / running 여부 / left/right tracked 여부... 매 프레임 로그를 찍지는 마라

`HandSubsystemBootDiagnostics` 는 `Start` 에서 `Invoke(nameof(LogOnce), delaySeconds)` 로 3초 (`delaySeconds = 3f`) 뒤에 한 번만 실행된다. `OpenXR` 세션과 서브시스템이 올라올 시간을 주려는 것이다. `SubsystemManager.GetSubsystems` 로 모든 `XRHandSubsystem` 을 모은 뒤, 실행 중인 것을 먼저 고르고, 없으면 첫 번째(존재하지만 아직 안 도는 것)를 고른다. 그리고 `[Gundam][HandDiag]` 접두어로 네 줄을 찍는다. 앞의 로거와 달리 이 프로젝트의 어떤 스크립트도 거치지 않고 런타임 서브시스템을 직접 본다는 점이 중요하다. 이 프로젝트의 스크립트는 모두 그보다 뒤에 있기 때문이다.

클래스 주석은 결과를 네 경우로 나누어 해석하는 법까지 적어 두었다. 이건호가 로그를 받아 들고 어디가 문제인지 바로 가를 수 있게 한 것이다.

로그 결과	뜻	추가 경고
found=false	<code>XR Hands</code> 가 서브시스템을 아예 만들지 않음. <code>Hand Tracking Subsystem</code> 기능이 켜져 있다면 빌드·패키지 문제일 가능성	<code>OpenXR Android</code> 탭의 <code>Hand Tracking Subsystem</code> 확인
found=true, running=false	서브시스템은 있으나 런타임이 시작하지 않음. 헤드셋/OS가 이 앱에 손 데이터를 넘기지 않는 경우(컨트롤러 모드, OS 손 추적 꺼짐)	"대개 이 프로젝트 밖의 문제"
running=true, 양손 tracked=false	런타임은 손을 보고하려 하지만 지금은 못 봄(시야 밖, 순간 상실)	양손을 헤드셋 앞에 확실히 두고 다시 확인

생성기 주석은 이 로그를 "기지에서 돌린 뒤 가장 먼저 볼 로그"라고 부른다. 서브시스템이 안 돌거나 손을 못 보고 있다면, 씬과 스크립트를 어떻게 고치든 소용없기 때문이다. 손 메시 문제와 조종간 문제가 같은 시기에 겹쳐 있었으므로, 원인을 위아래로 가르는 이 단계는 꼭 필요했다.

4. 조이스틱을 빼고, 손으로 움켜쥐는 구형 조종간으로

처음 모습: 팔걸이 옆의 기계식 조이스틱

처음 조종간은 이름 그대로 "조이스틱"처럼 생겼었다. `CreateJoystick` 위의 주석이 사라진 장식을 하나하나 적어 두었다. 콘솔 하우징, 연결 팔, 래칫 칼라, 표시등이 있었고, 손잡이는 의자 팔걸이 높이에서 옆으로 뻗은 수평 캡슐 막대였다. 입력 방식도 지금과 달랐다. 여러 주석이 "기울이기/비틀기(tilt/twist)" 계산을 언급한다. `JoystickLever`가 손잡이의 `localRotation`을 매 프레임 덮어써 기울였고, 손목을 비트는 각도도 읽었다. 그 흔적이 아직 코드 곳곳에 남아 있다. `GundamHeadCam360`은 자기 방식을 "JoystickLever의 손목 비틀기 기준 캡처와 같은 아이디어"라고 설명하고, `CreateJoystick` 안에는 "JoystickLever overwrites handleGo's own localRotation every frame for tilt"라는, 이제는 맞지 않는 옛 주석이 남아 있다.

그런데 입력은 처음부터 조이스틱이 아니었다. 실제 스틱이나 게임패드 축을 읽은 적은 없고, 늘 추적된 손(근접 + 집기)을 읽었다. 문제는 생김새였다. 기계 장치처럼 보이니 손을 뻗어 쥐는 물건이 아니라 "조종간"으로 읽혔다. 이건호는 이렇게 요청했다.

요청 · 조이스틱으로 조종하는게 아니고 손으로 하는거니까 조이스틱 빼고 손으로 잡을수있게해줘

Claude는 정적인 장식(하우징, 막대, 칼라, 표시등)을 모두 지웠다. 손이 실제로 닿는 부분(손잡이와 손가락 버튼)만 공중에 떠 있게 남겼다. 생성기 함수 이름은 `CreateJoystick`, 컴포넌트 이름은 `JoystickLever`, 씬 오브젝트 이름은 `LeftJoystick / RightJoystick` 그대로다. 이름을 바꾸면 다른 스크립트의 참조가 깨지기 때문이다. 이후 대화에서도 "LeftJoystick"은 이 손으로 쥐는 조종간을 가리키는 이름으로 쓰인다.

막대에서 구로: "사진과 같이" 움켜쥐기

다음으로 이건호는 주먹으로 둥근 손잡이를 감싸 쥔 참고 사진을 붙여 보냈다.

요청 · 구형으로 만들어서 손으로 움켜쥐고 사진과 같이 잡고

그래서 손잡이는 수평 캡슐에서 지름 0.10m의 구(`_GripBall`)로 바뀌었다. 주먹 하나로 감쌀 크기다. 왼쪽은 파란 발광 재질(`leftAccentMat`, 방출색 (0.15, 0.55, 1)), 오른쪽은 빨간 발광 재질(`rightAccentMat`, 방출색 (1, 0.2, 0.15))이다. 생성기 주석은 이를 "steering (blue)" / "weapon (red)"으로 구분한다. 왼쪽은 이동, 오른쪽은 무기다.

손가락마다 버튼

이건호는 이 구에 손가락별 버튼을 달아 쓰겠다고 했다.

요청 · 손가락별로 버튼이있어서 사용할거야

Claude는 구를 감싸 쥐었을 때 네 손가락이 자연스럽게 놓이는 앞면에, 위에서 아래로 비스듬히 휘어 내려가는 줄로 작은 구 네 개를 배치했다. 엄지 자리에는 위쪽에 하나를 더 놓았고, 오른쪽 조종간에만 아래쪽에 방아쇠 모양 큐브를 달았다. 버튼은 왼쪽이 초록(`buttonGreenMat`), 오른쪽이 빨강(`buttonRedMat`)이다. 좌표는 모두 손잡이(`_Handle`) 기준 로컬 값이다.

오브젝트	로컬 위치 (m)	크기 (m)
<code>_GripBall</code>	(0, 0, 0)	0.10 구
<code>_ThumbButton</code>	(0, 0.045, -0.015)	0.022 구
<code>_IndexButton</code>	(-0.026, 0.030, 0.040)	0.016 구
<code>_MiddleButton</code>	(-0.009, 0.014, 0.049)	0.016 구
<code>_RingButton</code>	(0.009, -0.004, 0.050)	0.016 구
<code>_PinkyButton</code>	(0.026, -0.022, 0.044)	0.016 구
<code>_Trigger (오른쪽만)</code>	(0, -0.035, 0.045)	0.03 × 0.018 × 0.035 큐브
<code>_GripPoint</code>	(0, 0, 0)	빈 오브젝트(손바닥 기준점)

주석에 따르면 이 시점의 버튼은 모양과 물리 콜라이더만 있는 자리표시자다. 어떤 동작에도 연결돼 있지 않다. 손가락 버튼에 실제 기능이 붙는 것은 훨씬 뒤(9/28 무렵 `JoystickFingerButtons.cs`)의 일이다. 다만 이 구들은 `CreatePrimitive`로 만들어져 기본 `SphereCollider`를 갖고 있다. 이것이 뒤에서 XRI 상호작용기가 잡을 대상(콜라이더)으로 다시 쓰인다.

계층 구조: Mount → Pivot → Handle → GripPoint

`CreateJoystick`은 조종간 하나를 네 단계 계층으로 만든다.

```
GameObject baseGo = new GameObject(name + "_Mount");           // JoystickLever가 붙는 곳
baseGo.transform.localPosition = localPos;
GameObject pivotGo = new GameObject(name + "_Pivot");          // 움직이지 않는 기준(Base)
pivotGo.transform.localPosition = new Vector3(0f, 0.02f, 0.05f);
GameObject handleGo = new GameObject(name + "_Handle");        // 실제로 미끄러지는 Stick
handleGo.transform.localPosition = Vector3.zero;
// ... GripBall, 버튼들을 handleGo 아래에 ...
GameObject gripPointGo = new GameObject(name + "_GripPoint");  // 손바닥이 맞춰질 점
JoystickLever lever = baseGo.AddComponent<JoystickLever>();
lever.pivot = pivotGo.transform; lever.handle = handleGo.transform;
lever.gripPoint = gripPointGo.transform;
lever.leftHandTracker = left;   lever.rightHandTracker = right;
```

손잡이 구와 버튼은 모두 `_Handle`의 자식이므로, 핸들이 움직이면 한 덩어리로 따라간다. `_GripPoint`는 뒤에서 설명할 "손과 손잡이가 정확히 붙어야 한다"는 보고 때문에 생겼다. 구의 중심, 곧 주먹이 감쌀 때 자연스럽게 가운데가 되는 자리에 둔다.

왼손은 왼쪽만, 오른손은 오른쪽만

`CreateJoystick`의 마지막 두 인자는 이 조종간을 잡을 수 있는 손이다. 생성기는 왼쪽에는 `(leftHand, null)`, 오른쪽에는 `(null, rightHand)`를 넘긴다. 근거는 이 요청이다.

요청 · LeftJoystick: 왼손으로 잡음... RightJoystick: 오른손으로 잡음

주석은 이전 상태도 적어 두었다. 원래 RightJoystick은 왼손 추적기도 받고 있었는데, 손 전용 규칙이 생기기 전에 남은 것이었다. 이 요청으로 조종간마다 자기 손만 받도록 고쳤다.

위치: 팔걸이에서 "디스플레이 3개 사이"로, 그리고 조금 위로

처음 조종간은 의자 팔걸이(옆 볼스터) 근처에 있었다. 앞 장에서 만든 세 개의 SYSTEM CHECK 화면이 무릎 위 계기판 역할을 맡게 되자, 이건호는 조종간 위치를 그쪽으로 옮기자고 했다.

요청 · 조종장치에 위치가 디스플레이 3개 사이에 그쯤에 있어야할거같아

Claude는 화면 좌표부터 계산했다. 화면은 `CreatePolarScreen`으로 눈 위치를 중심으로 한 원호 위에 놓인다. $x = \text{반지름} \times \sin(\text{각도})$, $z = \text{eyeZ} + \text{반지름} \times \cos(\text{각도})$ 이고, $\text{eyeZ} = -0.15$, 높이 $\text{baseY} = 0.92$ 다. 이 요청을 처리할 당시 반지름은 0.55였으므로 가운데 화면은 (0, 0.92, 0.4), 좌우 화면($\pm 40^\circ$)은 (∓ 0.3536 , 0.92, 0.2713)에 있었다. 주석에 적힌 숫자 그대로다. 조종간은 그 세 화면 사이의 틈에 오도록 (± 0.22 , **0.82**, **0.18**)에 두었다. 옛 팔걸이 자리보다 가운데에 가깝고, 화면보다 약간 아래·앞쪽이다.

Claude는 이 결정의 한계도 그대로 적었다. 예전 팔걸이 위치는 화면과 겹쳐 보이지 않도록 잡기 반경 + 프레임 반경인 약 0.35만큼 일부러 떼어 둔 자리였다. 새 위치는 화면 패널에 눈에 띄게 가깝다. "에디터에서 이 각도로 손잡이가 화면 프레임을 뚫고 들어가 보이는지 확인할 수 없으니, 화면에 박혀 보이면 말해 달라, 조금 뒤로 빼겠다"고 남겼다.

이건호의 다음 보고는 높이에 관한 것이었다.

문제 보고 · 조종기를 조금 위쪽으로 올린다

Y만 0.05m 올려 0.82 → **0.87**로 바꾸고, X·Z는 그대로 두었다. 최종 호출은 다음과 같다.

```
JoystickLever leftStick = CreateJoystick("LeftJoystick", interior.transform,
    new Vector3(-0.22f, 0.87f, 0.18f), false, leftAccentMat, buttonGreenMat,
    leftHand, null);
JoystickLever rightStick = CreateJoystick("RightJoystick", interior.transform,
    new Vector3(0.22f, 0.87f, 0.18f), true, rightAccentMat, buttonRedMat,
    null, rightHand);
ship.leftStick = leftStick;
```

숫자가 어긋나 보이는 이유

커밋 01의 `BuildSystemCheckDisplay` 를 보면 화면 반지름은 0.55가 아니라 **0.63**이다. "화면이 너무 가까이 붙어 보인다"는 보고로 화면을 0.08m 뒤로 민 결과다(앞 장 참고). 조종간 주석의 화면 좌표는 그 이전 값으로 계산된 것이고, 조종간 위치 (± 0.22 , 0.87, 0.18)는 화면을 민 뒤에도 바뀌지 않았다. 따라 만들 때는 화면 반지름을 바꾸면 조종간과 화면의 상대 위치도 함께 달라진다는 점을 기억해 두자.

5. 조종간을 움직이는 방식 — 기울이기에서 밀고 당기기로

요청: 진짜 조종간처럼

손잡이가 구로 바뀌자 움직이는 방식도 다시 정해야 했다. 이견호의 요청은 `JoystickLever` 클래스 주석에 남아 있다.

요청 · 실제 비행기/로봇 조종간처럼 손으로 잡고 앞뒤/좌우로 밀고 당기는 방식

예전 `JoystickLever` 는 기준점에서 손잡이를 각도로 기울였다. 새 모델은 위치 기반이다. 주석은 이를 "position-based redesign"이라 부른다. 규칙은 이렇다.

- 기준점(Base, `pivot`)은 절대 움직이지 않는다. 움직이는 것은 Stick(`handle`)뿐이다.
- Stick은 기준점의 로컬 X(좌우)와 Z(앞뒤 밀기) 평면 안에서만 미끄러진다. Y(높이)는 바뀌지 않고, 제자리에서 회전하지도 않는다.
- 놓으면 중심으로 스프링처럼 되돌아간다.
- 출력 `tiltInput` 은 원래 이름과 모양을 그대로 유지한다(X = 좌우, Y = 앞뒤, 각각 -1~1). 그래서 `ShipMovementController`, `WeaponAimFireController`, `CockpitHUD` 는 한 줄도 고치지 않고 계속 동작한다. 각도 대신 잘린(clamped) 위치에서 계산할 뿐이다.

이동 범위와 잡기 관련 기본값은 다음과 같다.

필드	값	의미
<code>grabRadius</code>	0.22 m	손바닥이 기준점에서 이 거리 안에 있어야 잡기 시작 (자체 판정)
<code>pinchToGrabThreshold</code> / <code>pinchToReleaseThreshold</code>	0.5 / 0.3	집기로 잡기 시작 / 놓기
<code>gripToGrabThreshold</code> / <code>gripToReleaseThreshold</code>	0.55 / 0.35	쥐기로 잡기 시작 / 놓기
<code>lateralRange</code>	0.08 m	좌우 이동 한계(±)
<code>depthRange</code>	0.08 m	앞뒤 이동 한계(±)
<code>returnLerpSpeed</code>	5	놓았을 때 중심 복귀 속도

잡기 시작과 놓기의 문턱값을 다르게 둔 것은 히스테리시스다. 0.55에서 잡힌 손이 0.35 아래로 내려가야 놓이므로, 문턱 근처에서 손이 조금 떨어져도 잡혔다 놓였다는 반복하지 않는다.

"손과 손잡이가 정확히 붙어야 한다"

위치 기반으로 바꾼 뒤 이건호가 보낸 보고는 길고 구체적이었다. 주석에 남은 원문이다.

문제 보고 · 손과 조종간 손잡이가 정확히 붙어야 한다... Grab이 시작되는 즉시 GripPoint가 실제 손바닥 위치에 정확히 맞는다... Grab 중 손이 움직이면 손잡이가 손을 정확하게 따라간다

이 보고로 `gripPoint` 라는 별도 자식이 생겼다. "핸들 Transform 전체를 손 위치에 복사하는 것"과 "핸들 안의 특정 점이 손바닥과 일치하는 것"은 다르다는 구분이다. 커밋 01 시점의 `Update` 는 이 보고를 다음처럼 해석해 구현했다.

잡는 순간에는 스냅. 잡힌 첫 프레임에 보간 없이 Stick을 옮겨, GripPoint가 손의 현재 위치에 정확히 오도록 한다(이동 범위 안으로 잘라서). 잡는 순간 손과 손잡이 사이에 틈이 보이지 않게 하려는 것이다.

잡고 있는 동안에는 강체처럼 1:1. 매 프레임 손의 이동량을 보간 없이 그대로 더한다(역시 범위 안으로 자름). 범위 안에 있는 한 GripPoint가 지연이나 미끄러짐 없이 손에 붙어 다닌다. 부드럽게 만드는 처리는 놓았을 때 중심으로 돌아가는 동작에만 쓴다.

```
Vector3 handLocalNow = pivot.InverseTransformPoint(_activeHand.PalmPosition);
Vector3 gripOffset = gripPoint != null ? gripPoint.localPosition : Vector3.zero;
if (!wasGrabbed)
{
    // 이번 프레임에 막 잡음 - 즉시 스냅
    _handStartLocal = handLocalNow;
    _handleStartLocalPos = ClampToRange(handLocalNow - gripOffset);
    handle.localPosition = _handleStartLocalPos;
}
else
{
    // 잡은 뒤 손 이동량을 1:1로 (Y는 버림)
    Vector3 localDelta = handLocalNow - _handStartLocal;
    Vector3 wanted = _handleStartLocalPos + new Vector3(localDelta.x, 0f, localDelta.z);
    handle.localPosition = ClampToRange(wanted);
}
```

`ClampToRange` 는 X를 중심 $\pm$ `lateralRange` , Z를 중심 $\pm$ `depthRange` 로 자르고, Y는 언제나 중심 높이로 고정한다. 놓았을 때는 `lerpT = 1 - exp(-returnLerpSpeed × Δt)` 로 중심을 향해 보간한다. 프레임률과 상관없이 같은 빠르기로 돌아오게 하는 지수 감쇠 방식이다. 마지막으로 중심에서 벗어난 거리를 범위로 나눠 `tiltInput = (offset.x / lateralRange, offset.z / depthRange)` 를 -1~1로 만든다.

왜 매 프레임 InverseTransformPoint인가

이 코드에서 가장 공들인 결정은 손 위치를 "기준점 로컬 좌표로, 매 프레임 새로" 계산한다는 점이다. 잡을 때 월드 좌표를 저장해 두고 그 차이를 쓰지 않는다. 클래스 주석이 이유를 길게 설명한다. 조종간의 기준점은 `MobileSuitRoot` 의 자손이고, 왼쪽 조종간을 밀면 바로 그 `MobileSuitRoot` 가 콧빔채로 이동한다. 월드 좌표 차이로 계산하면 기체가 움직인 만큼이 손이 움직인 것처럼 섞여 들어간다. 그러면 실제 손 입력이 없는데도

다른 쪽 조종간이, 혹은 그 조종간 자신이 기체가 움직일 때마다 저절로 밀려 나간다. 손 추적기도 같은 기체(XR Origin) 안에 함께 타고 있으므로, 둘 다 기준점 로컬 좌표로 바꾸면 공통으로 움직인 부분은 자동으로 상쇄된다.

이 "즉시 스냅"은 오래가지 않았다

잡는 순간 GripPoint를 손바닥에 스냅하는 이 방식은 커밋 01 시점의 구현이다. 다음 장에서 다룰 9월 23일 작업(커밋 03)에서 "조종간을 잡는 순간 조종간 전체가 내 몸 쪽/손 쪽으로 순간적으로 끌려온다"는 보고가 들어오고, 이 스냅은 그 버그의 원인으로 지목되어 지워진다. 잡은 지점의 오프셋을 유지하는 방식으로 바뀌는 과정은 그 장에서 이어진다.

에디터에서 보이는 기즈모

Claude는 에디터 화면을 볼 수 없지만, 이견호가 씬 뷰에서 확인할 수 있도록 `OnDrawGizmosSelected` 를 넣었다. 조종간을 선택하면 기준점 둘레에 반투명 노란 구(반지름 `grabRadius`)가 잡기 가능 범위를 보여 주고, 하늘색 납작한 상자(가로 `lateralRange×2`, 세로 `depthRange×2`)가 손잡이가 움직일 수 있는 평면을 보여 준다.

6. "손이 만들어졌고 근데 그걸로 안에있는 컨트롤러가 잡히지는 않아"

손 메시가 실기기에서 제대로 보이기 시작한 뒤에도 문제가 남았다. 이 장의 중심이 되는 보고서.

문제 보고 · 손이 만들어졌고 근데 그걸로 안에있는 컨트롤러가 잡히지는 않아

여기서 "컨트롤러"는 콕핏 안의 조종간이다. 손은 보이는데 조종간을 쥐어도 반응이 없었다. Claude는 이 문제를 두 갈래로 공략했다. 하나는 자체 판정에 쓰는 손 제스처 신호를 고치는 것이고, 다른 하나는 XRI의 공식 상호작용기를 잡기 신호로 끌어오는 것이다.

원인 하나: 집기가 아니라 쥐기였다

원래 자체 판정은 `PinchAmount`, 곧 엄지 끝과 검지 끝 사이 거리만 봤다. 그런데 조종간은 이견호가 "구형으로 만들어서 손으로 움켜쥐고"라고 요청한, **손 전체로 감싸 쥐는** 구였다. `HandJointTracker`에 추가된 주석이 원인을 설명한다. 공을 주먹으로 감싸면 엄지는 한쪽을, 나머지 네 손가락은 반대쪽을 감싸므로 엄지 끝과 검지 끝은 꽤 멀리 떨어진 채로 있다. 손이 분명히 무언가를 쥐고 있는데도 `PinchAmount`는 낮게 나온다. 주석은 이것 하나만으로도 "손 메시가 제대로 보이기 시작한 뒤에도 조종간이 잡히지 않은 이유"를 설명할 수 있다고 적는다.

그래서 실제 동작에 맞는 신호로 `GripAmount`를 추가했다(1절의 계산). 네 손가락 끝이 손바닥 쪽으로 얼마나 밀려 들어왔는지를 본다. 손가락 끝-손바닥 거리 `0.11m`(`openGripDistance`, 인스펙터 범위 `0.05~0.18`)면 펼친 손, `0.045m`(`fullGripDistance`, 범위 `0.01~0.08`)면 꽉 쥐 주먹이다. 정밀한 두 손가락 집기에는 여전히 `PinchAmount`가 맞는 신호이므로 지우지 않고 둘 다 남겼다. `JoystickLever.CanGrab`은 이제 둘 중 하나만 넘어도 잡기를 허용한다.

```
bool CanGrab(HandJointTracker hand)
{
    if (hand == null || !hand.IsTracked) return false;
    bool gripping = hand.GripAmount >= gripToGrabThreshold; // 0.55
    bool pinching = hand.PinchAmount >= pinchToGrabThreshold; // 0.5
    if (!gripping && !pinching) return false;
    return Vector3.Distance(hand.PalmPosition, pivot.position) <= grabRadius; // 0.22m
}
```

원인 둘의 대비: XRI의 진짜 손 상호작용기 쓰기

이건호는 이어서 이렇게 요청했다. 생성기와 두 스크립트에 나뉘어 남은 원문을 합치면 다음과 같다.

요청 · XR Interaction Toolkit의 실제 Hand Interactor/Direct Interactor를 사용... 현재 프로젝트의 XR Hands 구조와 충돌하지 않는다면

요청 · Grab 판정이 제대로 되도록

요청 자체에 "충돌하지 않는다면"이라는 조건이 있었으므로, Claude는 먼저 조사부터 했다. 가져온 프리팹 파일 `Assets/Samples/XR Interaction Toolkit/3.3.2/Hands Interaction Demo/Prefabs/XR Origin Hands (XR Rig).prefab` 을 직접 열어 보니, 리그에 이미 "Left Hand" 와 "Right Hand" 오브젝트가 있고, 각각 공식 지원되는 "Near-Far Interactor" 자식을 갖고 있었다. 이 상호작용기는 Galaxy XR 손 추적에서 집기/ 쥐기 선택을 XRI의 검증된 로직으로 읽는다. `HandJointTracker` 와 같은 손 데이터를 다른, 더 "공식적인" 경로로 읽는 셈이다.

결론은 "충돌하지 않는다"였다. 손 추적, Hand Visualizer, OpenXR/XR Hands 설정은 하나도 건드리지 않는다. 조종간 손잡이에 물리 효과가 없는 `XRSimpleInteractable` 을 하나 붙여 그 상호작용기가 선택할 대상을 만들어 주고, 선택 여부를 `JoystickLever` 에 알려 줄 뿐이다.

왜 XRGrabInteractable이 아니라 XRSimpleInteractable인가

XRI로 물체를 잡는 가장 흔한 방법은 `XRGrabInteractable` 이다. 하지만 그 컴포넌트는 잡힌 물체를 손에 붙여 직접 움직이고 돌린다. 이 조종간은 범위 안에서만 미끄러지고 놓으면 돌아오는, 고유한 움직임 계산을 갖고 있다. `XRSimpleInteractable` 은 물체를 움직이지 않고 선택·호버 이벤트만 낸다. 그래서 XRI는 "이 조종간이 선택됐는가"라는 예/아니오 신호로만 쓰고, 실제 움직임은 100% `JoystickLever` 가 계산한다. 클래스 주석은 이 분리를 여러 번 강조한다.

WireJoystickHandInteractors와 AttachHandInteractable

이 연결은 XR Origin이 생긴 뒤에야 할 수 있다. 조종간을 만들 때는 리그가 아직 없기 때문이다. 그래서 `BuildCockpitScene` 은 `CreateXROrigin` 과 `BuildHandVisualizer` 다음에 `WireJoystickHandInteractors(xrOrigin, leftStick, rightStick)` 를 부른다. 이 함수는 `FindDeepChild` 로 "Left Hand"/"Right Hand"를 찾고, 그 아래에서 다시 "Near-Far Interactor"를

찾는다. 넷 중 하나라도 없으면 경고만 남기고 돌아간다. 이때도 `JoystickLever`의 자체 판정은 그대로 동작한다.

`AttachHandInteractable`은 손잡이(`lever.handle`)에 `XRSimpleInteractable`을 붙이고, 손잡이 아래 모든 콜라이더(구 손잡이와 손가락 버튼들의 `SphereCollider`)를 `SerializedObject`로 `m_Colliders` 배열에 직접 넣는다. 컴포넌트가 자식 콜라이더를 스스로 모으는 시점에 기대지 않으려는 것이다. 따로 잡기용 콜라이더를 만들 필요가 없다. 마지막으로 `HandExclusiveGrabAdapter`를 붙이고, 조종간과 "허용된 상호작용기"를 지정한다.

HandExclusiveGrabAdapter — 손 전용 규칙을 XRI에서도

XRI의 Near-Far Interactor는 이름처럼 가까워서 잡기뿐 아니라 멀리서 광선으로 고르기도 한다. 그대로 두면 오른손이 광선으로 왼쪽 조종간을 선택할 수 있다. 어댑터는 선택 이벤트에서 상호작용기의 Transform을 `onlyAllowedInteractor`와 비교하고, 다르면 바로 선택을 취소한다.

```
void OnSelectEntered(SelectEnterEventArgs args)
{
    if (onlyAllowedInteractor != null && args.interactorObject.transform !=
        onlyAllowedInteractor)
    {
        // 반대 손(또는 원거리 광선)이 선택함 - 즉시 강제 해제
        if (_interactable.interactionManager != null)
            _interactable.interactionManager.CancelInteractableSelection((IXRSelectInteractable)_interactable);
        return;
    }
    _currentInteractor = args.interactorObject;
    if (joystick != null) joystick.NotifyXRIGrabbed(true);
}
```

잘못된 손이 잡은 채로 두면 올바른 손이 영영 잡지 못하게 막아 버린다. 그래서 무시하지 않고 강제로 해제한다. `XRSimpleInteractable`은 어디에도 물리적으로 붙지 않으니 이렇게 해도 안전하다고 주석은 적는다. 이 손 구분을 XRI의 Interaction Layer Mask 대신 Transform 비교로 한 이유도 있다. 그래야 Project Settings나 XRI 레이어 설정을 하나도 건드리지 않고 동작하기 때문이다. 선택 해제 이벤트는 자신이 기억한 상호작용기에서 온 것일 때만 `NotifyXRIGrabbed(false)`로 전달한다.

두 신호를 합치는 규칙

`JoystickLever`는 이제 잡기 신호를 두 개 받는다.

신호	출처	역할
<code>_xriGrabbed</code>	Near-Far Interactor → <code>XRSimpleInteractable</code> → <code>HandExclusiveGrabAdapter</code> → <code>NotifyXRIGrabbed</code>	우선 경로. 공식 선택 로직
<code>CanGrab / StillGrabbing</code>	<code>HandJointTracker</code> 의 손바닥 거리 + <code>GripAmount/PinchAmount</code>	대비 경로. XRI 연결이 없어도 동작

잡기 시작은 둘 중 하나면 된다. XRI가 먼저 선택했다면 어느 손을 추적할지는 이 조종간에 연결된 추적기로 정한다. 조종간마다 추적기가 하나만 들어 있으니(`leftHandTracker` 와 `rightHandTracker` 가 동시에 null이 아닌 경우는 없다) 헛갈릴 일이 없다. 반대로 놓기는 **두 신호가 모두 놓았다고 할 때만** 일어난다 (`(!_xriGrabbed && !StillGrabbing)`). 한쪽 신호가 한 프레임 먼저 끊겨도, 아직 잡고 있는 다른 쪽을 떼어쓰지 않게 하려는 것이다.

9/22 시점에 확인되지 않은 것

커밋 01의 코드만으로는 이 두 갈래 수정 뒤 이견호가 실기기에서 조종간을 실제로 잡았는지 알 수 없다. 주석에도 "잡힌다"는 확인은 없다. `HandExclusiveGrabAdapter.cs` 의 생성 시각이 9/23 16:15로 남아 있는 것은 이 파일이 커밋 01 이후 다시 쓰였다는 뜻이고, 실제로 다음 커밋(9/23 13:20)에서 잡기 관련 코드가 크게 손질된다. 이 장에서 다른 연결은 "잡히게 만들기 위한 첫 번째 완성된 시도"로 읽는 것이 정확하다.

7. 왼쪽 조종간으로 기체 움직이기

ShipMovementController — 밀면 전진, 옆으로 밀면 선회

`MobileSuitRoot` 에 붙은 `ShipMovementController` 는 왼쪽 조종간의 `tiltInput` 을 읽어 기체를 움직인다. 이 매핑은 앞 장에서 만든 대시보드의 텔레메트리 표시 "`THROTTLE {l.y} TURN {l.x}`"가 원래부터 의도했던 것이라고 주석은 적는다.

- 앞뒤로 밀고 당기기(`tiltInput.y`) = 전진/후진
- 좌우로 밀기(`tiltInput.x`) = 좌우 선회(yaw)

조종간이 이제 비틀리지 않고 미끄러지기만 하므로, 선회는 조종간의 **좌우 위치**로 한다. 옆걸음(strafe)은 없다. 오른쪽으로 밀면 옆으로 가면서 동시에 도는 식이 되어 서로 싸우기 때문이다. 선회만으로도 진행 방향을 바꿀 수 있으니 이 요청을 충족한다는 설명이다.

요청 · 왼쪽으로 밀기 -> Gundam 왼쪽 이동/원하는 방향 전환

속도는 최대 전진 `maxMoveSpeed = 8` m/s, 최대 선회 `maxTurnSpeed = 60` 도/초다.

"머리 시야까지 같이 돈다" — 원인과 해결

이 클래스 주석에서 "IMPORTANT"로 표시된 부분은 그 전의 보고 하나에서 나왔다. 조종간이 아직 비틀기 입력을 받던 때다.

문제 보고 · 왼손으로 LeftJoystick을 잡고 돌리면 XR Origin / Main Camera의 머리 시야까지 같이 돌아간다

원인은 계층 구조에 있었다. 예전 코드는 기체를 돌릴 때 이 `GameObject`에 `transform.Rotate(...)` 를 직접 불렀다. 그런데 XR Origin(그리고 그 안의 Main Camera)과 콕핏 내부 전체가 바로 이 `MobileSuitRoot` 아래에 있다(`CreateXROrigin(suitRoot.transform, ...)`, `interior` 도 그 자식). 루트를 돌리면 그 아래

모든 것, 곧 조종사의 시야도 함께 돈다. VR에서 조종사가 실제로 고개를 돌리지 않았는데 시야가 회전하는 것은 대표적인 멀미 유발 요인이다. 해결은 "Main Camera는 실제 HMD 추적으로만 돈다"를 보장하는 방향이어야 했다.

해결책은 기체의 방향을 실제 회전값이 아니라 `float` 하나(`HeadingYaw`)로 들고 다니는 것이다. 이 값은 이동 방향을 계산하는 데만 쓰고, 이 `GameObject`의 `transform.rotation`에는 절대 쓰지 않는다. 위치(`transform.position`)는 평소처럼 갱신하므로, 같은 부모-자식 구조 덕분에 콧빔과 조종사가 기체와 함께 이동한다.

```
Vector2 raw = leftStick.tiltInput;
Vector2 t = new Vector2(ApplyDeadZone(raw.x), ApplyDeadZone(raw.y));

HeadingYaw += t.x * maxTurnSpeed * Time.deltaTime; // 회전은 float에만 누적
Quaternion heading = Quaternion.Euler(0f, HeadingYaw, 0f);
Vector3 headingForward = heading * Vector3.forward;

CurrentSpeed = t.y * maxMoveSpeed;
transform.position += headingForward * (CurrentSpeed * Time.deltaTime); // 위치만 이동
```

`HeadingYaw`는 공개 속성으로 내놓았다. 나침반 표시나 나중의 외부 시점처럼, 일부러 바뀌지 않게 만든 `transform.rotation` 대신 "기체가 어느 쪽을 향하는지" 알아야 할 스크립트를 위해서다. 그 결과 이 시점의 기체는 시야를 돌리지 않고 진행 방향만 바꾼다. 콧빔 내부도, 바깥 건담 모델(`ExternalGundam`, 회전 identity 고정)도 조종간 때문에 회전하지 않는다.

"움직여도 이동하지 않는다" — 감사와 데드존

이어서 이런 보고가 들어왔다.

문제 보고 · LeftJoystick을 움직여도 Gundam/MobileSuit가 실제로 이동하지 않는다

Claude는 이동 경로 전체를 처음부터 끝까지 다시 확인했고, 그 결과를 주석에 검증 목록처럼 남겼다. 생성기에서 `ship.leftStick = leftStick`이 `CreateJoystick("LeftJoystick", ...)` 바로 다음에 연결돼 있다. `tiltInput`의 X/Y 의미는 위치 기반 `JoystickLever`의 출력과 맞고 이미 -1~1로 잘려 있다. 속도 값(8 m/s, 60°/s)은 0이 아니다. `Update`는 실행될 때마다 `transform.position`을 쓴다. 회전 시절의 죽은 코드나 남은 필드도 없었다. 요청 대비 실제로 빠져 있던 것은 데드존 하나였다.

`inputDeadZone = 0.08` (범위 0~0.3). 각 축의 절댓값이 이보다 작으면 0으로 본다. 조종간을 중심 근처에 가볍게 쥐고 있을 때 손 떨림으로 기체가 슬금슬금 밀리지 않게 하려는 것이다. 단순히 잘라 내기만 하면 데드존 경계에서 값이 0에서 0.08로 툭 튀므로, 남은 구간을 다시 0~1로 늘린다. 끝까지 밀면 여전히 정확히 1이 나온다.

```
float ApplyDeadZone(float v)
{
    float mag = Mathf.Abs(v);
    if (mag < inputDeadZone) return 0f;
    float rescaled = (mag - inputDeadZone) / (1f - inputDeadZone);
    return Mathf.Sign(v) * Mathf.Clamp01(rescaled);
}
```

밖에서 보는 건담도 따라오게 — ExternalGundamFollower

같은 증상의 다른 면을 짚은 보고가 하나 더 있었다.

문제 보고 · LeftJoystick을 움직여도 실제 Gundam이 움직이는 것이 화면에서 보이지 않는다

이번 원인은 이동 코드가 아니라 "누가 바깥 화면을 찍고 있는가"였다. 앞 장에서 다룬 것처럼 콕핏 돔과 왼쪽 보조 화면에 비치는 바깥 풍경은 ExternalGundam의 Head 뼈에 붙은 HeadCam이 만든다. 그런데 ExternalGundam은 기체 회전을 물려받지 않도록 일부러 월드 공간에 고정해 둔 오브젝트였다. 그래서 회전뿐 아니라 이동도 따라가지 않았다. MobileSuitRoot는 제대로 움직이는데, 바깥 화면을 찍는 카메라가 제자리에 있으니 화면에는 아무 변화가 없었던 것이다.

최소한의 수정으로 ExternalGundamFollower를 만들었다. 처음 실행될 때 MobileSuitRoot와의 위치 차이(_offset)를 한 번 기억하고, 매 프레임 `target.position + _offset`으로 옮긴다. 위치만 따라가고 회전은 건드리지 않는다. Head 뼈의 회전은 여전히 GundamHeadCam360이 조종사의 고개 방향에 맞춰 맡는다. Update가 아니라 LateUpdate를 쓴 것은 ShipMovementController가 그 프레임의 이동을 끝낸 뒤에 따라가게 해서, 한 프레임 늦게 따라오는 일을 없애기 위해서다. 생성기는 PlaceExternalGundam이 돌려준 instance에 이 컴포넌트를 붙이고 `target = suitRoot.transform`을 연결한다.

계기판 표시 — CockpitHUD

조종간 상태는 가운데 SYSTEM CHECK 화면 아래쪽 Telemetry 텍스트에 나온다. CockpitHUD가 매 프레임 "GUNDAM COCKPIT", `THRATTLE {l.y×100}% TURN {l.x×100}%, AIM X {r.x×100} Y {r.y×100}`를 쓰고, 잡혀 있는 조종간마다 [L GRIP], [R GRIP]을 붙인다. 잡혔는지를 눈으로 확인할 수 있는 거의 유일한 표시라서, 잡기 문제를 시험할 때 유용한 단서가 된다.

8. 첫 무기 — 오른쪽 조종간으로 겨누는 포탑

첫날부터 있던 무기 뼈대

WeaponAimFireController.cs와 HitTarget.cs는 생성 시각이 9/21 16:58로, 스크립트 폴더가 생긴 순간과 같다. 프로젝트 첫 골격에 이미 "오른쪽 조종간으로 겨누고 쏜다"는 구상이 들어 있었다는 뜻이다. 클래스 요약 주석도 단순하다. 오른쪽 조종간으로 총 피벗을 겨누고, 소총 손잡이를 앞으로 밀듯 조종간을 문턱 이상 앞으로 밀면 발사한다. 투사체 프리팹이 있으면 그것을 쏘고, 없으면 레이캐스트로 HitTarget을 맞힌다.

포탑 만들기

생성기는 `MobileSuitRoot` 아래 로컬 (0, 1.0, 2.4)에 `GunTurret` 을 놓는다. 그 아래 회전축 `GunPivot` 이 있고, 거기에 기본 도형으로 만든 포신이 붙는다.

오브젝트	모양	로컬 위치	크기 / 회전
GunHousing	실린더, hullMat	(0, 0, 0.1)	(0.14, 0.14, 0.2), X 90°
Barrel	실린더, gearMat	(0, 0, 0.45)	(0.07, 0.5, 0.07), X 90°
MuzzlePoint	빈 오브젝트	(0, 0, 0.95)	총구, 발사 기준점

`WeaponAimFireController` 는 `MobileSuitRoot` 에 붙고 `rightStick` , `gunPivot` , `muzzlePoint` 가 연결된다. `projectilePrefab` 은 비어 있으므로 이 시점의 사격은 레이캐스트 방식이다.

겨누기와 발사

매 프레임 오른쪽 조종간의 `tiltInput` 으로 좌우각(`_yaw`)과 상하각(`_pitch`)을 누적하고, 각각 $\pm 45^\circ$, $\pm 30^\circ$ 로 자른 뒤 `gunPivot.localRotation` 에 넣는다. 끝까지 밀었을 때 겨누는 속도는 `aimSpeed = 90` 도/초다. 조종간이 잡혀 있고 앞으로 민 값(`t.y`)이 `pushToFireThreshold = 0.85` 를 넘으면 `TryFire` 를 부른다.

```
_yaw = Mathf.Clamp(_yaw + t.x * aimSpeed * Time.deltaTime, -maxYaw, maxYaw);
_pitch = Mathf.Clamp(_pitch - t.y * aimSpeed * Time.deltaTime, -maxPitch, maxPitch);
gunPivot.localRotation = Quaternion.Euler(_pitch, _yaw, 0f);
_cooldownTimer -= Time.deltaTime;
if (rightStick.isGrabbed && t.y > pushToFireThreshold) TryFire();
```

코드 그대로 읽으면, 같은 앞뒤 축이 상하 조준과 발사를 함께 맡는다. 앞으로 끝까지 밀면 포가 한쪽으로 기울면서 동시에 연사가 시작되는 구조다. 이 시점의 무기가 "프로토타입"이었음을 보여 주는 대목이고, 나중에 손가락 버튼으로 발사를 옮기는 이유가 된다(그 변화는 뒤 장의 일이다).

`TryFire` 는 쿨다운과 탄약을 확인하고, 총구 위치에서 앞쪽으로 `raycastRange = 200` m 레이캐스트를 쏜다. 맞은 콜라이더에 `HitTarget` 이 있으면 `OnHit()` 을 부른다. 시각 효과는 아직 `Debug.DrawLine` (하늘색, 0.15초)뿐이라서 에디터 켜진 뷰에서만 보인다.

무기 목록과 탄약

무기는 `WeaponDef` (이름, 최대 탄약, 발사 간격) 배열로 정의돼 있다.

무기	maxAmmo	fireCooldown
MACHINE GUN	30	0.15 초
BEAM RIFLE	6	0.6 초

Awake 에서 현재 무기(인덱스 0, 머신건)의 발사 간격과 탄약을 적용한다. 쏠 때마다 탄약이 하나씩 줄고, 0이 되면 더 쏘지 않는다. 재장전은 아직 없다. 다음 무기로 바꾸는 CycleWeapon() 은 있지만, 주석대로 "아직 어떤 물리 조작에도 연결되지 않았다". 무기 이름과 탄약은 CockpitHUD 가 가운데 화면의 WeaponName 과 AmmoReadout (AMMO 030/030 형식)에 표시한다. 예전 물리 계기판을 지우고 이 정보를 세 화면이 대신 맡게 된 것도 이견호의 요청이었다.

요청 · 데시보드 역할이 아까 만든 디스플레이 3개가 하는거야

표적 — HitTarget

HitTarget 은 가장 단순한 적이다. 맞으면 자기 Renderer와 Collider를 끄고, respawnDelay = 2 초 뒤 다시 켜다. 연속으로 맞으면 CancelInvoke 로 이전 예약을 지우고 다시 2초를 켜다. 생성기는 월드 공간의 Targets 아래에 0.8m 큐브 표적 여섯 개를 만든다. 주황 발광 재질(targetMat, 방출색 (1, 0.35, 0.05))이다.

```
System.Random rnd = new System.Random(1234); // 시드 고정: 다시 빌드해도 같은 배치
for (int i = 0; i < 6; i++)
{
    float side = (i % 2 == 0) ? -1f : 1f;
    float x = side * (float)(3.5 + rnd.NextDouble() * 9.0); // |x| 3.5~12.5
    float y = (float)(rnd.NextDouble() * 6.0 - 1.0); // -1~5
    float z = 12f + i * 6f; // 12, 18, ... 42
    GameObject t = CreateCube($"Target_{i}", targetsRoot.transform, new Vector3(x, y, z),
    Vector3.one * 0.8f, targetMat);
    t.AddComponent<HitTarget>();
}
```

표적은 왼쪽·오른쪽을 번갈아 가며, 앞 디스플레이의 시야 원뿔 바깥 양옆과 위아래로 퍼지도록 놓였다. 디스플레이 바로 뒤에 겹쳐 쌓이지 않게 한 것이다. 기체에 붙이지 않았으므로 기체가 움직이면 표적과의 거리가 실제로 바뀐다. 움직이는 느낌을 주기 위해 반지름 25~70m 구면 위에 0.25m 큐브 별 60개 (Starfield)도 뿌렸다. 별은 콜라이더를 지워 레이캐스트에 걸리지 않게 했다. 이 별은

UnityEngine.Random 을 써서 빌드할 때마다 위치가 바뀐다.

9. 타겟팅 HUD — 링이 적을 보고 조여 들어간다

9월 22일 아침 9시 55분 무렵 TargetingHUD.cs 가 생겼다. 앞 장에서 만든 커다란 궤도 링(OrbitHUD, 중심 (0, 1.45, 2.4), 반지름 1.5, 눈금 60개)에 기능을 주는 작업이다. 이견호는 건담 록핏 HUD가 적을 락온하는 장면의 스크린샷을 보내며 이렇게 말했다.

요청 · 저 링이 이런식으로 적을 보면 타겟팅이 되야함

그리고 두 가지를 덧붙였다.

요청 · 내가 오른손 손잡이로 조종할수있어야해

설계

조준 기준은 기체 정면이 아니라 **총의 회전축**(`aimPivot = WeaponAimFireController.gunPivot`)이다. 오른쪽 조종간으로 포를 돌리면 조준선도 함께 쏘고 지나가고, 그 아래 걸린 적의 표식이 줄어들며 락온된다. 순수한 시각 센서이며 직접 쏘지는 않는다.

필드	값	의미
<code>ringRadius</code>	1.5	OrbitHUD 링 반지름과 같아야 함. 표식이 링 가장자리에서 멈춤
<code>detectionRange</code>	60 m	탐지 거리
<code>fieldOfViewDeg</code>	90°	조준 방향 기준 탐지 시야
<code>aimShrinkAngleDeg</code>	24°	이 각도 안에 들어오면 줄어들기 시작
<code>lockAngleDeg</code>	6°	이 각도 안이면 락온
<code>lockedMarkerScale</code>	0.45	락온 시 표식 크기 배율
<code>rescanInterval</code>	0.5 초	표적 목록 다시 찾기 주기

매 프레임 하는 일

0.5초마다 썬의 모든 `HitTarget` 을 `FindObjectsOfType` 로 다시 모은다. 그다음 표적마다 다음 순서로 거른다. 꺼져 있거나 방금 맞아서 `Renderer`가 꺼진(재생성 대기 중) 표적은 건너뛰다. 거리가 60m를 넘어도 건너뛰다. 방향을 조준 축의 로컬 좌표로 바꿔 뒤쪽($z \leq 0.05$)이거나 시야 90°의 절반인 45°를 벗어나면 건너뛰다.

남은 표적은 단순 원근 투영으로 링 평면에 찍는다. 링이 조준 축에서 앞으로 떨어진 거리를 `centerZ` 라 할 때 배율 `centerZ / localDir.z` 를 방향의 x·y에 곱하고, 결과가 링 반지름보다 크면 반지름 위로 끌어당긴다. 시야 가장자리의 적은 날아가 버리지 않고 링 가장자리에 붙어 보인다.

```
float scale = centerZ / localDir.z;
Vector2 p = new Vector2(localDir.x * scale, localDir.y * scale);
if (p.magnitude > ringRadius) p = p.normalized * ringRadius;
marker.localPosition = new Vector3(p.x, p.y, 0f);
```

모든 표적을 한 바퀴 돈 **뒤에**, 가장 정면에 가까운 하나만 골라 줄어들게 한다. 처음 가까웠던 하나가 아니라 보이는 전체 중 가장 중앙에 있는 것을 고르기 위해서다. 줄어들기는 24°에서 6°까지 연속으로 이어진다.

`InverseLerp(24, 6, angle)` 로 0~1을 얻고 크기를 1에서 0.45까지 보간한다. 조준하는 동안 표식이 서서히 조여 드는 것처럼 보이게, 한 번에 푹 바뀌지 않도록 했다. 6° 안에 들어오면 표식 재질을 밝은 "락온" 재질로 바꾸고, 같은 자리에 작은 고리(lock ring)를 쳐서 같은 배율로 줄인다.

표식은 생성기가 만들고, 스크립트는 켜고 끄기만

주석은 이 스크립트가 GameObject를 만들거나 지우지 않는다고 강조한다. 표식 풀(markers, markerRenderers, lockRings)과 재질(markerMat, lockedMat)은 생성기가 미리 만들어 넣어 주고, 런타임 스크립트는 이미 있는 것을 켜고 끄고 옮기고 크기만 바꾼다. 생성기가 켜는 모든 도형을 소유하고, 런타임 스크립트는 움직이기만 한다는 이 프로젝트의 원칙을 따른 것이다.

커밋 01의 생성기에는 연결 코드가 없다

그런데 커밋 01의 GundamCockpitSetup.cs에는 TargetingHUD를 붙이거나 표식 풀을 만드는 코드가 보이지 않는다. 이후의 모든 커밋 스냅샷에서도 생성기는 이 클래스를 참조하지 않는다. 스크립트 파일은 끝까지 남아 있지만, 적어도 이 스냅샷의 "Build Cockpit Scene" 결과에는 링 위의 락온 표식이 생기지 않는다. 이 시점에 썸을 다른 방법으로 연결했는지는 코드만으로 확인할 수 없다. 또 HUD가 찾는 대상은 HitTarget 뿐이라서, 아래에서 배치할 자쿠(EnemyMarker만 붙음)는 아직 이 HUD의 탐지 대상이 아니다.

10. 첫 적 — 자쿠를 우주에 세우다

모델 도착: "자쿠는 적으로 만들거야"

9월 22일 오전 10시 30분 무렵 건담 모델이, 12시 57분 무렵 자쿠 모델이 프로젝트에 들어왔다. 건담 모델을 콕핏 앞에 세우고 그 머리에 카메라를 다는 과정은 앞 장의 주제다. 여기서는 텍스처와 적 배치만 본다. 이건호는 두 모델의 텍스처를 받아 두고 이렇게 요청했다.

요청 · 자쿠 모델링이랑 건담 색깔 다운해둬 자쿠는 적으로 만들거야 자쿠를 적으로 배치해줘

범위에 대한 단서도 함께 남아 있다. 주석이 인용한 조각은

요청 · 지금은... 먼저 완성해라

이다. 이것을 근거로 Claude는 AI·조준·체력·전투를 넣지 않고, "배치"와 "적이라는 꼬리표"까지만 했다.

생성기 맨 위에 경로 상수가 추가됐다.

```
const string ZakuModelPath = "Assets/Models/Zaku/Green Zaku Mobile Suit-aca4bc150af72115520493e85c7e2457.fbx";
const string ZakuTexturePath = "Assets/Models/Zaku/ZakuTexture.png";
const float MobileSuitTargetHeight = 18f; // 건담과 같은 "모빌슈트 키" 약속
// 건담 쪽: "Assets/Models/Gundam/Mobile Suit Gundam-baseColor.png"
```

EnemyMarker — 빈 클래스의 역할

EnemyMarker.cs (12:59 무렵)는 본문이 비어 있는 MonoBehaviour다. 주석이 이유를 적는다. 요청은 자쿠를 장식이 아니라 "적으로" 배치하라는 것이었다. 그래서 배치되는 순간부터 코드와 에디터 계층에서 적임을 알아볼 수 있게 꼬리표를 달았다. 별도 파일로 둔 데도 의도가 있다. 나중에 실제 적 행동을 이 파일에 더할 때,

씬 생성기를 건드리지도, 생성기에 덮어써지지도 않게 하려는 것이다. 실제로 체력은 뒤에 `EnemyHealth.cs` (9/28 무렵)로 따로 생긴다.

PlaceZakuEnemy

`BuildCockpitScene` 은 `PlaceExternalGundam()` 바로 다음에 `PlaceZakuEnemy()` 를 부른다. 함수가 하는 일은 이렇다.

불러오기. `AssetDatabase.LoadAssetAtPath` 로 FBX를 읽는다. 실패하면, 즉 파일을 막 복사해 넣어 Unity가 아직 임포트하지 않았다면, "Assets > Refresh를 하거나 프로젝트를 다시 열고 Build Cockpit Scene을 다시 실행하라"고 경고하고 그냥 돌아간다. 나머지 씬 생성은 실패시키지 않는다. 건담 배치와 같은 방어 방식이다.

배치. `PrefabUtility.InstantiatePrefab` 으로 만들어 이름을 `ZakuEnemy` 로 바꾸고 월드 공간에 둔다. 조종사의 기체가 아니므로 `MobileSuitRoot` 아래에 넣지 않는다. 위치는 (0, 0, 40), 회전은 Y 180°다. 건담이 Z=20에 서 있으니 그보다 더 멀리, 서로 겹치지 않고 우주 공간에서 마주 보는 두 기체로 읽히게 했다. 180°는 플레이어 쪽을 바라보게 하려는 것이다.

크기. 건담은 FBX 원본 높이 0.985469를 알고 있어서 18m로 맞추는 배율을 상수로 계산했다. 자쿠는 원본 크기를 미리 알 수 없으므로, 손으로 짐작한 숫자를 넣지 않고 임포트된 모델의 실제

`SkinnedMeshRenderer.bounds.size.y` 를 재서 `18 / 높이` 로 배율을 정한다. 짤 수 없으면 크기가 틀릴 수 있으니 직접 조정하라고 경고한다.

텍스처와 꼬리표. `ZakuTexture.png` 를 `ApplyBaseColorTexture` 로 입히고, `instance.AddComponent<EnemyMarker>()` 로 적 표시를 단다.

버그: "자쿠 색깔이 건담한테도 들어갔는데"

처음 배치한 뒤 이견호의 보고다.

문제 보고 · 자쿠 색깔이 건담한테도 들어갔는데

원인은 공유 재질이였다. 건담과 자쿠 FBX는 둘 다 자기 재질 없이 임포트되어, 같은 기본(텍스처 없는) 재질 하나를 함께 가리키고 있었다. 예전 `ApplyBaseColorTexture` 는 `renderer.sharedMaterials[i]` 를 그 자리에서 고쳤다. 공유 에셋을 바꿨으니 자쿠에 입힌 텍스처가 같은 재질을 쓰는 건담에도 그대로 묻어났다. 건담 텍스처를 먼저 입히고 자쿠 텍스처를 나중에 입히면 마지막 것이 둘 다를 덮는다.

수정은 "고치기 전에 복사"다. 모델마다, 재질 슬롯마다 `new Material(원본)` 으로 자기만의 복사본을 만들고, 이름에 모델 라벨을 붙인 뒤 ("`...` (`ZakuEnemy`)") 그 복사본에만 텍스처를 넣는다.

```
foreach (Renderer renderer in root.GetComponentsInChildren<Renderer>(true))
{
    Material[] materials = renderer.sharedMaterials;
    for (int i = 0; i < materials.Length; i++)
    {
        if (materials[i] == null) continue;
        Material uniqueMat = new Material(materials[i]); // 공유 재질을 건드리지 않고 복사
        uniqueMat.name = materials[i].name + " (" + label + ")";
        uniqueMat.mainTexture = texture;
        materials[i] = uniqueMat;
    }
    renderer.sharedMaterials = materials;
}
```

텍스처 속성 이름을 `_BaseMap` 처럼 셰이더에 맞춰 직접 쓰지 않고 `Material.mainTexture` 를 쓴 점도 눈여겨볼 만하다. Unity는 이 속성을 셰이더가 "주 텍스처"로 지정한 속성(URP Lit이면 `_BaseMap`)으로 알아서 연결한다. 그래서 같은 함수를 건담과 자쿠에 똑같이 쓸 수 있다.

버그: "밖에 자쿠가 보이지 않아"

자쿠를 세웠는데 콕핏 안에서는 보이지 않았다.

문제 보고 · 밖에 자쿠가 보이지 않아

원인을 이해하려면 앞 장의 결정을 떠올려야 한다. 조종사는 돔 안에 갇혀 있고, 돔 표면은 건담 머리 카메라의 영상을 보여 주는 "화면"이다. 조종사의 눈(메인 카메라)은 돔 바깥을 직접 보지 못한다. 그런데 그때 돔에 입힌 것은 시야 60°짜리 평면 영상 하나(`headCamTex`)를 구 전체에 늘려 붙인 것이었다. 그 영상에는 머리 카메라가 그 순간 정면으로 보고 있던 좁은 원뿔 안의 것만 들어 있다. 40m 앞에 선 자쿠가 그 원뿔 밖에 있으면 이미지에 아예 없고, 아무리 늘려 붙여도 되살아나지 않는다. 생성기의 예전 주석은 이 방식을 "약간 왜곡되는 정도"의 한계로 적어 두었는데, 실제로는 먼 물체를 가리는 결함이었다고 "UPDATE per report"로 고쳐 적었다.

해결은 이미 매 프레임 렌더링되고 있던 360° 큐브맵을 돔이 직접 쓰게 하는 것이었다. `GundamHeadCam360` 이 `Camera.RenderToCubemap` 으로 스카이박스용 큐브맵을 그리고 있었다. 이 큐브맵을 `Assets/Models/Gundam/GundamHeadCam360.renderTexture` 에 저장되는 에셋(면당 512)으로 만들고, 손으로 쓴 작은 URP 셰이더 `Custom/CockpitDomeCubemap` 이 돔 중심에서 각 표면 점으로 향하는 실제 월드 방향으로 큐브맵을 샘플링한다. 돔 중심이 조종사의 머리 가까이에 있으니 이 방향은 "조종사가 그쪽을 보는 방향"과 거의 같다. 모든 방향이 늘어나지 않은 제 내용을 보여 주고, 머리 카메라가 어느 쪽을 보고 있었든 빠지는 것이 없다. 셰이더나 큐브맵을 찾지 못하면 예전 평면 영상으로, 그것도 없으면 정적인 깃데기 재질로 물러난다. 평면 영상으로 물러날 때는 "자쿠 같은 먼 물체가 잘 안 보일 것"이라고 경고한다. 돔과 셰이더의 세부 구현은 앞 장의 범위이므로 여기서는 자쿠 문제와 이어지는 부분만 적었다.

한 가지 더 짚어 둘 점이 있다. 건담 본체는 레이어 30(`GundamBodyLayer`)에 두고 조종사 카메라의 `cullingMask` 에서 빼서, 조종사가 자기 기체를 밖에서 보지 않게 했다. 자쿠에는 이 레이어 처리를 하지 않았다. 머리 카메라는 원래 모든 것을 보므로, 자쿠는 돔 영상에 정상적으로 찍힌다.

11. 9월 22일이 끝날 때 남아 있던 것

커밋 01 스냅샷을 다시 훑으면, 이틀 동안 "손으로 잡고, 움직이고, 겨누고, 적을 본다"는 뼈대가 모두 자리를 잡았다. 동시에 코드와 주석이 스스로 "아직"이라고 적어 둔 것도 많다. 다음 장으로 넘어가기 전에 정리해 둔다.

항목	이 시점의 상태	근거
조종간 잡기	GripAmount 추가 + XRI Near-Far Interactor 연결까지 완료. 실기기에서 확실히 잡혔다는 기록은 없음	HandJointTracker·JoystickLever 주석, 다음 커밋의 대규모 수정
잡는 순간 스냅	GripPoint를 손바닥에 즉시 스냅	9/23 "순간적으로 끌려온다" 보고로 제거됨(다음 장)
조종간 위치	(±0.22, 0.87, 0.18). 화면 프레임과 겹쳐 보일 가능성을 Claude가 확인 못 함	생성기 주석 "tell me and I'll pull it back"
손가락 버튼, 방아쇠	모양과 콜라이더만. 기능 없음	CreateJoystick 주석
무기 교체	CycleWeapon() 있으나 조작에 미연결	WeaponAimFireController 주석
발사 방식	오른쪽 조종간을 0.85 이상 앞으로 밀기(상하 조종과 같은 축)	WeaponAimFireController.Update
타겟팅 HUD	스크립트는 완성. 생성기에 연결 코드 없음. HitTarget만 탐지	GundamCockpitSetup.cs 전체
자쿠	위치·크기·텍스처·EnemyMarker까지. AI·체력·전투 없음. 사격 판정 대상 아님	EnemyMarker, PlaceZakuEnemy 주석
기체 회전	시야는 돌지 않고 진행 방향(HeadingYaw)만 바뀜	ShipMovementController

이 목록에서 Claude의 작업 방식도 한 가지 드러난다. 에디터 화면도 헤드셋 화면도 볼 수 없는 Claude는, 확인할 수 없는 가정을 할 때마다 그 사실을 주석에 적었다. "이 위치가 화면을 뚫고 들어가 보이면 말해 달라", "모텔이 누워 있으면 보정 회전을 넣겠다", "피드가 다시 회색으로 뿌옇게 나오면 비율을 조정하겠다" 같은 식이다. 그리고 그 확인을 이견호의 다음 테스트에 맡겼다. 원인을 위아래로 가르는 진단 로그를 먼저 심은 것도 같은 맥락이다. 이 상태가 9월 23일 오전 10시 6분 커밋 01로 처음 Git에 들어간다. 그 뒤 9월 23일 작업에서 나오는 보고들, 곧 조종간이 잡는 순간 손 쪽으로 끌려오는 문제, 두 조종간이 서로의 손 움직임에 반응한다는 문제, 사람마다 다른 팔 길이에 맞춰 조종간 위치를 정하는 보정(CalibrationManager)이 다음 장의 이야기다.

이날 한 일, 그리고 남은 기록

9월 23일은 이 프로젝트의 작업이 처음으로 Git 저장소에 담긴 날이다. 그 전 이틀(9월 21~22일) 동안 이견호와 Claude는 콧핀 씌, 손 추적, 조종간 두 개, 외부 건담 모델과 머리 카메라, 적 자쿠까지 만들어 두었지만, 그때까지는 버전 관리 없이 파일만 쌓여 있었다. 이날 아침 저장소를 만들고 지금까지의 작업을 통째로 커밋한 뒤부터 변경이 하나하나 diff로 남기 시작한다. 그래서 이 장은 처음으로 "무엇이 지워지고 무엇이 들어왔는지"를 코드로 직접 확인하면서 따라갈 수 있는 장이다.

이날 작업은 크게 세 덩어리다. 오전에는 Galaxy XR 헤드셋으로 직접 잡아 본 조종간(JoystickLever)의 느낌을 다듬었다. 잡는 순간 조종간이 손 쪽으로 끌려오는 문제, 양손이 서로의 조종간에 반응한다는 의심, 지연과 과민한 감도, 손을 밀어도 움직이지 않는 문제가 차례로 나왔다. 그와 함께 사람마다 팔 길이가 다르니 조종간 위치를 직접 정하게 해 달라는 요청에서 CalibrationManager가 생겼고, 이 기능은 두세 번 망가졌다가 다시 설계됐다. 오후에는 실제 기기 테스트에서 나온 조종간 문제를 더 깊이 고쳤고, 오른손 조종간이 잘 안 잡히는 문제를 해결했다. 그다음 시선으로 조준하고 엄지로 쏘는 헤드 발칸을 처음 만들었고, 마지막으로 콧핀 앞 디스플레이 세 개에 STATUS·전술·WEAPON이라는 역할을 각각 나눠 맡겼다.

근거가 되는 기록은 커밋 네 개다. 그런데 커밋 시각과 실제 작업 시각이 꼭 맞지는 않는다. 특히 04 커밋은 9월 28일 아침 09:31에 찍혔지만, 그 안에 든 파일들의 생성 시각을 보면 대부분 9월 23일 오후에 만든 것이다. 그래서 이 장에서는 04 커밋의 내용도 9월 23일 오후 작업으로 다룬다.

시각	근거	내용
09:50	커밋 00	저장소 초기화(빈 커밋)
10:06	커밋 01	9/21~9/22 작업 전체가 처음 들어옴
10:09	커밋 02	.gitignore 추가
10:09~13:20	커밋 03	조종간 오프셋 방식·입력 안정화, CalibrationManager 신설과 재설계, 콧핀 안 자기 건담 숨김, 패키지 1개 추가
14:20 무렵	파일 시각	CalibrationManager.cs 다시 쓰임(조종간 하나마다 확인받도록 변경)
16:15 무렵	파일 시각	HandExclusiveGrabAdapter.cs 다시 쓰임(오른손 조종간이 잘 안 잡히던 문제 수정)
16:40 무렵	파일 시각	CockpitHUDManager · CockpitStatusHUD · CockpitWeaponHUD 생성

9/28 09:31 커밋 위 오후 작업, 그리고 HeadVulcanController, JoystickLever 재작성,
04 ShipMovementController 옆 이동이 함께 커밋됨

1. Git 저장소 만들기 — 09:50, 10:06, 10:09

첫 커밋(00)은 오전 09:50에 찍혔고 파일이 하나도 없는 빈 커밋이다. 저장소만 만들어 두고 기준점을 하나 찍은 것이다. 16분 뒤인 10:06의 커밋 01에는 9월 21일부터 22일까지 만든 것이 한꺼번에 들어온다. 이 시점 스냅샷의 Assets/Scripts/Cockpit 폴더에는 JoystickLever, HandJointTracker, HandExclusiveGrabAdapter, ShipMovementController, WeaponAimFireController, CockpitHUD, TargetingHUD, EnemyMarker, HitTarget, GundamHeadCam360, ExternalGundamFollower, HandTrackingStatusLogger, HandSubsystemBootDiagnostics 까지 13개 스크립트가 있다. 여기에 씬 전체를 코드로 만드는 에디터 스크립트 Assets/Editor/GundamCockpitSetup.cs 와 셰이더 CockpitDomeCubemap.shader 가 더해진다. diff 경로가 gundam game/Assets/... 로 시작하는 것을 보면, 저장소 최상위 아래에 Unity 프로젝트 폴더 gundam game 이 통째로 들어 있는 구조다.

3분 뒤 10:09의 커밋 02는 .gitignore 파일 하나만 추가한다. 내용은 15줄이다.

```
[Ll]ibrary/
[Tt]emp/
[Oo]bj/
[Bb]uild/
[Bb]uilds/
[Ll]ogs/
[Uu]ser[Ss]ettings/

.vs/
.idea/

*.csproj
*.sln
*.user
*.userprefs
```

Unity 프로젝트를 Git에 올릴 때 가장 먼저 해야 하는 일은 Unity가 자동으로 다시 만드는 폴더를 빼는 것이다. Library/ 는 임포트된 에셋 캐시라서 수 GB까지 커지고, Temp/, Obj/, Logs/ 는 실행할 때마다 바뀐다. UserSettings/ 는 개인 에디터 설정이다. Build/, Builds/ 에는 APK 같은 빌드 결과물이 들어간다. .vs/ 와 .idea/ 는 Visual Studio와 Rider의 작업 폴더이고, *.csproj 와 *.sln 은 Unity가 IDE를 위해 자동으로 만드는 프로젝트 파일이다. 패턴마다 [Ll] 처럼 대소문자를 둘 다 받게 쓴 것은 운영체제에 따라 폴더 이름의 첫 글자가 다르게 생길 수 있어서이다. 앞에 / 를 붙이지 않았으므로 이 패턴들은 어느 깊이에서든 맞는다. 저장소 최상위가 아니라 gundam game/Library/ 에 있는 폴더도 제외된다는 뜻이다. 반대로 Assets/, Packages/, ProjectSettings/ 는 그대로 추적된다. 실제로 이후 diff에는 Packages/manifest.json 의 변경도 기록된다.

따라 해 보기

Unity 프로젝트를 새 저장소에 올릴 때는 커밋하기 전에 `.gitignore` 부터 두는 편이 안전하다. 이 프로젝트는 01 커밋 뒤에 `.gitignore` 를 추가했지만, 스냅샷에 남은 파일은 `Assets` 와 `Packages` 아래 소스뿐이어서 캐시 폴더가 커밋에 섞였는지는 기록으로 확인할 수 없다.

2. 조종간 다듬기 (오전)

2.1 출발점: 10:06 커밋의 조종간 구조

이날 아침 조종간이 어떤 상태였는지부터 짚어 두어야 이후 변경을 따라갈 수 있다. 조종간은

`GundamCockpitSetup.CreateJoystick` 이 만든다. 계층 구조는 `LeftJoystick_Mount` →

`LeftJoystick_Pivot` → `LeftJoystick_Handle` → 그 밑의 `GripBall` 과 손가락 버튼들이다. Mount는

콕핏 내부(`CockpitInterior`) 기준으로 왼쪽은 `(-0.22, 0.87, 0.18)`, 오른쪽은 `(0.22, 0.87, 0.18)` 에 놓인다. Pivot은 Mount에서 `(0, 0.02, 0.05)` 만큼 떨어져 있다. 손으로 쥐는 공(`GripBall`)은 지름 0.10m 구다. 그 위에 엄지 버튼(0.022m), 앞면에 검지·중지·약지·새끼 버튼(각 0.016m)이 붙고, 오른쪽 조종간에만 트리거 큐브가 하나 더 있다. 왼쪽은 파란색(조향), 오른쪽은 빨간색(무기)이다.

`JoystickLever` 는 Mount에 붙는다. 이 스크립트가 움직이는 것은 Handle 하나뿐이다. Handle은 Pivot 기준으로 X(좌우)와 Z(앞뒤) 평면에서만 미끄러지고, 높이(Y)는 바뀌지 않으며 회전하지 않는다. 이동 범위는 좌우 `lateralRange = 0.08 m`, 앞뒤 `depthRange = 0.08 m`다. Handle이 중심에서 벗어난 거리를 이 범위로 나눈 값이 `tiltInput` (X, Y 각각 -1~1)이다. `ShipMovementController` 는 왼쪽 조종간의 이 값으로 기체를 움직이고, `WeaponAimFireController` 는 오른쪽 조종간의 이 값으로 포탑을 조준한다.

"잡았다"는 판정은 두 길로 들어온다. 하나는 XR Interaction Toolkit(XRI) 샘플 리그 "XR Origin Hands (XR Rig)"에 이미 들어 있는 손마다의 `Near-Far Interactor` 가 Handle에 붙은 `XRSimpleInteractable` 을 선택하는 길이다. 이 신호는 `HandExclusiveGrabAdapter` 를 거쳐 `NotifyXRIGrabbed` 로 전달된다. 다른 하나는 직접 만든 예비 판정이다. 손바닥이 Pivot에서 `grabRadius = 0.22 m` 안에 있고,

`HandJointTracker` 가 계산한 쥐기 정도(`GripAmount`)가 0.55 이상이거나 꼬집기 정도(`PinchAmount`)가 0.5 이상이면 잡은 것으로 본다. 놓을 때는 각각 0.35와 0.3 아래로 떨어져야 한다. 잡는 문턱과 놓는 문턱을 다르게 둔 것은 경계값 근처에서 잡힘과 놓임이 번갈아 깜빡이지 않게 하려는 장치다.

항목	값(10:06 기준)
<code>grabRadius</code>	0.22 m
<code>gripToGrabThreshold</code> / <code>gripToReleaseThreshold</code>	0.55 / 0.35
<code>pinchToGrabThreshold</code> / <code>pinchToReleaseThreshold</code>	0.5 / 0.3
<code>lateralRange</code> / <code>depthRange</code>	0.08 m / 0.08 m
<code>returnLerpSpeed</code> (놓았을 때 복귀)	5

2.2 잡는 순간 조종간이 끌려온다 — 스냅 방식에서 오프셋 방식으로

오전에 이건호가 헤드셋을 쓰고 조종간을 잡아 본 뒤 첫 번째로 보고한 문제다.

문제 보고 · 조종간을 잡는 순간 조종간 전체가 내 몸 쪽/손 쪽으로 순간적으로 끌려온다... 손이 조종간의 어디를 잡았든 현재 조종간 위치를 그대로 유지해야 한다... 조종간을 손 위치로 스냅하지 않는다

원인은 바로 앞서 이건호의 다른 요청을 그대로 구현한 데 있었다. 10:06 커밋의 `JoystickLever` 설명 주석에는 그 요청이 남아 있다.

요청 · 손과 조종간 손잡이가 정확히 붙어야 한다... Grab이 시작되는 즉시 GripPoint가 실제 손바닥 위치에 정확히 맞는다... Grab 중 손이 움직이면 손잡이가 손을 정확하게 따라간다

. Claude는 이 요청을 따라 `gripPoint` 라는 별도 자식 Transform을 두고, 잡는 첫 프레임에 Handle을 손바닥 위치로 즉시 옮겼다(스냅). 코드는 이랬다.

```
// 10:06 커밋 — 잡는 첫 프레임에 손 위치로 '스냅'
Vector3 handLocalNow = pivot.InverseTransformPoint(_activeHand.PalmPosition);
Vector3 gripOffset = gripPoint != null ? gripPoint.localPosition : Vector3.zero;
if (!wasGrabbed)
{
    _handStartLocal = handLocalNow;
    _handleStartLocalPos = ClampToRange(handLocalNow - gripOffset);
    handle.localPosition = _handleStartLocalPos; // ← 여기서 된다
}
```

문제는 손바닥 관절 위치(`PalmPosition`)가 사람이 실제로 공을 감싼 위치와 같지 않다는 점이다. 손은 공의 옆이나 위를 쥐기도 하고, 손 추적값 자체에도 약간의 치우침이 있다. 그런데 잡는 순간 Handle을 그 손바닥 좌표로 옮기니, 잡을 때마다 조종간이 몸 쪽이나 손 쪽으로 확 끌려왔다. 이동 범위가 $\pm 8\text{cm}$ 여서 그 안에서 최대한 손 쪽으로 붙는 모양이 됐고, 잡자마자 `tiltInput` 이 0이 아닌 값을 내니 기체가 의도와 상관없이 움직이기 시작할 수도 있었다.

Claude는 스냅을 완전히 지우고 **오프셋 방식**으로 바꿨다. 잡는 첫 프레임에는 기준점 두 개만 기록하고 Handle은 건드리지 않는다. 하나는 그 순간 손의 Pivot 기준 위치 `_grabStartHandLocal`, 다른 하나는 그 순간 Handle이 있던 위치 `_grabStartJoystickLocal` 이다. 그 뒤 프레임부터는 "손이 잡은 뒤로 얼마나 움직였나"(`handMovement`)만 계산해 시작 위치에 더한다.

```
// 13:20 커밋 — 첫 프레임에는 기준점만 기록, Handle은 그대로
if (!wasGrabbed)
{
    _grabStartHandLocal = handLocalNow;
    _grabStartJoystickLocal = handle.localPosition;
    _trackedTargetLocal = handle.localPosition;
    _lastRawHandLocal = handLocalNow;
    _lastValidHandMovement = Vector3.zero;
}
// 이후 프레임: 목표 = 시작 위치 + (지금 손 - 잡을 때 손)
```


이렇게 하면 손이 공의 어디를 잡았든 잡은 순간의 손과 공 사이 상대 관계가 잡고 있는 동안 그대로 유지된다. 손바닥 좌표에 일정한 치우침이 있어도 두 값을 빼는 순간 상쇄된다. `gripPoint` 는 지우지 않고 남겼지만, 툴팁을 "그립 공의 시각적 중심. 표시용일 뿐 Handle을 움직이거나 맞추는 데 절대 쓰지 않음"으로 바꿔 역할을 순수한 표시용으로 낮췄다.

Pivot 기준 좌표로 계산하는 원칙은 그대로 유지했다. 조종간의 Pivot은 `MobileSuitRoot` 의 자손이고 기체가 움직이면 함께 움직인다. 손의 월드 좌표 변화를 그대로 쓰면 기체가 이동한 거리까지 손이 움직인 것으로 읽힌다. 매 프레임 `pivot.InverseTransformPoint` 로 손 위치를 Pivot 기준으로 다시 구하면, 손 추적기도 같은 곡궤를 타고 움직이므로 공통 이동이 저절로 빠진다. 오프셋 방식의 `handMovement` 는 이런 Pivot 기준 값 두 개의 차이이므로 같은 상쇄 효과를 그대로 물려받는다. 이 원칙이 오후에 한 번 더 문제가 되는데, 그 이야기는 5장에서 다룬다.

2.3 "두 조종간이 동시에 반응한다" — 점검하고, 한 겹 더 막기

두 번째 보고는 양손을 같이 쓸 때의 문제였다.

문제 보고 · 왼손으로 `LeftJoystick`을 잡고 오른손으로 `RightJoystick`을 잡으면 두 조종간이 동시에 반응하거나 서로의 손 움직임에 반응함

양손 분리는 이미 여러 층으로 되어 있었다. 첫째, 씬을 만들 때 왼쪽 조종간에는 `leftHand` 만, 오른쪽 조종간에는 `rightHand` 만 넘기고 반대 손 자리에는 `null` 을 넣는다(`CreateJoystick(..., leftHand, null)`, `CreateJoystick(..., null, rightHand)`). 오른쪽 조종간이 예전에 왼손 추적기도 받던 흔적이 있었는데, 이는 이미 고쳐져 있었다. 둘째, XRI 쪽에는 `HandExclusiveGrabAdapter` 가 있다.

`HandExclusiveGrabAdapter` 는 10:06 커밋에 이미 들어 있던 스크립트다. 파일 시각이 16:15 무렵인 것은 오후에 다시 쓰였기 때문이다(5.4절). 설계 요점은 이렇다. XRI에는 잡은 물체를 손에 붙여 움직이는 `XRGrabInteractable` 이 있지만, 이 프로젝트는 일부러 `XRSimpleInteractable` 을 쓴다. Simple 쪽은 물체를 스스로 움직이거나 돌리지 않고 선택·호버 이벤트만 낸다. 그래서 조종간의 실제 위치 계산은 100% `JoystickLever` 가 맡고, XRI는 "올바른 손이 잡았는가"라는 예/아니오 신호만 준다.

`GundamCockpitSetup.WireJoystickHandInteractors` 는 리그 안에서 "Left Hand", "Right Hand" 아래의 "Near-Far Interactor"를 이름으로 찾는다. `AttachHandInteractable` 은 Handle에 `XRSimpleInteractable` 과 어댑터를 붙이고, 어댑터의 `onlyAllowedInteractor` 에 그 손의 인터랙터를 넣는다. 인터랙터블의 충돌체 목록(`m_Colliders`)은 자동 수집 타이밍에 맡기지 않고 `SerializedObject` 로 Handle 아래 `GripBall`과 버튼 구들의 충돌체를 직접 채운다. 어댑터의 `OnSelectEntered` 는 선택한 인터랙터가 허용된 것이 아니면 `interactionManager.CancelInteractableSelection` 으로 즉시 선택을 풀어 버린다. 잘못된 손이 쥔 채로 남아 올바른 손을 막는 일이 없도록 하기 위해서다. Interaction Layer Mask 설정을 바꾸지 않고 코드만으로 손을 가른 것이므로 프로젝트 설정은 건드리지 않는다.

Claude는 이 층들을 처음부터 끝까지 다시 읽었다. `HandJointTracker` 의 손 구분, `CreateJoystick` 의 `null` 자리, 어댑터의 `onlyAllowedInteractor` 비교, `JoystickLever` 의 `null` 검사까지 모두 확인했다. 주석에 남긴 결론은 "한 조종간이 다른 손의 데이터를 읽을 수 있는 지점을 찾지 못했다"였다. 그래도 Claude는 "XRI가 잡은 뒤에는 공유되거나 따로 찾아 온 손 위치에 기대지 말라"는 취지에 맞춰 보강을 한 겹 더 넣었다.

어댑터가 `NotifyXRIGrabbed(true, args.interactorObject.transform)` 로 실제 선택한 인터랙터의 Transform을 넘기고, `JoystickLever` 는 그 Transform을 `_grabbedInteractor` 에 저장한다. 위치 계산도 새 함수 `GetActiveHandWorldPosition()` 을 거치게 했는데, XRI로 잡힌 경우에는 그 인터랙터의 위치를 직접 읽는다.

```
Vector3 GetActiveHandWorldPosition()
{
    if (_xriGrabbed && _grabbedInteractor != null) return _grabbedInteractor.position;
    return _activeHand != null ? _activeHand.PalmPosition : Vector3.zero;
}
```

미리 말해 두면

이 보강은 오후 실기 테스트에서 오히려 "앞으로는 밀리지도 않음" 문제의 원인으로 밝혀져 통째로 제거된다. 인터랙터 Transform이 실제 손과 1:1로 움직인다는 가정을 검증하지 않은 채 위치 소스로 썼기 때문이다(5.3절).

2.4 늦고, 민감하고, 가끔 튕다 — 입력 안정화

세 번째 보고는 느낌에 관한 것이었다.

문제 보고 · 손 움직임을 조이스틱이 살짝 늦게 따라오는 느낌... 감도가 너무 민감함... 가끔 손 추적값이 순간적으로 튀면서 기체가 갑자기 엉뚱한 방향으로 마구 날아감

한 문장 안에 서로 다른 문제가 세 개 들어 있다. 지연(늦게 따라옴), 감도(너무 민감함), 튕(추적값이 한 프레임에 크게 점프함)이다. 지연을 줄이려면 필터를 약하게 해야 하고, 튕을 막으려면 필터를 강하게 해야 한다. Claude는 이 둘이 부딪치지 않게 장치를 역할별로 나눴다. 13:20 커밋에 들어간 `JoystickLever` 의 새 "Input Stabilization" 필드는 다음과 같다.

필드	값	역할
<code>positionSensitivity</code>	0.65	손 이동량에 곱하는 배율. 같은 손 움직임에 조종간이 덜 움직인다(1:1보다 30~40% 둔함). 거리를 줄일 뿐 지연은 만들지 않는다.
<code>smoothingSpeed</code>	25	Handle이 목표 위치를 따라가는 지수 평활 속도. 유일한 지연 원인이며, 작은 떨림만 거르도록 빠르게 잡았다(툴팁 권장 20~30).
<code>maxHandDeltaPerFrame</code>	0.15m	날것의 손 위치가 한 프레임에 이보다 많이 뛰면 추적 끊김으로 보고 버린다. 속도 제한이 아니라 이상값 검사다.
<code>deadZone</code>	0.08	최종 <code>tiltInput</code> 에 적용하는 데드존. 안쪽은 0, 바깥쪽은 다시 -1~1로 늘려 경계에서 값이 튀지 않게 한다.
<code>returnLerpSpeed</code>	5 → 6	놓았을 때 중심으로 돌아가는 속도. 조금 빨라졌다.

튕 방지는 "이번 프레임 손 위치 - 지난 프레임 손 위치"의 크기를 보고 판단한다. 0.15m를 넘으면 이번 프레임의 손 이동량 대신 지난번에 유효했던 이동량(`_lastValidHandMovement`)을 그대로 쓴다. 조종간이 움직일 수 있는 범위가 $\pm 8\text{cm}$ 에 불과하므로 아무리 빠르게 밀어도 실제 손이 한 프레임에 15cm를 움직일 일은 거의 없다. 이 문턱은 그보다 넉넉히 위에 잡혀 있다. 손 추적 자체가 잠깐 끊긴 프레임

(IsActiveHandTrackingValid() 가 거짓)에는 목표를 갱신하지 않고 그대로 둔다. 0이나 엉뚱한 기본값으로 떨어지지 않게 하기 위해서다.

```
// 13:20 커밋 - 목표는 매 프레임 바로 대입, 실제 Handle만 부드럽게 접근
_trackedTargetLocal = ClampToRange(_grabStartJoystickLocal + handMovement *
positionSensitivity);
...
float smoothT = 1f - Mathf.Exp(-smoothingSpeed * Time.deltaTime);
handle.localPosition = Vector3.Lerp(handle.localPosition, _trackedTargetLocal, smoothT);
```

$1 - \exp(-\text{속도} \times dt)$ 는 프레임레이트가 달라도 같은 느낌으로 따라가게 하는 지수 평활 공식이다. 놓았을 때 중심으로 돌아가는 코드에서 이미 쓰던 방식을 가져왔다. 데드존은 위치가 아니라 최종 입력값에 적용했고, ShipMovementController.ApplyDeadZone 과 같은 "0으로 만든 뒤 남은 구간을 다시 늘리는" 방식을 쓴다. 그래서 데드존 경계를 막 넘는 순간 입력이 0에서 갑자기 0.08로 튀지 않고 0부터 연속적으로 올라간다.

```
static float ApplyDeadZoneRescale(float v, float deadZone)
{
    float magnitude = Mathf.Abs(v);
    if (magnitude <= deadZone) return 0f;
    if (deadZone >= 1f) return 0f;
    float rescaled = (magnitude - deadZone) / (1f - deadZone);
    return Mathf.Sign(v) * Mathf.Clamp01(rescaled);
}
```

마지막으로 두 축을 합친 벡터의 길이가 1을 넘으면 정규화하는 안전장치를 하나 더 붙였다. 축마다 이미 Mathf.Clamp 를 하고 있지만, 대각선으로 밀었을 때 합친 크기가 1.41까지 커질 수 있는 경우를 막는 이중 방어다.

2.5 "손을 앞으로 밀어도 거의 안 움직인다" — 이중 지연 제거

안정화를 넣은 뒤 정반대 방향의 보고가 들어왔다.

문제 보고 · 손을 앞으로 밀어도 조종간이 거의 안 움직이거나 입력이 제대로 나오지 않는다

13:20 커밋의 주석은 이 두 보고를 함께 적고 "두 보고는 신호를 이중으로 늦추지 않는 것으로 화해시켰다"고 정리한다. 커밋에 남은 코드는 이미 고친 뒤의 모습이지만, 주석이 첫 시도를 설명해 준다. 처음 넣은 안정화에서는 튜브 검사가 날것의 손 위치가 아니라 목표 위치 자체가 한 프레임에 얼마나 움직일 수 있는지를 제한했다. 그 위에 평활까지 한 번 더 걸려 있었다. 손 추적 데이터는 매끄러운 연속값이 아니라 어느 정도 띄엄띄엄한 점프로 들어오는 경우가 많다. 그러니 정상적인 손 이동도 그 프레임당 제한에 계속 걸렸고, 결과적으로 "거의 안 움직이는" 증상이 났다.

고친 구조는 앞 절에서 본 그대로다. (1) 이상값 검사는 날것의 손 위치끼리 비교한다. (2) 목표는 매 프레임 바로 대입하고 별도의 프레임당 이동 제한은 두지 않는다. (3) 감도 배율은 거리만 줄인다. (4) 지연은 마지막 평활 한 곳에서만 생긴다. 주석은 이 프레임당 목표 제한을 "Removed"라고 분명히 적어 둔다. 지연을 만드는

장치가 두 개 겹치면 각각은 작아도 합치면 손을 못 따라온다는 것이 이 시행착오의 교훈이다. 그리고 이 감도 배율과 평활도 오후 실기 테스트에서 결국 지워진다(5.2절).

3. 조종간 캘리브레이션 — CalibrationManager

3.1 요청

조종간의 느낌을 다듬는 사이 이견호는 조종간의 위치 자체에 대한 요청을 했다.

요청 · 사람마다 팔 길이와 앉는 위치가 다르기 때문에 게임 시작 전에 사용자가 자신의 몸에 맞게 왼쪽/오른쪽 조종간 위치를 설정할 수 있어야 한다

조종간 위치는 `CreateJoystick` 에 $(\pm 0.22, 0.87, 0.18)$ 로 박혀 있었다. 10:06 커밋 주석을 보면 이 위치도 "디스플레이 3개 사이"로 옮기고, 이어서 "조금 위쪽으로" Y를 0.82에서 0.87로 올리는 식으로 한 사람의 체형에 맞춰 손으로 조정해 온 값이다. 다른 사람이 앉으면 이 숫자가 맞을 리 없다. 그래서 게임 안에서 각자 조종간을 옮겨 놓을 수 있게 하자는 것이다.

Claude가 만든 `CalibrationManager` 는 `MobileSuitRoot` 에 붙는다. 조종간 두 개와 손 추적기 두 개를 참조한다. 핵심 원칙은 `JoystickLever` 의 내부(Pivot, Handle, 잡기 계산)를 전혀 건드리지 않는 것이다. 대신 조종간 전체의 뿌리인 `Mount`(`JoystickLever` 컴포넌트가 붙은 `GameObject` 자체)를 옮긴다. 잡혔는지는 `JoystickLever` 의 공개 속성 `isGrabbed` 를 읽기만 해서 판단한다. 조종간을 특별한 "캘리브레이션 모드"로 숨기거나 끄지 않는다. 씬이 시작될 때부터 평소처럼 보이고 평소처럼 잡힌다. 그 상태에서 잡고 있는 동안에만 이 스크립트가 `Mount` 전체를 손을 따라 끌고 다닌다. 그러는 동안 `JoystickLever` 의 $\pm 8\text{cm}$ Handle 추적은 아래에서 그대로 돌아간다. Handle 위치는 언제나 그 프레임의 Pivot 기준으로 다시 계산되므로, `Mount`가 움직여도 서로 싸우지 않고 함께 실려 간다. 오른쪽 조종간은 오른손으로만, 왼쪽은 왼손으로만 옮긴다는 양손 분리도 그대로 따른다.

3.2 첫 설계와 그 결과 — 커밋에 남지 않은 두세 번의 실패

13:20 커밋에 들어간 `CalibrationManager.cs` 는 이미 여러 번 고쳐진 뒤의 모습이다. 첫 설계는 파일 안의 "REDESIGNED", "FIXED" 주석과 `GundamCockpitSetup` 의 주석을 통해서만 복원할 수 있다. 정리하면 첫 버전은 이랬다.

부분	첫 설계	무엇이 문제였나
저장	캘리브레이션 결과를 <code>PlayerPrefs</code> 에 저장하고, 다음 실행 때 저장 값이 있으면 안내 과정 없이 바로 <code>Complete</code> 로 건너뛴	한 번 잘못 저장된 낮은 위치가 매번 다시 불러와졌고, 게임 안에서 다시 할 방법이 없었음
안내 표시	XR 카메라에 붙인 머리 고정 Canvas(불투명, 시야 정중앙)	고개를 숙여 조종간을 보면 안내판이 그대로 따라와 조종간을 가림
잡기	매 프레임 손바닥 절대 좌표를 그대로 <code>Mount</code> 위치로 사용	잡는 순간 큰 점프, 이어서 안전 제한에 걸려 그 자리에 고정됨

3.3 "조종간 자체가 화면에 보이지 않는다"

캘리브레이션을 넣은 직후 이견호의 보고다.

문제 보고 · LeftJoystick/RightJoystick 조종간 자체가 화면에 보이지 않는다... CalibrationManager를 추가한
이후부터 발생

원인은 안내판이었다. 첫 버전의 안내 Canvas는 XR 리그의 카메라 밑에 자식으로 붙어 있었다. 불투명한 판이 시야 정중앙에 떠서 시선을 따라다녔다. 조종간을 잡으려고 아래를 내려다보면 판도 같이 내려와서 조종간을 그대로 덮었다. 조종간 자체에는 아무 문제가 없었던 셈이다. 수정은 안내판을 카메라가 아니라 콧속 내부(`interior`)에 고정하는 것이었다. 이렇게 하면 카메라를 찾는 코드도 필요 없어진다.

3.4 "새로 이상한 디스플레이 만든거 지우고"

두 번째 안내판은 콧속에 고정은 됐지만, 기존 디스플레이를 만드는 `CreatePolarScreen` 으로 프레임과 화면이 있는 물리적인 판을 하나 더 세운 형태였다. 가림 문제는 사라졌지만, 이견호에게는 갑자기 콧속에 생긴 낯선 콘솔 화면으로 보였다. 이 요청은 다음 절의 긴 요청 앞부분에 함께 들어 있다. 세 번째 버전인 `BuildCalibrationPromptUI` 는 물리적 소품을 전부 없앴다. 배경 이미지도, 프레임도, 화면 큐브도 없이 글자만 떠 있다.

```
canvasGo.transform.SetParent(interior, false);
canvasGo.transform.localPosition = new Vector3(0f, 1.25f, 0.32f);
canvasGo.transform.localScale = Vector3.one * 0.0009f;
canvasRect.sizeDelta = new Vector2(560, 160);
Text prompt = CreateUIText("PromptText", canvasGo.transform, Vector2.zero,
    new Vector2(540, 140), 26, TextAnchor.MiddleCenter,
    new Color(0.85f, 0.95f, 1f), "COCKPIT CALIBRATION");
Outline outline = prompt.gameObject.AddComponent<Outline>();
outline.effectColor = new Color(0f, 0f, 0f, 0.85f);
```

위치 `(0, 1.25, 0.32)` 는 조종간 높이($y=0.87$)와 디스플레이 묶음 높이($y=0.92$)보다 위다. 앞뒤로는 조종간 ($z=0.18$)과 디스플레이(약 0.48) 사이에 놓인다. 배경이 없으니 글자 가독성은 검은색 `Outline` (반투명도 0.85 , 거리 $2, -2$ 픽셀)으로 확보했다. 회전은 기존 디스플레이들과 같은 "회전 없음" 규칙을 써서 조종사 쪽을 향한다.

3.5 "조종기 위치가 아래로 내려갔잖아" — 저장 기능 제거

이 무렵 이견호가 남긴 요청은 길고, 이 기능이 어떻게 동작해야 하는지를 분명히 정의한다.

문제 보고 · 새로 이상한 디스플레이 만든거 지우고 조종기 위치가 아래로 내려갔잖아... 앱을 시작하면 조종기 위치를 어디에 둘건지 물어보고 내가 둔곳에 고정되 되고 시작하는 거야 그래야 팔길이 상관없이 조종하지

"아래로 내려갔다"의 원인은 `PlayerPrefs`였다. 끌어서 놓는 방식이 아직 새로 생겼을 때 한 번 한 캘리브레이션이 의도보다 낮은 위치를 저장했다. 그 뒤로는 앱을 켤 때마다 저장값이 조용히 불러와졌고, 안내 과정은 건너뛰어졌다. 그러니 조종간은 늘 낮은 위치에 박혀 있었고 게임 안에서 다시 할 방법도 없었다.

Claude는 저장 기능을 아예 없앴다. 디스크에 아무것도 쓰지 않으니 오래된 값이 남아 문제를 일으킬 여지 자체가 없다. 앱을 켤 때마다 예외 없이 같은 안내 과정이 처음부터 돈다. 이견호의 말대로 "내가 둔곳에 고정되고 시작"하는 것이다. `CreateJoystick`이 정한 원래 위치는 이제 출발점일 뿐이고, 매번 캘리브레이션이 덮어쓴다. 원래 위치는 저장용이 아니라 안전 제한의 기준점으로만 `Start()`에서 기억해 둔다.

```
void Start()
{
    if (leftStick != null) _leftDefaultLocalPos = leftStick.transform.localPosition;
    if (rightStick != null) _rightDefaultLocalPos = rightStick.transform.localPosition;
    // 저장값을 불러오거나 건너뛰지 않는다 - 항상 처음부터
    CurrentState = State.WaitingForLeftGrab;
    UpdatePromptText();
}
```

바로 이어서 흐름을 더 구체적으로 정한 요청이 들어왔다.

요청 · 일단 게임을 시작하면 조종기에 위치를 설정하겠다고 하고 내가 조종기 위치를 손으로 잡어서 옮길거야 그리고 확인을 누르면 게임이 시작되는거고

그전까지는 오른쪽 조종간을 놓는 순간 캘리브레이션이 저절로 끝났다. 이제 두 개를 다 놓은 뒤 `WaitingForConfirm` 단계가 생기고, "확인" 버튼을 눌러야 끝난다. 버튼은 `BuildCalibrationConfirmButton`이 만든다. 두 조종간 사이, 약간 위 앞쪽인 `(0, 1.0, 0.28)`에 지름 0.07m 초록 구(`buttonGreenMat`, 조종간 버튼과 같은 재질)를 놓는다. 그 0.09m 위에는 "확인"이라는 글자(크기 30, 0.0007 배율 Canvas)를 띄운다. 구와 글자를 하나의 루트 `GameObject` 아래에 묶어 `SetActive`한 번으로 함께 켜고 끌 수 있게 했다. 처음에는 꺼진 상태로 만들어진다.

XRI 인터랙터를 새로 붙이지 않은 것도 의도한 선택이다. 버튼 판정은 `HandJointTracker`가 이미 공개하고 있는 값만으로 하겠다는 것이었고, 손 추적 코드는 건드리지 않는다.

3.6 "바닥쪽으로 내려가서 안올라옴" — 절대 좌표에서 상대 이동으로

확인 버튼까지 붙인 뒤 기기에서 해 본 결과다.

문제 보고 · 위치를 바꾸려고 조종간을 잡는순간 바닥쪽으로 내려가서 안올라옴 옮겨지지도 않고 그리고 확인 버튼도 안눌림

2.2절에서 조종간 Handle이 겪은 것과 똑같은 종류의 버그가 Mount 단위에서 다시 나타난 것이다. 첫 버전의 `DragMountToHand`는 매 프레임 "지금 손바닥이 있는 곳에 Pivot을 정확히 놓는다"는 식으로 Mount를 손바닥 절대 좌표에 놓았다. 잡은 순간의 손과 조종간 사이 상대 위치는 기억하지 않았다. 손바닥 관절 좌표는 실제로 쥔 지점과 어긋나 있고, 그 어긋남이 잡는 순간 큰 점프로 나타났다. 점프가 크다 보니

`maxOffsetFromDefault = 0.6 m` 안전 제한에 걸렸다. 그 뒤로는 프레임마다 다시 계산한 위치가 여전히 제한을 비슷한 만큼 넘어서, 조종간은 매번 같은 제한 지점에 묶여 꼼짝하지 않는 것처럼 보였다. 확인 버튼도 같은 이유로 실패했다. 손바닥 절대 좌표를 버튼의 고정된 월드 좌표와 직접 비교했으니, 같은 오차 때문에 손이 아무리 다가가도 "충분히 가깝다"는 판정이 나지 않았다.

Claude가 주석에 적었듯이, `JoystickLever` 는 바로 이 함정을 피하려고 손의 **변화량**만 쓰도록 이미 고쳐져 있었다. 캘리브레이션은 같은 `PalmPosition` 을 읽으면서도 그 교훈을 적용하지 않았던 것이다. 수정은 `JoystickLever` 의 오프셋 방식을 그대로 옮겨 오는 것이었다.

```
Vector3 handLocalNow = mountParent.InverseTransformPoint(hand.PalmPosition);
if (justGrabbed)
{
    _grabStartHandLocal = handLocalNow;           // 잡은 순간 손 위치
    _grabStartMountLocal = mount.localPosition;   // 잡은 순간 Mount 위치
    return;                                       // 이 프레임엔 움직이지 않음
}
Vector3 handMovement = handLocalNow - _grabStartHandLocal;
Vector3 newMountLocalPos = _grabStartMountLocal + handMovement;
// 기본 위치에서 0.6m 넘게 벗어나지 않게 안전 제한
Vector3 offsetFromDefault = newMountLocalPos - defaultLocalPos;
if (offsetFromDefault.magnitude > maxOffsetFromDefault)
    newMountLocalPos = defaultLocalPos + offsetFromDefault.normalized * maxOffsetFromDefault;
mount.localPosition = newMountLocalPos;
```

좌표는 Mount의 부모(콧핏 내부) 기준으로 잡는다. 콧핏 내부와 Mount는 모두 회전이 없는 상태로 만들어지므로 회전 계산 없이 그대로 빼고 더하면 된다는 것도 주석에 확인해 두었다. 추적이 한 프레임 끊기면(`!hand.IsTracked`) 그 프레임은 아무것도 하지 않고 Mount를 마지막 위치에 둔다. "잡기 시작"은 `isGrabbed` 가 직전 프레임에 거짓이었다가 이번 프레임에 참이 된 순간으로 판단한다. 한 번에 한 조종간만 끌리므로 시작 기준점 필드는 한 쌍만 공유해도 된다.

확인 버튼은 위치 판정을 완전히 버렸다. 대신 `JoystickLever` 의 잡기 판정이 이미 잘 쓰고 있는 손 모양 신호를 그대로 쓴다. 확인 단계에서 어느 손이든 주먹을 쥐거나(`GripAmount ≥ 0.55`) 꼬집으면(`PinchAmount ≥ 0.5`) 확인으로 본다. 버튼 구는 "여기로 손을 뺄라"는 시각적 표시로만 남는다.

```
bool IsHandPressingConfirm(HandJointTracker hand)
{
    if (hand == null || !hand.IsTracked) return false;
    return hand.GripAmount >= confirmGripThreshold
        || hand.PinchAmount >= confirmPinchThreshold;
}
```

버튼이 연결되지 않은 경우(`confirmButton == null`)에는 확인 단계를 바로 통과시킨다. 없는 버튼을 기다리다 캘리브레이션이 영원히 멈추는 일을 막기 위해서다. 또 `CalibrationManager` 는 `ShipMovementController` 나 `WeaponAimFireController` 를 전혀 참조하지 않는다. 캘리브레이션 중에도 이동이나 사격을 막지 않는다. `CurrentState` 와 `IsCalibrationComplete` 는 나중에 다른 스크립트가 반응할 수 있도록 공개만 해 두었다.

3.7 13:20 시점의 흐름

13:20 커밋의 상태 머신은 네 상태다. `WaitingForLeftGrab` → (왼쪽을 잡아 옮기고 놓음) → `WaitingForRightGrab` → (오른쪽을 잡아 옮기고 놓음) → `WaitingForConfirm` → (손을 쥐거나 꼬집음) → `Complete`. 안내 문구는 단계마다 바뀐다.

상태	안내 문구(줄바꿈은 / 로 표시)
<code>WaitingForLeftGrab</code>	COCKPIT CALIBRATION / 지금부터 조종간 위치를 설정합니다 / 왼손으로 왼쪽 조종간을 잡아 / 원하는 위치로 옮긴 뒤 놓으세요
<code>WaitingForRightGrab</code>	COCKPIT CALIBRATION / 오른손으로 오른쪽 조종간을 잡아 / 원하는 위치로 옮긴 뒤 놓으세요
<code>WaitingForConfirm</code>	COCKPIT CALIBRATION / 위치가 마음에 드시면 / 앞의 확인 버튼을 눌러 시작하세요
<code>Complete</code>	안내와 버튼 모두 숨김

3.8 오후: "한쪽씩 설정할때마다 물어봐야함"

`CalibrationManager.cs` 파일 시각이 14:20 무렵인 것은 이 변경 때문으로 보인다. 13:20 커밋에 이미 들어 있는 파일이 다시 쓰였다는 뜻이다. 이견호의 요청은 짧다.

요청 · 한쪽씩 설정할때마다 물어봐야함 됐냐고 위치

두 조종간을 다 놓은 뒤 한 번에 확인받던 방식을 조종간 하나마다 따로 확인받는 방식으로 바꿨다. 상태가 다섯 개가 됐다. `WaitingForConfirm` 은 `WaitingForLeftConfirm` 과 `WaitingForRightConfirm` 둘로 나뉘었다.

```
public enum State
{
    WaitingForLeftGrab,
    WaitingForLeftConfirm,    // 새로 추가
    WaitingForRightGrab,
    WaitingForRightConfirm,   // WaitingForConfirm을 대체
    Complete
}
...
case State.WaitingForLeftConfirm: UpdateWaitingForConfirm(State.WaitingForRightGrab); break;
case State.WaitingForRightConfirm: UpdateWaitingForConfirm(State.Complete); break;
```

놓았을 때의 분기는 `CurrentState = isLeft ? State.WaitingForLeftConfirm : State.WaitingForRightConfirm;` 한 줄로 줄었다. 확인 함수는 다음 상태를 인자로 받는 형태 (`UpdateWaitingForConfirm(State nextState)`)로 일반화됐다. 확인 뒤에는 바로 숨기지 않고 `UpdatePromptText()` 를 불러 다음 단계 문구를 보여 준다. 확인 버튼은 확인 단계에서만 켜지고, 잡기 단계로 돌아가면 다시 꺼진다. 안내 문구도 바뀌었다. 확인 방법이 이제 "버튼을 누르는" 것이 아니라 손 모양이라는 점이 문구에 드러난다.

상태	안내 문구
----	-------

WaitingForLeftConfirm

COCKPIT CALIBRATION / 왼쪽 조종간 위치가 괜찮으면 / 손을 쥐거나 꼬집어 확인하세요

WaitingForRightConfirm

COCKPIT CALIBRATION / 오른쪽 조종간 위치가 괜찮으면 / 손을 쥐거나 꼬집어 확인하세요

확인할 점

확인 판정이 "어느 손이든 쥐거나 꼬집기"이므로, 조종간을 놓은 직후 손이 아직 쥔 모양이면 다음 프레임에 곧바로 확인으로 넘어갈 수 있다. 조종간을 놓았다는 것 자체가 JoystickLever 의 놓기 문턱(쥐기 0.35, 꼬집기 0.3) 아래로 손이 펴졌다는 뜻이긴 하다. 하지만 반대 손은 그 조건에 묶여 있지 않다. 이 시점 코드에 이를 따로 막는 장치는 없다.

4. 콕핏 안에서 내 건담은 보이지 않게

13:20 커밋에는 조종간과 상관없는 변경이 하나 더 있다. 건담 머리에 붙은 카메라(HeadCam)의 컬링 마스크다. 배경부터 정리하면 이렇다. 외부 건담 모델(ExternalGundam)은 플레이어의 기체와 위치가 동기화되어 우주 공간에 서 있다. 모델의 Head 뼈에 붙은 카메라가 렌더 텍스처와 360도 큐브맵을 채운다. 그 영상이 콕핏 돔과 디스플레이에 "건담의 눈으로 본 바깥"으로 나온다. 플레이어 자신의 XR 카메라에서는 이 건담 몸체가 보이면 안 되므로, 몸체 전체를 레이어 30(GundamBodyLayer)에 두고 XR 카메라의 cullingMask 에서 이 레이어를 빼 두었다. 레이어 30을 고른 것은 Unity가 기본으로 아무것도 할당하지 않는 레이어라서 이름 없이도 충돌할 가능성이 낮기 때문이다.

그런데 머리 카메라 쪽은 바로 직전 요청 때문에 이 레이어를 일부러 보이게 해 두었다. 10:06 커밋의 주석에 남은 그 요청은

요청 · 건담에 머리는 안보이는데 몸통은 보였으면 좋겠어

였다. 머리 카메라는 얼굴 앞면 바로 앞에 앞을 보고 있으므로 머리는 렌즈 옆이나 뒤에 있어 안 보이고 몸통과 어깨만 화면 아래쪽에 보일 것이라는 위치 계산에 기댄 처리였다. Claude는 주석에 "직접 눈으로 확인할 수 없는 가정"이라고 적어 두었다.

이건호는 이를 뒤집었다.

요청 · 그냥 콕핏안에서는 내건담에 모습이 보이면안됨

그래서 머리 카메라도 메인 카메라와 똑같이 레이어 30을 뺀다. 18줄짜리 설명 주석이 지워지고 한 줄이 들어갔다.

```
headCam.cullingMask &= ~(1 << GundamBodyLayer);
```

이제 자기 건담은 직접 보는 시야에서도, 머리 카메라 영상에서도, 그 영상으로 만드는 360도 스카이박스과 콕핏 돔 배경에서도 전혀 보이지 않는다. 다만 PlaceExternalGundam 안의 SetLayerRecursively 위 주석에는 아직 "머리 카메라는 일부러 이 레이어를 빼지 않는다"는 옛 설명이 남아 있다. 코드는 바뀌었고 주석 하나가 미처 따라오지 못한 것이다.

같은 커밋의 `Packages/manifest.json`에는 `"com.unity.ai.assistant": "2.19.0-pre.2"` 한 줄이 추가됐다. Unity 에디터의 AI 어시스턴트 패키지이며, 게임 코드에서 이 패키지를 쓰는 곳은 없다.

5. 오후: 실제 기기에서 다시 잡아 본 조종간

13:20 이후 `JoystickLever.cs`는 세 번 더 크게 바뀐다. 이 변경들은 9월 28일 09:31 커밋에 한꺼번에 들어 있지만, 주석에 남은 보고 세 개가 순서대로 쌓여 있어 흐름을 복원할 수 있다. 오전의 수정들이 실제로 기체를 몰면서 쓸 때는 충분하지 않았다는 것이 오후의 주제다.

5.1 움직이는 중에는 조종간이 손을 못 따라온다 — 실행 순서

문제 보고 · 조종간을 잡고 이동할때 조종간이 내손따라서 제대로 안움직임... 특정방향으로는 안밀림... 로봇이 맘대로 다른데로 가기도함... 움직일때는 오른손 조종장치가 안작동함

핵심은 "이동할때"라는 조건이다. 기체가 서 있을 때는 멀쩡하고 움직일 때만 이상했다. Claude가 찾은 원인은 Unity의 스크립트 실행 순서였다. `JoystickLever`는 손 위치를 Pivot 기준으로 바꿔서 기체의 이동을 상쇄한다(2.2절). 이 상쇄가 맞으려면 손 추적값과 Pivot의 Transform이 같은 시점의 `MobileSuitRoot` 위치를 반영해야 한다. 손 추적 데이터는 XR 서브시스템을 통해 프레임 초반에 들어온다. 반면 일반 `MonoBehaviour.Update()`들의 실행 순서는 따로 정하지 않으면 보장되지 않는다. 어떤 프레임에 같은 루트에 붙은 `ShipMovementController.Update()`가 먼저 돌면, `JoystickLever`가 Pivot을 읽을 때는 기체가 이미 이번 프레임만큼 이동한 뒤다. 손바닥 위치는 아직 이동 전 기준이다. 그 차이가 정확히 기체가 한 프레임에 이동한 거리만큼 손 움직임으로 잘못 읽힌다. 기체가 빨리 갈수록 커지는 가짜 입력이 생기고, 그 입력이 다시 기체를 움직이는 되먹임이 생긴다. "로봇이 맘대로 다른데로" 가는 증상과 맞아떨어진다.

수정은 클래스에 속성 한 줄을 다는 것이다.

```
[DefaultExecutionOrder(-100)]
public class JoystickLever : MonoBehaviour
```

실행 순서 -100은 기본값(0)인 `ShipMovementController`보다 항상 먼저 돈다는 뜻이다. 그러면 두 조종간이 Pivot을 읽는 시점에 기체는 아직 이번 프레임에 움직이지 않았고, 손 추적값도 같은 "아직 안 움직인" 상태를 반영한다. 컴포넌트 추가 순서 같은 우연에 기대던 상쇄가 이제 매 프레임 보장된다.

5.2 Galaxy XR 테스트 — 1:1 재작성과 회전 잠금

이어진 실기 테스트 보고는 길고 항목별 요구까지 달려 있었다.

문제 보고 · 왼손으로 LeftJoystick을 잡고 밀면 건담 이동은 정상... 하지만 LeftJoystick 자체가 손을 따라오지 않는다... 오른손으로 RightJoystick을 잡으려고 하면 잘 안 잡힌다... 오른쪽 조종간을 잡아도 손을 따라가는 것이 아니라 갑자기 크게 꺾이거나 이상한 방향으로 회전한다

주석에 따르면 이 보고에는 A부터 H까지 항목이 붙어 있었다. A는 잡을 때 스냅 금지, B는 1:1 추적과 불필요한 평활 금지, C는 회전 금지, D는 축 대응, E는 양손 분리, F는 감사, H는 이동·무기 스크립트를 건드리지 말 것이다. F 항목은 원문이 인용돼 있다.

요청 · 이 세 시스템이 동시에 조종간 Transform을 움직이고 있는지 반드시 확인해라

. 세 시스템이란 `JoystickLever`, `HandExclusiveGrabAdapter`, XRI 인터랙터블이다.

감사(F). Claude는 `HandExclusiveGrabAdapter.cs` 와 `GundamCockpitSetup` 의 `AttachHandInteractable / WireJoystickHandInteractors` 를 다시 읽었다. 어댑터는 `NotifyXRIGrabbed` 만 호출하고 위치나 회전은 어디서도 읽거나 쓰지 않는다. `XRSimpleInteractable` 은 `XRGrabInteractable` 과 달리 물체를 붙이거나 움직이는 동작이 없다. 폴더 안 모든 스크립트에서 `transform.position/rotation/localPosition/localRotation` 대입을 찾아보니 Handle을 움직이는 곳은 `JoystickLever` 한 곳뿐이었다. 즉 프로젝트 코드 안에서는 움직이는 주체가 하나뿐이다. 그런데도 오른쪽 조종간이 눈에 띄게 꺾였으니, 원인은 이 코드 밖(예를 들어 인터랙터 쪽 인스펙터 설정)에 있을 수 있다. Claude는 그 원인을 특정하지 못했다고 솔직하게 적고, 원인과 상관없이 증상 자체를 막는 장치를 넣었다.

회전 잠금(C). 처음 프레임에 Handle의 회전, Pivot의 위치와 회전을 기억해 둔다. 그리고 `LateUpdate()` 에서 잡고 있든 아니든 매 프레임 무조건 그 값으로 되돌린다. `LateUpdate` 는 렌더링 전 마지막 갱신이므로, 프레임 중 무엇이 건드렸든 마지막 말은 이 코드가 한다.

```
void LateUpdate()
{
    if (!_centerCaptured) return;
    if (handle != null) handle.localRotation = _centerLocalRotation;
    if (pivot != null)
    {
        pivot.localPosition = _pivotDefaultLocalPos;
        pivot.localRotation = _pivotDefaultLocalRotation;
    }
}
```

이를 위해 `CaptureCenterIfNeeded()` 가 `_centerLocalRotation`, `_pivotDefaultLocalPos`, `_pivotDefaultLocalRotation` 도 함께 기억하도록 확장됐고, Pivot이 없으면 기억하지 않도록 null 검사도 추가됐다. 헤더와 툴팁도 "Base는 움직이지도 돌지도 않는다. 매 프레임 기본값으로 붙잡아 둔다"로 바뀌었다.

1:1 추적(B). 오전에 "늦고 민감하다"는 보고 때문에 넣은 감도 배율 0.65와 평활 속도 25가 이번에는 "손을 따라오지 않는다"의 원인이 됐다. 배율이 1보다 작으면 손이 10cm 갈 때 조종간은 6.5cm만 간다. 잡은 지점이 손에서 점점 멀어진다. 평활은 그 위에 지연을 더한다. 둘 다 "손보다 뒤처지는" 느낌의 실제 원인이었고, 이번 보고가 명시적으로 1:1과 최소 평활을 요구했으므로 Claude는 둘 다 지웠다.

`positionSensitivity`, `smoothingSpeed` 필드와 `_trackedTargetLocal` 목표 변수가 모두 사라졌다. 계산 결과를 Handle 위치에 바로 대입한다.

```
// 9/23 오후 - 감도 배율 없음, 평활 없음, 범위 제한만 하고 바로 대입
else if (IsActiveHandTrackingValid())
{
    Vector3 rawHandFrameDelta = handLocalNow - _lastRawHandLocal;
    rawHandFrameDelta.y = 0f;
    Vector3 handMovement = handLocalNow - _grabStartHandLocal;
    handMovement.y = 0f;    // D: Y는 입력에 쓰지 않음
    if (rawHandFrameDelta.magnitude > maxHandDeltaPerFrame)
        handMovement = _lastValidHandMovement;
    else
        _lastValidHandMovement = handMovement;
    _lastRawHandLocal = handLocalNow;
    handle.localPosition = ClampToRange(_grabStartJoystickLocal + handMovement);
}
// 추적이 끊긴 프레임: Handle을 건드리지 않고 마지막 위치 유지
```

남은 보호 장치는 프레임 간 0.15m 이상값 검사 하나다. 헤더 이름도 "Grab tracking safety (position only - not smoothing)"으로 바꿔 이것이 평활이 아님을 분명히 했다. 놓았을 때 중심으로 돌아가는 `returnLerpSpeed = 6`의 보간만 그대로 남았다. 놓은 뒤의 연출일 뿐 잡고 있을 때의 추적이 아니기 때문이다. A(스냅 금지)는 오전에 만든 오프셋 방식이 이미 맞았으므로 그대로 두었다. D와 E도 이미 요구대로였다. 손의 x, z가 Handle의 x, z로 가고, Y는 입력에 쓰이지 않으며, 양손 분리는 앞서 본 층들이 맞는다. 오전 커밋의 긴 "Input stabilization" 주석은 지워지고, 그 자리를 이 Galaxy XR 테스트의 수정 기록이 차지했다.

5.3 재테스트 — 오전의 보강이 원인이었다

1:1로 다시 쓴 뒤에도 실기 결과는 좋지 않았다.

문제 보고 · 조종이 제대로 안되고 내가 지정한 위치에 조종장치가 고정되지도 않음... 조종장치에 움직임이랑 건담에 움직임이랑 다름... 조종장치가 앞으로는 밀리지도 않음

Claude는 원인을 2.3절에서 넣은 보강에서 찾았다. XRI로 잡힌 경우 `GetActiveHandWorldPosition()`은 손바닥 위치가 아니라 `_grabbedInteractor.position`, 즉 XRI 샘플 리그의 `Near-Far Interactor` GameObject 위치를 읽고 있었다. 이 Transform이 실제 손과 1:1로 움직인다는 가정은 한 번도 검증된 적이 없었다. 프로젝트에서는 샘플 리그가 내부적으로 그 Transform을 어떻게 움직이는지 실행해 볼 수 없고 C# 코드만 읽을 수 있다. 예를 들어 리그가 원거리 레이 조준을 위해 그 Transform의 방향만 바꾸고 가까이서 주먹을 쥔 때는 위치를 손만큼 옮기지 않는다면, 보고된 증상이 모두 설명된다. 조종간이 손 위치에 고정되지 않고, 손 움직임과 조종간 움직임이 다르고, 그 Transform이 거의 움직이지 않는 축(앞으로 밀기)은 아예 반응이 없다.

게다가 이 보강은 이견호가 이 조종간에 대해 처음 낸 보고에서 정한 구조와도 어긋났다. 주석에 인용된 그 원칙은

요청 · XRI = 잡았는지/놓았는지 이벤트만 담당, JoystickLever = 실제 조종간 위치 계산 담당

이다. 반면 손바닥 위치(`PalmPosition`)는 이미 손 메시 표시, 쥐기·꼬집기 판정, 캘리브레이션 끝기에서 제대로 동작한다는 것이 확인된 값이었다.

그래서 `_grabbedInteractor` 필드를 지우고, 두 함수는 무조건 `_activeHand` (왼손 또는 오른손 `HandJointTracker`)만 읽도록 되돌렸다.

```
public void NotifyXRIGrabbed(bool grabbed, Transform interactorTransform = null)
{
    _xriGrabbed = grabbed;    // interactorTransform은 받기만 하고 저장하지 않음
}
Vector3 GetActiveHandWorldPosition()
{
    return _activeHand != null ? _activeHand.PalmPosition : Vector3.zero;
}
bool IsActiveHandTrackingValid()
{
    return _activeHand != null && _activeHand.IsTracked;
}
```

`NotifyXRIGrabbed`의 두 번째 인자는 어댑터 쪽 호출 코드와 맞추기 위해 남겨 두었지만 어디에도 저장하거나 읽지 않는다. 잡았는지 판단하는 데는 여전히 XRI 신호를 먼저 쓴다. 바뀐 것은 이동량을 어디서 가져오느냐 하나뿐이다. 오전의 "안전을 위한" 보강이 오후 버그의 원인이 되어 몇 시간 만에 사라진 셈이다. 검증하지 않은 데이터 소스를 "더 직접적"이라는 이유로 끌어오면 안 된다는 것이 이 일의 교훈이다.

5.4 "RightJoystick은 잘 안 잡히고" — 정확히 같은 객체에서 그 아래 어디든으로

Galaxy XR 보고에 들어 있던 "오른손으로 RightJoystick을 잡으려고 하면 잘 안 잡힌다"는 증상은 `HandExclusiveGrabAdapter`에서 고쳤다. 파일 시각 16:15 무렵이 이 수정이다. 주석에 인용된 보고는

문제 보고 · RightJoystick은 잘 안 잡히고

이다.

어댑터는 선택한 인터랙터의 Transform이 `onlyAllowedInteractor`와 **정확히 같은 객체**인지 비교하고 있었다. 이 객체는 썬 생성 때 `FindDeepChild`로 찾은 "Near-Far Interactor" `GameObject`다. 그런데 XRI가 실제로 선택 이벤트를 그 객체 자신이 아니라 그 아래 자식(근거리/원거리 하위 인터랙터 같은 것)에서 보낸다면, 올바른 오른손으로 잡아도 "다른 인터랙터"로 판정된다. 그러면

`CancelInteractableSelection`으로 강제 취소된다. 이견호에게는 그것이 "잘 안 잡힘"으로 느껴졌을 것이다. 이번에도 Claude는 샘플 프리팹이 실행 중에 선택을 어떻게 보내는지 직접 확인할 수 없다는 점을 주석에 적었다.

```
// 이전: 정확히 같은 Transform만 허용
if (onlyAllowedInteractor != null && args.interactorObject.transform != onlyAllowedInteractor)
// 이후: 허용된 인터랙터 자신이거나 그 하위 객체면 허용
if (onlyAllowedInteractor != null &&
    !args.interactorObject.transform.IsChildOf(onlyAllowedInteractor))
```

`Transform.IsChildOf` 는 자기 자신에 대해서도 참을 돌려준다. 그래서 이전에 통과하던 경우는 그대로 통과하고, 그 아래 어디서 온 선택도 받아들인다. 그래도 양손 분리는 깨지지 않는다. 왼손 인터랙터는 "Left Hand" 아래에, 오른손 인터랙터는 "Right Hand" 아래에 완전히 따로 있으므로 한쪽이 다른 쪽의 검사를 통과할 길이 없다.

6. 헤드 발칸 — 바라보는 곳을 쏜다

6.1 요청과 기존 포탑의 보존

조종간이 어느 정도 자리를 잡자 무기 이야기가 나왔다. 그때까지 오른쪽 조종간은 `WeaponAimFireController` 로 콧 앞의 달린 별도의 포탑(`GunTurret` → `GunPivot` → `MuzzlePoint`, 기체 기준 $(0, 1.0, 2.4)$)을 조종했다. 조종간을 85% 이상 앞으로 밀면 발사했고, 무기 목록은 "MACHINE GUN"(30발, 0.15초)과 "BEAM RIFLE"(6발, 0.6초)이었다. 이견호의 새 요청은 조준 방식을 근본적으로 바꾸는 것이었다.

요청 · 오른손은 조준 장치를 잡아서 조준하는 것이 아니라, '내가 바라보는 방향'으로 건담 머리의 헤드발칸을 조준하고, 오른손 엄지를 구부리는 동작으로 발사한다.

요청에는 조건이 몇 개 더 붙어 있었고 `HeadVulcanController` 주석에 그대로 남아 있다. 첫째는

요청 · 삭제하지 말고 기존 코드는 보존한다... 이번에는 Head Vulcan 조작에서 사용하지 않게 분리한다

, 둘째는

요청 · 새로운 MediaPipe나 별도 손 추적 시스템을 추가하지 않는다

이다.

Claude는 포탑 쪽 연결 코드를 읽고 분리할 것이 처음부터 없다는 것을 확인했다.

`WeaponAimFireController` 는 콧 앞의 포탑을 움직일 뿐 건담의 머리와는 연결된 적이 없었다. 그래서 포탑, `WeaponAimFireController`, 오른쪽 조종간 연결은 하나도 지우지 않고 그대로 두었다. 새 `HeadVulcanController` 에는 `JoystickLever`, `RightJoystick`, `WeaponAimFireController` 에 대한 참조가 한 줄도 없다.

6.2 총구 두 개를 머리에 붙이기

총구는 `GundamCockpitSetup` 에서 만든다. 머리 카메라를 붙일 때 찾았던 같은 `Head` 뼈를 `FindDeepChild` 로 다시 찾는다. 위치는 메시의 실제 렌더링 경계(`SkinnedMeshRenderer.bounds`)로 계산한다. 높이는 머리 카메라와 같은 "전체 높이의 92%" 눈높이 추정치를 쓴다. 앞쪽으로는 높이의 5%만큼 내밀고, 좌우로는 메시 폭의 28%만큼 벌린다.


```
float vulcanEyeHeight = vulcanBounds.min.y + vulcanBounds.size.y * 0.92f;
float vulcanForwardClearance = vulcanBounds.size.y * 0.05f;
float vulcanSideOffset = vulcanBounds.size.x * 0.28f;
GameObject muzzleL = new GameObject("HeadVulcanMuzzle_L");
muzzleL.transform.position = new Vector3(vulcanBounds.center.x - vulcanSideOffset,
    vulcanEyeHeight, vulcanBounds.center.z + vulcanForwardClearance);
muzzleL.transform.SetParent(vulcanHead, true); // 월드 위치 유지한 채 뼈에 붙임
```

먼저 월드 좌표에 놓은 뒤 `SetParent(head, true)` 로 붙이는 것은 머리 카메라에서 쓴 기법과 같다. 뼈의 로컬 스케일을 추측할 필요가 없어진다. 총구가 머리 뼈를 따라가므로 이진호가 고개를 돌려 건담 머리가 돌면 총구도 함께 돈다. 총구의 방향은 의미가 없다. 탄은 항상 카메라 정면으로 날아가기 때문이다.

6.3 엄지가 얼마나 굽었나 — 손 크기에 맞춰 스스로 맞춰지는 계산

"새 손 추적 시스템 금지" 조건에 따라 `HeadVulcanController` 는 오른손 추적기(`RightHandTracker`)에 이미 붙어 있는 `XRHandTrackingEvents` 컴포넌트를 그대로 쓴다. 그 컴포넌트의 `jointsUpdated` 이벤트에 리스너를 하나 더 다는 것이다(`SubscribeToRightHand`). `UnityEvent`는 리스너를 여러 개 받을 수 있으므로 `HandJointTracker`의 기존 리스너와 충돌하지 않는다. 왼손 데이터는 절대 읽지 않는다.

요청은 엄지 끝 좌표 하나의 거리만 보지 말고 엄지 관절들(`ThumbTip/ThumbDistal/ThumbProximal`)을 기준 관절과 함께 쓰라는 것이었다. 기준 관절로는 요청이 허용한

요청 · `IndexMetacarpal` 또는 `Palm`

가운데 `Palm`을 골랐다. `HandJointTracker`의 쥐기 계산이 이미 손바닥을 기준으로 쓰고 있기 때문이다. 계산은 이렇다.

```
float thumbReach; // 이 손의 엄지가 펴졌을 때 길이
if (gotProximal) thumbReach = Vector3.Distance(proximalPose.position, tipPose.position);
else if (gotDistal) thumbReach = Vector3.Distance(distalPose.position, tipPose.position) * 1.6f;
else thumbReach = 0.06f;
thumbReach = Mathf.Max(thumbReach, 0.02f);
...
float extendedDistance = baseToPalm + thumbReach; // 펴졌을 때 끝~손바닥
float curledDistance = baseToPalm * 0.35f; // 접었을 때 끝~손바닥
_thumbBendAmount = Mathf.Clamp01(Mathf.InverseLerp(extendedDistance, curledDistance,
    tipToPalm));
```

엄지 뿌리(`Proximal`)에서 끝까지의 거리를 "이 사람 엄지의 길이"로 삼는다. 그래서 손 크기가 달라도 같은 기준으로 굽힘 정도를 낸다. 고정된 월드 거리 하나를 모든 사람에게 쓰는 방식을 피한 것이다. `Proximal`이 없으면 `Distal`에서 끝까지의 거리에 1.6을 곱하고, 둘 다 없으면 0.06m를 쓴다. `baseToPalm`은 엄지 뿌리와 손바닥 사이 거리다. 엄지 끝이 손바닥에서 "뿌리 거리 + 엄지 길이"만큼 떨어져 있으면 굽힘 0, "뿌리 거리의 35%"까지 들어오면 굽힘 1이다. 엄지 끝이나 손바닥 관절을 못 얻은 프레임은 `_thumbTracked = false`로 두어 발사하지 않는다.

6.4 발사, 연사 간격, 임시 탄환

발사 조건은 `_thumbTracked` 이고, 굽힘이 `thumbBendThreshold = 0.6` 이상이며, 쿼다운이 끝났을 때다. 쿼다운은 `fireRate = 0.15` 초다. "직전 프레임에 굽혀 있었나"를 따지는 래치는 따로 두지 않았다. 엄지를 굽힌 채로 있으면 0.15초마다 계속 쏘고(폭주 연사는 없음), 폼다가 다시 굽히면 쿼다운만 끝났으면 바로 쏜다. 상태 변수가 적어 꼬일 일도 적다.

발사 방향은 요청대로 코드에 그대로 옮겼다.

```
Vector3 fireDirection = mainCamera.transform.forward;
Quaternion fireRotation = Quaternion.LookRotation(fireDirection);
if (alternateMuzzles && muzzleLeft != null && muzzleRight != null)
{
    Transform muzzle = (_muzzleToggle % 2 == 0) ? muzzleLeft : muzzleRight;
    _muzzleToggle++;
    SpawnBullet(muzzle, fireDirection, fireRotation);
}
```

`mainCamera` 는 XR 리그 안의 카메라, 즉 이진호의 머리(HMD)다. 탄은 건담 머리의 총구 위치에서 나오지만 방향은 이진호가 바라보는 방향이다. `alternateMuzzles = true` 이면 왼쪽과 오른쪽 총구가 번갈아 쏘고, 거짓이면 두 총구가 동시에 쏜다. 탄 속도는 `bulletSpeed = 80` , 수명은 `bulletLifetime = 3` 초다.

탄환 모델은 아직 없었다. 요청 자체가 이를 허용했다.

요청 · 실제 탄환 모델이 없다면 작은 Sphere 또는 Capsule을 임시 탄환으로 만들어도 된다. 중요한 것은 먼저 기능을 확인하는 것이다.

그래서 `projectilePrefab` 이 비어 있으면 지름 0.04m 구를 만들고, 충돌체를 제거하고, 중력 없는 `Rigidbody` 에 `linearVelocity` 를 준다. 충돌체를 뺀 이유는 탄이 건담이나 콧을 밀거나 자기 비행을 막지 않게 하기 위해서다. 그래서 이 첫 구현의 탄환은 아무것에도 맞지 않는다. "쏘는 것이 보이는지"를 먼저 확인하는 단계였다.

썬 생성 코드는 카메라나 오른손 이벤트 컴포넌트를 찾지 못하면 경고를 남긴다. `Head` 뼈가 없으면 총구 배치 자체를 건너뛴다. 모델 FBX가 아직 없는 환경에서도 썬 생성이 멈추지 않게 하려는 것이다.

7. 디스플레이 3개 — STATUS, 전술, WEAPON

7.1 그때까지의 세 화면

콧 정면 허벅지 위에는 "SYSTEM CHECK" 스타일의 디스플레이 묶음(`BuildSystemCheckDisplay`)이 있었다. 조종사 눈 위치를 중심으로 반지름 0.63m 호 위에 놓고, 높이 0.92, 위로 36도 기울어 있다. 가운데 `SysCheck_Center` (0.42×0.42m), 왼쪽 -40도 `SysCheck_Left` (0.2×0.24m), 오른쪽 +40도 `SysCheck_Right` 로 구성된다. 아래에는 빨간 강조 막대가 있고, 양옆 바깥에는 눈금 사다리가 서 있다. 가운데 화면에는 원형 눈금 다이얼과 "PILOT: ---", 포탑 무기 이름과 탄수가 표시됐다. 왼쪽 화면은 건담

머리 카메라 영상 전체, 오른쪽 화면은 "AUX-R"이라는 장식뿐이었다. 10:06 커밋 주석에 따르면 이전에 있던 물리 대시보드는 이미

요청 · 데시보드 역할이 아까 만든 디스플레이 3개가 하는거야

라는 요청으로 없어진 상태였다. 즉 이 세 화면이 계기판 역할을 해야 했는데, 실제로 의미 있는 정보를 주는 화면은 사실상 하나뿐이었다.



가운데 전술 HUD(SPEED km/h, STATUS NORMAL, 조준점)와 오른쪽 WEAPON 디스플레이. 이날 처음 역할이 정해진 디스플레이 3개의 최종 모습(무기 터치 버튼은 9월 29일에 추가).

7.2 역할 나누기와 파일 분리 설계

16:40 무렵 생긴 세 파일 `CockpitHUDManager.cs`, `CockpitStatusHUD.cs`, `CockpitWeaponHUD.cs`가 이 작업의 결과다. 주석으로 보면 요청은 세 화면에 역할을 나누고, 그 구현을

`CockpitHUDManager` / `CockpitStatusHUD` / `CockpitWeaponHUD` 세 파일로 나누라는 것이었다. 주석은 화면을 `LeftDisplay`, `FrontDisplay`, `RightDisplay`라고 부르고, 각각 실제 `GameObject` 이름이 `SysCheck_Left`, `SysCheck_Center`, `SysCheck_Right`라고 밝혀 둔다.

화면	실제 이름	새 역할	담당 스크립트
LeftDisplay	<code>SysCheck_Left</code>	GUNDAM STATUS — 게이지 9줄	<code>CockpitStatusHUD</code>
FrontDisplay	<code>SysCheck_Center</code>	전술 HUD — SPEED, STATUS, 조준점, HEAD CAM 인셋, THROTTLE/TURN/HEADING	<code>CockpitHUD</code> (확장)
RightDisplay	<code>SysCheck_Right</code>	WEAPON — 무기 이름, 탄수, 탄 게이지, READY/EMPTY/RELOADING	<code>CockpitWeaponHUD</code>
(공유 데이터)	<code>MobileSuitRoot</code>	HP/SHIELD/ENERGY/부위별 체력	<code>CockpitHUDManager</code> + <code>GundamVitals</code>

설계의 중심은 데이터와 표시를 떼어 놓는 것이다. 이 시점에는 기체가 맞는 데미지 시스템이 아예 없었다. 이견호는 그 점을 알고 이렇게 요청했다.

요청 · 아직 실제 부위별 데미지 시스템이 없으면 우선 전체 HP/상태를 기반으로 동작하는 구조로 만들고, 나중에 실제 데미지 시스템과 연결할 수 있도록 분리해줘

그래서 `GundamVitals` 는 `MonoBehaviour` 가 아닌 순수 데이터 클래스(`[System.Serializable]`)로 만들었다. `CockpitHUDManager` 는 그 인스턴스 하나를 소유하는 껍데기 컴포넌트다. UI 참조는 하나도 갖지 않는다. 전술 화면의 상태 표시와 STATUS 화면의 게이지가 같은 인스턴스를 읽으므로 두 화면의 숫자가 어긋날 수 없다.

```
[System.Serializable]
public class GundamVitals
{
    public enum BodyPart { Head, Body, LeftArm, RightArm, LeftLeg, RightLeg }
    public enum Status { Normal, Warning, Critical }
    public float MaxHP = 1000f;      public float HP = 1000f;
    public float MaxShield = 500f;   public float Shield = 500f;
    public float MaxEnergy = 100f;   public float Energy = 100f;
    [Range(0f, 1f)] public float warningThreshold = 0.6f;
    [Range(0f, 1f)] public float criticalThreshold = 0.3f;
    readonly float[] _partOverride = { -1f, -1f, -1f, -1f, -1f, -1f };
    public float PartFraction(BodyPart part)
    {
        float o = _partOverride[(int)part];
        return o >= 0f ? o : HPFraction; // 덮어쓴 값이 없으면 전체 HP를 따라감
    }
}
```

부위별 체력은 "덮어쓴 값"이 -1이면 전체 HP 비율을 그대로 보여 준다. 그래서 데미지 시스템이 없어도 화면은 완전히 동작한다(지금은 모두 100%). 나중에 진짜 부위별 데미지가 생기면 `SetPartFraction(part, 0~1)` 을 부르지만 하면 그 부위 게이지가 따로 움직인다. 음수를 넘기면 다시 전체 HP를 따라가게 된다. 전체 HP는 `ApplyDamage / Heal` 로 바꾼다. HUD 쪽은 그 차이를 알 필요가 없다. 상태 판정은 HP 비율이 0.3 이하면 CRITICAL, 0.6 이하면 WARNING, 그 위는 NORMAL이다. 게이지 색도 같은 기준의 공용 함수 `StatusColor` 가 정한다. 0.3 이하는 빨강 (1, 0.25, 0.2), 0.6 이하는 노랑 (1, 0.8, 0.2), 그 위는 초록 (0.3, 1, 0.4) 이다. STATUS와 WEAPON 두 화면의 모든 게이지가 이 함수를 쓰므로 콧 전체의 색 언어가 하나로 통일된다.

7.3 왼쪽: GUNDAM STATUS

`BuildStatusScreenUI` 는 왼쪽 화면에 380×440 Canvas(배율 0.00047)를 세우고 맨 위에 "GUNDAM STATUS" 제목을 단다. 그 아래 y=155부터 34씩 내려가며 HP, SHIELD, ENERGY, HEAD, BODY, L ARM, R ARM, L LEG, R LEG 9줄의 게이지를 만든다. 한 줄은 `BuildGaugeRow` 가 만든다. 300×22 어두운 배경 막대, 그 위에 왼쪽부터 차오르는 `Image.Type.Filled` 막대(`FillMethod.Horizontal` , `OriginHorizontal.Left`), 그 위에 "HP 100%" 같은 글자(크기 13)다. 채움 막대는 pivot을 왼쪽 끝 (0, 0.5) 에 두고 `anchoredPosition` 을 막대 폭의 절반만큼 왼쪽으로 옮겨 배경과 정확히 겹치게 했다.

UI를 만드는 일은 에디터 스크립트가 하고, 매 프레임 갱신은 `CockpitStatusHUD` 가 한다. 이 컴포넌트는 줄마다 `GaugeRefs` (채움 Image + Text) 한 쌍을 받아 `Update()` 에서 비율과 색, 글자를 쓴다. 게임 로직, 이동, 손 추적, 무기 로직은 하나도 건드리지 않는 순수한 렌더러다.

```
static void SetGauge(GaugeRefs g, string label, float frac01)
{
    if (g == null) return;
    frac01 = Mathf.Clamp01(frac01);
    Color c = GundamVitals.StatusColor(frac01);
    if (g.fill != null) { g.fill.fillAmount = frac01; g.fill.color = c; }
    if (g.text != null) { g.text.text = $"{label} {Mathf.RoundToInt(frac01 * 100f)}%"; }
}
```

7.4 가운데: 전술 HUD, 중앙 조준점, HEAD CAM 인셋

가운데 화면에서는 포탑 무기 이름과 탄수 자리를 속도와 상태가 차지했다. `SpeedReadout` ("SPEED 0.0 m/s", 크기 26)과 `StatusReadout` ("STATUS: NORMAL", 크기 20, 초록)이다. 원형 다이얼 중심에는 조준점을 그렸다. 요청 문구는

요청 · 간단한 중앙 조준점

이다. 다이얼 중심 (0, 10) 에 50×3짜리 가로 막대와 3×50짜리 세로 막대를 겹친 십자다. 색은 연두 (0.6, 1, 0.7, 0.85) 이고 데이터는 없는 순수 장식이다.

왼쪽 화면을 STATUS에 내주면서 머리 카메라 영상이 갈 곳이 없어졌다. 이견호의 지시는 분명했다.

요청 · 기존 HeadCam 기능은 삭제하지 말고 FrontDisplay 중앙 전술 HUD의 한쪽 구석에 작은 인셋 영상으로 옮겨줘

`BuildHeadCamInsetUI` 는 가운데 Canvas의 오른쪽 위 모서리(anchor·pivot 모두 (1,1))에서 안쪽으로 20씩 들어온 자리에 200×200 틀을 만든다. 틀 위쪽에 "HEAD CAM" 글자(크기 11)를 넣고, 나머지 공간을 `RawImage` 로 채운다. 이 `RawImage` 가 보여 주는 텍스처는 머리 카메라가 이미 매 프레임 그리고 있는 같은 렌더 텍스처다. 새 카메라나 새 렌더 텍스처는 만들지 않았다. 같은 텍스처를 읽는 화면이 하나 더 생겼을 뿐이다. 예전 왼쪽 화면용 함수 `BuildHeadCamScreenUI` 는 더 이상 호출되지 않지만 지시대로 지우지 않고, "더 이상 호출되지 않지만 삭제하지 말라는 지시에 따라 그대로 둔다"는 주석만 달았다. 렌더 텍스처가 없으면 (FBX가 아직 없을 때) 인셋은 만들지 않는다.

`CockpitHUD` 도 "대체가 아니라 확장"됐다. `ship` , `hudManager` , `speedText` , `statusText` 필드가 생겼다. 아래쪽 텔레메트리 문구에서는 오른쪽 조종간 값을 보여 주던 "AIM X/Y" 줄과 "[R GRIP]" 표시가 빠지고, 대신 기체 방향 "HEADING"이 들어갔다. 이제 오른손은 조종간이 아니라 헤드 발칸 엄지로 쓰기 때문이다.

```
float headingDeg = ship != null ? NormalizeDegrees(ship.HeadingYaw) : 0f;
infoText.text =
    "GUNDAM COCKPIT\n" +
    $"THROTTLE {l.y * 100f:0}%   TURN {l.x * 100f:0}%\n" +
    $"HEADING {headingDeg:000}°" +
    (leftStick != null && leftStick.isGrabbed ? " [L GRIP]" : "");
...
speedText.text = $"SPEED {Mathf.Abs(ship.CurrentSpeed):0.0} m/s";
```

HeadingYaw 는 계속 더해지는 값이라 360도를 넘거나 음수가 될 수 있다. NormalizeDegrees 는 이를 0~360으로 접는다. 상태 문구는 GundamVitals.OverallStatus 에 따라 NORMAL/WARNING/CRITICAL로 바뀌고 색도 HP 비율 색을 따른다. 포탑용 필드 weapon , weaponNameText , ammoText 는 "legacy/optional"이라는 헤더로 남겼다. 썬 생성 코드가 더 이상 연결하지 않을 뿐이다. Update() 는 원래 이 값들이 null이면 아무것도 하지 않으므로 고장이 아니라 그냥 아무 일도 하지 않는 상태가 된다. 포탑과 WeaponAimFireController 는 그대로 동작한다.

7.5 오른쪽: WEAPON, 그리고 헤드 발칸의 탄약 상태

오른쪽 화면은 헤드 발칸의 상태를 보여 준다. BuildWeaponScreenUI 는 "WEAPON" 제목 아래에 무기 이름 ("HEAD VULCAN", 크기 22), 탄수("AMMO 200/200", 크기 18, 주황), 300×22 탄 게이지, 상태 글자 ("READY", 크기 24)를 쌓는다. CockpitWeaponHUD 가 이를 매 프레임 갱신한다. 상태 글자는 READY는 초록, EMPTY는 빨강, RELOADING은 노랑이다.

그런데 6장에서 만든 HeadVulcanController 에는 탄약 개념이 없었다. 이견호는 여기서도 선을 그었다.

요청 · Head Vulcan 발사 시 탄약 수가 감소하도록... 단, 기존 발사 동작 자체는 변경하지 말고, HUD가 읽을 수 있는 상태값만 노출

그래서 탄약은 발사를 막는 장치가 아니라 HUD에 보여 주기 위한 별도 장부로 붙였다. maxAmmo = 200 , emptyHoldDuration = 0.4 초, reloadDuration = 2.5 초다. Fire() 는 탄을 다 생성한 뒤 마지막에 ConsumeAmmoForShot() 을 한 번 부른다. 탄이 0이 되면 상태가 Empty 가 되고, 0.4초 뒤 Reloading , 다시 2.5초 뒤 200발로 채워지며 Ready 가 된다. 이 타이머는 Update() 안에서 발사 로직과 완전히 따로 돈다.

```
void ConsumeAmmoForShot()
{
    if (CurrentAmmo > 0) CurrentAmmo--;
    if (CurrentAmmo <= 0 && State == FireReadyState.Ready)
    {
        CurrentAmmo = 0;
        State = FireReadyState.Empty;
        _stateTimer = emptyHoldDuration;
    }
}
```

주석이 분명히 말하듯 탄이 떨어져도 엄지를 굽히면 여전히 쏜다. 바뀌는 것은 HUD가 읽는 CurrentAmmo 와 State 값뿐이다. "기존 발사 동작 자체는 변경하지 말고"라는 요청을 문자 그대로 지킨 구조다. 실제로 탄 부족 때문에 사격을 멈추게 하는 일은 이 시점에는 하지 않았다.

이 순서 때문에 썬 생성 코드에 작은 연결 문제가 생긴다. WEAPON 화면은 `BuildSystemCheckDisplay` 에서 먼저 만들어지는데, `HeadVulcanController` 는 한참 뒤 XR 리그가 생긴 다음에야 만들어진다. 그래서 `BuildSystemCheckDisplay` 는 `out CockpitWeaponHUD weaponHUD` 로 컴포넌트를 돌려주기만 하고, 헤드 발칸 연결 블록에서 `weaponHUD.headVulcan = headVulcan;` 으로 나중에 이어 준다. `CockpitHUDManager` 는 반대로 의존하는 것이 없으므로 `MobileSuitRoot` 를 만든 직후, `ShipMovementController` 바로 다음에 붙인다. 그래야 디스플레이를 만들 때 넘겨줄 수 있다.

작업 순서를 짐작할 근거

`HeadVulcanController` 의 탄약 필드 주석에는 "ADDED for the Cockpit HUD"라고 적혀 있다. WEAPON 화면 주석에는 "기존 Head Vulcan 연결 블록이 이제 weaponHUD.headVulcan도 설정한다"고 적혀 있다. 그러니 헤드 발칸이 먼저 만들어졌고, 디스플레이 세 개 작업(16:40 무렵) 때 탄약 상태가 덧붙었다고 볼 수 있다.

8. "옆으로가 잘 안감" — ShipMovementController

04 커밋에 함께 들어온 마지막 변경은 기체 이동이다. 그때까지 왼쪽 조종간의 좌우 입력은 오직 방향 전환(요)에만 쓰였다. `HeadingYaw += t.x * maxTurnSpeed * Time.deltaTime` (`maxTurnSpeed = 60` 도/초)로 방향 값만 쌓이고, 실제 이동은 앞뒤 입력에만 따랐다(`maxMoveSpeed = 8` m/s). 이때 방향 값은 `Transform`을 회전시키지 않는 그냥 숫자다. 예전에 기체를 직접 돌렸더니 그 아래에 붙은 XR 카메라까지 돌아 멀미를 일으켰기 때문이다. 이견호는 이렇게 보고했다.

문제 보고 · 옆으로가 잘 안감. 옆으로 해도 앞으로 하고나서 옆으로 해야 옆으로감. 그냥 옆으로 해도 갈 수 있게 해야함

원인은 바로 그 구조였다. 좌우로만 밀면 보이지 않는 방향 값만 바뀌고 기체는 제자리에 있다. 앞으로 가고 있을 때 좌우로 밀어야 비로소 진행 방향이 휘면서 옆으로 가는 것처럼 보였다. 수정은 기존 회전을 없애지 않고 그 위에 옆 이동(스트레이프)을 더하는 것이다.

```
Vector3 headingForward = heading * Vector3.forward;
Vector3 headingRight = heading * Vector3.right;
Vector2 moveInput = t.sqrMagnitude > 1f ? t.normalized : t;
Vector3 planarVelocity = (headingForward * moveInput.y + headingRight * moveInput.x) *
maxMoveSpeed;
CurrentSpeed = planarVelocity.magnitude;
transform.position += planarVelocity * Time.deltaTime;
```

이제 좌우로 밀면 방향도 돌고 그 방향의 오른쪽으로 바로 미끄러진다. 대각선 구석으로 밀었을 때 두 축이 합쳐져 더 빨라지지 않도록 입력 길이가 1을 넘으면 정규화한다. `CurrentSpeed` 도 앞뒤 추력값(`t.y * maxMoveSpeed`, 뒤로 가면 음수)에서 실제 평면 속도의 크기(항상 0 이상)로 바뀌었다. 예전 값으로는 옆으로만 움직일 때 전술 화면의 SPEED가 0으로 나왔을 것이다. 바뀐 주석이 이 점을 "HUD 속도계"를 들어 설명하므로, 이 변경은 디스플레이 작업과 같은 흐름에서 나온 것으로 보인다. 클래스 맨 위의 설명도 "좌우는 방향 전환 전용, 따로 옆 이동 없음"이라던 옛 문구가 새 동작 설명으로 바뀌었다.

조작 정리(9/23 저녁 기준)

왼쪽 조종간을 앞뒤로 밀면 현재 방향으로 전진·후진한다. 좌우로 밀면 그 방향으로 돌면서 옆으로 이동한다. 오른손은 조종간 대신 엄지를 굽혀 헤드 발칸을 쓰고, 조준은 머리로 한다. 오른쪽 조종간과 포탑은 그대로 남아 있지만 HUD에는 더 이상 연결되지 않는다. 앱을 켤 때마다 왼쪽→확인→오른쪽→확인 순서로 조종간 위치를 정한다.

9. 이날의 시행착오 정리

9월 23일은 같은 종류의 실수가 모양을 바꿔 여러 번 나타난 날이다. 가장 두드러진 것은 **손 추적값을 절대 좌표로 쓰는 것**의 위험이다. 조종간 Handle은 잡는 순간 손바닥 좌표로 스냅했다가 끌려왔다.

캘리브레이션은 Mount를 손바닥 좌표에 두었다가 바닥에 박혔다. 확인 버튼은 손바닥과 버튼의 거리를 잴다가 놓리지 않았다. 세 경우 모두 "잡은 순간을 기준으로 한 변화량"이나 "손 모양 신호"로 바꾸자 해결됐다.

두 번째는 **지연 장치 겹치기**다. 틸을 막으려던 프레임당 목표 제한과 평활이 겹쳐 조종간이 손을 못 따라왔다. 그것을 걷어낸 뒤에도 남은 감도 배율과 평활이 실기 테스트에서 다시 지적돼, 결국 1:1 직접 대입과 이상값 검사 하나만 남았다.

세 번째는 **검증하지 않은 데이터 소스**다. 양손 간섭을 막겠다고 오전에 끌어온 XRI 인터랙터 Transform이 오후의 "앞으로는 밀리지도 않음"을 낳았다. 이견호가 처음부터 정해 둔 "XRI는 이벤트만, 위치 계산은 JoystickLever"라는 분리로 돌아가자 정리됐다. 같은 맥락에서 Claude는 직접 볼 수 없는 부분(샘플 리그 내부 동작, 머리 카메라 화각 안에 무엇이 들어오는지 등)을 코드 주석에 "확인할 수 없는 가정"이라고 반복해서 적어 두었다.

보고/요청	처음 시도	바뀐 방식
잡는 순간 끌려옴	GripPoint를 손바닥에 스냅	잡은 순간 기준점만 기록, 이후 손 변화량을 더함
양손이 서로 반응	XRI 인터랙터 Transform을 위치 소스로(보강)	(오후) 보강 제거, 위치는 항상 HandJointTracker 손바닥
늦음·민감·틸	목표 프레임당 제한 + 평활	원시 손 위치 이상값 검사 + 감도 0.65 + 평활 25 + 데드존 0.08
앞으로 밀어도 안 움직임	(위의 이중 지연)	목표 제한 제거 → (오후) 감도·평활도 제거, 1:1 직접 대입
이동 중 조종간 이상	실행 순서 미지정	[DefaultExecutionOrder(-100)]
오른쪽 조종간이 꺾이고 회전	"아무도 회전을 안 쓴다"는 가정	LateUpdate 에서 Handle 회전·Pivot 전체를 매 프레임 고정
오른쪽 조종간이 잘 안 잡힘	인터랙터 Transform 정확히 일치 비교	IsChildOf 비교
조종간이 안 보임	카메라에 붙은 불투명 안내판	코핏에 고정 → 소품 없는 떠 있는 글자
조종간이 아래로 내려감	PlayerPrefs 저장/불러오기, 저장 있으면 건너뛸	저장 제거, 매 실행마다 처음부터

바닥으로 내려가 안 움직임, 확인 안 눌림	손바닥 절대 좌표로 Mount 배치, 거리로 버튼 판정	변화량 기반 끌기, 쥐기·꼬집기로 확인
한쪽씩 물어봐야 함	마지막에 한 번 확인	조종간마다 확인 상태 추가(5상태)
콧속 안에서 내 건담이 보임	머리 카메라는 몸통을 보이게 둬	머리 카메라도 레이어 30 제외
옆으로 잘 안 감	좌우 입력 = 방향 전환만	방향 전환 + 옆 이동, 속도는 평면 속도 크기

이날 저녁의 콧속은 아침과 꽤 달라졌다. 조종간은 손을 1:1로 따라가며 돌지도 끌려오지도 않는다. 앱을 켜면 내 팔에 맞게 조종간을 옮겨 놓고 한쪽씩 확인한다. 고개를 돌려 겨누고 엄지를 굽혀 머리 발칸을 쏜다. 정면 세 화면은 기체 상태, 속도와 방향, 무기 상태를 나눠 보여 준다. 그다음 작업은 닷새 뒤인 9월 28일에 이어진다.

9월 28일은 커밋이 가장 촘촘하게 찍힌 날이다. 오전 9시 31분 커밋(04)부터 11시 14분(05), 11시 23분(06), 12시 26분(07), 12시 55분(08), 13시 03분(09), 13시 06분(10)까지 일곱 번 커밋했다. 06과 09는 썬 파일만 바뀌었으니 코드가 실제로 달라진 커밋은 다섯 개다. 이날 생긴 파일들의 시각도 남아 있다.

`HeadCamManualLook.cs` 는 10시 50분 무렵, 돔 셰이더 `CockpitDomeCubemap.shader` 는 11시 13분 무렵(수정 시각일 수 있다), `JoystickFingerButtons.cs` 는 13시 04분 무렵, 적 체력 스크립트 `EnemyHealth.cs` 는 14시 21분 무렵이다. `EnemyHealth.cs` 는 이날 커밋되지 않고 다음 날(9월 29일 11시 52분) 커밋 11에 들어갔다.

하루의 흐름은 이렇다. 아침 커밋에는 조종간을 움직이는 시스템 셋이 서로 충돌하는지 점검한 결과와 역할 분리가 담겨 있다. 오전에는 오른쪽 조종간으로 콕핏 바깥 화면을 돌리는 기능을 세 번 고쳐 만들었고, 그 바람에 왼쪽 조종간의 이동 기준이 두 번 바뀌었다. 점심 무렵에는 헤드 발칸 문제를 몰아서 풀었다. 썬도 안 보이고, 안 나가고, 남은 탄수가 화면에 안 뜨는 문제였다. 그러다 화면 글자가 안 보이던 오래된 문제의 원인도 찾아냈다. 오후에는 조종간 구체에 손가락마다 버튼을 달아 엄지 버튼으로 발칸을 쏘게 했고, 콕핏 앞의 원형 링(OrbitHUD)이 적을 잡으면 적 크기로 오므라드는 락온 기능을 만들었다. 마지막으로 자쿠에게 체력 1000을 줬다. 이 장에서는 이 과정을 시간 순서대로, 각 단계에서 무엇이 틀렸고 어떻게 고쳤는지 diff를 근거로 따라간다.

1. 아침의 출발점: 조종간 Transform을 건드리는 시스템이 셋이다

9월 28일 첫 커밋(04, 09:31)은 다른 커밋보다 크다(diff 약 10만 8천 바이트). BRIEF에 적힌 대로 커밋은 작업이 끝난 뒤에 찍히므로, 이 커밋의 변경 상당수는 9월 23일 오후에 했을 수 있다. 다만 이날 오전 작업이 모두 이 커밋의 상태에서 출발하므로, 이 장도 여기서 시작한다. 핵심은 `JoystickLever.cs` 클래스 주석에 새로 붙은 긴 AUDIT(감사) 기록이다.

1-1. 문제 보고: 조종간이 손을 안 따라오고, 오른쪽은 제멋대로 꺾인다

Galaxy XR 실기 테스트에서 나온 보고는 다음과 같다(`JoystickLever.cs` 주석에 원문이 있다).

문제 보고 · 왼손으로 LeftJoystick을 잡고 밀면 건담 이동은 정상... 하지만 LeftJoystick 자체가 손을 따라오지 않는다... 오른손으로 RightJoystick을 잡으려고 하면 잘 안 잡힌다... 오른쪽 조종간을 잡아도 손을 따라가는 것이 아니라 갑자기 크게 꺾이거나 이상한 방향으로 회전한다

같은 보고에는 항목별 요구사항(A~H)이 붙어 있었다. 주석은 각 항목을 "item A", "item B"처럼 짚어 가며 답한다. 그중 F 항목이 이 절의 제목이 된 요구다.

요청 · 이 세 시스템이 동시에 조종간 Transform을 움직이고 있는지 반드시 확인해라

세 시스템은 다음과 같다. ① XR Interaction Toolkit(XRI): 조종간 손잡이에 붙은

`XRSimpleInteractable`, 그리고 그 위에서 "올바른 손만 잡게" 거르는 `HandExclusiveGrabAdapter`. ②

캘리브레이션: 게임 시작 때 조종간 받침(Mount) 전체를 손으로 끌어 옮기게 하는 `CalibrationManager`. ③

조종간 자체: 손잡이(handle)를 손 움직임에 맞춰 $\pm 8\text{cm}$ 범위에서 미끄러뜨리는 `JoystickLever`.

1-2. 감사 결과: 코드에서 손잡이를 움직이는 곳은 한 군데뿐이었다

Claude는 `HandExclusiveGrabAdapter.cs`와 `GundamCockpitSetup.cs`의 `AttachHandInteractable`,

`WireJoystickHandInteractors`를 처음부터 끝까지 읽었다. 확인한 내용은 세 가지다. 첫째,

`HandExclusiveGrabAdapter`가 하는 일은 `NotifyXRIGrabbed(bool, Transform)` 호출 하나뿐이고,

`transform.position`이나 `rotation`을 읽거나 쓰는 곳은 없다. 둘째, 이 프로젝트는 일부러

`XRGrabInteractable`을 쓰지 않고 `XRSimpleInteractable`을 쓴다. 후자에는 물체를 끌고 가는 동작

(attach)이 없고 select/hover 이벤트만 낸다. 셋째, 폴더 안 스크립트 전체에서

`transform.position/rotation/localPosition/localRotation`에 값을 넣는 곳을 찾아보니, 조종간 손잡이를 움직이는 곳은 `JoystickLever` 자신뿐이었다. 캘리브레이션은 손잡이가 아니라 그 위의 Mount를 옮긴다. `JoystickLever`는 손 위치를 언제나 pivot 기준 로컬 좌표로 계산하므로, Mount가 움직여도 함께 실려 갈 뿐 서로 싸우지 않는다. `CalibrationManager` 주석에도 "it simply rides along rather than fighting it"이라고 적혀 있다.

그런데 실기에서는 RightJoystick이 눈에 띄게 꺾였다. 프로젝트 코드로는 설명이 안 되는 증상이었다.

주석은 Interactor 쪽 Inspector 설정처럼 프로젝트 스크립트 밖에 원인이 있을 수 있다고 인정하고, 원인을 못 찾더라도 증상이 생길 수 없게 만드는 쪽을 택했다. 이것이 "Fix C"다.

```
// Fix C: 손잡이 회전과 pivot의 로컬 Transform 전체를
// 처음 캡처한 기본값으로 매 프레임 되돌린다 (잡았든 안 잡았든)
void LateUpdate()
{
    if (!_centerCaptured) return;
    if (handle != null) handle.localRotation = _centerLocalRotation;
    if (pivot != null)
    {
        pivot.localPosition = _pivotDefaultLocalPos;
        pivot.localRotation = _pivotDefaultLocalRotation;
    }
}
```

`LateUpdate`는 렌더링 직전에 마지막으로 값을 쓰는 자리다. 그 프레임에 누가 조종간을 돌려 놓았든 여기서 원래대로 돌아간다. 기본값은 `CaptureCenterIfNeeded()`가 첫 프레임에 손잡이 위치와 회전, pivot의 위치와 회전을 한 번에 저장해 둔다.

1-3. 역할 분리: XRI는 잡았는지만, 위치 계산은 JoystickLever가

Fix C 뒤에 다시 실기 테스트를 하자 다른 보고가 나왔다.

문제 보고 · 조종이 제대로 안되고 내가 지정한 위치에 조종장치가 고정되지도 않음... 조종장치에 움직임이랑 건담에 움직임이랑 다름... 조종장치가 앞으로는 밀리지도 않음

원인은 `GetActiveHandWorldPosition()` 이었다. 이 함수는 XRI로 잡은 경우 손 위치로 `_grabbedInteractor.position`, 즉 XRI 샘플 리그의 "Near-Far Interactor" GameObject의 Transform을 썼다. 그 Transform이 실제 손을 1:1로 따라간다는 가정은 검증된 적이 없었다. 주석은 원거리 레이 조준을 위해 방향만 바뀌고 주먹 쥐기 같은 근거리 동작에는 그만큼 평행이동하지 않을 수도 있다고 적었다. 그렇다면 조종간이 손 위치에 고정되지 않고, 손 움직임과 어긋나고, 그 Transform이 거의 움직이지 않는 축(앞으로 밀기)은 아예 반응하지 않는다. 보고된 증상과 정확히 맞는다.

이 문제는 맨 처음 이 조종간에 대해 나온 보고에서 이미 원칙으로 정해져 있었다.

요청 · XRI = 잡았는지/놓았는지 이벤트만 담당, JoystickLever = 실제 조종간 위치 계산 담당

그래서 `_grabbedInteractor` 필드를 지웠다. `NotifyXRIGrabbed` 는 기존 호출부와 시그니처를 맞추려고 Transform 인자를 그대로 받지만 저장하지 않는다. 위치는 XRI로 잡았든 제스처로 잡았든 항상 `HandJointTracker.PalmPosition` 을 쓴다. 이 값은 손 메시, GripAmount/PinchAmount, 캘리브레이션 드래그에서 이미 정확하다고 확인된 값이다.

시스템	맡은 일(9/28 아침 기준)	Transform에 쓰는 것
XRI + HandExclusiveGrabAdapter	올바른 손이 잡았는지, 놓았는지(예/아니오 신호)	없음
CalibrationManager	시작할 때 Mount(조종간 받침 전체)를 손으로 끌어 배치	Mount 위치(잡고 있는 동안만)
JoystickLever	손잡이 위치 계산(X/Z, 1:1), tiltInput 출력	handle.localPosition의 X/Z, LateUpdate에서 회전 고정

1-4. 같은 커밋의 다른 조종간 수정들

같은 커밋에는 원인을 하나 더 찾은 기록이 있다. 보고는

문제 보고 · 조종관을 잡고 이동할때 조종관이 내손따라서 제대로 안움직임... 특정방향으로는 안밀림... 로봇이 맘대로 다른데로 가기도함... 움직일때는 오른손 조종장치가 안작동함

이었다. 원인은 실행 순서였다. 조종간, XR Origin, 손 트래커는 모두 `MobileSuitRoot` 아래에 있고, 그 루트를 `ShipMovementController` 가 움직인다. 손 추적 데이터는 프레임 앞쪽에 들어오는데, `ShipMovementController.Update()` 가 `JoystickLever.Update()` 보다 먼저 도는 프레임에는 pivot은 이미 이동했고 손 샘플은 이동 전 값이다. 그래서 기체가 움직이는 동안 한 프레임 이동 거리만큼 어긋남이 생겼다. `JoystickLever` 클래스에 `[DefaultExecutionOrder(-100)]` 을 붙여 항상 먼저 돌게 해서 해결했다.

Fix B는 1:1 추종이다. 그전에는 `positionSensitivity = 0.65` 로 손 움직임을 줄이고 `smoothingSpeed = 25` 로 지수 보간을 해서, 둘 다 "손보다 늦게 따라오는" 원인이었다. 둘을 지우고 `handle.localPosition =`

`ClampToRange(_grabStartJoystickLocal + handMovement)` 로 매 프레임 바로 대입한다. 남은 안전장치는 한 프레임에 손이 `maxHandDeltaPerFrame = 0.15 m` 넘게 순간이동하면 그 샘플을 버리는 글리치 가드 하나다.

`HandExclusiveGrabAdapter` 도 고쳤다. 보고는

문제 보고 · RightJoystick은 잘 안 잡히고

였다. 이전 코드는 select 이벤트를 낸 Interactor의 Transform이 허용된 Interactor와 정확히 같은 객체여야 통과시켰다. XRI가 실행 중에 그 자식 객체에서 이벤트를 내면 올바른 오른손도 "다른 손"으로 판정해 강제로 취소했다. `Transform.IsChildOf` 로 바꿔 허용된 Interactor 계층 아래에서 온 이벤트면 받아들인다. 왼손과 오른손 Interactor는 "Left Hand"/"Right Hand"라는 서로 다른 하위 트리에 있으므로 손 구분은 그대로 유지된다.

캘리브레이션에도 요청이 하나 반영됐다.

요청 · 한쪽씩 설정할때마다 물어봐야함 됐냐고 위치

. 이전에는 두 조종간을 다 놓은 뒤 마지막에 한 번 확인했다. 이제는 상태가 `WaitingForLeftGrab` → `WaitingForLeftConfirm` → `WaitingForRightGrab` → `WaitingForRightConfirm` → `Complete` 로 다섯 단계다. 확인 단계에서는 "왼쪽 조종간 위치가 괜찮으면 / 손을 쥐거나 꼬집어 확인하세요" 같은 안내가 뜨고, 어느 손이든 `grip 0.55` 또는 `pinch 0.5` 이상이면 확인으로 친다.

마지막으로 왼쪽 조종간의 좌우 이동이다.

문제 보고 · 옆으로가 잘 안감. 옆으로 해도 앞으로 하고나서 옆으로 해야 옆으로감. 그냥 옆으로 해도 갈 수 있게 해야함

. 이전에는 조종간을 좌우로 밀면 보이지 않는 진행 방향 `HeadingYaw` 만 돌았고 위치는 그대로였다. 그래서 앞으로 가는 중에만 방향이 꺾이며 옆으로 가는 것처럼 보였다. 04 커밋은 회전은 그대로 두고 그 방향의 오른쪽 벡터로 즉시 옆 이동(strafe)하는 항을 더했다. 대각선 입력은 길이 1로 잘라 모서리로 밀어도 더 빨라지지 않게 했다. 이 "회전은 그대로 두고"가 몇 시간 뒤 문제가 된다(3절).

2. 오른쪽 조종간으로 바깥 화면 돌리기: 세 번의 시도

2-1. 요청

이 무렵 콕핏 구조를 정리하면 이렇다. 파일럿(이건호)의 머리, 즉 XR Main Camera는 불투명한 구형 돔 `Cockpit_Dome` 안에 있다. 돔 안쪽에 비치는 바깥 세상은 콕핏 앞 약 20m에 서 있는 외부 건담 (`ExternalGundam`)의 머리 뼈(Head) 밑에 달린 카메라 `HeadCam` 이 찍는다. `GundamHeadCam360` 이 매 `LateUpdate` 마다 `headCam.RenderToCubemap(...)` 으로 면당 512픽셀 큐브맵을 굽고, 돔의 셰이더 `Custom/CockpitDomeCubemap` 이 그 큐브맵을 표면 방향으로 샘플링한다. `GundamHeadCam360` 은 이건호가

고개를 돌리면 건담 머리 뼈도 따라 돌린다(이전 요청 "내가 머리를 돌리면 건담 머리도 돌아야해"). 오른쪽 조종간은 원래 `WeaponAimFireController`의 포탑 조준에 쓰였다.

이날 오전의 요청은 오른쪽 조종간으로 바깥 화면을 돌리는 것이었다. 최종 클래스인 `CockpitViewController` 주석에 남은 원문은 다음과 같다.

요청 · 오른손으로 RightJoystick을 잡고 움직이면 '건담 콕핏에서 바라보는 외부 화면'이 움직여야 한다...
RightJoystick.tiltInput.x -> Yaw, RightJoystick.tiltInput.y -> Pitch

2-2. 시도 ①: GundamHeadCam360 안에 수동 시점 오프셋 (되돌림)

처음 시도는 머리 추적 스크립트 `GundamHeadCam360`에 오른쪽 조종간으로 움직이는 "manual look" 오프셋을 끼워 넣는 것이었다. 이 코드는 커밋에 남지 않았다. 05 커밋의 `GundamHeadCam360.cs`에 붙은 NOTE만 있다. 더 정확한 후속 요청이 와서 되돌렸고, 그래서 이 클래스는 이름 그대로 "머리 추적과 360 큐브맵 공급"만 한다는 내용이다.

2-3. 시도 ②: HeadCamManualLook, 시점 카메라만 돌린다

후속 요청은 무엇을 돌려야 하는지를 꼭 집었다(`HeadCamManualLook.cs` 주석).

요청 · 실제로 회전시킬 대상은 콕핏 외부를 보여주는 시점용 Camera이다... 그 Camera를 정확히 찾아서 그 Camera의 회전만 RightJoystick 입력으로 제어한다.

요청에는 세부 조건도 있었다.

요청 · Yaw는 360도 자유롭게 회전할 수 있게 한다

,

요청 · pitch = -60 ~ +60도

, 그리고 예시

요청 · tiltInput.x = +1 -> 오른쪽으로 회전

,

요청 · tiltInput.y = +1 -> 위쪽으로 회전

. 10시 50분 무렵 생긴 `HeadCamManualLook`는 `HeadCam` GameObject에 붙어 자기 `localRotation`만 바꾼다.


```

void LateUpdate()
{
    if (rightStick == null) return;
    if (!_haveStart) { _startLocalRotation = transform.localRotation; _haveStart = true; }
    Vector2 t = rightStick.tiltInput;
    // Yaw: 제한 없음(360도)
    _yaw += t.x * lookSpeed * Time.deltaTime;
    // Pitch: ±maxPitch로 제한, 앞으로 밀면(+y) 위를 본다
    _pitch = Mathf.Clamp(_pitch - t.y * lookSpeed * Time.deltaTime, -maxPitch, maxPitch);
    transform.localRotation = _startLocalRotation * Quaternion.Euler(_pitch, _yaw, 0f);
}

```

설계에서 따진 점은 셋이다. 첫째, `HeadCam`의 부모는 머리 뼈이고 그 뼈는 `GundamHeadCam360`이 머리 추적으로 계속 돌린다. 이 스크립트는 부모를 건드리지 않고 자기 로컬 회전만 쓰므로, Unity가 부모 회전 위에 합성해서 실제 머리 추적과 수동 시점이 서로 싸우지 않고 더해진다. 둘째, 카메라의 원래 로컬 회전이 항등이라는 보장이 없으므로 처음 값을 `_startLocalRotation`에 저장하고 그 위에 곱한다. 덮어쓰면 조종간이 가운데에 있어도 엉뚱한 쪽을 볼 수 있다. 셋째, 속도는 `lookSpeed = 60` 도/초, 상하 한계는 `maxPitch = 60` 도다. Unity에서는 X축 음수 회전이 위를 보는 방향이라 `_pitch - t.y`로 부호를 뒤집었다.

2-4. 왜 돔이 돌지 않았나: RenderToCubemap은 카메라 회전을 무시한다

실기에서 돌아간 것은 FrontDisplay 구석의 작은 "HEAD CAM" 화면뿐이었고, 이걸 돌리려 한 돔은 그대로였다. Claude가 코드에서 확인한 원인은 05 커밋 `GundamCockpitSetup.cs`와 `CockpitViewController.cs` 주석에 적혀 있다. `HeadCam` 하나가 출력 두 개를 낸다.

출력	만드는 방법	보이는 곳	카메라 회전 반영
평면 <code>RenderTexture</code> <code>GundamHeadCam.renderTexture</code>	<code>headCam.targetTexture</code>	FrontDisplay의 HEAD CAM 인셋(RawImage)	반영됨
큐브 <code>RenderTexture</code> <code>GundamHeadCam360.renderTexture</code>	<code>headCam.RenderToCubemap</code> (매 <code>LateUpdate</code>)	Cockpit_Dome, 파일럿을 둘러싼 360도 화면	반영 안 됨

RenderToCubemap의 성질

`Camera.RenderToCubemap`은 카메라 위치에서 월드 축 방향(+X, -X, +Y, -Y, +Z, -Z)으로 여섯 면을 찍는다. 카메라를 어떻게 돌려 놓아도 면은 항상 월드 축에 맞춰져 있고, 카메라 회전은 결과에 영향을 주지 않는다. 그래서 `HeadCam`을 아무리 돌려도 돔이 샘플링하는 큐브맵 내용은 똑같다. 돔을 돌리려면 큐브맵을 읽을 때 방향 벡터를 돌려야 한다.

2-5. 시도 ③: CockpitViewController와 돔 셰이더의 _ViewRotQ

세 번째 해법은 같은 회전을 두 곳에 동시에 보내는 것이다. 새 컴포넌트 `CockpitViewController`는 `MobileSuitRoot` 아래에 같은 이름의 `GameObject`로 생성돼 Hierarchy에서 바로 찾을 수 있다. 하는 일은 두 가지다. (1) `HeadCam`의 `localRotation`을 돌린다. 평면 인셋용이다. (2) 같은 회전을 쿼터니언으로 돔

머티리얼의 `_ViewRotQ` 에 넣는다. 동용이다. `HeadCamManualLook` 은 이제 붙이지 않는다. 같이 붙이면 `HeadCam`이 두 번 돌기 때문이다. 파일은 쓰지 않은 채 남겨 두었다.

```
Vector2 input = rightJoystick.tiltInput;
Yaw += yawInput * yawSpeed * Time.deltaTime;           // 제한 없음
if (Yaw > 360f || Yaw < -360f) Yaw %= 360f;
Pitch = Mathf.Clamp(Pitch + pitchInput * pitchSpeed * Time.deltaTime, minPitch, maxPitch);
// 월드 위쪽 축 기준 yaw, 그다음 돌아간 오른쪽 축 기준 pitch → 롤 없음
Quaternion look = Quaternion.Euler(-Pitch, Yaw, 0f);
if (viewCamera != null && _haveCameraStart)
    viewCamera.transform.localRotation = _cameraStartLocalRotation * look;
if (_domeMat != null)
    _domeMat.SetVector(ViewRotQId, new Vector4(look.x, look.y, look.z, look.w));
```

수치는 `yawSpeed = 60` 도/초, `pitchSpeed = 45` 도/초, `minPitch = -60`, `maxPitch = 60` 이다. 이 컨트롤러는 **속도형**이다. 조종간을 기울인 정도가 회전 속도가 되므로, 오른쪽으로 계속 밀면 계속 돌고, 손을 놓아 조종간이 가운데로 돌아오면 그 방향에 그대로 멈춘다. 처음 방향으로 튕겨 돌아가지 않는다.

`JoystickLever` 가 데드존과 놓았을 때의 복귀를 이미 처리하므로 가운데는 정확히 0이다. 체감상 방향이 반대라면 바꿀 수 있게 `invertYaw`, `invertPitch` 옵션도 넣었다.

돔 머티리얼은 `domeRenderer.material` (인스턴스)로 가져온다. `.sharedMaterial` 을 쓰면 Play 중에 바꾼 회전이 씬에 저장된 머티리얼 원본에 그대로 기록되기 때문이다. 옛 셰이더라서 `_ViewRotQ` 속성이 없으면 경고를 남기고 인셋만 돌린다.

셰이더 쪽은 이렇게 바뀌었다. 속성 블록에 `_ViewRotQ("View Rotation (quaternion xyzw)", Vector) = (0, 0, 0, 1)` 을 추가했다. 기본값 (0,0,0,1)은 항등 회전이라 값을 안 넣는 씬은 전과 똑같이 보인다. 프래그먼트 셰이더는 샘플링 방향을 이 쿼터니언으로 돌린다.

```
// Unity의 Quaternion * Vector3와 같은 계산
// → C#에서 Quaternion.Euler로 만든 값이 셰이더에서도 똑같이 돈다
float3 RotateByQuat(float3 v, float4 q)
{
    float3 t = 2.0 * cross(q.xyz, v);
    return v + q.w * t + cross(q.xyz, t);
}
half4 Frag(Varyings IN) : SV_Target
{
    // 돔의 방향 d를 보면 월드의 (viewRotation * d) 방향이 보인다
    float3 dir = RotateByQuat(normalize(IN.dirWS), _ViewRotQ);
    half3 col = SAMPLE_TEXTURECUBE(_CubeTex, sampler_CubeTex, dir).rgb;
    return half4(col * _Exposure, 1.0);
}
```

셰이더 주석의 예를 들면, yaw를 +30도로 돌리면 돔의 정면에 월드에서 오른쪽 30도 방향이 보인다.

`CockpitViewController` 가 건드리지 않는 것도 주석에 적혀 있다. `MobileSuitRoot` 의 위치와 회전, XR Origin, Main Camera와 HMD 추적, 머리 뼈, `LeftJoystick`, `ShipMovementController`, `JoystickLever` 의 잡기 로직은 손대지 않는다. `WeaponAimFireController` 는 같은 `RightJoystick`을 계속 읽는다. 시점 컨트롤러는 그 입력을 함께 읽는 쪽이 하나 더 생긴 것이지 대체한 것이 아니다.

설정 스크립트는 돔을 `interior.transform.Find("Cockpit_Dome")` 로 찾아 연결한다. 못 찾으면 "only the FrontDisplay HEAD CAM inset will turn" 경고를 남긴다. 07 커밋에서 이 컨트롤러에 `LookRotation` 프로퍼티가 추가되는데, 돔에 보낸 것과 같은 쿼터니언을 다른 스크립트(발칸 조준, 락온 링, 이동)가 읽게 하려는 것이다. 이 값은 뒤의 여러 절에서 다시 나온다.

3. "앞으로 밀면 옆으로 가고": 이동 기준을 두 번 고치다

3-1. 첫 번째: 보이지 않는 HeadingYaw를 없앤다 (05, 11:14)

시점 회전과 같은 커밋에 왼쪽 조종간 보고가 함께 들어왔다.

문제 보고 · 왼손 조종기로 어느정도 조종을하면 조종이 제대로 안됨 앞으로 밀면 옆으로 가고 옆으로 밀면 앞으로가고... 처음에만 멀정하고

원인은 1절 끝에서 말한 04 커밋의 구조였다. 좌우 입력이 들어오면 `HeadingYaw += t.x * maxTurnSpeed * dt` 로 진행 방향을 계속 돌렸고, 앞/옆 이동은 그 돌아간 방향을 기준으로 계산했다. 그런데 화면에 보이는 것 중 `HeadingYaw` 를 따라 도는 것은 하나도 없다. 이 클래스는 VR 멀미 때문에 `MobileSuitRoot` 의 회전을 일부러 건드리지 않으므로, 콧도 돔도 이번호의 시야도 늘 같은 방향을 본다. 좌우로 밀 때마다 보이지 않는 이동 기준만 초당 60도씩 돌았고, 1.5초쯤 좌우 입력이 쌓이면 90도가 어긋난다. 그러면 조종간을 앞으로 밀면 화면에서는 옆으로 가고 옆으로 밀면 앞으로 간다. 방향이 쌓이기 전인 "처음에만 멀정"한 것까지 보고와 정확히 맞았다.

```
// 이동 기준을 이 Transform 자신의 고정된 forward/right로
// (콧도와 시야가 향하는 방향과 같다)
Vector3 suitForward = transform.forward;
Vector3 suitRight = transform.right;
Vector2 moveInput = t.sqrMagnitude > 1f ? t.normalized : t;
Vector3 planarVelocity = (suitForward * moveInput.y + suitRight * moveInput.x) * maxMoveSpeed;
planarVelocity.y = 0f;
CurrentSpeed = planarVelocity.magnitude;
transform.position += planarVelocity * Time.deltaTime;
```

왼쪽 조종간의 좌우는 이제 아무것도 회전시키지 않는다. 둘러보기는 오른쪽 조종간 몫이 됐다. `HeadingYaw` 프로퍼티는 HUD의 HEADING 표시가 컴파일되도록 남겨 두었다(값은 0으로 고정). `maxTurnSpeed` (60)도 Inspector에서 필드가 사라지지 않게 쓰지 않은 채 남겼다. 이전 "옆으로 안감" 보고 때 넣은 `[ShipMove]` 디버그 로그는 지웠다. 최고 속도는 `maxMoveSpeed = 8 m/s`다.

3-2. 두 번째: 시선의 yaw를 기준으로 (07, 12:26)

시점을 돌릴 수 있게 되자 같은 문제가 반대쪽에서 나타났다. 오른쪽 조종간으로 옆을 보고 있으면 화면의 "앞"은 더 이상 기체의 고정된 forward가 아니다.

요청 · 움직이는거를 콧빔에서의 시선을 기준으로 해야할거 같아 옆을 바라볼때는 앞으로 밀어도 옆으로 가는거 처럼 되니까

ShipMovementController 에 viewController 참조를 추가하고, 이동 기준 벡터를 시점의 Yaw만큼 돌렸다.

```
float viewYaw = viewController != null ? viewController.Yaw : 0f;
HeadingYaw = viewYaw; // HUD의 HEADING이 실제 진행 방향을 표시
Quaternion viewHeading = Quaternion.AngleAxis(viewYaw, Vector3.up);
Vector3 suitForward = viewHeading * transform.forward;
Vector3 suitRight = viewHeading * transform.right;
```

Yaw만 쓰고 Pitch는 무시한다. 위나 아래를 보면서 앞으로 밀어도 기체가 상승하거나 하강하지 않게 하려는 것이다. 왼쪽 조종간은 여전히 아무것도 회전시키지 않고, 이 스크립트는 여전히 transform.rotation 에 쓰지 않는다. 결국 왼쪽 조종간의 "앞"은 지금 콧빔이 보고 있는 방향이 됐다. 하루 사이에 이동 기준은 보이지 않는 HeadingYaw에서 기체의 고정 forward로, 다시 시선의 yaw로 바뀌었다. 기준을 늘 화면에 보이는 것과 맞춘다는 원칙은 같다.

4. 눈앞의 막대기 치우기

05 커밋에는 짧은 정리 요청도 하나 있다.

요청 · 눈앞에있는 막대기로 만들어진 총열같이 생긴놈을 지우고

Claude는 이것이 오른쪽 조종간 포탑(GunTurret)의 메시라고 확인했다. 기체 루트에서 2.4m 앞, 파일럿 정면 시야 한가운데에 원기둥 두 개 GunHousing (지름 0.14, 길이 0.2)과 Barrel (지름 0.07, 길이 0.5)이 있었다. 이 두 CreateCylinder 호출만 지웠다. 같은 요청에서 조준·발사 기능 자체는 지우지 말라고 했으므로 GunPivot 과 MuzzlePoint 는 빈 Transform으로 남기고, WeaponAimFireController 연결도 그대로 두었다. 그 원기둥들에만 쓰이던 hullMat / gearMat 는 이 블록에서 더 쓰이지 않지만 다른 곳에서 쓰는 머티리얼이라 그대로 두었다.

5. 헤드 발칸 탄수: 120발을 60발로 표기한다

5-1. 탄수 사양

헤드 발칸 HUD는 9월 23일 오후 무렵 RightDisplay의 WEAPON 화면(CockpitWeaponHUD)으로 만들어졌고, 최대 탄수 maxAmmo 는 200이었다. 이날 사양이 정해졌다.

요청 · 총알 개수는 120발이야 머리 한쪽당 60발씩해서 총 120발이야 그니까 동시에 양쪽에서 나가는거니까 60발로 표기하면될듯

이 문장에서 두 가지가 바뀌었다. 먼저 `alternateMuzzles` 기본값이 `true` (좌우 번갈아 발사)에서 `false` (양쪽 동시 발사)로 바뀌었다. 다음으로 `maxAmmo` 가 200에서 60이 됐다. 주석의 계산은 "한 번 쏠 때 좌우 1발씩 2발이 나가므로, HUD 탄수는 개별 탄이 아니라 **발사 횟수**를 센다. $60회 \times 2발 = 120발$ "이다. 07 커밋에서 `RightDisplay`의 초기 문구도 "AMMO 200/200" 에서 "AMMO 060/060" 으로 고쳤다. 탄이 떨어지면 `emptyHoldDuration = 0.4` 초 동안 EMPTY를 보여 준 뒤 RELOADING으로 바뀌고, `reloadDuration = 2.5` 초 뒤 60으로 채워진다. 이 탄수 계산은 HUD 표시용으로만 쓰이고 발사 자체를 막지는 않는다(9월 23일 요청 "기존 발사 동작 자체는 변경하지 말고, HUD가 읽을 수 있는 상태값만 노출"에 따른 구조다).

5-2. 정면 디스플레이에도 탄수를

문제 보고 · 헤드발칸이 몇발 남았는지 내앞에 디스플레이에 표시가 안됨

(줄여서

문제 보고 · 내 앞에 디스플레이에 표시가 안됨

으로도 주석에 남아 있다). `RightDisplay`는 정면에서 40도 옆에 있다. 머리로 조준하는 중에는 시선을 돌려야 보인다. 그래서 같은 정보를 정면 화면 `SysCheck_Center` (`FrontDisplay`)에도 띄웠다. `BuildSystemCheckDisplay` 에 `out Text vulcanAmmoText` 를 하나 추가하고, `StatusReadout(y=-40)` 바로 아래 `y=-80`에 600×28 크기, 글자 16, 주황색(1, 0.7, 0.3) 텍스트 `VulcanAmmoReadout` 를 만들었다. 주석에는 위치를 정한 근거도 있다. 눈금 링(`BuildTickRing`)은 반지름 230에 중심 `y=10`이므로 아래 가장자리가 약 `y=-220`이고, 그 안쪽에 넉넉히 들어간다.

```
// CockpitHUD.Update()
string stateLabel = headVulcan.State switch
{
    HeadVulcanController.FireReadyState.Empty => "EMPTY",
    HeadVulcanController.FireReadyState.Reloadng => "RELOADING",
    _ => "READY",
};
vulcanAmmoText.text = $"{headVulcan.weaponName} AMMO
{headVulcan.CurrentAmmo:00}/{headVulcan.maxAmmo:00} [{stateLabel}]";
vulcanAmmoText.color = GundamVitals.StatusColor(headVulcan.AmmoFraction);
```

탄이 줄수록 `GundamVitals.StatusColor` 에 따라 색이 바뀐다. 연결 순서에도 신경 썼다.

`hud.vulcanAmmoText` 는 디스플레이를 만들 때 바로 연결하지만, `headVulcan` 은 설정 스크립트 뒷부분에서야 생기므로 `hud.headVulcan = headVulcan` 은 그 시점에 따로 넣었다. 그런데 이렇게 만들어도 실기에서는 글자가 보이지 않았다. 원인은 이 기능이 아니라 디스플레이 쪽에 있었다.

6. 파랑기만 한 디스플레이: 캔버스가 화면 상자 안에 묻혀 있었다

07 커밋의 `CreateWorldCanvas` 주석은 "오래된" 보고 두 개를 함께 해결했다고 적는다.

문제 보고 · 디스플레이가 파랑기만 해 / 글자가 전혀 안 보임

문제 보고 · 디스플레이 3개 있는곳에 발칸 총알 개수가 안나옴

이번에는 Claude가 Editor에서 빌드된 GundamCockpit 씬을 직접 들여다보고 원인을 확인했다. 콕핏 앞 디스플레이 세 개는 `CreatePolarScreen` 이 만든다. 가운데가 $0.42 \times 0.42\text{m}$, 좌우가 $0.2 \times 0.24\text{m} (\pm 40^\circ)$ 인 납작한 큐브다. 화면 큐브의 두께 방향 스케일은 0.02이고, 크기는 큐브의 `localScale` 로 정해진다. 그런데 `CreateWorldCanvas` 는 이 스케일이 걸린 큐브를 anchor로 받아 캔버스를 그 자식으로 넣고, 로컬 $z=-0.015$ 에 두었다.

스케일된 부모 안의 로컬 좌표

큐브의 z 스케일이 0.02이므로 로컬 $z=-0.015$ 는 실제로 $0.015 \times 0.02 = 0.0003\text{m}$, 즉 큐브 중심에서 0.3mm 떨어진 곳이다. 큐브 앞면은 중심에서 두께의 절반인 10mm 앞에 있다. 캔버스는 불투명한 화면 큐브 속에 있었고, 그래서 화면은 파란 판만 보이고 글자는 하나도 안 보였다. 부모의 비균일 스케일도 물려받아 옆 화면의 캔버스는 폭이 3.6cm로 찌그러져 있었다.

해결 방법은 캔버스의 부모를 스케일이 없는 상위 객체로 바꾸는 것이다.

```
Transform panelRoot = anchor.parent; // CreatePolarScreen이 만든 패널 루트(스케일 없음)
if (panelRoot != null)
{
    canvasGo.transform.SetParent(panelRoot, false);
    float frontFaceOffset = anchor.localScale.z * 0.5f + 0.002f; // 앞면 + 2mm
    canvasGo.transform.localPosition = anchor.localPosition
        + anchor.localRotation * new Vector3(0f, 0f, -frontFaceOffset);
    canvasGo.transform.localRotation = anchor.localRotation;
}
else { /* 부모가 없으면 옛 방식 그대로 */ }
```

캔버스는 이제 화면 큐브의 실제 앞면에서 2mm 앞에 큐브와 같은 방향으로 놓인다. `worldScale` 은 처음부터 스케일 없는 부모를 전제로 계산된 값이었다. 가운데 화면은 $760 \times 0.00054 = 0.41\text{m}$ 로 0.42m 화면을, 옆 화면은 $380 \times 0.00047 = 0.18\text{m}$ 로 0.20m 화면을 채운다. 그래서 위치만 고치자 UI가 원래 의도대로 화면을 가득 채웠다. 정면 화면의 탄수 표시, RightDisplay의 WEAPON 화면, 상태 화면이 모두 이 함수로 만들어지므로 한 번에 살아났다.

7. 발칸이 안 보인다, 안 나간다

헤드 발칸(HeadVulcanController)은 "내가 바라보는 방향으로 건담 머리의 헤드발칸을 조준하고, 오른손 엄지를 구부리는 동작으로 발사한다"는 요청으로 만들어져 9월 28일 아침 커밋(04)에 처음 들어왔다. 조준 방향은 HMD(Main Camera)의 forward, 발사 조건은 오른손 엄지 굽힘 정도가 `thumbBendThreshold = 0.6` 이상일 때, 연사 간격은 `fireRate = 0.15` 초, 탄속은 80m/s, 수명은 3초였다. 이날 이 무기에 대한 보고가 네 번 이어졌다.

7-1. 보이지 않는다 (05)

문제 보고 · 헤드발칸이 발사가 되는지 안보임

. 총구는 외부 건담의 머리에 있고 파일럿 카메라에서 약 20m 떨어져 있다. 임시 탄환은 기본 머티리얼을 쓴 0.04 크기 구체였으니 그 거리에서는 거의 안 보이는 회색 점이였다. 첫 처방은 두 가지였다. 밝은 주황색 발광 머티리얼(`MakeEmissiveMat`, 기본색 (0.2, 0.08, 0.01), 발광 (1, 0.55, 0.15))을 `bulletMat` 으로 연결했고, 크기를 `bulletScale = 0.12` 로 키웠다.

7-2. 리스너가 저장되지 않았다 (07)

문제 보고 · 발칸이 엄지를 움직였을때 발사가 되야하는데 발사가 안됨

. 여기서는 씬 파일을 열어 근본 원인을 찾았다. 저장된 `GundamCockpit` 씬에서 `RightHandTracker`의 `XRHandTrackingEvents.jointsUpdated` 리스너가 0개였다.

에디터에서 붙인 `AddListener`는 씬에 남지 않는다

원래 `SubscribeToRightHand` 는 씬을 만드는 에디터 스크립트(`GundamCockpitSetup`) 안에서 `jointsUpdated.AddListener(...)` 를 붙였다. `AddListener` 는 런타임 전용 리스너라 씬 파일에 직렬화되지 않는다. 저장된 씬을 Play하거나 헤드셋 빌드로 실행하면 그 리스너는 없고, 엄지 콜백 `OnRightHandJointsUpdated` 가 한 번도 불리지 않는다. 엄지는 끝까지 "추적 안 됨"이고 `Fire()` 는 절대 실행되지 않는다. `HandJointTracker` 가 멀쩡했던 것은 실행 중에 스스로 구독하기 때문이다.

고친 방법은 이벤트 소스를 직렬화 필드 `rightHandEvents` 로 저장하고, 실제 구독은 런타임 `OnEnable` 에서 하는 것이다. 해제는 `OnDisable` 에서 한다. `SubscribeToRightHand` 는 이제 참조만 저장한다. 필드가 비어 있는 옛 씬에서도 동작하도록 `FindRightHandEvents()` 가 씬을 뒤져 오른손 `HandJointTracker` 옆의 `XRHandTrackingEvents` 를 우선 찾고, 없으면 오른손 이벤트 아무것이나 쓴다. 덕분에 씬을 다시 빌드하지 않아도 동작한다.

7-3. 엄지를 구부려도 안 나간다 (07)

리스너를 고치고 씬을 다시 빌드해 `rightHandEvents` 가 연결된 것까지 확인했지만, 다음 보고는

문제 보고 · 엄지를 구부려도 발사가안됨

이었다. 이번에는 굽힘을 재는 방식이 문제였다. 원래 계산은 엄지 끝과 손바닥 중심의 거리였다. 펴진 기준 거리를 `baseToPalm + thumbReach` (엄지가 손바닥에서 일직선으로 뻗었다는 가정), 접힌 기준을 `baseToPalm × 0.35` 로 잡고 그 사이를 0~1로 환산했다. 그런데 이 "펴진" 기준이 너무 길어서 보통으로 엄지를 구부려도 (끝이 손바닥에서 5~6cm) 값이 0.5 정도에 그쳐 문턱 0.6에 못 미쳤다.

그래서 손 크기나 손바닥 관절 위치에 영향을 받지 않는 두 번째 척도를 더했다. 엄지 관절 사슬이 얼마나 접혔는지를 각도로 잴다.


```

Vector3 seg1 = proximalPose.position - metacarpalPose.position;
Vector3 seg2 = distalPose.position - proximalPose.position;
Vector3 seg3 = tipPose.position - distalPose.position;
float curl = Vector3.Angle(seg1, seg2) + Vector3.Angle(seg2, seg3);
ThumbCurlDegrees = curl;
angleBend = Mathf.Clamp01(Mathf.InverseLerp(straightThumbCurlDegrees, bentThumbCurlDegrees,
curl));
// ...
_thumbBendAmount = Mathf.Max(distanceBend, angleBend); // 둘 중 큰 값

```

곧게 편 엄지는 합계 10~30도, 일부러 구부리면 60도 이상이 나온다. `straightThumbCurlDegrees = 25`, `bentThumbCurlDegrees = 80` 으로 두면 문턱 0.6은 약 58도에 해당한다. 런타임이 metacarpal 관절을 주지 않으면 마지막 관절(IP)의 접힘만 재고 2배 해서 쓴다. 두 척도 중 큰 값을 쓰므로 전에 발사되던 동작은 지금도 발사되고, 문턱값은 바꾸지 않았다.

7-4. 쏘면 보여야 한다: 예고탄, 수렴, 총구 섬광 (07)

문제 보고 · 헤드발칸에서 총알이 안나가는거 같아 총알을 발사하면 내가 볼수있어야함

. 0.12m 구체로 키워어도 여전히 안 보였다. 이건호는 바깥을 직접 보지 않고 면당 512픽셀 큐브맵이 그려진 돔을 통해서만 본다. 그 큐브맵을 찍는 HeadCam은 총구 바로 옆에 있다. 시점에서 정면으로 멀어지는 0.12m 구체는 순식간에 2픽셀 크기가 되고, 그것도 정면으로만 보인다. 그래서 세 가지를 바꿨다.

바꾼 것	필드와 값	효과
예고탄 모양	tracerLength = 4m, tracerWidth = 0.25m (구체 → Cylinder, X축 90도 눕혀 진행 방향으로 정렬, 길이 절반만큼 앞으로 밀어 총구에서 시작)	길고 가는 빔줄기
수렴 조건	convergenceDistance = 80m, maxAimDistance = 300m	좌우 탄이 옆에서 시선 중앙으로 모여드는 게 보임
총구 섬광	muzzleFlashSize = 0.8m, muzzleFlashDuration = 0.06s	발사마다 총구에서 짧게 번쩍임

수렴 조건은 시점 회전과 맞물려 있다. 이건호가 실제로 조준하는 시선은 `mainCamera.forward` 가 아니라 돔이 보여 주는 방향, 즉 `viewController.LookRotation * mainCamera.forward` 이고, 그 시선은 HeadCam에서 출발한다. 이 시선으로 레이캐스트를 쏘서 5m보다 먼 곳에 맞으면 그 거리, 아무것도 안 맞으면 80m 앞을 조준점으로 잡는다. 좌우 총구는 각자 그 점을 향해 쏜다(`DirectionFrom`). `viewController` 가 없는 옛 씬에서는 전처럼 모든 탄이 `mainCamera.forward` 로 나간다. 설정 스크립트는 `suitRoot.GetComponentInChildren<CockpitViewController>(true)` 로 컨트롤러를 찾아 연결하고, 못 찾으면 경고를 남긴다.

"안 쏜 것"과 "쐈는데 안 보이는 것"을 헤드셋 안에서 바로 구분할 수 있도록 진단 표시도 넣었다.

`HeadVulcanController` 에 읽기 전용 상태값 `IsThumbTracked`, `ThumbBend`, `ThumbCurlDegrees`, `LastFireTime` 을 공개했다. 정면 디스플레이의 탄수 표시는 두 줄이 됐고(높이 52, y=-92), 둘째 줄에 `THUMB`

45%/60% 처럼 굽힘 대 문턱을 보여 준다. 엄지가 추적되지 않으면 `THUMB --`, 발사 후 0.2초 동안은 `FIRE!` 를 붙인다.

8. 손가락 버튼 다섯 개: JoystickFingerButtons

8-1. 요청과 첫 번째 버전 (07, 12:26)

조종간 구체에는 이전 요청(

요청 · 손가락별로 버튼이있어서 사용할거야

)으로 이미 자리표시용 버튼 구체 다섯 개가 있었다. 엄지 0.022, 나머지 0.016 크기였고 아무 기능도 연결되지 않았다. 점심 무렵 요청이 왔다.

요청 · 지금 조종기에 있는 버튼 5개를 손가락 끝마디 위치로 정확히 옮겨줘 그리고 엄지 부분에 버튼이 눌렸을때 발칸이 나가게하자

새 컴포넌트 `JoystickFingerButtons` 는 각 조종간의 Base GameObject에 붙는다. 첫 버전은 두 부분으로 되어 있다.

배치. 고정 좌표로는 실제 손에 맞출 수 없으므로 추적된 진짜 손을 쓴다. 조종간을 잡고 `settleTime = 0.3` 초가 지나 주먹이 다 쥐어지면, 각 손가락 끝마디의 가운데(Distal 관절과 Tip의 중점, Distal이 없으면 Tip)를 구체 표면으로 투영해서 그 손가락의 버튼을 옮긴다. 버튼 중심이 표면 위에 오도록 `local.normalized * gripRadius` (반지름 0.05, GripBall 지름 0.10)에 둔다. 다음에 잡을 때까지 버튼은 그 자리에 있다.

누르기. 구체는 가상이라 실제로 밀어낼 표면이 없다. 그래서 손가락 끝마디가 구체 중심 쪽으로 쉬던 자리보다 `pressDepth = 0.015 m(15mm)` 더 들어오면 눌림, 그 절반 아래로 나오면 해제(히스테리시스)로 판정했다. 쉬는 깊이는 눌리지 않은 동안 `restAdaptSeconds = 2` 초 시간상수로 천천히 따라가서, 왼 손을 살짝 고쳐 잡아도 유령 입력이 생기지 않게 했다. 눌린 버튼은 발광 머티리얼(`pressedMaterial`)로 바뀌고 0.8배로 작아진다.

손 데이터는 각 조종간 손의 `HandJointTracker` 옆에 있는 `XRHandTrackingEvents` 를 다시 쓴다. 7-2에서 배운 대로 구독은 런타임 `OnEnable` 에서 한다. 관절 위치는 XR Origin 공간 값이므로 `hand.transform.parent` 로 월드 좌표로 바꾼다.

발사 쪽은 `HeadVulcanController` 에 `fireButtons` 필드가 생겼고, `RightJoystick`의 `JoystickFingerButtons` 가 연결된다. 발사 조건은 다음과 같다.

```
bool buttonFire = fireButtons != null && fireButtons.ThumbPressed;
bool bendFire = (fireOnThumbBend || fireButtons == null)
    && _thumbTracked && _thumbBendAmount >= thumbBendThreshold;
if ((buttonFire || bendFire) && _cooldownTimer <= 0f) { Fire(); /* ... */ }
```

오전에 고친 엄지 굽힘 발사는 지우지 않았다. `fireOnThumbBend` (기본 꺼짐) 뒤로 옮겼고, 버튼이 연결되지 않은 옛 씬에서는 자동으로 이 방식으로 돌아간다. 정면 디스플레이 둘째 줄은 버튼 상태로 바뀌었다. 오른쪽 조종간을 안 잡았으면 `THUMB BTN: GRAB R-STICK`, 잡고 맞추는 중이면 `THUMB BTN: FITTING...`, 맞춘 뒤에는 `THUMB BTN ON 12/15mm` 처럼 눌림 여부와 밀어 넣은 깊이 대 기준 깊이를 mm로 보여 준다.

8-2. 잘못 이해한 우회로: 손바닥을 얹는 조종기와 "네모난 판"

07과 08 사이, 커밋에 남지 않은 짧은 우회가 있었다. 흔적은 `JoystickLever.cs` 의 주석과 08 커밋 설정 스크립트의 주석에 있다. 먼저 이런 요청이 왔다.

요청 · 조종기 버튼 위치가 손을 책상에 올려두었을때 손가락 마디 끝에 있다고 생각하고 두어야해

그리고 이견호가 명시적으로 고른 방식은

요청 · 손바닥 올리면 잡히게

였다. Claude는 이것을 손을 책상에 얹듯 **평평하게** 조종기 위에 올려놓는 방식으로 받아들였다. 손바닥은 돔 위에, 손가락 끝은 버튼 위에 놓는 식이다. 손을 편 상태에서는 `GripAmount/PinchAmount`가 거의 0이라 주먹이나 꼬집기로 잡는 기존 판정으로는 잡을 수 없다. 그래서 `JoystickLever` 에 두 번째 잡기 방법을 더했다. 손바닥이 손잡이 중심에서 `palmRestGrabRadius = 0.11 m` 안에 들어오면 잡힘, `palmRestReleaseRadius = 0.16 m` 밖으로 나가면 놓임이다(히스테리시스). 이때 손가락 끝이 닿을 자리에 버튼을 놓는 판(plate)도 만들어졌던 것으로 보인다.

헤드셋에서 본 결과에 대한 답은 분명했다.

문제 보고 · 네모난 판이 생김 저거지워 그리고 구체 조종기 유지해 구체 조종기를 잡았을때 손가락 끝마디에 버튼이 있는거라고

손을 펴서 얹는 것이 아니었다. 조종기는 주먹으로 쥐는 구체이고, 쥐었을 때 각 손가락 끝마디가 닿는 구체 표면에 버튼이 있어야 했다. 08 커밋에서 판은 사라졌고, 손바닥 얹기 잡기는 코드는 남기되 `allowPalmRestGrab = false` 로 꺼졌다. 커 두면 손이 구체 근처를 지나가기만 해도 잡히기 때문이다. 주석은 이 일을 "the flat-hand idea was a misunderstanding"이라고 적었다.

기본 배치도 "구체를 쥐 손"에 맞게 다시 계산했다. 손바닥은 뒤쪽 위에, 네 손가락 끝은 구체 앞면의 적도 바로 아래에 나란히(검지가 엄지 쪽, 새끼가 가장 아래), 엄지 끝마디는 안쪽 옆에 온다. 오른손은 엄지 쪽이 -X, 왼손은 +X이므로 `handX` 로 좌우를 뒤집는다.

```
float handX = isRight ? 1f : -1f;
const float ballRadius = 0.05f;
// 방향 벡터를 정규화해 구체 표면(반지름 0.05)에 놓는다
thumb : new Vector3(-0.85f * handX, 0.25f, 0.46f).normalized * ballRadius
index : new Vector3(-0.40f * handX, -0.15f, 0.904f).normalized * ballRadius
middle : new Vector3(-0.13f * handX, -0.20f, 0.971f).normalized * ballRadius
ring : new Vector3(0.13f * handX, -0.22f, 0.967f).normalized * ballRadius
pinky : new Vector3(0.36f * handX, -0.30f, 0.883f).normalized * ballRadius
```

(위 코드는 다섯 개의 `CreateSphere` 호출에서 위치 인자만 모은 것이다.) 잡은 뒤에 실제 손가락 위치로 다시 맞추는 기능은 그대로 유지된다.

8-3. 인식 개선: 손을 기준으로 재고, 두 신호를 쓰고, 떨림을 걸러낸다 (08, 12:55)

엄지 버튼 발사는 동작했다. 다음 보고는 정확도에 관한 것이었다.

문제 보고 · 엄지로 나가는거 확인됐는데 인식이 안좋은거 같아

같은 무렵 확장성 요구도 나왔다.

요청 · 조종장치에 손가락 별로 버튼이 다 있어야해 그래야 나중에 다른기능을 추가함

08 커밋 클래스 주석은 첫 버전의 약점을 넷으로 정리했다.

첫째, 손가락 끝과 **구체 중심** 사이 거리를 잴다. 그런데 구체는 손을 좌우·앞뒤(X/Z)로만 따라가고 이동 한계에서 멈추므로, 손 전체를 움직이거나 들어 올리기만 해도 그 거리가 바뀌었다. 둘째, 15mm는 주먹 안에 끼인 엄지가 움직이기에 너무 큰 거리였다. 셋째, 관절 위치는 떨리고, 한 프레임의 잡음이 상태를 뒤집었다. 넷째, 맞추는 순간에 추적되지 않은 손가락은 쉬는 기준값을 못 받아서 그 잡기 동안 내내 놀리지 않았다.

새 방식은 모든 것을 **손 자신을 기준으로**, 같은 관절 샘플 안에서 잰다. 그러면 조종간이나 손 전체의 움직임이 상쇄된다. 신호는 두 가지다.

a) **밀기**: 손가락 끝마디가 **손바닥 중심(Palm 관절)**에 쉬던 때보다 가까워진 거리. 기준 `pressDepth = 0.010 m(10mm)`.

b) **접기**: 손가락 관절 사슬이 쉬던 때보다 더 접힌 각도. 기준 `pressCurlDegrees = 18` 도. 왼 구체의 버튼을 누르는 동작은 대부분 손끝을 까딱이는 것이라, 끝마디가 거의 움직이지 않아도 이 신호로 잡을 수 있다.

```
// 손가락마다 관절 4개: 엄지 Metacarpal-Proximal-Distal-Tip,
// 나머지 Proximal-Intermediate-Distal-Tip
Vector3 padLocal = g2 ? (p2.position + p3.position) * 0.5f : p3.position;
if (gotPalm) _rawPalmDist[i] = Vector3.Distance(padLocal, palm.position);
if (g0 && g1 && g2)
{
    Vector3 s1 = p1.position - p0.position;
    Vector3 s2 = p2.position - p1.position;
    Vector3 s3 = p3.position - p2.position;
    _rawCurl[i] = Vector3.Angle(s1, s2) + Vector3.Angle(s2, s3);
}
```

판정은 둘 중 하나라도 기준을 넘으면 눌림(OR)이고, 해제는 둘 다 절반 아래로 내려와야 한다. 두 신호 모두 `smoothingSeconds = 0.04` 초 저역 필터를 거친다. 쉬는 기준값은 손가락마다 "잡은 뒤 settle이 끝나고 그 손가락이 처음 추적된 프레임"에 따로 잡는다(`_fitted[i]`). 맞추는 순간 하나가 가려져 있어도 나중에 보이면 그때 기준을 받는다. 버튼 위치 재배치도 이 순간 손가락마다 한다(`refitOnGrab`, 기본 켜짐). 쉬는 기준의 재적응은 "확실히 놓인" 상태(두 신호 모두 기준의 30% 미만)에서만 해서, 반쯤 누른 채 오래 있어도

그 상태가 새 기준이 되지 않게 했다. 손가락 추적이 잠깐 끊기면 그 손가락의 상태는 유지한다. 누르고 있던 것을 떨어뜨리지도 않고, 새로 눌림을 만들지도 않는다.

확장성을 위해 손가락마다 `FingerButtonEvents` (UnityEvent `onPressed` / `onReleased`)를 Inspector에 노출했다. `thumbEvents`, `indexEvents`, `middleEvents`, `ringEvents`, `pinkyEvents` 다. C# 이벤트 `ButtonChanged(Finger, bool)` 과 `IsPressed(Finger)` 도 있다. 이 컴포넌트는 두 조종간 모두에 붙어 있으므로, 나중에 어느 손 어느 손가락에든 이 클래스를 고치지 않고 새 기능을 연결할 수 있다. 정면 디스플레이에는 mm와 함께 `14/18deg` 같은 접힘 각도도 표시하게 했다.

8-4. "엄지를 펴도 발칸이 나갈때가 있어": 감도 조정 (10, 13:06)

마지막 보고는 반대 방향, 즉 오발이었다.

문제 보고 · 엄지를 펴도 발칸이 나갈때가 있어 약간 감도 수정이 필요함

원인은 8-3의 OR 판정이었다. 10mm 또는 18도 중 하나만 넘어도 눌렀으므로, 주먹 안에 반쯤 가려진 엄지에서 흔히 생기는 관절 하나의 잡음만으로 발칸이 나갔다. 커밋 10은 판정을 이렇게 바꿨다.

항목	8-3 (12:55)	8-4 (13:06)
판정 방식	밀기 $\geq 10\text{mm}$ 또는 접기 $\geq 18\text{도}$	점수 = 밀기/pressDepth + 접기/pressCurlDegrees $\geq$ pressScore 1.6
기준값	10mm / 18도	12mm / 22도 (조금 더 뻗뻗하게)
퍼는 신호	고려 안 함	둘 중 하나라도 -0.35 점보다 낮으면(펴지거나 손바닥에서 멀어지면) 눌림 불가, 눌러 있으면 즉시 해제
유지 시간	즉시	조건이 pressHoldSeconds 0.06초 동안 유지돼야 눌림
해제	두 신호 모두 절반 아래	점수 $<$ releaseScore 0.6
글리치	필터에 그대로 반영	한 샘플이 필터값에서 3cm 또는 40도 넘게 튀면 무시(연속 2프레임까지, 3프레임째부터는 실제 변화로 받아들임)
필터	0.04초	0.05초
전체 감도	없음	sensitivity 0.3~2 (1이 기본, 낮을수록 세게 눌러야 함)

```
float s = Mathf.Max(0.05f, sensitivity);
float pushPts = push / Mathf.Max(1e-4f, pressDepth / s);
float curlPts = curlPush / Mathf.Max(0.1f, pressCurlDegrees / s);
float score = Mathf.Max(0f, pushPts) + Mathf.Max(0f, curlPts);
// 퍼는 중이면 절대 눌림이 아니다: "엄지를 펴도 발칸이 나갈때가 있어"
bool extending = pushPts < -0.35f || curlPts < -0.35f;
bool downNow = !extending && score >= pressScore;
if (!_pressed[i])
{
    _downTime[i] = downNow ? _downTime[i] + Time.deltaTime : 0f;
    if (_downTime[i] >= pressHoldSeconds) SetPressed(i, true);
}
else if (extending || score < releaseScore) { SetPressed(i, false); _downTime[i] = 0f; }
```

점수 1.6의 의미는 툴팁에 적혀 있다. "완전히 접고 조금 밀기" 또는 "확실히 밀고 조금 접기"처럼 두 신호가 같이 나와야 한다는 뜻이다. 가장 중요한 변경은 **퍼는 신호 거부(extension veto)**다. 엄지를 퍼는 동작에서는 끝마디가 손바닥에서 멀어지거나 관절이 퍼지므로 둘 중 하나가 음수로 크게 나온다. 다른 신호가 아무리 잡음으로 높게 나와도 이 경우는 놀림이 될 수 없다. 글리치 거부는 주먹 속에 가려진 관절이 다시 추정되면서 한 샘플만 크게 튀는 경우를 노린 것이다. 짧은 틱은 건너뛰고, 지속되는 변화는 그대로 받아들인다. 쉬는 기준의 재적응 조건도 점수가 $\text{releaseScore} \times 0.5$ (0.3점) 미만일 때로 바뀌었다. 정면 디스플레이는 SCORE 1.2/1.6 (8mm 15deg) 처럼 점수, 밀기, 접기를 함께 보여 줘서 헤드셋을 쓴 채 감도를 맞출 수 있게 했다. 이 커밋(13:06)이 9월 28일의 마지막 코드 커밋이다.

9. OrbitHUD 링이 적을 붙잡는다: OrbitHUDTargetLock

9-1. 링의 구조

콕핏 정면에는 이전 요청(

요청 · 콕핏에 외관에 구형이 있어서 그 구형에 밖에 보이면서 저런 표식으로 궤도를 알려주는 시스템이 있어야해

)으로 만든 큰 원형 HUD `OrbitHUD` 가 떠 있다. `BuildOrbitHUD` 가 콕핏 내부 기준 (0, 1.45, 2.4)에 반지름 1.5m 링을 만든다. 6도마다 하나씩 60개 자리가 있다. 15개마다 하나(4곳)는 사선 막대 두 개로 된 `OrbitHUD_Gate` 이고, 나머지 56개는 `OrbitHUD_Tick_i` 큐브다(5개마다 긴 눈금 0.16, 나머지 0.09). 이름이 붙은 것은 눈금뿐이라 이견호의 요청은 그중 하나를 짚었다.

요청 · `OrbitHUD_Tick_3` 이거 만들어 둔게 적을 포착하면 적 사이즈로 모여서 줄어들어야해 원형으로 적을 타겟하는거야

08 커밋에서 `BuildOrbitHUD` 가 링 루트 Transform을 돌려주게 바뀌었고, 그 위에 `OrbitHUDTargetLock` 이 붙는다. 이 컴포넌트에는 파일럿 Main Camera, `CockpitViewController`, 돔 Transform, `ringRadius = 1.5`, 잠김 색 머티리얼(붉은 발광, 기본색 (0.15, 0.02, 0.02), 발광 (1, 0.2, 0.15))이 연결된다.

9-2. 원리 ①: 적이 "어디에 보이는가"를 돔을 거꾸로 따라가 구한다

단순히 적의 월드 위치를 카메라로 투영하면 틀린다. 이견호는 세상을 직접 보지 않고 돔에 비친 HeadCam의 큐브맵을 보며, 그 영상은 오른쪽 조종간으로 돌아가 있다. 그래서 `ProjectPoint` 는 돔의 매핑을 거꾸로 따라간다.

```

Vector3 toPoint = worldPoint - ViewOrigin(); // HeadCam에서 대상까지
Vector3 domeDir = Quaternion.Inverse(look) * toPoint.normalized; // 그 월드 방향을 보여 주는 돔 방향
Vector3 local = dome.InverseTransformDirection(domeDir);
Vector3 nObj = Vector3.Scale(dome.lossyScale, local).normalized;
Vector3 shownPoint = dome.TransformPoint(nObj * 0.5f); // 그 방향이 그려진 돔 표면의 점
Vector3 ray = (shownPoint - eye).normalized; // 파일럿 눈 → 그 점
Vector3 n = transform.forward;
float denom = Vector3.Dot(ray, n);
if (denom <= 1e-4f) return false; // 링 평면 반대쪽
float tPlane = Vector3.Dot(transform.position - eye, n) / denom;
Vector3 hl = transform.InverseTransformPoint(eye + ray * tPlane);
onPlane = new Vector2(hl.x, hl.y); // 링 로컬 평면 좌표

```

단계는 이렇다. (1) 시점 원점(HeadCam)에서 대상까지의 월드 방향을 구한다. (2) 돔의 방향 d에는 월드 방향 `LookRotation * d`가 보이므로, 역회전을 걸어 그 월드 방향을 보여 주는 돔 방향을 얻는다. (3) 돔은 비균일 스케일이 걸린 구(지름 1인 프리미티브)이고 표면 법선 방향으로 큐브맵을 샘플링한다. 그래서 $n_{world} \propto R \cdot (n_{obj} / S)$ 이고, 거꾸로 $n_{obj} \propto S \cdot (R^{-1} \cdot d)$ 다. 이 식으로 그 방향이 그려진 돔 표면의 실제 점(반지름 0.5)을 구한다. (4) 이견호의 눈(Main Camera)에서 그 점으로 가는 광선이 링 평면과 만나는 곳이 링 로컬 (x, y)다. 돔이 없으면 눈에서 5m 앞을 대신 쓴다.

9-3. 원리 ②: 겹보기 크기도 같은 매핑으로

적이 얼마나 크게 보이는지도 같은 경로로 켤다. 대상의 렌더러 바운드를 모두 합친 상자에서 중심, 위쪽 끝(중심 + 위 $\times$ extents.y), 옆쪽 끝(중심 + 수평 옆 방향 $\times$ max(extents.x, extents.z))을 각각 `ProjectPoint`로 투영한다. 링 평면에서 중심과 두 끝 사이 거리 중 큰 값에 `sizeMargin = 1.1`을 곱하면 잠긴 원의 반지름이 된다. 돔의 늘어남까지 들어간 값이라 이견호가 돔에서 보는 크기와 맞는다. 거리가 0.5m보다 가깝거나 `maxRange = 500 m`보다 멀면 후보에서 뺀다.

9-4. 원리 ③: 포착, 눈금 모이기, 잠김 색

후보. `rescanInterval = 0.5` 초마다 씬에서 `EnemyMarker` (자쿠 등)를 다시 찾는다. 연습용 표적 `HitTarget` 은 `includeHitTargets = false` 라 기본으로는 빠진다.

포착. 링 평면에 투영한 적 중심이 링 중심에서 `ringRadius  $\times$  captureRadiusFactor(1.0)` 안에 있으면 포착 후보다. 이미 잡은 대상은 `releaseRadiusFactor(1.25)` 까지 놓지 않는다(히스테리시스). 후보가 여러이면 링 중심에 가장 가까운 것을 고르되, 지금 잡은 대상은 거리를 절반으로 쳐서 우선한다. 그래서 대상이 이리저리 바뀌지 않는다.

모이기. 각 눈금은 `Awake` 에서 원래 위치, 스케일, 중심에서의 방향(dir), 머티리얼을 기억한다. 포착 중에는 원의 중심 `_center` 와 반지름 `_radius` 가 `followSharpness = 12` 로 적의 투영 중심과 겹보기 크기(최소 `minLockRadius = 0.08 m`, 최대 링 반지름)를 지속적으로 따라간다. 진행도 `_t` 는 `lockTime = 0.5` 초에 걸쳐 1로, 놓치면 `releaseTime = 0.4` 초에 걸쳐 0으로 간다.


```
float e = _t * _t * (3f - 2f * _t); // smoothstep
float tickScale = Mathf.Lerp(1f, lockedTickScale, e); // 0.5배까지 작아짐
Vector3 locked = new Vector3(_center.x + k.dir.x * _radius,
                             _center.y + k.dir.y * _radius, k.pos.z);
k.t.localPosition = Vector3.Lerp(k.pos, locked, e);
k.t.localScale = k.scale * tickScale;
```

그 결과 링 전체가 각 눈금의 원래 각도를 유지한 채 적의 위치로 미끄러지며 적 크기의 작은 원으로 옴트라들고, 눈금도 절반 크기로 작아진다. 처음 포착할 때는 원을 링 중심과 원래 반지름에서 시작하게 해서 "링이 적에게 모여드는" 모습이 보인다. **잠김 색**. e가 0.98 이상이 되면 모든 눈금과 게이트의 렌더러가 붉은 `lockedMaterial` 로 바뀐다. 적이 링 밖으로 나가거나 사라지면 원래 모양과 색으로 돌아간다. 다른 스크립트가 쓸 수 있게 `CurrentTarget`, `LockProgress`, `IsLocked` (진행도 0.99 이상이고 대상이 있을 때)를 공개했다. 이 컴포넌트는 링의 자식 눈금만 움직이고 바꾸며, 적, 카메라, `MobileSuitRoot`, 시점은 건드리지 않는다.

10. 자쿠에게 체력을: EnemyHealth (14:21 무렵)

오후 2시 21분 무렵 `EnemyHealth.cs` 가 생겼다. 이날은 커밋되지 않았고, 다음 날 오전 커밋 11(9월 29일 11:52)에 자쿠 AI, 탄환 충돌 등과 함께 들어갔다. 여기서는 `EnemyHealth` 와 그 연결부만 다룬다. 요청 원문은 설정 스크립트 주석에 남아 있다.

요청 · 자쿠에게 움직임을 주고 체력을 1000으로 설정해줘 그리고 헤드 발칸에 데미지는 1이야

파괴 뒤 동작은

요청 · 폭발 후 재등장

이라는 짧은 문구로 클래스 주석에 남아 있다.

10-1. 체력과 피해

`maxHealth = 1000` 이다. 피해는 `TakeDamage(int amount, Vector3 hitPoint)` 로 들어온다. 헤드 발칸 탄환 (`HeadVulcanBullet`)이 이 객체의 콜라이더에 닿을 때마다 `HeadVulcanController.damage(1)`만큼 부르고, 양쪽 총구가 동시에 쏘므로 한 번 발사에 최대 2가 들어갈 수 있다. 이미 죽었거나 피해량이 0 이하면 무시하고 `false`를 돌려준다. 체력은 0 아래로 내려가지 않고, 맞은 시각 `LastHitTime` 이 기록되며, `Damaged`, `Died`, `Respawned` C# 이벤트로 다른 스크립트(자쿠 AI 등)에 알린다.

맞을 대상도 필요했다. 설정 스크립트는 자쿠 루트에 세로(Y) `CapsuleCollider` 를 붙인다. 높이는 측정한 모델 높이, 반지름은 그 0.22배, 중심은 높이의 절반이다. 루트 스케일 덕분에 실제로는 약 18m가 된다. 주석에 따르면 콜라이더가 없으면 발칸 탄이 맞을 것이 없었다.

10-2. 폭발과 재등장

```
IEnumerator ExplodeAndRespawn()
{
    Vector3 center = transform.position + Vector3.up * (barHeightAboveRoot * 0.45f);
    SpawnExplosion(center);
    // 불덩이가 기체를 덮을 때까지 잠깐 기다렸다가 숨긴다
    yield return new WaitForSeconds(explosionDuration * 0.25f);
    SetBodyVisible(false);
    yield return new WaitForSeconds(Mathf.Max(0f, respawnDelay));
    transform.SetPositionAndRotation(_spawnPosition, _spawnRotation);
    CurrentHealth = maxHealth;
    IsDead = false;
    SetBodyVisible(true);
    Respawned?.Invoke(this);
}
```

폭발은 발광 구체 `explosionFireballs = 7` 개로 만든다. 첫 번째는 중심에 `explosionSize = 22 m` 크기로, 나머지는 반경 0.35배 안의 임의 위치에 0.35~0.7배 크기로, 0~0.4배 지연을 두고 생긴다. 각 불덩이는 `explosionDuration = 1.2` 초의 70% 동안 처음 30%에서 빠르게 커지고 나머지 동안 작아지며 사라진다. 기체를 숨길 때 GameObject를 끄지 않고 렌더러와 콜라이더만 끈다. 그래야 이 코루틴과 자쿠 AI가 계속 돈다. 콜라이더가 꺼져 있으므로 숨은 동안에는 맞지도 않는다. `respawnDelay = 5` 초 뒤 처음 생성된 위치와 회전으로 돌아오고 체력이 다시 차오른다.

10-3. HP 바

1씩 깎이는 것이 헤드셋에서 보이도록 기체 위에 HP 바를 띄웠다. 바는 기체의 자식이 아니라 별도 루트 객체(<이름>_HPBar)다. 자식으로 넣으면 기체의 렌더러 바운드에 포함되어, 9절의 락온 링이 원 크기를 쥔 때 HP 바까지 적의 몸으로 계산하게 된다. 위치는 매 `LateUpdate` 마다 기체 발밑에서 `barHeightAboveRoot` 만큼 위로 맞춘다. 설정 스크립트는 이 값을 모빌슈트 목표 높이 + 2m, 즉 머리 바로 위로 준다. 바는 수평으로만 회전해 이견호의 건담(`faceTowards`, 없으면 AI의 target, 그것도 없으면 `Camera.main`)을 향한다.

구성은 이렇다. 검은 배경 큐브(폭 10.3, 높이 1.1), 체력 비율만큼 왼쪽부터 채워지는 Fill 큐브(`barWidth = 10`, `barThickness = 0.8`), 바 중심에서 1.7m 위(`barThickness × 0.5 + 1.3`)에 레거시 내장 폰트 `LegacyRuntime.ttf` 로 그린 `TextMesh` 라벨 `HP 997/1000` 이다. 채움 색은 탄수 표시와 같은 `GundamVitals.StatusColor` 를 따른다. 라벨을 못 만들면 경고만 남기고 바만 보여 준다. 바의 머티리얼은 폭발용 발광 머티리얼(기본색 (0.35, 0.12, 0.02), 발광 (1, 0.5, 0.1))을 복제해 색만 바꾼 것이다. 체력이 바뀐 프레임에만 다시 그린다(`_shownHealth` 비교).

락온 링도 같이 고쳤다. `OrbitHUDTargetLock` 의 후보 루프에 "EnemyHealth 가 있고 IsDead 면 건너뛰다"는 세 줄이 추가됐다. 파괴되어 재등장을 기다리는 자쿠는 보이지 않으니 잠글 수도 없어야 하기 때문이다.

정리: 9월 28일의 커밋과 남은 것

커밋(시각)

주요 변경

계기가 된 요청·보고

04 (09:31)	조종간 세 시스템 감사, LateUpdate 회전 고정, 실행 순서 -100, 1:1 추종, XRI는 잡기 신호만, IsChildOf, 조종간별 확인 단계, 옆 이동 추가	"이 세 시스템이 동시에...", "XRI = 잡았는지/놓았는지...", "한쪽씩 설정 할때마다...", "옆으로가 잘 안감..."
05 (11:14)	CockpitViewController와 _ViewRotQ(HeadCamManualLook 대체), 이동 기준을 고정 forward로, 포탑 메시 삭제, 탄수 60(120발), 정면 탄수 표시, 탄환 발광과 0.12 크기	"오른손으로 RightJoystick을 잡고...", "앞으로 밀면 옆으로 가고...", "총알 개수는 120발이야..."
07 (12:26)	이동을 시선 yaw 기준으로, 캔버스가 화면 큐브 속에 묻히던 문제 수정, 발칸 리스너 런타임 구독, 각도 기반 엄지 굽힘, 예광탄·수렴·섬광, JoystickFingerButtons 첫 버전, 엄지 버튼 발사	"움직이는거를 콕핏에서의 시선을 기준으로...", "디스플레이가 파랑기만 해...", "엄지 부분에 버튼이 눌렸을 때 발칸이 나가게하자"
08 (12:55)	네모난 판 제거, 구체 위 기본 배치, 손바닥 엮기 잡기 끄, 손 기준 인식(밀기+접기), 손가락별 이벤트, OrbitHUDTargetLock	"네모난 판이 생김 저거지워...", "인식이 안좋은거 같아", "OrbitHUD_Tick_3 이거 만들어 둬게..."
10 (13:06)	점수 합산, 펄스 신호 거부, 0.06초 유지, 글리치 거부, sensitivity	"엄지를 펴도 발칸이 나갈때가 있어..."
(14:21 무렵, 9/29 커밋 11)	EnemyHealth: 체력 1000, 폭발 후 5초 뒤 재등장, HP 바, 죽은 적은 락온 제외	"자쿠에게 움직임을 주고 체력을 1000으로...", "폭발 후 재등장"

이날 작업에는 반복되는 패턴이 하나 있다. 실패의 원인이 대부분 "코드가 가정한 것"과 "이건호가 헤드셋에서 실제로 보는 것"의 차이였다. 돌린 카메라는 돔에 반영되지 않았다. 이동 기준은 보이지 않는 HeadingYaw를 따랐다. 캔버스는 화면 큐브 안에 숨어 있었다. 탄환은 큐브맵 해상도에서는 점보다 작았다. 에디터에서 붙인 리스너는 씬에 저장되지 않았다. 손가락 깊이는 손이 아니라 움직이는 구체를 기준으로 켜다. 그때마다 Claude는 씬을 열어 확인하거나(리스너 0개, 캔버스 위치), 화면에 진단 수치를 띄우는 방식(THUMB %, mm, deg, SCORE)으로 추측을 측정으로 바꿨다. 락온 링이 돔 매핑을 거꾸로 따라가 "보이는 위치"를 계산하는 것도 같은 원칙이다. 다음 날(9월 29일)은 상하 이동용 T자 레버, 빔 사벨, 자쿠의 반격으로 이어진다.

9월 29일은 이 프로젝트에서 커밋이 가장 촘촘하게 찍힌 날이다. 11시 52분(1b536cc), 12시 48분(953897d), 16시 24분(26e8970), 17시 15분(70313ec), 18시 00분(81e4831)에 다섯 번 커밋이 찍혔다. 전날까지 이 게임은 "조종석에 앉아 레버를 잡고 움직이며, 앞에 있는 자구를 머리 발칸으로 겨눌 수 있는" 정도였다. 자구는 서 있기만 했고 맞아도 아무 일이 없었다. 이날 하루 동안 위아래 이동 레버, 움직이고 반격하는 적, 우주 배경, 무기 세 가지(헤드 발칸·빔 샷벨·빔 라이플), 무기를 고르는 터치 화면, 적의 머신건, 플레이어 체력이 모두 들어왔다. 저녁 6시가 되자 "적을 고르고, 무기를 바꾸고, 맞히고, 맞는" 전투가 처음으로 한 바퀴 돌아가게 되었다.

이 장은 시간 순서대로 다섯 덩어리로 나눈다. 커밋은 작업을 *마친 뒤* 찍히므로, 각 커밋에 들어온 변경은 바로 앞 커밋 이후에 한 작업이다. 그 사이의 순서는 파일 생성 시각(이건호의 PC 기준)으로 짐작할 수 있다. 오전 커밋 하나에 들어온 변경이 가장 많은데, Models/Debris 폴더가 10시 06분 무렵, VerticalTLever.cs 가 10시 18분 무렵, VerticalThrustController.cs 가 10시 57분 무렵에 생겼다. 그 밖의 오전 작업은 파일 시각으로 순서를 가릴 수 없어서, 주제별로 묶어 설명한다.

시간대	커밋	주요 내용
오전 (~11:52)	1b536cc	수직 T자 레버와 상승·하강, 조종관 잡힘 문제, 밖에서 보이는 콕핏 숨기기, 건담·자쿠 크기 맞추기, 자쿠 체력과 전투 기동, 시작 거리, 발칸 재장전 중 발사 금지, 별·먼지와 우주 잔해
점심 (~12:48)	953897d	빔 샷벨 모드 첫 버전(손에 쥐는 전용 스틱), 건담 오른팔 메시, 무기 터치 디스플레이, 터치가 안 되던 원인(XRHandSpace), 디스플레이 축소, 캐노피 테두리선
오후 (~16:24)	26e8970	빔 샷벨을 오른쪽 조종관으로 조작(찌르기·휘두르기), 락온 대상 시선 고정, 손에 쥘 샷벨, 손잡이 레이저 방향, 샷벨 데미지 200
늦은 오후 (~17:15)	70313ec	빔 라이플(양손 파지, 락온 사격, 500 데미지, 15발), HEAD VULCAN 버튼, 팔만 보이는 메시 개선, 몸이 시선 방향으로 도는 건담
저녁 (~18:00)	81e4831	건담 속도 40 m/s, 자쿠 3마리와 자연스러운 움직임, 자쿠 머신건, 플레이어 체력 3000, 라이플 재충전 5초, 락온 지연과 락온 전 사격 빛나감

오전 — 위아래로 나는 레버, 그리고 움직이기 시작한 자쿠

1. 새로운 수직 이동 T자 레버

전날까지 건담은 왼쪽 조종간(LeftJoystick)으로 앞뒤·좌우로만 움직였다. ShipMovementController는 평면 이동만 하는 컴포넌트였고, 자기가 쓰는 Y값을 매 프레임 0으로 맞춰다. 우주에서 싸우는데 위아래로 움직일 수 없는 것이다. 이날 아침 이건호의 첫 요청은

요청 · 새로운 수직 이동 T자 레버

였다. 이 레버를 어디에 둘지에 대해서는

요청 · 플레이어가 콕핏 좌석에 정상적으로 앉았을 때의 어깨 높이를 기준으로 배치

라는 조건이 붙었다. 진행 방식에 대한 지시도 있었다.

요청 · 이번 단계에서는 실제 Gundam의 Y축 이동까지 연결하지 말고

. 레버는 먼저 독립된 입력 장치로 만들어 손으로 잡히고 움직이는 것부터 확인하고, 건담을 실제로 띄우는 일은 다음 단계로 미루라는 뜻이다.

그래서 Claude는 새 파일 `VerticalTLever.cs` 를 기존 코드와 완전히 떨어진 장치로 만들었다. 클래스 주석에는 "JoystickLever, ShipMovementController, WeaponAimFireController, HandJointTracker, XR 리그, 카메라를 건드리지 않는다. 이번 단계는 VerticalInput 값만 만들고, 아직 아무도 그것을 읽지 않는다"라고 적혀 있다. 오브젝트 구조는 다음과 같다.

오브젝트	역할
VerticalTLever	장착부(mount). 원점이 손잡이 가운데이고, 스크립트가 여기 붙는다.
VerticalTLever_Base	손잡이 아래의 고정 블록(5×3×9 cm). 경첩 역할을 하고 움직이지 않는다.
VerticalTLever_Stem	보이기만 하는 막대(굵기 1.8 cm, 길이 16 cm). 매 프레임 Base에서 Handle 쪽으로 다시 겨워지므로, 레버가 앞뒤로 기우는 것처럼 보인다.
VerticalTLever_Handle	움직이는 부분. 장착부의 로컬 Z(앞뒤)로만 미끄러지고, 옆이나 위로는 움직이지 않는다.
VerticalTLever_Crossbar	가로로 누운 T자 손잡이(캡슐, 길이 14 cm, 굵기 3.5 cm). 손으로 위에서 덮어 쥐는 크기이고, 잡기 충돌체가 붙는다.

잡기 규칙 — 왼손만, 그리고 한 손에 하나만

잡기 판정은 기존 조종간과 같은 두 신호를 쓴다. 첫째는 XR Interaction Toolkit이다. Handle에 `XRSimpleInteractable` 을 붙이고, 그 충돌체 목록(`m_Colliders`)에는 Crossbar의 충돌체만 넣었다. 이 인터랙터블은 스스로 아무것도 움직이지 않고 "선택되었다"는 사실만 알려 준다. 선택 이벤트가 오면 `onlyAllowedInteractor`, 즉 왼손의 `Near-Far Interactor` 아래에서 온 것인지 `IsChildOf` 로 확인한다. 다른 인터랙터라면 `CancelInteractableSelection` 으로 즉시 취소한다. 조종간에 쓰던 `HandExclusiveGrabAdapter` 와 같은 규칙이다. 에디터 쪽 `WireVerticalLeverInteractor` 는 XR Origin 아래에서 "Left Hand" → "Near-Far Interactor"를 찾아 이 필드에 넣는다. 찾지 못하면 XRI 경로를 꺼버려서, 다른 인터랙터가 잘못 선택할 여지를 없앴다.

둘째는 손 추적 대체 경로다. 왼손 `HandJointTracker` 가 Crossbar에서 `grabRadius` 0.12 m 안에 있고, 쥐기 정도(Grip)가 0.55 이상이거나 집기(Pinch)가 0.5 이상이면 잡은 것으로 본다. 놓는 기준은 Grip 0.35 미만이면 Pinch 0.3 미만일 때다. 잡는 문턱과 놓는 문턱을 다르게 두어(히스테리시스), 경계에서 잡힘과 놓임이 깜빡이지 않게 했다. 기존 `JoystickLever` 와 같은 숫자다. 거리 판정은 Crossbar의 중심점이 아니라 캡슐의 긴 축 선분까지의 거리로 계산한다. 그래서 가로 막대의 양 끝을 잡아도 잡힌다.

가장 신경 쓴 부분은 "한 손이 두 장치를 동시에 움직이면 안 된다"는 점이다. T자 레버는 왼쪽 조종간 근처에 있다. 왼손이 조종간을 잡은 채로 레버 쪽으로 미끄러지면 두 장치가 함께 움직일 수 있다. 그래서 `blockWhileGrabbed` 배열에 `LeftJoystick` 을 넣고, 그중 하나라도 잡혀 있으면 레버는 XRI 선택이든 손 추적이든 잡히지 않게 했다.

잡는 순간 튀지 않게

레버는 잡는 순간 손이 있는 곳으로 순간이동하면 안 된다. 가로 막대의 왼쪽, 가운데, 오른쪽 어디를 잡든 손잡이는 그 자리에 있어야 하고, 그 뒤로는 손의 움직임만 따라가야 한다. 그래서 잡은 첫 프레임에는 기준점 두 개만 저장하고 손잡이는 움직이지 않는다. 하나는 장착부 로컬 공간에서 손바닥의 Z 위치이고, 다른 하나는 손잡이의 Z 위치다.

```
// 잡은 첫 프레임: 기준점만 저장, 손잡이는 그대로
_grabStartHandZ = hz;
_grabStartHandleZ = handle.localPosition.z;
...
// 이후 프레임: 손이 움직인 만큼만, ±travel 안에서
if (Mathf.Abs(hz - _lastRawHandZ) > maxHandDeltaPerFrame)
    delta = _lastValidDelta; // 한 프레임에 15cm 이상 튀면 추적 오류로 무시
else
{
    delta = hz - _grabStartHandZ;
    _lastValidDelta = delta;
}
float z = Mathf.Clamp(_grabStartHandleZ + delta,
    _handleRestLocal.z - travel, _handleRestLocal.z + travel);
```

움직이는 범위 `travel` 은 0.035 m다. 앞으로 3.5 cm, 뒤로 3.5 cm, 합쳐서 7 cm이고 그 밖으로는 그냥 멈춘다. 출력은 -1(끝까지 당김 = 하강)부터 +1(끝까지 뺄 = 상승)까지이고, 가운데 10%는 데드존(`deadZone` 0.1)이라 0이다. 손을 놓으면 `returnSpeed` 8로 지수적으로 가운데에 돌아가고, 0.5 mm 안으로 들어오면 정확히 중앙에 붙는다. 잡고 있지 않을 때 `VerticalInput` 은 늘 0이다.

어깨 높이에 놓기

"어깨 높이"는 고정된 월드 좌표로 정하지 않고, 실제로 만들어진 좌석에서 계산하게 했다 (`CreateVerticalTLever`). 먼저 좌석 방석(`Seat_Pan`)의 윗면 높이를 구하고, 거기에 성인이 앉았을 때 방석에서 어깨까지의 높이 0.58 m를 더한다. 앞뒤 위치는 기울어진 등받이(`Seat_Back`)의 앞면이 그 높이에서 어디에 있는지 회전까지 반영해 구하고, 거기서 어깨 두께 0.08 m를 앞으로 더한다. 좌우는 어깨 반폭 0.19 m만큼 왼쪽이다. 손잡이는 왼쪽 어깨에서 앞으로 0.34 m, 바깥쪽으로 0.14 m, 아래로 0.02 m 떨어진 곳에 둔다. 주석에 따르면 팔꿈치를 편하게 굽혔을 때 손이 닿는 자리로, 아래팔은 수평, 손목은 곧게 유지된다. 동시에 왼쪽 조종간보다 높고 뒤쪽이며, 왼쪽 디스플레이를 보는 시선도 가리지 않는다.

상수	값	의미
<code>SeatedShoulderAboveSeat</code>	0.58 m	방석 윗면에서 앉은 어깨까지
<code>SeatedShoulderHalfWidth</code>	0.19 m	몸 중심에서 어깨까지
<code>ShoulderDepthFromBackrest</code>	0.08 m	등받이 앞면에서 어깨 관절까지

TLeverReachForward	0.34 m	어깨 앞쪽으로
TLeverOutboard	0.14 m	어깨 바깥쪽으로
TLeverBelowShoulder	0.02 m	어깨보다 살짝 아래

사람마다 앉은키가 다르므로 실행 중 보정도 넣었다(`adaptToPilotHeadHeight`). 머리 추적이 유효해진 뒤 처음 1.5초 동안 조종사의 눈높이를 평균 낸다. 콕핏 기준 0.6~2.2 m 밖의 값은 추적이 아직 시작되지 않은 것으로 보고 버린다. 그 평균에서 눈-어깨 거리 0.27 m를 빼 실제 어깨 높이를 구하고, 레버 높이를 그쪽으로 옮긴다. 다만 좌석 기준 값에서 ± 0.12 m까지만 움직이고, 아무도 잡고 있지 않을 때만 초당 0.25 m로 천천히 옮긴다.

2단계 — 레버를 건담에 연결하다

레버 동작이 확인되자 다음 요청이 왔다.

요청 · 레버를 앞으로 올리면 건담이 위로 날아야 하는데 ... 아래로는 내려와야하고

. 레버의 입력을 실제 건담의 상승과 하강으로 연결하라는 것이다. 이 연결도 기존 이동 코드를 고치지 않고 새 컴포넌트 `VerticalThrustController` 로 했다. 이 컴포넌트는 `ShipMovementController` 와 같은 `MobileSuitRoot` 에 붙어서 같은 Transform에 세로 이동만 *더한다*. 평면 이동은 여전히 `ShipMovementController` 가 맡으므로 두 움직임이 그대로 합쳐진다. 바깥에 보이는 건담(`ExternalGundam`) 과 그 머리에 달린 `HeadCam` 은 이미 `ExternalGundamFollower` 를 통해 `MobileSuitRoot` 의 위치를 따라가고 있었다. 그래서 따로 손대지 않아도 외부 화면이 함께 오르내렸다.

```
float input = lever != null ? Mathf.Clamp(lever.VerticalInput, -1f, 1f) : 0f;
float target = input * maxClimbSpeed;
CurrentVerticalSpeed = Mathf.MoveTowards(CurrentVerticalSpeed, target,
    acceleration * Time.deltaTime); // 12 m/s² - 출발·정지를 부드럽게
Vector3 p = transform.position;
p.y += CurrentVerticalSpeed * Time.deltaTime;
```

최대 상승 속도는 `verticalThrust.maxClimbSpeed = ship.maxMoveSpeed` 로 정했다. 수평 최고 속도와 같은 값이고, 이때는 8 m/s였다. 레버를 가운데로 돌리거나 놓으면 가속도 12 m/s²로 줄어들다가 멈추므로 흘러가는 일이 없다. 고도 제한 필드(`minAltitude / maxAltitude`)도 있지만 기본값은 둘 다 0, 즉 제한 없음이다. 왼쪽 디스플레이의 HEADING 줄 끝에는 `ALT 12m +3.5` 처럼 고도와 세로 속도를 붙였고, 레버를 잡고 있을 때는 `[T +0.6]` 처럼 레버 입력값도 보이게 했다. 줄을 새로 만들지 않고 기존 줄에 붙인 것은 패널을 세 줄로 유지하기 위해서였다.

2. "갑자기 조종관이 안잡힘" — 콕핏을 밖에서 숨기다가 생긴 일

같은 오전에 문제 보고가 하나 있었다.

문제 보고 · 콕핏이 내가 건담 안에서 보이거든 밖에 있는 콕피트가 이거 안보이게해야함

. 이 게임의 구조를 알아야 이해되는 문제다. 조종사는 원점(MobileSuitRoot)에 놓인 콕핏 안에 앉아 있다. 바깥 풍경은 건담 머리 뼈에 붙은 카메라 HeadCam 이 찍어서 콕핏 돔(Cockpit_Dome) 안쪽에 비춘다. 그런데 콕핏은 건담 머리에서 약 20 m 떨어진 곳에 실제 물체로 존재한다. 그래서 HeadCam이 바깥을 비출 때 그 콕핏이 "우주에 떠 있는 물체"로 찍혀 돔에 나타난 것이다.

해법은 건담 건담 자기 몸을 숨기던 방식(GundamBodyLayer = 30)을 거꾸로 적용하는 것이었다. 이름 없는 레이어 29를 CockpitInteriorLayer 로 정하고, 좌석·돔·프레임·디스플레이·조종간·OrbitHUD 등 CockpitInterior 아래의 모든 것을 이 레이어에 넣었다. 그리고 HeadCam의 컬링 마스크에서만 이 레이어를 뺐다. 조종사 자신의 XR 카메라는 이 레이어를 계속 그리므로 콕핏 안은 그대로 보인다.

그런데 이렇게 하자 이견호가 곧바로 다음 문제를 보고했다.

문제 보고 · 갑자기 조종관이 안잡힘

. 원인은 손의 XRI Near-Far Interactor에 있었다. 이 인터랙터의 근거리 탐색기(near caster)는 물리 마스크가 Default 레이어 하나로만 되어 있어서, Default에 있는 잡기 충돌체만 찾는다. 콕핏 전체를 레이어 29로 옮기면서 조종간의 그림 공, 손가락 버튼, 트리거 같은 잡기 충돌체도 함께 옮겨 갔고, 손이 그것들을 찾지 못하게 된 것이다. Claude는 XR 리그 설정은 건드리지 않고, 레이어를 옮기는 함수 쪽을 고쳤다.

```
// XR 인터랙터블에 속한 충돌체는 현재 레이어(Default)를 유지
static void SetLayerRecursivelyExceptColliders(GameObject go, int layer)
{
    bool grabCollider = go.GetComponent<Collider>() != null
        && go.GetComponentInParent<XRBaseInteractable>(true) != null;
    if (!grabCollider) go.layer = layer;
    for (int i = 0; i < go.transform.childCount; i++)
        SetLayerRecursivelyExceptColliders(go.transform.GetChild(i).gameObject, layer);
}
```

이 호출은 씬 빌드의 맨 마지막에 둔다. 그래야 그 전에 CockpitInterior 아래에 만들어진 모든 물체(T자 레버 포함)가 빠짐없이 적용된다. 잡기 충돌체는 대부분 눈에 보이지 않는 부품이어서 Default 레이어에 남아도 HeadCam 화면에 문제가 될 만한 것이 없었다.

3. 건담과 자쿠의 크기 맞추기 — 정점을 직접 재다

다음 보고는 크기였다.

문제 보고 · 건담이랑 자쿠에 사이즈가 같아야해 건담이 너무 큰거같아

. 두 기체 모두 목표 키는 MobileSuitTargetHeight = 18 m였다. 하지만 맞추는 방법이 서로 달랐다. 건담은 9월 22일에 FBX의 "원래 높이" 0.985469를 상수로 박아 넣고 18/0.985469배로 키웠다. 자쿠는 자기 SkinnedMeshRenderer.bounds의 높이로 나눠 키웠다. 빌드된 씬에서 재 보니 건담은 21.0 m, 자쿠는 정확히 18.0 m였다. 하드코딩한 원래 높이가 Unity가 FBX를 실제로 가져오는 방식과 맞지 않았던 것이다.

첫 수정은 건담도 자쿠처럼 "스케일 1에서 렌더러 경계 상자를 재서 18 m로 맞추기"로 바꾸는 것이었다. 그러자 다음 보고가 왔다.

문제 보고 · 지금 크기가 머리하나는 차이 나는데?

이번에는 경계 상자 방식 자체가 문제였다. 코드 주석에 남은 Claude의 진단에 따르면,

`SkinnedMeshRenderer`의 `bounds`는 여유 공간이 붙은 느슨한 상자이고 그 여유가 모델마다 다르다. 스케일 1에서 건담의 상자는 높이 1.150, 자쿠의 상자는 1.253이었다. 실제 정점이 차지하는 높이는 각각 0.9855와 0.9804였다. 그래서 "경계 상자 기준 18 m"는 실제 몸으로 따지면 건담 15.4 m, 자쿠 14.1 m가 되었다. 자쿠가 머리 하나만큼 작아 보인 이유다.

최종 해법은 새 함수 `MeasureMeshHeight` 다. 스킨드 메시를 `BakeMesh`로 굽고 모든 정점을 월드 좌표로 옮긴 뒤, 최저점과 최고점의 차이를 잰다.

```
foreach (SkinnedMeshRenderer smr in root.GetComponentsInChildren<SkinnedMeshRenderer>(true))
{
    Mesh baked = new Mesh();
    smr.BakeMesh(baked, false);
    // BakeMesh 결과에는 이미 월드 스케일이 들어 있으므로 위치·회전만 적용
    Matrix4x4 l2w = Matrix4x4.TRS(smr.transform.position, smr.transform.rotation, Vector3.one);
    foreach (Vector3 v in baked.vertices)
    {
        float y = l2w.MultiplyPoint3x4(v).y;
        if (y < minY) minY = y;
        if (y > maxY) maxY = y;
    }
}
```

함정이 하나 있었다. `BakeMesh`의 결과에는 이미 렌더러의 월드 스케일이 반영되어 있다. 여기에 `localToWorldMatrix`를 곱하면 스케일이 두 번 들어간다. 그래서 위치와 회전만 담은 TRS 행렬을 따로 만들었다. 주석에는 에디터에서 검증한 결과도 적혀 있다. 루트 스케일과 결과가 선형으로 비례해서, 건담이 ×1에서 0.9855, ×10에서 9.855였다. 이제 건담과 자쿠 모두 스케일을 1로 되돌리고, 정점 높이를 잰 뒤 18/측정값으로 키운다. 측정에 실패하면 경고를 남기고 예전 상수로 돌아간다. 이렇게 해서 두 기체의 실제 몸이 모두 18 m가 되었다.

4. 자쿠에게 체력과 움직임을

이어서 적에 대한 요청이 왔다.

요청 · 자쿠에게 움직임을 주고 체력을 1000으로 설정해줘 그리고 헤드 발칸에 데미지는 1이야

· 움직임을 종류는

요청 · 전투 기동

, 체력이 0이 되었을 때는

요청 · 폭발 후 재등장

을 골랐다. 그때까지 자쿠는 `EnemyMarker` 태그만 달린 조각상이었다. 충돌체가 없어서 발칸 탄이 맞을 대상도 없었다.

체력은 `EnemyHealth` 가 맡는다. 이 클래스의 자세한 구조는 다른 장에서 다루므로 여기서는 요점만 적는다. 최대 체력은 1000이다. 0이 되면 구체 7개로 된 폭발(가장 큰 것이 지름 22 m)이 일어나고, 몸의 렌더러와 충돌체가 꺼지며, 5초 뒤 처음 위치에서 체력이 가득 찬 채로 다시 나타난다. 머리 위 2

m(`MobileSuitTargetHeight + 2`)에는 `HP 1000/1000` 숫자가 달린 체력 막대가 떠서 건담 쪽을 향한다. 1씩 깎이는 것을 헤드셋 안에서 직접 볼 수 있게 하려는 것이다. 맞을 수 있도록 자쿠 루트에 세로 캡슐 충돌체도 붙였다. 높이는 측정한 몸 높이 전체, 반지름은 그 22%다. 죽어서 재등장을 기다리는 적은

`OrbitHUDTargetLock` 이 락온 후보에서 뺀다.

발칸 데미지 1은 `HeadVulcanController.damage` 로 넣었다. 탄에는 새 컴포넌트 `HeadVulcanBullet` 을 붙였다. 예광탄 자체에는 충돌체가 없다. 충돌체가 있으면 건담을 밀거나 서로 막을 수 있기 때문이다. 대신 매 프레임 탄의 앞 끝이 지난 구간 전체에 레이를 쏜다(`Physics.RaycastAll`). 80 m/s로 날아도 얇은 대상을 뚫고 지나가는 터널링이 없다. 맞는 대상은 `EnemyHealth` 와 연습용 `HitTarget` 뿐이다. 콕핏 내부 부품 같은 다른 충돌체는 무시하고 통과한다. 두 머리 총구가 한 번에 같이 쏘므로, 한 번 발사에 최대 2발이 맞을 수 있다.

ZakuCombatAI — 전투 기동

움직임은 새 파일 `ZakuCombatAI.cs` 가 맡는다. 처음 설계는 건담과 일정 거리(`preferredDistance` 60 m, 허용 오차 8 m)를 유지하면서 좌우로 옆걸음질하는 것이었다. 이 버전을 본 이견호의 보고는

문제 보고 · 자쿠가 너무 거리유지함

이었다. 항상 같은 거리에서 옆으로만 움직이니 예측하기 쉽고 위협적이지 않았던 것이다. 그래서 `varyDistance` (기본 켜짐)를 넣었다. 기동을 새로 고를 때마다 싸울 거리도 새로 고른다.

기동 종류	확률	목표 거리	비고
돌격(CHARGE)	25%	18~26 m	접근 속도 ×2
후퇴(RETREAT)	15%	95~120 m	멀리 물러남
보통	나머지 60%	30~85 m	범위 안에서 무작위
최소 거리	—	15 m	이보다 가까우면 접근 속도의 1.5배로 강하게 물러남

기동은 1.5~4초마다 다시 고른다. 20% 확률로 옆걸음 없이 멈추고, 나머지는 왼쪽이나 오른쪽을 고른다. 그중 25%는 1.8배 빠른 대시다. 매 프레임 원하는 속도는 "목표를 향한 방향(radial)"과 "그에 수직인 방향(tangent)"을 합쳐 만든다.

```
Vector3 tangent = Vector3.Cross(Vector3.up, radial); // 자쿠가 건담을 볼 때 오른쪽
float wanted = Mathf.Max(minDistance, varyDistance ? CurrentPreferredDistance :
preferredDistance);
float approach = Mathf.Clamp((dist - wanted) / Mathf.Max(0.1f, distanceTolerance), -1f, 1f)
* approachSpeed * _approachMul;
if (dist < minDistance) approach = -approachSpeed * 1.5f; // 너무 가까움 - 강하게 후퇴
Vector3 desired = tangent * (_strafeDir * strafeSpeed * _speedMul) + radial * approach;
Velocity = Vector3.MoveTowards(Velocity, desired, acceleration * dt);
```

옆걸음 속도는 9 m/s, 접근 속도는 11 m/s, 가속도는 8 m/s²다. 가속도를 낮게 둔 것은 무거운 기체처럼 느껴지게 하려는 것이다. 회전은 초당 90°로 항상 건담 쪽을 향한다. 높이는 처음 Y에 고정되어 있다.

자쿠 FBX에는 뼈대는 있지만 애니메이션 클립이 없다. 그래서 걷는 동작은 `LateUpdate` 에서 뼈를 직접 돌려 만드는 절차적 애니메이션이다. 매 프레임 먼저 모든 뼈를 시작 자세(rest pose)로 되돌리고, 그 위에 이번 프레임의 회전을 더한다. 회전이 프레임마다 쌓여 자세가 틀어지는 일을 막기 위해서다. 움직임의 크기는 모두 속도에 비례하고, 멈춰 있을 때는 가벼운 흔들림만 남는다.

항목	값	동작
<code>strideLength</code>	7 m	한 걸음 거리. 위상은 속도/보폭· π 로 진행
<code>legSwingDegrees</code>	28°	허벅지(LeftUpLeg/RightUpLeg)를 진행 방향 축으로 반대 위상으로 흔들
<code>kneeBendDegrees</code>	40°	무릎은 cos 위상의 양수 쪽에서만 굽힘
<code>armSwingDegrees / elbowBendDegrees</code>	18° / 15°	팔은 다리와 반대로 흔들고 팔꿈치는 늘 조금 굽힘
<code>torsoLeanDegrees</code>	6°	진행 방향으로 상체를 기울임. 정지 시 $\sin(1.3t) \cdot 1.2^\circ$ 흔들림
<code>hipBob</code>	0.35 m	걸음마다 엉덩이가 오르내림
<code>hitFlinchDegrees</code>	3°	맞을 때마다 상체가 뒤로 움찔(<code>Damaged</code> 이벤트)

흔드는 축이 몸의 오른쪽이 아니라 `Cross(up, 진행 방향)` 인 점이 핵심이다. 자쿠는 늘 건담을 보고 있지만 실제로는 옆으로 걷는 경우가 많다. 다리를 진행 방향으로 흔들어야 게처럼 미끄러지지 않고 걷는 것처럼 보인다. 체력이 0이면 AI와 걷기 모두 멈추고, 재등장하면 속도를 0으로 초기화한 뒤 기동을 새로 고른다.

시작 거리와 발칸 재장전

요청 · 시작할때 조금 거리가 있어야할거같아

. 자쿠는 Z = 40에 있었고 건담은 Z = 20에 서 있어서, 시작하자마자 20 m 앞에 적이 있었다. 이를 `ZakuSpawnZ = 100`으로 옮겨 80 m에서 시작하게 했다. 시작하면 자쿠가 스스로 교전 거리까지 다가온다.

문제 보고 · 발칸이 재장전될때는 안나가야함

. 원래 `HeadVulcanController` 의 탄약은 HUD 표시용이었다. 주석에 "탄이 떨어져도 엄지를 굽히면 이전처럼 계속 발사된다. 화면의 숫자만 줄어든다"고 일부러 적어 둘 정도였다. 이제 탄약이 실제로 발사를

막도록 바꿨다. 다만 옛 동작으로 돌아갈 수 있게 스위치로 남겼다.

```
// blockFireWhenNotReady = true(기본): EMPTY·RELOADING 중엔 발사 안 함
bool ammoReady = !blockFireWhenNotReady
    || (State == FireReadyState.Ready && CurrentAmmo > 0);
if ((buttonFire || bendFire) && _cooldownTimer <= 0f && ammoReady)
{
    Fire();
}
```

탄약은 60발(두 총구가 동시에 쏘므로 실제 120발)이다. 다 쓰면 0.4초 동안 EMPTY가 표시되고, 이어서 2.5초 동안 RELOADING이 된 뒤 다시 가득 찬다. 이 3초가 채 안 되는 동안에는 이제 탄이 나가지 않는다.

5. 별과 먼지, 그리고 멀리 떠 있는 잔해

움직임을 느끼는 감각에 관한 요청도 있었다.

요청 · 배경에 별들이 더 있어야할거같아 움직이는게 안느껴져서

. 기존 **Starfield**는 시작 지점 주변 25~70 m에 흩어진 큐브 60개가 전부였다. 건담이 조금만 멀리 가면 주변에 스쳐 지나갈 것이 하나도 없으니, 실제로는 움직여도 멈춘 것처럼 보였다. Claude는 기존 **Starfield**는 그대로 두고 **SpaceDustField**를 새로 만들었다. 두 겹으로 되어 있다.

가까운 먼지(시차 효과). 한 변이 40 m인 타일을 $3 \times 3 \times 3$, 즉 27개로 배치해 HeadCam을 둘러싼다. 타일은 월드에 고정되어 있으므로, 건담이 움직이면 먼지 입자가 실제 시차를 갖고 흘러 지나간다. 카메라가 다른 격자 칸으로 넘어가면 27개 타일을 새 칸 주위로 다시 맞춘다. 그래서 어느 방향으로 가도 먼지가 끊기지 않는다. 각 칸의 무늬는 미리 만든 4가지 변형 중 하나다. 무늬는 칸 좌표를 해시해서 고르므로, 같은 칸은 항상 같은 무늬다. 타일을 다시 맞출 때 먼지가 바뀌거나 튀지 않는다.

```
int VariantFor(Vector3Int c)
{
    int h = (c.x * 73856093) ^ (c.y * 19349663) ^ (c.z * 83492791);
    h &= 0x7fffffff;
    return h % _tileMeshes.Length;
}
```

먼 별(하늘). 450~900 m 거리의 구 껍질 위에 별을 흩뿌린 메시 하나를 두고, 이 메시가 카메라 위치를 그대로 따라가게 했다. 하늘처럼 늘 같은 거리에 있으므로, 이동할 때는 움직이지 않고 시점을 돌릴 때만 또렷하게 보인다. 입자 하나는 정점 6개, 삼각형 8개짜리 정팔면체다. 모든 먼이 바깥을 향하도록 감는 순서를 맞췄다. 메시는 실행할 때 만들어지므로 씬 파일에는 저장되지 않는다.

이 먼지 오브젝트는 레이어 28(**SpaceBackdropLayer**)에 두었다. 조종사 자신의 XR 카메라만 이 레이어를 건너뛴다. 콧빛은 불투명한 돔으로 막혀 있어서 원래 보일 일이 없지만, 원점 근처를 떠다니는 먼지가 콧빛 안에 떠 보이는 일을 확실히 막기 위해서다. 바깥 화면을 그리는 HeadCam에는 그대로 보인다.

처음 숫자를 넣고 테스트한 이견호의 반응은 짧았다.

문제 보고 · 너무 많은데?

주석에 바뀐 숫자가 그대로 남아 있다.

필드	처음	"너무 많은데?" 이후
particlesPerTile	220 (×27 타일)	55
farStarCount	2500	700

먼지 입자 크기(0.15~0.45 m)와 먼 별 크기(2.5~6 m)는 그대로 두었다.

우주 잔해 — VARCO 3D 모델을 멀리 배경으로

같은 무렵 이견호는

요청 · 우주에 건물을 넣으려고 하는데 우주에 잔해같은거 거슬리지 않게

라고 요청했고, 배치 방식으로는

요청 · 멀리 배경으로

를 골랐다. `Assets/Models/Debris` 폴더(10시 06분 무렵 생성)에는 VARCO 3D로 만든 콜로니·정거장·전함 파편 모델이 들어 있었다. 에디터 함수 `PlaceSpaceDebris` 는 이 폴더의 모델을 모두 읽어 이름순으로 정렬하고, 전투 구역 중심 (0, 10, 40)을 둘러싸는 6곳에 하나씩 놓는다.

방위각	고도각	거리	크기
35°	12°	560 m	260 m
105°	-16°	650 m	180 m
160°	9°	500 m	150 m
215°	24°	720 m	300 m
275°	-9°	580 m	200 m
330°	-26°	620 m	160 m

각 파편은 시각적 중심에 피벗을 둔 빈 오브젝트 아래에 놓아, 제자리에서 굴러가도록 했다. 크기는 경계 상자의 가장 긴 변이 목표 크기가 되게 맞췄다. 충돌체는 지우고 그림자는 켜다. 머티리얼은 원본을 건드리지 않도록 복사본을 만들고, 기본 색에 0.6을 곱해 어둡게 했다. 배경에 물리나 보이고 튀지 않게 하려는 것이다. 모두 `SpaceBackdropLayer` 에 두었고, 900 m 가까이 떨어진 잔해와 먼 별이 보이도록 HeadCam의 원거리 클립을 2000 m 이상으로 늘렸다.

잔해 묶음 전체에는 `SpaceDebrisField` 가 붙어서, 카메라 이동량의 90%(`followFactor` 0.9)를 따라 움직인다. 0이면 월드에 고정되고 1이면 함께 움직여서 무한히 먼 것처럼 보이는데, 0.9는 몇 킬로미터 떨어진

물체처럼 아주 천천히 흘러가 보이게 한다. 또 아무리 날아가도 잔해에 닿지 않으니, 잔해가 전투 한가운데로 끼어들지 않는다. 각 파편은 자기 무작위 축을 중심으로 초당 0.3~1.5°씩 천천히 돈다.

```
transform.position = _startPos + (viewer.position - _viewerStart) * followFactor;
```

오전 커밋(11:52)에는 이 모든 것이 들어 있었다. 파일 수로 11개, diff로 약 2000줄이다. 바로 다음 작업은 무기였다.

점심 — 빔 샤벨 모드 첫 버전과 무기 터치 화면

11시 55분 무렵 `Models/BeamSaber` 폴더가 생겼다. 빔 샤벨 손잡이 모델(`BeamSaberHilt.fbx`)과 그 기본 색 텍스처다. 요청은

요청 · 빔샤벨 모드

였고, 조작 방식으로

요청 · BEAM SABER 모드에서는 사용자의 오른손에 전용 컨트롤 스틱을 하나 쥐어주자

(선택지로는

요청 · 전용 컨트롤 스틱

)가 정해졌다. 팔에 대해서는

요청 · 건담의 오른팔이 사용자의 손 움직임을 따라가도록

,

요청 · 기존 건담 모델의 팔을 활용

이라고 했다. 무기를 고르는 방식은

요청 · 디스플레이는 실제 XR 손가락 터치 방식으로 구현

이었다.

1. WeaponModeController — 오른손의 주인을 바꾸는 스위치

이 단계의 무기는 BEAM RIFLE(기본)과 BEAM SABER 두 가지다. 다만 BEAM RIFLE은 아직 실제 라이플이 아니라 "기본 조종간 모드"의 이름이었다. 오른쪽 조종간으로 조준하고 시점을 돌리며, 엄지 버튼으로 헤드 발칸을 쏜다. BEAM SABER로 바뀌면 `RightJoystick` 이 오른손을 받지 못하게 막아야 했다. Claude는 조종간 코드를 고치지 않고 런타임에 참조 두 개만 바꿔 끼웠다.

- `rightStick.rightHandTracker` 를 저장해 두고 null로 비운다. 조종간이 손 추적으로 직접 잡는 경로가 끊긴다.
- 조종간 손잡이의 `XRSimpleInteractable` 을 끈다. XRI로 잡는 경로도 끊긴다.

다른 무기를 고르면 저장해 둔 값을 되돌린다. 오른손이 조종간을 쥔 채로 BEAM SABER를 누르면, 손을 놓을 때까지 전환을 미룬다(`_pending`). 쥐고 있는 조종간을 억지로 뺏지 않기 위해서다. 미뤄지는 동안 화면에는 `(RELEASE R-STICK)` 이 표시된다.

2. BeamSaberControlStick — 손에 쥐는 전용 스틱(첫 버전)

BEAM SABER 모드에서만 나타나는 길이 14 cm, 굵기 3.4 cm의 원통 스틱이다. 위에 분홍빛 캡이 달려 있다. 모드가 시작되면 오른손 손바닥 위치에 똑바로 세워져 나타난다. 손이 추적되지 않으면 콕핏 로컬 (0.28, 1.0, 0.05)에 나타난다. 레버와 달리 3차원으로 자유롭게 움직이는 손잡이지만, 잡는 순간 튀지 않는 원칙은 같다. 잡을 때 손바닥 위치 A와 스틱 위치 B, 손바닥 회전과 스틱 회전을 저장한다. 이후로는 $\text{스틱 위치} = B + (\text{손바닥} - A)$, $\text{스틱 회전} = (\text{손바닥 회전} \times \text{잡을 때 손바닥 회전의 역}) \times \text{잡을 때 스틱 회전}$ 으로 계산한다. 놓으면 그 자리에 머문다. 잡기 신호는 오른손 Near-Far Interactor 또는 오른손 추적(반경 0.10 m)이고, 다른 인터랙터의 선택은 취소한다.

3. 건담 팔만 보여 주는 메시

조종사에게는 자기 건담의 몸이 보이지 않는다. 몸 전체가 HeadCam에서 빠진 `GundamBodyLayer` 에 있기 때문이다. 이 상태에서 오른팔만 보이게 하려고, Claude는 `BuildGundamRightArmView` 로 두 번째 `SkinnedMeshRenderer` 를 만들었다. 건담 메시에서 오른팔 뼈(RightShoulder, RightArm, RightForeArm, RightHand)에 주로 붙은 삼각형만 골라 새 메시지를 만들고, 같은 실제 뼈 배열과 bindpose로 스키닝한다. 이 렌더러를 보이는 레이어(0)에 두면, 진짜 뼈를 움직였을 때 진짜 팔 모양이 그대로 따라 움직인다. 메시는 `Assets/Models/Gundam/GundamRightArm_View.asset` 으로 저장해 두고, BEAM SABER 모드에서만 켜다. 이 첫 버전의 삼각형 선택 기준은 "세 정점 모두 가장 큰 가중치를 가진 뼈가 팔 뼈일 것"이었다. 이 기준은 오후에 다시 바뀐다.

빔 샤벨(`BuildBeamSaber`)은 오른손 뼈의 자식으로 붙였다. 손잡이 모델을 1.8 m 길이로 맞추고, 길이 9 m, 반지름 0.28 m의 빔을 달았다. 빔은 바깥의 분홍 발광 원통, 안쪽의 밝은 코어, 끝의 구로 되어 있다. 손 뼈의 `lossyScale`을 역으로 걸어 루트의 1단위가 월드 1 m가 되게 했다.

팔을 움직이는 것은 `BeamSaberArmController` 다. 매 프레임 다음 순서로 계산한다. ① 조종사 오른쪽 어깨 (눈 위치 + (0.19, -0.27, -0.04))에서 스틱까지의 벡터를 콕핏 공간에서 구한다. ② 돔의 시점 회전 (`CockpitViewController.LookRotation`)을 적용해, 조종사가 보는 "앞"이 화면의 앞이 되게 한다. ③ 이를 (건담 팔 길이 / 사람 팔 길이 0.60 m) 비율로 키워 건담 어깨에 더한 점을 손 목표로 삼는다. ④ 2본 IK(two-bone IK)로 위팔과 아래팔을 굽혀 손을 거기에 보낸다. 팔꿈치는 아래·바깥·뒤로 굽는다. ⑤ 샤벨은 스틱의 긴 축 방향을 가리킨다. HeadCam이 화면을 그리기 전에 팔 자세가 정해지도록 실행 순서를 -50으로 앞당겼다.

4. 무기 터치 디스플레이 — 그리고 "디스플레이가 터치가안됨"

무기 선택 버튼은 오른쪽 디스플레이(SysCheck_Right)의 기존 `Weapon_Canvas`에 UI 사각형으로 올렸다. `WeaponTouchPanel`이 그 캔버스에 붙어 양손 검지 끝을 감시한다. 손가락 데이터는 새로 만들지 않고, 이미 `LeftHandTracker`/`RightHandTracker`에 붙어 있던 `XRHandTrackingEvents.jointsUpdated`에 리스너를 하나 더 다는 방식으로 받는다. 이 이벤트는 `HandJointTracker`와 `JoystickFingerButtons`도 쓰고 있어서, 비활성화할 때는 자기 리스너만 떼도록 했다.

누름 판정은 "화면 앞에서 들어오는 손가락"만 받는다.

```
float front = -local.z * scale; // 유리면 앞쪽으로 몇 m
if (front > armDistance) f.armed = true; // 먼저 2cm 이상 앞에 있어야 '장전'
...
if (f.armed && front <= touchDepth // 표면 1cm 안으로 들어오고
    && front > -behindTolerance // 3cm 이상 뒤로 뚫고 나가지 않았고
    && Time.time - _lastPressTime > pressCooldown) // 0.4초 간격
{
    f.armed = false;
    if (weapons != null) weapons.SelectWeapon(b.mode);
}
```

손가락이 버튼 위 5 cm 안으로 오면 버튼이 밝아지고, 눌리면 0.18초 동안 흰색으로 번쩍이며 3 mm 들어간다. 선택된 무기 버튼은 초록색으로 남는다. 디스플레이 뒤로 손을 돌리거나 옆에서 감싸도 눌리지 않는다. 처음부터 앞에 있지 않은 손가락은 장전되지 않기 때문이다.

그런데 이렇게 만든 버튼이 눌리지 않았다.

문제 보고 · 디스플레이가 터치가안됨

. 원인을 찾아 보니 손 좌표계에 관한 오래된 오해가 있었다. XR Hands가 주는 관절 자세는 XR Origin의 추적 공간, 즉 XR Origin 아래 `Camera Offset` 오브젝트(`XROrigin.CameraFloorOffsetObject`) 기준이다. 이 리그는 Device 추적 모드라서 `Camera Offset`이 XR Origin 루트보다 1.36 m 위에 있다. 그런데 `HandJointTracker`는 XR Origin 루트 바로 아래에 붙어 있었다. 그래서 그 `PalmPosition`이 월드에서 1.36 m 낮게 나왔다. 조종간, T자 레버, 손가락 버튼은 손의 움직임(차이값)이나 방향만 쓰기 때문에 지금까지 아무도 이 차이를 알아채지 못했다. 디스플레이 터치처럼 손을 실제 물체의 위치와 비교하는 기능에서 처음 드러난 것이다.

Claude는 `HandJointTracker`를 고치지 않았다. 그것을 고치면 잘 돌아가던 조종간들이 흔들릴 수 있었다. 대신 정적 도우미 클래스 `XRHandSpace`를 새로 만들어, 트래커가 이미 저장해 둔 원시 로컬 자세를 올바른 공간으로 다시 변환하게 했다.

```

public static Transform TrackingSpace(Component c)
{
    XROrigin origin = c.GetComponentInParent<XROrigin>();
    if (origin != null && origin.CameraFloorOffsetObject != null)
        return origin.CameraFloorOffsetObject.transform; // Camera Offset
    if (origin != null) return origin.transform;
    return c.transform.parent;
}
public static Vector3 PalmPosition(HandJointTracker h)
{
    Transform s = TrackingSpace(h);
    return s != null ? s.TransformPoint(h.transform.localPosition) : h.PalmPosition;
}

```

터치 패널의 손가락 끝 변환과 샤벨 스틱의 손바닥 위치·회전이 모두 이 함수를 거치도록 바꿨다. 터치를 조절할 수 있게, 가장 가까운 손가락이 화면에서 몇 mm 떨어져 있는지도 SELECT 줄에 [R 12mm] 처럼 표시했다(`showTouchDebug`).

5. 디스플레이를 반으로 줄이고 조종간 위로

터치가 되자 이번에는 위치가 문제였다.

문제 보고 · 앞에 디스플레이를 더 작게 ... 조종기랑 너무 겹쳐서 누를수가 없어

. 크기 요구는

요청 · 조종기 사이 공간에 들어올 사이즈로

였고, 선택지로

요청 · 절반 크기 + 조종기 위로

를 골랐다. `ShrinkSystemCheckDisplay` 는 먼저 원래 크기의 디스플레이 묶음에서 양옆 화면의 아래 끝 높이를 켤다. 다음으로 두 조종간 장착부에서 가장 높은 렌더러, 즉 그립 공의 위 끝을 켤다. 그런 다음 묶음 전체를 콕핏 원점 기준으로 0.5배로 줄이고, 옆 화면의 아래 끝이 그립 위 끝보다 0.08 m 위에 오도록 올린다. 그립을 쥔 주먹 관절이 걸리지 않을 여유다. 앞뒤로는 가운데 화면이 손이 닿던 $Z = 0.40$ 에 머물도록 맞췄다. 양옆의 눈금 사다리(`SysCheck_TickLadder_L/R`)는 별도 오브젝트라 같은 변환을 따로 적용했다. 이 축소는 모든 것을 만든 뒤 맨 마지막에 하므로, 디스플레이 위에 올라간 HUD 글자, 헤드캠 인셋, 무기 터치 버튼이 함께 줄어든다.

디스플레이가 반으로 줄면 버튼도 반으로 준다. 그래서 버튼은 캔버스 단위 340×72 로 높게 만들어 손가락 크기를 유지했다. 버튼 가장자리 밖으로 1.5 mm의 여유 판정 영역(`hitMargin 0.0015`)도 두었다.

6. 캐노피 테두리선

같은 커밋에 들어온 작은 요청이 하나 더 있다.

요청 · 콕핏 태두리선을 더얇게하고 아주조금 빛나게해줘

, 그리고

문제 보고 · 선이 끈어진거 없이

. 돔 안쪽의 캐노피 프레임은 곡면 원시 도형이 없어서 짧은 직육면체 마디를 이어 만든 선이다. 선이 점선처럼 끊겨 보였던 원인은 이랬다. 예전 마디는 경도선 20개, 적도 24개로 나눈 현(chord)이었고, 그 양 끝이 불투명한 돔 표면 위에 정확히 놓여 있었다. 그래서 이음매 근처마다 돔이 선을 가렸다.

항목	전	후
경도·적도 분할	20 / 24 마디	48 / 48 마디
선 굵기	0.025 m	0.012 m
돔에서 안쪽으로	0 (표면 위)	0.02 m 안쪽
마디 길이	점 사이 거리	거리 + 굵기(이음매에서 겹침)
발광색	(0.08, 0.1, 0.14) — 거의 검정	(0.18, 0.3, 0.42) — 옅은 푸른빛

오후 — 스틱을 버리고 조종관으로 휘두르는 빔 샤벨

12시 48분 커밋과 16시 24분 커밋 사이에는 세 시간 반 남짓의 간격이 있다. 이 커밋에서 점심때 만든 "손에 쥐는 전용 스틱"이 사라진다.

1. 오른쪽 조종관으로 찌르고 휘두르기

요청은 이랬다.

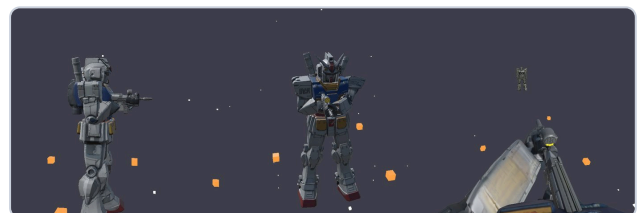
요청 · 빔샤벨 조종기를 빼고 오른쪽 조종관으로 밀고 당기기는 찌르기 옆으로 당기면 옆으로 휘둘러

. 공중에 떠 있는 스틱을 손으로 잡아 휘두르는 대신, 이미 손에 익은 오른쪽 조종간의 기울기로 칼질을 하겠다는 것이다. 에디터에서는 스틱을 만드는 코드 약 40줄을 통째로 지우고

`modes.rightStickDrivesSaber = true` 로 바꿨다. `BeamSaberControlStick` 클래스 파일은 남겨 두었다. 이 플래그를 끄면 예전 방식(조종간 차단 + 손 스틱)으로 돌아간다.



빔 샤벨 손잡이 모델의 방향 맞추기 렌더(세 방향).



양손으로 빔 라이플을 든 건담의 IK 자세 확인 렌더.

이제 BEAM SABER 모드에서도 오른손은 `RightJoystick` 을 그대로 쓴다. 대신 조종간 입력을 원래 받아 쓰던 쪽을 떼어 낸다. `WeaponAimFireController` 를 꺼서 앞으로 찌를 때 총이 나가지 않게 하고, `CockpitViewController.rightJoystick` 참조를 비워 조종간이 시점을 돌리지 않게 한다. 모드를 벗어나면 둘 다 되돌린다. 조종간을 쥔 채로 전환해도 되므로 "놓을 때까지 대기"는 필요 없어졌다.

`BeamSaberArmController` 에는 `rightStick` 필드와 `JoystickPose` 함수가 생겼다. 조종간의 `tiltInput` (x, y 각 -1~1)을 "손 위치"와 "칼날 방향"으로 바꾸는 함수다. 좌표는 조종사가 보는 시야 기준(x 오른쪽, y 위, z 앞)이고, 건담 오른쪽 어깨에서 팔 길이에 대한 비율로 나타낸다.

조종간	자세	손 위치(팔 길이 비율)	칼날 방향
가운데	GUARD 막기	(-0.10, -0.22, 0.55) 가슴 앞	(0, 0.8, 0.6) 위·앞
앞으로 밀 (+Y)	THRUST 찌르기	(-0.22, -0.06, 0.97) 팔을 쪽 뻗음	(-0.05, 0, 1) 정면
뒤로 당김 (-Y)	CHAMBER 장전	(0.10, -0.30, 0.22) 옆구리로 당김	(0, 0.15, 1) 앞(찌를 준비)
옆으로 (X)	SWING 휘두르기	가슴 앞 피벗 (-0.35, -0.12, 0) 중심, 반지름 0.62의 호	수평으로 눕혀 호를 따라 회전

```
// 밀기/당기기: GUARD → THRUST 또는 GUARD → CHAMBER
float py = Mathf.SmoothStep(0f, 1f, Mathf.Abs(y));
grip = Vector3.Lerp(guardGrip, y >= 0f ? thrustGrip : chamberGrip, py);
blade = Vector3.Slerp(guardBlade.normalized, (y >= 0f ? thrustBlade : chamberBlade).normalized, py);
// 옆: 가슴 둘레의 수평 호. 옆 기울기 각도 = 휘두르는 각도
float w = Mathf.SmoothStep(0f, 1f, Mathf.Clamp01(Mathf.Abs(x) / swingFullAt));
float ang = x * swingAngle * Mathf.Deg2Rad; // 끝까지 = ±80°
float radius = swingRadius * (1f + 0.25f * Mathf.Max(0f, y)); // 밀면서 휘두르면 더 멀리
Vector3 swingGrip = swingPivot + new Vector3(Mathf.Sin(ang), 0f, Mathf.Cos(ang)) * radius;
Vector3 swingBlade = new Vector3(Mathf.Sin(ang * 1.15f), 0.05f, Mathf.Cos(ang * 1.15f));
```

옆으로 45%(`swingFullAt`)만 기울여도 칼날은 완전히 수평으로 눕는다. 조종간을 한쪽 끝에서 다른 쪽 끝으로 넘기면 칼이 정면을 가로질러 160°를 베고 지나간다. 자세는 조종간을 계속 따라가므로, 조종간을 빠르게 튕기면 빠르게 벤다. 칼날 방향은 0.04초 시간 상수로 살짝 부드럽게 했다. `SaberRotation` 은 칼날을 +Y로 두고, 주먹 관절(+Z)의 방향을 자연스럽게 정한다. 칼이 서 있을 때는 앞을, 앞이나 옆으로 누우면 아래를 향한다.

2. "건담 손에 빔샤벨을 쥐고있어야"

첫 버전에서는 샤벨을 손 뼈 끝에서 0.55 m 떨어진 "주먹 중심"에 매 프레임 따로 갖다 놓았다. 손 모양과 상관없이 칼이 떠 있는 셈이다.

요청 · 건담 손에 빔샤벨을 쥐고있어야

라는 요구에 맞춰 관계를 뒤집었다. 샤벨은 손 뼈의 자식으로 쥔 주먹을 관통하는 고정 위치에 둔다. 그 위치 `BeamSaberGripInHand` = (-0.24, 0.13, 0.80)은 주석에 "모텔의 주먹 메시에서 쟀" 값이라고 적혀 있다. x는 손바닥·손가락 홈 쪽, y는 주먹 가운데, z는 손목에서 손가락 방향이다. 손잡이 축은 손 뼈의 +Y(검지·엄지 쪽)

라서 칼날이 실제로 쥐었을 때처럼 엄지 위로 나온다. 손잡이 길이도 1.8 m에서 2.4 m로 늘려 주먹 양쪽으로 튀어나오게 했다.

이제 칼 방향을 바꾸려면 손을 돌려야 한다. 원하는 샤�벨 회전에 손-샤�벨 고정 회전의 역을 곱해 손 회전을 구한다. 그 손 회전에서 주먹 오프셋을 빼 손목 목표를 구하고, 2본 IK로 팔을 거기에 맞춘다. 손목만 크게 비틀리면 어색하므로, 손이 아래팔 축을 중심으로 도는 성분(twist)을 쿼터니언에서 분리했다. 그중 절반 (`forearmTwistShare` 0.5)은 아래팔이 맡는다.

3. "손잡이에서 레이저가 이상하게 나오는데"

칼을 손에 쥐어 보니 다른 문제가 보였다.

문제 보고 · 손잡이에서 레이저가 이상하게 나오는데

. 첫 버전은 "손잡이 모델의 긴 축이 로컬 Z"라고 가정하고 X축으로 -90° 만 돌렸다. 그런데 이 모델(VARCO 3D로 생성)의 긴 축은 어느 로컬 축에도 맞지 않고 약 30° 기울어 있었다. 그래서 빔이 손잡이에서 비스듬히 튀어나왔다. Claude는 정점에서 진짜 긴 축을 계산했다. 정점을 최대 3만 개 정도로 줄여 뽑고, 공분산 행렬을 만든 뒤, 거듭제곱법(power iteration)을 64번 돌려 가장 큰 고유벡터, 즉 주축을 구한다.

```
// 공분산의 지배 고유벡터 = 손잡이의 긴 축
Vector3 a = new Vector3(0.3f, 0.5f, 0.8f).normalized;
for (int it = 0; it < 64; it++)
{
    Vector3 n = new Vector3(xx * a.x + xy * a.y + xz * a.z,
                           xy * a.x + yy * a.y + yz * a.z,
                           xz * a.x + yz * a.y + zz * a.z);
    a = n.normalized;
}
```

방향은 두 가지가 가능하므로, 어두운 둥근 방출구가 있는 쪽을 칼날 쪽으로 정했다. 모델 렌더를 확인해 보니 -90° 회전 뒤 방출구가 $-Z$ 를 향했고, 그 기준으로 축의 부호를 정한 뒤 정확히 $+Y$ 에 맞게 돌렸다. 이렇게 곧게 세운 다음에야 $+Y$ 방향 길이로 2.4 m 스케일을 맞추고, 단면 중심을 축 위에 올렸다. 예전에는 기울어진 상태의 경계 상자로 길이를 잴기 때문에 길이도 정확하지 않았다.

4. 빔 샤�벨 데미지 200

요청 · 빔샤�벨은 데미지 일단은 200

. 판정은 새 컴포넌트 `BeamSaberBlade`가 맡는다. 칼날을 따라 8개 점을 찍고 두 가지 검사를 한다. 하나는 각 점에서 반지름 0.58 m(빔 반지름 0.28 + 0.3)의 구 겹침 검사(`OverlapSphereNonAlloc`)다. 다른 하나는 각 점이 지난 프레임 위치에서 지금 위치까지 지나간 경로를 구로 쏘어 보는 검사(`SphereCastAll`)다. 두 프레임 사이에 적을 완전히 지나쳐 버리는 빠른 휘두르기도 맞게 하려는 것이다.

```
if (_lastHit.TryGetValue(hp, out float last) && Time.time - last < hitCooldown) return;
_lastHit[hp] = Time.time;           // 같은 적은 0.5초에 한 번만
hp.TakeDamage(damage, point);       // 200
Spark(point);                       // 지름 3m 섬광, 0.12초
```

칼을 적 몸속에 넣고 있으면 매 프레임 200씩 깎이는 사고가 날 수 있다. 그래서 적마다 마지막으로 맞은 시각을 기록하고 0.5초 동안은 다시 맞지 않게 했다. 다시 때리려면 빼서 다시 휘둘러야 한다. 체력 1000짜리 자쿠는 다섯 번 베면 터진다.

5. "락온된 상대에게 시선이 고정되서 따라가게하자"

조종간을 칼질에 쓰면 그 조종간으로 하던 시점 회전을 할 수 없다. 이건호는 바로 이 점을 짚었다.

요청 · 빔사벨 모드에서는 화면조작을 못하니까 락온된 상대에게 시선이 고정되서 따라가게하자

· `CockpitViewController`에는 자동 시점 모드가 추가되었다. `SetAutoLookTarget(worldPoint)`를 매 프레임 호출하면 조종간 입력 대신 그 점을 향해 Yaw·Pitch를 돌린다. 이때 최대 초당 150°, 수렴 속도 6으로 부드럽게 돌린다. `ClearAutoLook()`을 호출하면 조종간이 다시 시점을 맡는다. Yaw·Pitch 값은 그대로 유지되므로, 시점이 마지막으로 보던 곳에서 이어서 움직인다.

누구를 볼지는 새 컴포넌트 `SaberLookAssist`가 정한다. 실행 순서를 -20으로 두어 시점 컨트롤러보다 먼저 판단한다. 우선순위는 다음과 같다. ① OrbitHUD 링이 지금 락온한 적. ② 지금 락온이 없다면 마지막으로 락온했던 적. ③ 그 적도 없거나 죽었다면 0.5초마다 다시 찾는 가장 가까운 살아 있는 `EnemyMarker`. 시선은 적의 발(피벗)이 아니라 렌더러 경계 상자의 중심을 겨눈다. 그래야 몸 한가운데가 화면 중앙에 온다.

늦은 오후 — 빔 라이플, HEAD VULCAN 버튼, 시선을 따라 도는 몸

16시 37분 무렵 `Models/BeamRifle` 폴더가 생겼다. 요청은 길었다.

요청 · 다운로드에 빔라이플을 다운해두었거든 무기에 빔라이플을 추가하고 무기를 빔라이플로 변경하면 빔라이플을 양손으로 잡는거야 그리고 락온한 상대를 쏘거야 쏘는거는 오른손 엄지

· 수치로는

요청 · 빔 데미지는 500

,

요청 · 빔라이플은 15발이고 재장전까지 쿨타임은 1분이야 충전식이거든

이 주어졌다. 헤드 발칸에 대해서는

요청 · 빔라이플 모드에서는 헤드발칸은 사용안함

이라고 했고, 버튼 구성은 선택지

요청 · HEAD VULCAN 버튼 추가

로 정해졌다.

1. 무기가 세 개가 되다

지금까지 BEAM RIFLE이라는 이름은 "기준 조종간 모드"를 뜻했다. 그런데 진짜 라이플이 생겼으니 그 모드에는 새 이름이 필요했다. `WeaponModeController.Mode`에 `HeadVulcan = 2`를 추가했다. 기존 값 `BeamRifle = 0`, `BeamSaber = 1`은 이미 만들어진 씬과 맞도록 그대로 두었다. 시작 모드(`startMode`)는 HEAD VULCAN이다.

모드	오른쪽 조종간	오른손 엄지	보이는 것
HEAD VULCAN (기본)	조준·시점 회전	헤드 발칸	(예전과 같음)
BEAM RIFLE	시점 회전	빔 라이플 발사(발칸 컴포넌트 꺼짐)	양팔 + 라이플
BEAM SABER	찌르기·휘두르기	헤드 발칸	오른팔 + 샷벨, 시점은 락온 대상 고정

터치 버튼도 세 개가 되었다. 세로로 길게 쌓던 340×72 버튼 두 개를 없애고, 118×118 정사각형 세 개를 가로로 나란히 놓았다. 위치는 $x = -126, 0, 126$ 이고 $y = -138$, 글자 크기는 22다. 주석에 따르면 반으로 줄어든 디스플레이 위에서 한 번 약 2.8 cm 크기다. 글자는 `HEAD\nVULCAN` 처럼 두 줄로 썼다.

2. "일단 건담 팔만 보여야"

샷벨 모드의 팔 메시에서 지적이 나왔다.

문제 보고 · 일단 건담 팔만 보여야

. 점심때 만든 메시에는 `RightShoulder` 뼈에 붙은 삼각형이 포함되어 있었다. 그 뼈의 스키닝은 가슴과 목 둘레 장갑까지 덮고 있어서, 머리 카메라 바로 옆에 가슴 조각이 걸려 보였다. 게다가 "세 정점의 주 뼈가 모두 팔"이라는 선택 기준 때문에 팔과 어깨가 만나는 곳에 들쭉날쭉한 구멍이 났다. 두 가지를 고쳤다.

- 뼈 목록에서 `RightShoulder`를 빼고 `RightArm`, `RightForeArm`, `RightHand`만 남겼다.
- 선택 기준을 가중치 평균으로 바꿨다. 정점마다 네 개의 뼈 가중치 중 팔 뼈에 해당하는 몫을 더하고, 삼각형 세 정점의 평균이 0.6(`ArmViewMinBoneWeight`) 이상이면 남긴다.

메시를 만드는 함수는 `BuildGundamArmView(gundam, 뼈 이름들, 이름, 저장 경로)`로 일반화했다. 같은 함수로 왼팔 메시(`GundamLeftArm_View.asset`)도 만든다.

3. BeamRifleController — 양손으로 쥐는 라이플

라이플 모델은 대략 1 단위 길이라서 5.5배(`BeamRifleScale`)로 키워 5.5 m 라이플로 만들었다. 모델 공간을 정점으로 재 보니, 라이플이 Y축을 따라 서 있고 총구가 -Y, 위쪽이 +Z였다. 그래서 라이플 루트를 "+Z =

총구, +Y = 위, 1단위 = 1 m"로 정하고 `RifleModelToRoot` 회전으로 옮겼다. 주요 지점도 모델 좌표에서 읽어 변환했다.

지점	모델 좌표	용도
개머리(stock)	(0, 0.495, 0.07)	가슴에 붙이는 기준점
총구(muzzle)	(0, -0.50, 0.06)	빔이 나가는 곳
권총 손잡이(grip)	(0, 0.199, -0.130)	오른손
손잡이 바깥 방향	(0, 0.709, -0.706)	엄지 방향 계산
앞쪽 손잡이(fore-end)	(0, -0.15, 0.04)	왼손(손바닥 위로, 엄지는 총열 방향)

자세 계산은 이렇다. 개머리는 오른쪽 어깨 관절에서 시야 기준 (-1.3, -0.6, 0.3) m, 즉 가슴 앞에 붙인다. 총구 방향은 OrbitHUD 링이 락온한 적의 몸 중심을 향하되, 시야 방향에서 50°(`maxAimAngle`)를 넘지 않는다. 락온이 없으면 정면이다. 방향은 0.06초 시간 상수로 부드럽게 따라간다. 오른손 주먹은 권총 손잡이에, 왼손 주먹은 앞쪽 손잡이에 놓는다. 왼손 주먹의 손잡이 위치는 오른손의 `rightFistGrip` (샤벨과 같은 (-0.24, 0.13, 0.80))을 건담 몸의 좌우 대칭면에 비춰 구했다. 두 팔은 샤벨 때와 같은 방식으로 2본 IK를 풀고 비틀림을 아래팔에 나눠 준다. 오른팔은 샤벨용 팔 렌더러와 같은 메시를 쓰는 별도 렌더러 (`GundamRightArm_RifleView`)를 두었다. 샤벨 쪽이 자기 팔 메시를 끄고 켜는 것이 라이플을 든 팔까지 숨기지 않게 하기 위해서다. 실행 순서는 -45로, 매 프레임 오른팔을 시작 자세로 되돌리는 `BeamSaberArmController` (-50)보다 뒤에 와서 라이플 자세가 이기게 했다.

발사는 오른쪽 조종간의 엄지 버튼(`JoystickFingerButtons.ThumbPressed`)이다. 헤드 발칸이 쓰던 그 버튼이고, 그래서 이 모드에서는 `HeadVulcanController` 컴포넌트를 끈다. 버튼을 누르고 있으면 0.8초마다 한 발씩 나간다. 탄(`BeamRifleShot`)은 밝은 코어와 넓은 분홍 발광 캡슐 두 겹으로 된 길이 14 m의 빔이다. 초속 350 m로 날고, 매 프레임 지나간 구간을 반지름 1.0 m 구로 쏘어서 맞았는지 본다. 맞힐 수 있는 대상은 살아 있는 `EnemyHealth`와 `HitTarget` 뿐이다. 콕핏 부품, 자기 건담, 잔해는 통과한다. 맞으면 500 데미지와 지름 6 m의 섬광을 남기고 사라진다. 4초 동안 아무것도 못 맞히면 저절로 사라진다. 락온 상태에서는 총구에서 목표 중심을 향해 곧게 쏘는데, 목표가 총열 방향에서 15°를 넘게 벗어나 있으면 총열 방향으로 나간다.

에너지는 15발이다. 마지막 발을 쏘면 60초 동안 충전(RECHARGING)되고 다시 가득 찬다. 다른 무기를 들고 있어도 충전은 계속된다. 오른쪽 디스플레이의 무기 HUD에는 `ENERGY 15/15 LOCK`, `RECHARGING 42s`, `READY / FIRE`가 표시된다. 체력 1000짜리 자쿠는 라이플 두 발이면 터진다.

4. "건담이 내가 바라보는 방향으로 항상 몸이 돌아야함"

라이플이 시선 방향 기준 50° 안에서만 돌아가니, 몸이 시선을 따라오지 않으면 뒤를 볼 때 문제가 생긴다. 실제로 이 무렵 요청은

요청 · 건담이 내가 바라보는 방향으로 항상 몸이 돌아야함

이었다. 그때까지 `ExternalGundamFollower` 는 "위치만, 회전은 절대 건드리지 않음"을 원칙으로 하는 스크립트였다. 여기에 몸의 Yaw 회전 하나만 예외로 추가했다. 목표 각도는 `CockpitViewController.Yaw` 다. 조종간으로 돌린 시점이든, BEAM SABER에서 락온 대상을 따라간 시점이든 상관없다.

그냥 몸을 돌리면 머리에 달린 HeadCam이 원을 그리며 움직인다. 조종사의 360° 시야 전체가 함께 흔들리는 것이다. 그래서 회전을 몸 원점이 아니라 머리 뼈(`turnPivot`)를 지나는 세로축으로 잡았다.

```
float step = Mathf.DeltaAngle(_bodyYaw, wantYaw) * k;           // k = 1 - exp(-10*dt)
_bodyYaw += Mathf.Clamp(step, -maxStep, maxStep);              // 최대 360°/s
Quaternion rot = Quaternion.AngleAxis(_bodyYaw, Vector3.up) * _startRotation;
Vector3 pivotWorld = basePos + _startRotation * _pivotLocal;    // 돌지 않았을 때 머리 위치
transform.rotation = rot;
transform.position = pivotWorld - rot * _pivotLocal;             // 머리는 제자리, 몸만 돌
```

이렇게 하면 HeadCam은 그대로이고 그 아래에서 몸(과 BEAM SABER·RIFLE의 팔)만 돈다. 이 스크립트는 실행 순서 -60으로 가장 먼저 돌게 했다. 팔 IK(-50, -45)와 머리 카메라가 이번 프레임의 몸 방향을 보고 계산해야 하기 때문이다.

저녁 — 반격하는 자쿠 셋, 그리고 깎이는 내 체력

마지막 커밋(18:00)은 전투를 양방향으로 만든 커밋이다. 17시 53분 무렵 `Models/ZakuMachineGun` , 17시 57분 무렵 `PlayerHealth.cs` 가 생겼다.

1. 건담 속도를 높이다

요청 · 건담에 스피드를 더 빠르게 해줘 어느정도 미는정도에 따라서

· `ShipMovementController` 는 이미 왼쪽 조종간을 민 정도(데드존 이후)에 비례해 속도를 내고 있었다. 그래서 코드는 바꾸지 않고 최고 속도만 새 상수 `ShipTopSpeed` 로 8 m/s에서 40 m/s로 올렸다. 살짝 밀면 여전히 느리고, 끝까지 밀면 초속 40 m다. 에디터 코드에서 `verticalThrust.maxClimbSpeed = ship.maxMoveSpeed` 줄이 이 설정 뒤에 실행되므로, 오전에 만든 T자 레버의 최대 상승·하강 속도도 함께 40 m/s가 된다.

2. 자쿠 세 마리, 그리고 더 자연스러운 움직임

요청 · 자쿠에 움직임을 조금더 자연스럽게 해주고 3마리정도

, 그리고

요청 · 자쿠 ... 3마리정도 나오게 해줘 다른 곳에

· `PlaceZakuEnemy` 를 이름과 시작 위치를 받는 함수로 바꾸고 세 번 호출했다. 각 자쿠는 처음부터 건담을 바라보게 회전시킨다. 셋이 서로 다른 방향에 있으니 조종사는 고개를 돌려 찾아야 한다.

이름	시작 위치	건담(Z=20)에서
ZakuEnemy	(0, 0, 100)	정면 80 m
ZakuEnemy_2	(-85, 0, 75)	왼쪽 앞 약 100 m
ZakuEnemy_3	(95, 0, -10)	오른쪽 약 100 m

ZakuCombatAI에는 "Natural motion" 항목이 새로 붙었다. 이전 버전에서 딱딱해 보였던 부분을 하나씩 바꿨다.

- **속도 변화:** 일정한 가속도(MoveTowards)로 바뀌던 속도를 임계 감쇠 방식(Vector3.SmoothDamp, 0.8초)으로 바꿨다. 속도가 서서히 붙고 서서히 줄어든다.
- **옆걸음 방향 전환:** 왼쪽에서 오른쪽으로 바로 뒤집지 않는다. 1.2초(strafeReverseTime)에 걸쳐 멈춘 뒤 반대로 간다.
- **접근 곡선:** 거리 오차를 -1~1로 자르던 직선 대신 tanh를 써서, 원하는 거리에 가까워질수록 부드럽게 느려진다.
- **개체별 흔들림(wander):** 자쿠마다 다른 시드로 펄린 노이즈(진폭 0.35, 주파수 0.22)를 만들어 옆걸음과 접근 속도에 섞는다. 두 자쿠가, 그리고 같은 자쿠의 두 번의 움직임이 똑같지 않다.
- **서로 떨어지기:** 정적 목록 s_all로 모든 자쿠를 알고 있다. 40 m 안에 있는 다른 자쿠에게서 거리에 반비례하는 힘(최대 12)으로 밀려난다.
- **몸의 방향:** SmoothDampAngle (0.45초)로 돌고, 옆걸음 속도에 따라 진행 방향으로 최대 25° 몸을 튼다. 상체는 그 틀어진 각도의 70%만큼 건담 쪽으로 되돌리고, 머리는 남은 각도를 ±45° 안에서 맡아 건담을 계속 본다.
- **걸음의 무게 이동:** 걸음마다 엉덩이를 디딘 발 쪽으로 4° 굴린다.
- **엇갈림:** 첫 기동 타이머에 0.3~1을 곱해 세 마리가 같은 순간에 방향을 바꾸지 않게 했다.

```
Vector3 desired = tangent * ((_strafeCur + wStrafe) * strafeSpeed)
                + radial * (approach + wApproach * approachSpeed * 0.5f)
                + Separation();
desired.y = 0f;
Velocity = Vector3.SmoothDamp(Velocity, desired, ref _velRef,
                             Mathf.Max(0.05f, velocitySmoothTime));
```

3. 자쿠 머신건

이어서 자쿠가 반격하게 해 달라는 요청이 왔다.

요청 · 자쿠의 머신건을 다운로드 했어 이거를 자쿠손에 들려주고 사격을 하게 해줘 자쿠 머신건에 데미지는 50이야 탄환은 100발이야 재장전속도는 5초야 분당 280발에 속도로 총알이 나가

이 모델은 다행히 기본 방향이 이미 총구 +Z, 드럼 탄창 +Y였다. 그래서 FBX의 원래 회전을 그대로 쓰고, 9배로 키워 18 m 자쿠에 맞는 9 m 총으로 만들었다. 손잡이 (0, -0.113, -0.170)와 총구 (0, -0.015, 0.499)는 모델 좌표에서 읽었다. 권총 손잡이가 뒤로 기울어 있어서 엄지 방향은 (0, 1, 0.45)로 잡았다. 총을 쏠 자쿠 오른손 주먹 안의 위치(`ZakuFistGripUnscaled = (-0.005, 0.0045, 0.048)`)는 자쿠 주먹 메시에서 잤다. 총은 자쿠의 자식이라서 자쿠가 파괴되면 `EnemyHealth`가 몸과 함께 숨긴다.

사격 자세는 `ZakuMachineGun`이 만든다. 실행 순서는 100이라 `ZakuCombatAI`의 걷기 동작 위에 얹힌다. 건담이 260 m(`engageRange`) 안에 들어오면 0.5초에 걸쳐 총을 든다. 자쿠 오른팔의 2본 IK로, 어깨에서 조준 방향을 따라 팔 길이의 72% 앞, 12% 아래에 주먹을 놓는다. 조준점은 건담 발밑에서 10.8 m(18 m × 0.6) 위, 가슴 높이다. 사거리 밖으로 나가면 총을 내린다. 그 상태에서도 총은 매 프레임 손 위치에서 다시 계산해 주먹에 붙어 있으므로, 걸을 때 팔과 함께 흔들린다.



우주에서의 빔 라이플 사격 테스트. 분홍색 빔이 락온한 적을 향한다.

```
if (Reloading || gun == null) return;
if (Time.time < _burstResume || Time.time < _nextShot) return;
if (Vector3.Angle(gun.forward, to) > fireCone) return; // 총열이 10° 안에 들어와야
SpawnBullet(muzzle, dir); // 무작위 산포 최대 1.4°
_nextShot = Time.time + 60f / roundsPerMinute; // 280rpm → 약 0.214초 간격
if (--Ammo <= 0) { Ammo = 0; Reloading = true; _reloadEnd = Time.time + reloadTime; }
if (--_burstLeft <= 0)
{
    _burstLeft = Random.Range(burstRounds.x, burstRounds.y + 1); // 8~22발
    _burstResume = Time.time + Random.Range(burstPause.x, burstPause.y); // 0.8~2.4초
}
}
```

항목	값
데미지 / 탄창 / 재장전	50 / 100발 / 5초
연사 속도	분당 280발(발 간격 약 0.214초)
점사	8~22발 쏘고 0.8~2.4초 쉬
탄속 / 산포 / 수명	320 m/s / 최대 1.4° / 2.5초
예광탄	길이 5 m, 굵기 0.35 m, 주황 발광. 총구 섬광 1.6 m, 0.05초

산포를 둔 이유는 주석에 적혀 있다. 레이저처럼 정확하지 않게 해서, 움직이는 건담은 상당 부분을 피할 수 있게 하려는 것이다. 사격 자세는 이 시점에는 "오른손으로 총을 들어 겨누는" 한 팔 자세였다. 두 손으로 총을 받치고 몸을 비튼 본격적인 사격 자세는 다음 커밋에서 다시 다듬어진다(다음 장).

4. 플레이어 체력 3000

자쿠 탄이 맞을 대상도 필요했다.

요청 · 건담에 체력은 3000이고

. 새 컴포넌트 `PlayerHealth` 를 바깥 건담(`ExternalGundam`)에 붙였다. 여기서 Claude는 충돌체를 일부러 추가하지 않았다. 건담에 충돌체를 붙이면 자기 무기의 레이와 탄, XR 손 잡기, 콕핏 물리에 영향이 갈 수 있기 때문이다. 대신 몸을 "발밑 1 m에서 17 m(18 - 1)까지, 반지름 3.2 m의 세로 캡슐"이라는 수학적 부피로만 정의했다. 자쿠 탄(`ZakuBullet`)이 매 프레임 자기가 지난 선분과 이 캡슐 축 사이의 최단 거리를 직접 계산한다(`CapsuleHit`, 선분-선분 최근접점). 물리 엔진을 거치지 않으니 터널링도 없고, 다른 물체에 영향도 주지 않는다.

```
public bool TakeDamage(int amount, Vector3 point)
{
    if (IsDown || amount <= 0) return false;
    CurrentHealth = Mathf.Max(0, CurrentHealth - amount);
    SyncVitals(); // 콕핏 HUD의 공유 Vitals로 바로 반영
    if (CurrentHealth <= 0)
    {
        IsDown = true;
        _repairAt = Time.time + repairDelay; // 5초 뒤 완전 수리
    }
    return true;
}
```

체력은 `CockpitHUDManager.Vitals` (HP/MaxHP)에 그대로 넣었다. 그래서 STATUS 디스플레이의 HP·부위 게이지와 정면의 NORMAL / WARNING / CRITICAL 표시가 HUD 코드를 고치지 않아도 피해를 반영한다. 머신건 한 발이 50이므로 60발을 맞으면 0이 된다. 0이 되면 5초 동안 행동 불능 상태가 되어 더 맞지 않고, 그 뒤 전부 회복된다. 지금 단계에서는 게임 오버가 아니라 일시적으로 쓰러지는 처리다.

5. 라이프 다듬기 — 5초 충전, 느린 락온, 락온 전 사격은 빗나감

라이프는 이 커밋에서 두 번 더 손을 봤다. 먼저

요청 · 재장전 5초로 변경하자

. 60초 충전은 실제로 써 보니 너무 길었던 것이다. `rechargeTime` 이 5초가 되었고, 에디터에서도 `rc.maxShots = 15; rc.rechargeTime = 5f;` 를 명시해 넣었다.

다음은

요청 · 빔라이플을 들고있을때 락온이 되는 속도를 줄여줘 락온전에 쏘면 빗나가게

. 한 발 500짜리 무기가 링 안에 들어오자마자 0.5초 만에 락온되면 너무 쉽다. 라이플 모드가 켜지면 `OrbitHUDTargetLock.lockTime` 의 원래 값(0.5초)을 저장해 두고 2초(`rifleLockTime`)로 바꾼다. 다른 무기로 바꾸면 원래 값으로 돌린다. 락온이 완료되기 전에 쏘면 일부러 빗나가게 했다.

```
if (tgt == null)    // 아직 락온 완료 전
{
    if (acquiring != null && ...)    // 링이 잡아 가는 중인 적이 있으면
    {
        Bounds b = TargetBounds(acquiring);
        Vector3 side = Vector3.ProjectOnPlane(Random.onUnitSphere, (b.center -
muzzle).normalized);
        Vector3 missPoint = b.center + side.normalized * (b.extents.magnitude * 1.2f + 4f);
        dir = (missPoint - muzzle).normalized;
        ignore = acqHp;                // 그 적은 이 빔에 맞지 않음
    }
    else    // 잡아 가는 적도 없으면 총열에서 2.5~6° 흩뿌림
}
```

잡아 가는 중인 적이 있으면, 빔을 그 적의 경계 상자 밖(크기 $\times 1.2 + 4$ m)의 무작위 옆 지점으로 보낸다. 그리고 그 적을 `BeamRifleShot.ignore` 에 넣어, 빗맞힌 빔이 스치면서 판정이 걸리는 일도 막는다. 락온이 끝나야만 맞는다. HUD에는 진행률이 ENERGY 14/15 LOCKING 63% 처럼 표시되고, 완료되면 LOCK 으로 바뀐다.

정리 — 하루 동안 생긴 것들

이날 새로 생긴 스크립트는 19개다. 오전에 `VerticalTLever`, `VerticalThrustController`, `ZakuCombatAI`, `EnemyHealth`, `HeadVulcanBullet`, `SpaceDustField`, `SpaceDebrisField`, 점심에 `WeaponModeController`, `WeaponTouchPanel`, `XRHandSpace`, `BeamSaberControlStick`, `BeamSaberArmController`, 오후에 `BeamSaberBlade`, `SaberLookAssist`, 늦은 오후에 `BeamRifleController`, `BeamRifleShot`, 저녁에 `PlayerHealth`, `ZakuMachineGun`, `ZakuBullet` 가 생겼다. 그중 `BeamSaberControlStick` 은 같은 날 오후에 쓰이지 않게 되었지만, 파일과 되돌리는 플래그 (`rightStickDrivesSaber`)는 남았다.

하루의 작업을 돌아보면 몇 가지 원칙이 반복된다.

이날 반복된 설계 원칙

첫째, 기존 코드는 고치지 않고 옆에 붙인다. 세로 이동은 `ShipMovementController` 옆의

`VerticalThrustController` 로, 손 좌표 문제는 `HandJointTracker` 를 고치지 않고 `XRHandSpace` 로, 무기 모드는 조종간 코드를 고치지 않고 참조 몇 개를 바꿔 끼우는 방식으로 풀었다.

둘째, 잡는 순간에는 기준점만 저장한다. T자 레버와 샷벨 스틱 모두 손이 움직인 양만 따라가고, 한 프레임에 15 cm 넘게 튀면 추적 오류로 무시한다.

셋째, 빠른 물체의 판정은 지나간 구간 전체로 한다. 발칸 탄의 레이 스위프, 샷벨의 구 스위프, 라이플 빔의 `SphereCast`, 자쿠 탄의 선분-캡슐 계산이 모두 같은 생각이다.

넷째, 재지 않은 숫자를 믿지 않는다. 건담 크기는 하드코딩 값에서 경계 상자로, 다시 정점 측정으로 두 번 바뀌었다.

샷벨 손잡이는 축을 가정했다가 주축을 계산하는 방식으로 바뀌었다. 손 위치는 1.36 m 차이를 찾아낸 뒤에야 맞았다.

다음 날로 넘어간 것

저녁 커밋의 자쿠는 아직 한 손으로 총을 들고, T자 레버는 오전에 정한 위치 그대로이며, 우주는 별과 먼 잔해뿐이다.

두 손 사격 자세, 레버 위치 조정, 콜로니 배경은 9월 30일 작업에서 이어진다.

저녁 6시의 상태를 정리하면 이렇다. 이건호는 콕핏에 앉아 왼손으로 조종간과 T자 레버를 오가며 최대 초속 40 m로 3차원 공간을 날아다닌다. 오른쪽 디스플레이를 손가락으로 눌러 헤드 발칸, 빔 라이플, 빔 샷벨을 고른다. 세 방향에서 다가오고 물러나는 자쿠 세 마리를 락온해 쏘거나 벤다. 그동안 자쿠들은 머신건을 점사로 쏘아 건담의 체력 3000을 깎는다. "전투의 틀"이 이날 처음으로 완성되었다.

9월 30일은 이 프로젝트에서 "무대"가 생긴 날이다. 전날까지 건담은 별과 먼지, 멀리 떠 있는 잔해 몇 조각뿐인 빈 우주에서 자쿠 세 기와 싸웠다. 이날 이건호는 그 빈 공간에 건담이 들어가 싸울 수 있는 거대한 구조물을 요청했고, Claude는 그것을 지름 6.4km, 길이 12km짜리 실물 크기 스페이스 콜로니로 만들었다. 콜로니 안에는 수백 동의 빌딩이 선 도심과 고가도로, 교외와 농지가 있고, 바깥에는 선체 패널과 거울, 항구 크레인이 있으며, 가까운 쪽 끝판에는 전쟁으로 뚫린 거대한 파공(破孔)이 입구 노릇을 한다. 그리고 그 안에서는 중력이 작용하고, 건물에 부딪히고, 건물 뒤에 숨으면 자쿠가 나를 보지 못한다.

이날 작업은 두 커밋에 담겼다. 16번 커밋(13:14, 9a3baa5)은 오전 작업 전부를 담은 3,300줄 가까운 diff로, 콜로니 생성기 본체와 ColonyStructure · ColonyAtmosphere · SpawnSelectPanel · SuitCollision 네 개의 새 스크립트, 그리고 콕핏 원거리 배치·T레버 이동·빔사벨 락온 전환·자쿠 사격 자세가 함께 들어 있다. 17번 커밋(14:15, e832d01)은 그 위에 얹은 "전장 다듬기"로, 총알이 건물에서 멈추게 하고 자쿠가 건물 뒤의 건담을 모르게 하고 자쿠를 열 기로 늘리고 화질과 디스플레이 깜빡임을 고쳤다. 그리고 14:39 무렵부터는 커밋 없이 최적화 작업이 이어졌는데, 이 장 마지막 절에서 최신 코드를 근거로 따로 다룬다.

사용자 PC의 파일 생성 시각도 그날의 흐름을 어느 정도 보여 준다. 콜로니 메시지를 저장하는 Assets/Models/Colony 폴더가 10:50 무렵, 우주 먼지와 잔해 스크립트(SpaceDustField, SpaceDebrisField)가 다시 쓰인 것이 11:08 무렵, VARCO로 만든 건물 모델 폴더 Models/ColonyBuildings 가 11:37 무렵, ColonyAtmosphere 가 12:11 무렵, 외관 소품 폴더 Models/ColonyProps 가 12:20 무렵, ColonyStructure 가 13:20 무렵(17번 커밋에서 다시 쓰인 시각일 가능성이 높다), 최적화용 HeadCamInsetCube.shader 가 14:39 무렵, 축소 메시 폴더 Models/Optimized 가 14:42 무렵이다. 다만 생성 시각은 파일을 다시 쓸 때 바뀔 수 있으므로 "이 무렵" 정도로만 읽어야 한다.

1. "우주공간에 거대한 구조물이 있으면 좋을거같아"

요청의 연쇄

콜로니는 한 번의 요청으로 정해진 것이 아니다. BuildColony 위에 붙은 긴 주석은 이건호의 요청을 순서대로 기록하고 있는데, 그 흐름이 곧 설계가 커져 간 과정이다. 처음 요청은

요청 · 우주공간에 거대한 구조물이 있으면 좋을거같아 건담이 들어갈수있는 그래서 거기를 전장으로 사용하게

였다. 이어서 무엇을 만들지가 구체화되었다.

요청 · 콜로니를 엄청 크게 만들어서 콜로니 안에 도시를

. 시작 위치에서 얼마나 떨어져 있어야 하느냐에 대해서는

요청 · 조금 멀리있어도되 내가 가서 싸우면되니까

라고 답했다.

그런데 첫 결과물은 성에 차지 않았던 모양이다. 다음 요청은 요구 사항을 한꺼번에 올렸다.

문제 보고 · 더 커야하고 더 자세하고 더 콜로니 같아야하고 콜로니 안에서는 중력이 있을거야 진짜 엄청커야해 도시가 진짜크게 복잡한도시고 그래서 도시에서 싸우는느낌이 있게

. 여기에 충돌에 대한 한 마디,

요청 · 부딪히게

가 붙었다. 이 문장이 콜로니의 크기(실물 크기), 도시의 밀도(수백 동, 고가도로), 그리고 중력·충돌 시스템 (SuitCollision, ColonyStructure)을 한꺼번에 결정했다.

건물을 무엇으로 만들지에 대해서는 이견호가 길을 열어 주었다.

요청 · 콜로니 외관이랑 도시에 건물은 varko에서 만들어도되

. 그 결과가 11:37 무렵의 Models/ColonyBuildings 폴더다. 그다음 다시 품질 지적이 나왔다.

문제 보고 · 디테일을 더 살려줘 지금은 너무 비현실적이야

. 이 요청 때문에 텍스처가 "v3"로 다시 만들어지고, 대기 안개(ColonyAtmosphere)와 거리 소품이 추가되었다. 마지막으로 외관에 대해

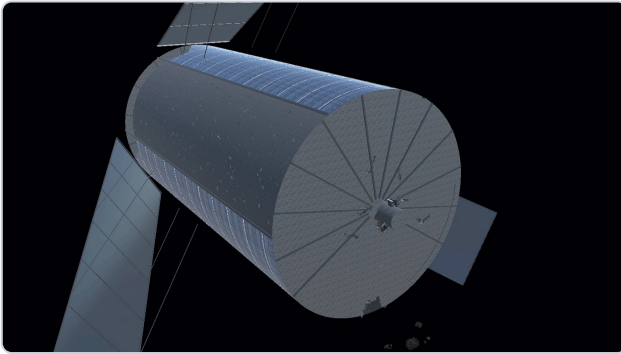
요청 · 파공 주변 잔해나 항구 크레인 같은 외관 소품

이 요청되어 12:20 무렵의 Models/ColonyProps 가 생겼다.

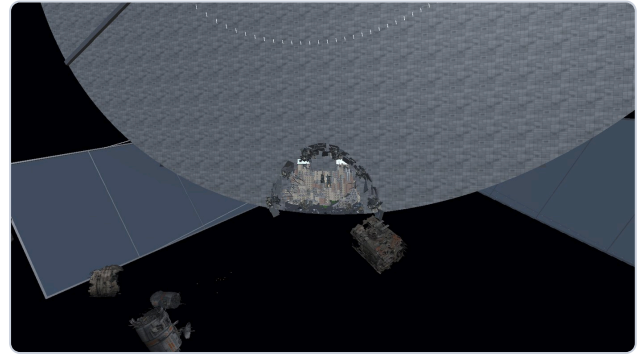
이 모든 단계의 중간 결과물은 커밋되지 않았다. 13:14 커밋에는 최종본만 들어 있으므로, 중간 시행착오는 코드에 남은 흔적(주석의 "v3", 쓰이지 않게 된 옛 함수, "was ..." 메모, "hairline cracks" 같은 수정 사유)으로만 확인할 수 있다. 아래에서는 그 흔적을 하나씩 짚어 가며 콜로니가 어떻게 만들어졌는지를 처음부터 따라간다.

설계: 건담 세계의 "아일랜드 3" 실린더

Claude가 고른 형태는 건담 작품 속 콜로니의 원형인 오닐 실린더, 이른바 "아일랜드 3(Island 3)"형이다. 긴 원통이 축을 중심으로 돌며 원심력으로 중력을 만들고, 원통 벽은 땅 띠 세 개와 창문 띠 세 개가 번갈아 있으며, 창 밖에 거울이 열려 햇빛을 안으로 반사한다. 주석은 이를 "A full-size Island-3 (O'Neill) colony like the ones in Gundam: 6.4 km across and 12 km long"이라고 적었다. 수치는 모두 상수로 정의되어 있다.



우주에서 본 콜로니. 원통 몸체와 열린 거울 판, 앞쪽 끝판.



끝판에 뚫린 파공(입구)과 주변 잔해·우주 잔해.

상수	값	의미
ColonyRadius	3200 m	원통 반지름(지름 6.4km)
ColonyLength	12000 m	원통 길이(+Z 방향)
ColonyNearZ	1500 m	가까운 끝판의 z. 건담은 z = 20에서 출발
ColonyBreachHalfWidth	330 m	끝판 파공의 반폭 ($ x < 330$)
ColonyBreachTop	430 m	파공 위쪽 끝 높이 (파공 바닥은 y = 0)
ColonyDowntownDepth	4600 m	도심이 끝판에서 안쪽으로 뻗는 깊이
ColonyDowntownHalf	1005 m	도심의 반폭
ColonyLandHalf	1550 m	바닥 땅 띠(선체 60°)의 반폭
BuildingTile	28 m	파사드 텍스처 한 장 = 8층 × 3.5m

"조금 멀리있어도되"라는 답에 맞춰 콜로니의 가까운 끝판은 출발점에서 1.5km 앞에 놓였다. 먼 끝판은 $1500 + 12000 = 13.5\text{km}$ 앞이다. 이 때문에 건담 머리 카메라(HeadCam)의 원거리 클립도 바뀌었다. 전날까지는 멀리 있는 잔해(최대 900m 정도)와 별만 보이던 되어 `farClipPlane` 이 2000이었는데, 이날 16000으로 늘어났다. 주석은 "the whole space colony (6.4 km across, far end 13.5 km away at the start)"라고 이유를 적었다. 이 16km라는 숫자는 오후에 디스플레이 깜빡임의 원인 중 하나가 된다.

좌표계: 축과 GroundY

콜로니 좌표계의 기준점은 `axis0 = (0, ColonyRadius, ColonyNearZ)` , 즉 가까운 끝판의 중심이다. 축 높이를 반지름과 같게 잡았기 때문에 원통의 가장 낮은 선(축 바로 아래)이 정확히 $y = 0$ 이 된다. 바깥 우주에서 건담이 날던 높이와 콜로니 바닥 높이가 맞아떨어지므로, 파공으로 수평 비행해 들어가면 그대로 도시 바닥 높이에 도착한다.

원통 안쪽 바닥은 평면이 아니라 곡면이다. 실제 콜로니처럼 땅이 양옆으로 휘어 올라가야 하기 때문이다. 그래서 x 위치마다 땅 높이를 구하는 함수가 필요했고, 생성기 쪽에는 `ColonyGroundY` , 런타임 쪽에는 `ColonyStructure.GroundY` 가 같은 식으로 들어갔다.

```
// 생성기: x에서 바닥 땅의 높이 (축 바로 아래 x=0이면 0)
static float ColonyGroundY(float x) =>
    ColonyRadius - Mathf.Sqrt(Mathf.Max(0f, ColonyRadius * ColonyRadius - x * x));
// x에서 땅의 각도(도). 270 = 축에서 바로 아래
static float LandAngle(float x) =>
    270f + Mathf.Asin(Mathf.Clamp(x / ColonyRadius, -1f, 1f)) * Mathf.Rad2Deg;
```

도심 반폭 1005m 지점의 땅은 $3200 - \sqrt{(3200^2 - 1005^2)} \approx 162\text{m}$ 높이까지 올라간다. 빌딩 하나의 양쪽 모서리가 서로 다른 높이의 땅에 걸치게 되므로, 건물 생성 코드는 낮은 쪽보다 3m 더 아래(`gLow`)에서 기초를 시작하고, 높이는 높은 쪽(`gHigh`)에서 잔다. 그래야 경사진 땅 위에 공중에 뜬 건물 모서리가 생기지 않는다.

2. 콜로니를 찍어 내는 도구: MeshAcc와 1km 청크

이 콜로니는 모델 파일이 아니라 전부 코드로 생성된다. 생성 결과는 수십만 개의 사각형이므로, 하나하나 GameObject로 만들면 드로우콜이 감당되지 않는다. 특히 이 게임은 HeadCam이 360° 큐브맵을 만들기 위해 매 프레임 장면을 여섯 번 그린다. 주석은 "Everything is procedural, low-poly and merged per material and per 1 km chunk (the head camera renders the scene six times a frame for the 360 view)"라고 설계 원칙을 적었다.

그 원칙을 구현한 것이 `MeshAcc` (메시 누적기)와 `ChunkedAcc` 다. `MeshAcc` 는 정점·법선·UV·삼각형 리스트를 들고 `Quad(a, b, c, d, normal, uv...)` 로 사각형을 하나씩 쌓는다. 정점이 65000개를 넘으면 인덱스 형식을 `UInt32` 로 바꾸고, 완성된 메시를 `Assets/Models/Colony/이름.asset` 으로 저장한 뒤 그림자 송출·수신을 끈 `MeshRenderer` 를 붙인다. `ChunkedAcc` 는 (재질 번호, z/1000) 쌍을 키로 `MeshAcc` 를 나눠 들고 있다가, 마지막에 `Colony_City_m{재질}_c{청크}` 라는 이름으로 한꺼번에 빌드한다. 재질별로 합쳐야 드로우콜이 줄고, 1km 청크로 나눠야 카메라 절두체 컬링이 먹힌다.

시행착오의 흔적: 삼각형 감김 방향

`Quad` 안의 주석이 흥미롭다. "Unity's front face of triangle (a,b,c) is along Cross(b-a, c-a); the diagonals give the same sign and also work for fan (degenerate) quads." 처음에는 네 점을 넘기는 순서로 앞면이 정해졌을 텐데, 콜로니는 안쪽에서 보는 면(바닥 땅)과 바깥에서 보는 면(선체)을 같은 함수로 만들기 때문에 순서를 매번 맞추기 어렵다. 그래서 두 대각선의 외적 `Cross(c - a, d - b)` 을 넘겨받은 법선과 내적해 부호를 보고, 음수면 삼각형 순서를 뒤집는다. 대각선을 쓰는 이유는 나무 피라미드처럼 두 점이 겹친 "부채꼴(퇴화) 사각형"에서도 부호가 올바르게 나오기 때문이다.

생성기는 다시 돌릴 때마다 청크 개수가 바뀔 수 있으므로, 시작할 때 `Assets/Models/Colony` 안의 기존 메시를 모두 지운다. 또 `BuildColonyPreview()` 라는 공개 함수가 따로 있다. `s_colonyPreview` 플래그를 켜고 `BuildColony()` 를 부르면, 메시와 텍스처를 에셋으로 쓰지 않고 메모리에만 둔 채 현재 장면에 콜로니만 지어 준다. 주석은 "for looking at it in a scratch scene. The real build is Gundam > Build Cockpit Scene"이라고 설명한다. 콕핏 전체를 다시 빌드하지 않고 콜로니 모양만 빨리 확인하기 위한 도구로, 디테일을 여러 번 고친 이날의 작업 방식을 짐작하게 한다.

기본 도형들

콜로니의 모든 형태는 몇 개의 도형 함수로 조립된다.

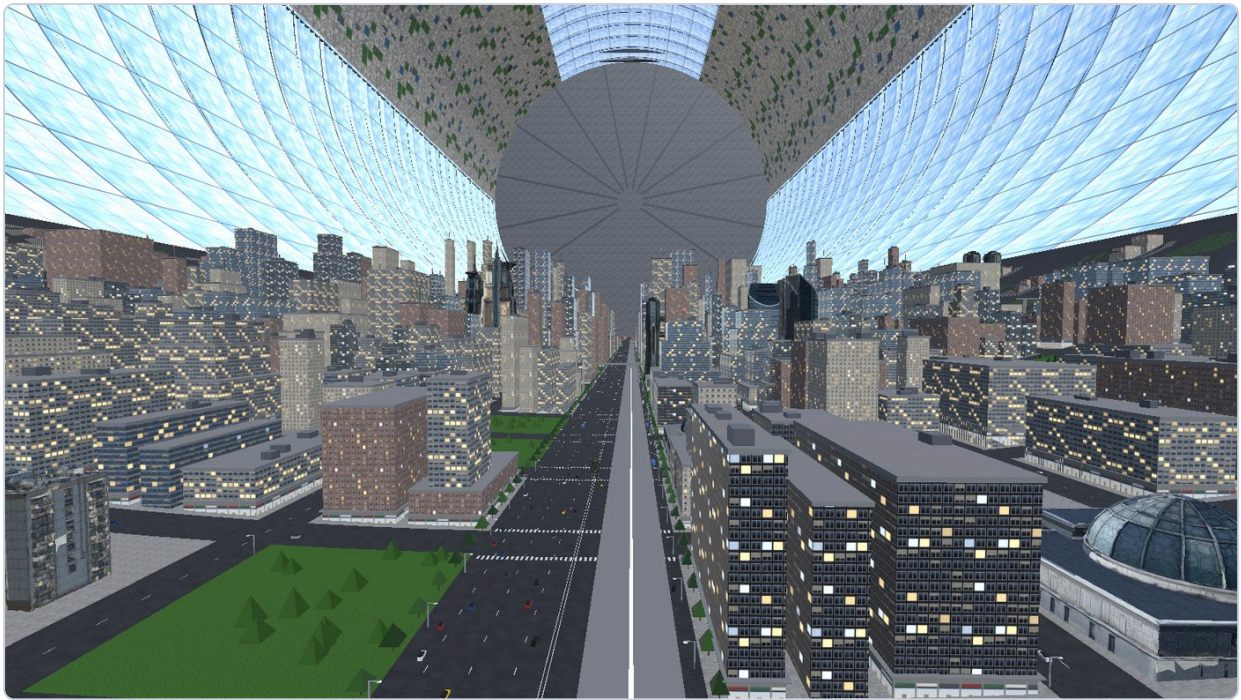
함수	만드는 것
AddCylinderBand	축 둘레의 원통 띠. 각도 a0~a1(0 = +X, 90 = 위), 길이 z0~z1, 안쪽/바깥쪽 향함, 둘레·길이 분할 수 지정
AddAnnulus	끝판 평면 위의 고리(조명 링, 스피들 뚜껑)
AddCapGrid	끝판을 극좌표 격자로. skip(x, y) 가 참인 칸은 빼서 파공을 뚫는다. 양면
AddBuildingBox	바닥 없는 상자. 파사드 UV를 크기에 비례해 28m 단위로, 지붕은 어두운 텍셀 한 점에
AddSolidBox / AddColorBox	6면 상자 / 모든 UV를 팔레트의 한 점에
AddOrientedBox	임의 방향 상자(파편, 거울 보, 자동차)
AddBandBox	위아래 없는 띠 상자(1층 상점가)
AddTree / AddTreeUV	줄기 위 사각뿔 나무
AddGableRoof	긴 변을 따라 용마루가 나는 박공지붕(처마 0.6m)
AddGroundPaint	흰 땅 위 0.15m에 붙는 도로 도색

시행착오의 흔적: 끝판의 머리카락 균열

AddCapGrid 는 고리마다 섹터 수가 다르다(바깥 고리일수록 많다). 그런데 각 고리의 가장자리는 곡선이 아니라 직선 현(弦)이라서, 섹터 수가 다른 두 고리의 경계가 정확히 맞지 않는다. 주석이 이 문제를 그대로 적어 두었다. "Rings have different sector counts, so their straight edges don't meet exactly - pull each ring's inner edge inside the previous ring's outer chords (hairline cracks let space show through)." 끝판에 머리카락처럼 가는 틈이 생겨 우주가 비쳐 보였다는 뜻이다. 해결은 각 고리의 안쪽 반지름을 이전 고리 현의 안쪽으로 당겨 겹치게 하는 것이다. $r_0 = r_0 * \cos(\pi / \text{prevSecs}) - 1.5$. 현의 가장 안쪽 지점은 반지름 $\times \cos(\pi/\text{섹터수})$ 이므로, 거기서 1.5m 더 안으로 들이면 틈이 사라진다.

3. 원통의 안쪽: 땅 띠 셋, 창 띠 셋

원통 안쪽 벽은 60°씩 여섯 띠로 나뉜다. 바닥(축에서 바로 아래, 270°)을 가운데로 하는 240°~300° 띠가 도시가 서는 "바닥 땅"이다. 그 양옆인 180°~240°와 300°~360°, 그리고 꼭대기 60°~120°가 창문 띠이고, 나머지 0°~60°와 120°~180°가 머리 위에 걸린 두 개의 "위쪽 땅"이다.



콜로니 내부 도심. 양옆 땅이 위로 말려 올라가 머리 위에서 만나고, 창문 띠로 빛이 들어온다. 멀리 반대쪽 끝판이 보인다.

```
// 안쪽 선체: 땅 3 + 창 3 (각도, 길이 0~L, 둘레 분할, 길이 12분할)
AddCylinderBand(bottomLand, axis0, R, 240f, 300f, 0f, L, 96, true, 40f, 12);
AddCylinderBand(upper, axis0, R, 0f, 60f, 0f, L, 32, true, 900f, 12);
AddCylinderBand(upper, axis0, R, 120f, 180f, 0f, L, 32, true, 900f, 12);
AddCylinderBand(windows, axis0, R, 300f, 360f, 0f, L, 32, true, 420f, 12);
AddCylinderBand(windows, axis0, R, 60f, 120f, 0f, L, 32, true, 420f, 12);
AddCylinderBand(windows, axis0, R, 180f, 240f, 0f, L, 32, true, 420f, 12);
```

바닥 땅은 둘레 96분할로 곡면을 매끄럽게 하고 아스팔트 텍스처(`Colony_Asphalt`, 256px, 펠린 노이즈 얼룩과 균열)를 40m마다 반복한다. 머리 위의 두 땅은 실제 건물을 세우는 대신 텍스처 한 장으로 처리했다. `Colony_UpperLandDay` 는 128px마다 큰 도로, 32px마다 작은 도로를 긋고, 블록마다 해시값으로 공원(18%), 연못(4%), 나머지는 작은 옥상 사각형들로 채운 "위에서 내려다본 낮의 도시와 농지" 그림이다. 900m마다 한 번 반복되며, 약간의 자체 발광(0.45)을 넣어 아래에서 올려다볼 때 어둡게 죽지 않게 했다.

흔적: 밤의 도시에서 낮의 도시로

`BuildColony` 머리 주석에는 아직 "The two land strips overhead are city-light textures"라고, 즉 도시 불빛(야경) 텍스처라고 적혀 있다. 그러나 실제 코드의 텍스처 이름은 `Colony_UpperLandDay` 이고, 바로 위 주석은 "a daytime city / farmland pattern (the air-haze from ColonyAtmosphere does the rest)"이다. 처음에는 머리 위 땅을 야경으로 그렸다가, "디테일을 더 살려줘" 단계에서 대기 안개와 함께 낮 풍경으로 바뀐 것으로 보인다. 머리 주석만 미처 고치지 못하고 남은 것이다.

창문 띠의 텍스처 `Colony_WindowSky` 는 하늘색 유리판에 펠린 노이즈로 얇은 구름을 얹고, 가장자리 6px의 굵은 구조 프레임과 64px마다 가는 멀리언(창살)을 그었다. 자체 발광 0.95, 매끄러움 0.9로, 거울이 반사해 들인 햇빛이 유리를 통해 비쳐 드는 느낌을 낸다. 콜로니 안에서 고개를 들면 휘어 올라간 땅과 빛나는 창 띠가 번갈아 머리 위를 지나가는 구조다.

4. 원통의 바깥: 선체, 거울, 끝판, 파공

선체와 골조

바깥 선체는 반지름 $R + 2m$ 에 깔린다. 땅 띠 셋(240° , 0° , 120° 에서 시작하는 60°)은 패넬 선체 텍스처 (`Colony_HullPanels`)로, 창 띠 셋은 바깥에서 본 유리(`windowOutMat`, 어두운 청회색, 금속성 0.6, 매끄러움 0.95)로 덮는다. 선체 텍스처는 크기가 섞인 패넬, 이음매, 열 개 중 하나꼴의 점검 해치, 리벳 줄, 펄린 노이즈 때를 256px에 그렸고 120m마다 반복한다.

여섯 띠의 경계마다 프레임 리브가 서 있다. $R + 30m$ 높이의 $\pm 0.6^\circ$ 윗면 띠와, $R + 2m$ 에서 $R + 30m$ 까지 올라가는 양 옆벽으로 이루어진 28m 높이의 골조다. 창 띠 위에는 바깥쪽 격자가 붙는다. 400m마다 14m 폭의 가로 프레임($R + 12m$, 아랫면까지 양면)과, 10° 마다 $\pm 0.15^\circ$ 폭의 세로 멀리언($R + 8m$)이다. 1000m마다(500m부터) 18m 폭의 조명 링이 원통을 한 바퀴 두르고(144분할, 발광 재질), 땅 띠 위에는 180m 간격으로 띠마다 다섯 개씩 방열판·배관·환기구 같은 상자 "그리블(greeble)"을 무작위 크기(반폭 8~42m, 높이 2~12m, 길이 10~70m)로 박아 넣었다.

거울 세 장

아일랜드 3형 콜로니의 상징인 거울은 창 띠 중심 각도(330° , 90° , 210°)마다 하나씩, 먼 끝($z = L$)에 경첩을 달고 28° 열린 상태로 만들었다. 거울 한 장의 반폭은 $R \times \sin 30^\circ \times 0.98$, 길이는 $L \times 0.92$ (약 11km)이다. 경첩은 선체 바깥 $R + 40m$ 에 있고, 거울 판은 그 경첩에서 콜로니 쪽($-Z$)으로 28° 벌어지며 뻗는다. 판 양면을 만들고, 네 모서리에 보(가장자리 보 $18 \times 14m$, 경첩 보 $22 \times 22m$), 뒷면에 가로 리브 다섯 개, 선체에서 거울 가장자리로 뻗는 버팀대 네 개(거울 길이의 0.45, 0.8 지점, 좌우)를 붙였다.

거울 재질의 주석은 이렇다. "Mirrors: dark, almost perfectly reflective panels (they show the stars and the sun's glare, not a glow of their own)." 어두운 청회색 바탕에 매끄러움 0.97, 금속성 1, 발광은 아주 약하게만 넣었다. 거울이 스스로 빛나는 판처럼 보이지 않게, 별과 햇빛을 반사하는 금속판으로 보이게 한 선택이다.

끝판, 스핀들, 그리고 파공

먼 끝판은 막힌 원판(16고리, 96섹터)이고, 가까운 끝판(36고리, 180섹터)에는 파공이 뚫린다. 파공 판정 함수는 단순한 사각형이 아니라 가장자리가 들쭉날쭉하다.

```
System.Func<float, float, bool> breach = (x, y) =>
{
    // 펄린 노이즈로 가장자리를 ±70m 들쭉날쭉하게
    float jag = (Mathf.PerlinNoise(x * 0.012f, y * 0.012f) - 0.5f) * 140f;
    return Mathf.Abs(x) < ColonyBreachHalfWidth + jag
        && y < ColonyBreachTop + jag * 0.8f;
};
AddCapGrid(caps, axis0, R + 2f, 0f, 36, 180, breach);
AddCapGrid(caps, axis0, R + 2f, L, 16, 96, null);
```

파공 바깥쪽 둘레에는 "안에서 바깥으로 터져 나간" 찢긴 판 90장을 무작위 크기(8~44m)로 기울여 붙였고, 파공 가장자리에서 구멍 안쪽으로 빠져나온 구조 거더 26개(길이 30~100m, 단면 4.4m)를 꽂았다. 이것들은 BreachWreckage 그룹에 들어가 안팎 어디서나 보인다.

끝판 바깥 중심에는 도킹 스피들이 튀어나와 있다. 가까운 쪽은 반지름 280m, 길이 900m, 먼 쪽은 반지름 360m, 길이 700m의 원통이고, 각각 조명 띠 세 개와 끝 뚜껑의 발광 고리가 붙는다. 가까운 끝판 바깥면에는 반지름 900~930m, 1800~1830m에 발광 고리가, 먼 끝판 안쪽면에는 1200~1260m, 2400~2440m에 빛나는 고리가 있다. 양 끝판 바깥에는 22.5° 간격으로 16개의 방사형 보강 살이 뻗는데, 파공이 있는 235°~305° 구간은 건너뛴다. 파공 위에 보가 걸쳐 있으면 입구가 막혀 보이기 때문이다.

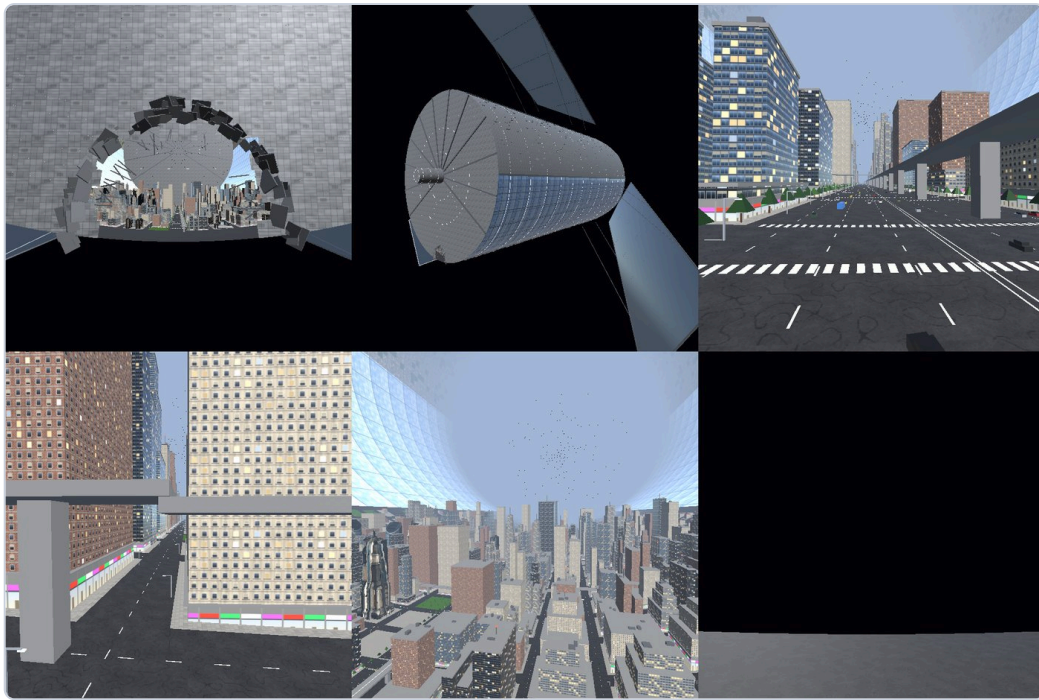
5. 도시: "복잡한도시고 그래서 도시에서 싸우는느낌이 있게"

도심의 격자

도시는 끝판에서 350m 들어간 지점($z_{City0} = z_N + 350$)에서 시작한다. 처음 350m는 파공으로 들어온 사람이 착지하는 입구 광장이다. 도심은 끝판에서 4600m 안쪽까지 이어지고, 가운데로 반폭 55m의 대로(boulevard)가 길게 뻗는다. 대로 양쪽으로 한 번 110m 블록이 30m 도로를 사이에 두고(간격 140m) 일곱 줄씩 늘어선다. 블록 하나하나의 땅의 곡률을 따라 흰 연석 높이(0.9m)의 판(포석 텍스처 또는 잔디)을 깔고 그 위에 건물을 세운다.

높이 분포는 도심 핵($core = (0, z_N + 2300)$)을 중심으로 한 가우스 함수다. 블록 중심에서 핵까지의 거리(x 방향은 1.3으로 나눠 조금 납작하게)를 $dist$ 라 하면 $hScale = \exp(-(dist/1100)^2)$ 이다. 핵 근처의 블록은 $hScale$ 이 1에 가까워 수백 미터짜리 마천루가 서고, 멀어질수록 낮아진다. 건물 높이는 $22 + \text{난수} \times 50 + hScale \times (60 + \text{난수} \times 360)$ 으로, 핵 근처에서는 최고 약 490m까지 올라간다.

블록마다 주사위를 굴린다. 7% 미만이면 공원(잔디판에 나무 14그루), 10% 미만이면 빈 광장, 22% 미만이고 VARCO 모델이 있으면 랜드마크 건물, 나머지는 필지를 1개·2개·4개로 나눠(가장자리 여백 6m, 필지 사이 8m) 절차적 건물을 세운다. 나눈 필지는 $hScale$ 에 0.6~0.85를 곱해 높이를 섞어 스카이라인이 단조롭지 않게 했다.



콜로니 테스트 렌더 모음. 파공 입구, 외부, 대로, 고층 건물 등 여러 지점에서 찍어 디테일을 확인했다.

건물 네 유형

유형	조건	형태
세트백 타워	type 1, 높이 > 80m	3단. 높이를 45%/33%/22%로 나눠 쌓고 단마다 가로세로 0.72배로 줄임. 단 사이에 콘크리트 텍
포디움 + 타워	type 2, 높이 > 60m	12~26m 저층부 위에, 필지의 45~60% 크기 타워를 무작위 위치에. 타워는 다른 파사드 스타일
쌍둥이 판상	type 3, 폭 > 40m	폭 18% 틈을 둔 두 동. 한쪽은 높이의 65~95%
단순 상자	그 밖	필지 전체를 채우는 직육면체

도시 건물에는 1층 상점가가 붙는다. 땅 위 6.5m까지 상점 정면 텍스처를 두른 띠(AddBandBox)를 감고, 그 위에 1.8m 내민 캐노피 텍을 얹는다. 지붕 모서리에는 코니스(처마 돌림띠)를 두르고, 옥상에는 1~3개의 설비 상자(4~13m)를 놓는다. 높이가 120m를 넘는 건물은 45% 확률로 12~47m 안테나를 세우고 끝에 붉은 항공 경고등(발광 재질)을 단다. 교외 주택은 상점가 대신 팔레트 색의 박공지붕(3~6m)을 쓴다.

모든 건물의 상자는 `collide(mn, mx)` 로 충돌 목록에 들어간다. 이 목록이 나중에 `ColonyStructure.boxMin/boxMax` 가 되어 건담이 부딪히고, 자쿠가 돌아가고, 총알이 멈추고, 시선이 막히는 모든 판정의 재료가 된다.

교외, 농지, 고가도로

도시 바깥(도시 반쪽 1005m 밖, 또는 4600m 너머)은 170m 격자에 120m 필지를 놓아 채운다. 35%는 농지(잔디판), 15%는 빈 땅, 나머지는 주택 1~3채(한 번 25~50m, 높이 8~28m, 주거용 또는 벽돌 파사드)다. 이

교외는 먼 끝판 400m 앞까지, 땅 띠 전체 폭(±1550m)에 깔린다. 도심에서 멀어지며 건물이 낮아지고 들판이 늘어나는 풍경이 "콜로니 같아야하고"에 대한 답이다.

대로 위에는 고가도로가 지난다. 대로 동쪽 절반(x 17~43m, 중심 x = 30)에 높이 30m, 두께 3m의 상판을 300m 구간씩 이어 붙이고, 100m마다 8×8m 교각을 세우고, 상판 가운데에 가로등 띠를 깔았다. 서쪽 절반을 비워 둔 이유를 주석은 "so the centre line stays open to walk/fly down"이라고 적었다. 건담이 대로 한가운데를 따라 날아갈 수 있도록 길을 남긴 것이다. 또 동서 방향 도로 두 곳(zN + 1455, zN + 3135) 위로 가로 고가도로가 도심 폭 전체를 가로지른다. 땅이 휘어 있으므로 60m 단위로 끊어 각 구간을 그 지점의 땅 높이 + 38m에 놓았고, 한 구간 걸러 교각을 세웠다. 고가도로 상판과 교각도 모두 충돌 상자다.

6. "디테일을 더 살려줘 지금은 너무 비현실적이야"

텍스처 v3와 남겨진 옛 함수

이 요청에 대한 답의 중심은 텍스처다. 코드에는 "Colony v3 detail helpers"라는 표제가 붙은 함수 묶음이 있고, 그 위에 이전 세대 함수 `MakeFacadeTexture` 가 아무 데서도 호출되지 않은 채 남아 있다. 두 함수를 비교하면 무엇이 바뀌었는지가 드러난다.

	옛 <code>MakeFacadeTexture</code>	v3 <code>MakeFacade</code>
해상도	128px, 창 칸 16px	256px, 창 칸 32px (= 28m에 8층 8칸)
창	커짐/꺼짐 두 색	하늘 반사 그라디언트, 프레임, 멀리언, 블라인드(35%), 대각선 반사 줄, 미세 노이즈
벽	단색	세로 띠 자국(떨린), 1층 띠, 발코니(주거), 벽돌 줄눈(벽돌)
발광	같은 텍스처를 발광 맵으로 (벽까지 빛 남)	별도 발광 맵: 켜진 방 창만 빛남

마지막 줄이 핵심이다. 이전에는 `MakeTexturedMat` 이 발광을 켜면 기본 텍스처를 그대로 `_EmissionMap` 에 넣었다. 그러면 벽까지 은은하게 빛나 건물이 야광 장난감처럼 보인다. 주석은 v3의 목표를 "emission maps that light only the lit rooms / shops - the walls themselves no longer glow"라고 명시한다.

`MakeFacade` 는 기본 텍스처와 함께 `_Emission` 텍스처를 따로 만들어, 불 켜진 방(스타일별 10~20%)의 창 안쪽 픽셀만 따뜻한 노란색(75%) 또는 차가운 흰색으로 칠하고 나머지는 검게 둔다. `MakeEmissionMapMat` 이 이 둘을 짝지어 재질을 만든다.

파사드 스타일은 다섯 가지다. 0 사무용 콘크리트, 1 유리 커튼월(매끄러움 0.85, 금속성 0.3), 2 주거(발코니), 3 벽돌, 4 어두운 현대식(금속성 0.4). 스타일마다 벽색 둘, 프레임색, 유리 위아래 색, 창 영역 픽셀 범위, 점등 비율, 띠 정도가 정해져 있다. 도심 건물은 다섯 중 무작위, 교외 주택은 주거나 벽돌이다. 상점 정면(`MakeStorefront`)은 한 장에 64px 폭 상점 넷을 그리고, 기둥, 위쪽 간판 띠(여섯 색), 차양 모서리, 문, 받침, 그리고 따뜻한 색으로 빛나는 진열창을 넣었다. 그 밖에 포석(32×16px 블록과 줄눈), 잔디, 그리고 자동차·지붕·기둥·나무 색을 한 장에 담은 4×4 팔레트 텍스처(64px)가 있다. 팔레트를 쓰면 색이 다른 수천 개의 작은 물체가 재질 하나, 드로우콜 하나로 합쳐진다.

결국 도시 재질은 14개로 정리되었다. 0 사무, 1 유리, 2 주거, 3 콘크리트, 4 포석, 5 잔디, 6 옥상 설비, 7 붉은 경고등, 8 가로등, 9 벽돌, 10 어두운 현대식, 11 상점 정면, 12 팔레트, 13 도로 도색. `ChunkedAcc`의 키가 이 번호다.

거리의 디테일: 총 27번

두 번째 답은 거리 수준의 소품이다. 주석은 "road paint and crossings, street lights, trees along the boulevard, and traffic left standing where it stopped when the fighting started (some of it burnt out)"라고 적었다. 전투가 시작된 순간 멈춰 선 채 버려진 차들이다.

대로에는 18m 간격의 차선 점선, 45m 단위의 이중 중앙선, 양쪽 45m 간격의 가로등(높이 10m 기둥, 3.2m 팔, 발광 등갓), $x = \pm 58.5m$ 보도를 따라 22m 간격의 가로수가 있다. 남북 도로 12개(대로에서 블록 하나 + 15m 떨어진 곳부터 140m 간격, 좌우 대칭)에도 차선 도색과 90m 간격으로 좌우를 번갈아 선 가로등이 있고, 블록 줄 사이 동서 도로가 대로와 만나는 곳에는 2.6m 간격의 얼룩말 횡단보도가 그려진다.

자동차는 승용차(길이 4.4~5.0m, 폭 1.8m, 높이 1.0m, 위에 캐빈 상자)와 버스(8%, $11.5 \times 2.5 \times 3.1m$)이고, 15%는 불탄 차(팔레트의 거의 검은 색)다. 버려진 느낌을 내려고 진행 방향에서 조금씩 틀어 놓았다. 보통 차는 $\pm 7^\circ$, 불탄 차는 $\pm 25^\circ$ 다. 대로 서쪽 차선 여섯, 동쪽 차선 다섯, 남북 도로, 동서 도로에 확률적으로 흩어진다.

이 거리 디테일은 수가 많고 작아서 멀리서는 보이지도 않는다. 그래서 따로 `StreetDetail` 그룹에 모아 레이어 27번(`ColonyDetailLayer`)에 두었고, HeadCam의 레이어별 컬링 거리(`layerCullDistances`)로 1400m 밖은 그리지 않는다(아래 대기 절에서 다룬다).

파공 바로 안쪽의 입구 광장에는 전쟁의 흔적을 뿌렸다. 160개의 콘크리트·선체 파편(크기 1.5~17.5m, 작은 것이 훨씬 많게 난수 두 개를 곱한 분포)을 무작위 각도로 기울여 놓았고, 70%는 그을린 콘크리트 색(팔레트), 30%는 선체 재질이다. 반폭이 9m를 넘는 큰 덩어리는 크기의 70%로 충돌 상자를 등록해 건물처럼 길을 막는다.

7. VARCO 건물과 외관 소품

"콜로니 외관이랑 도시에 건물은 varko에서 만들어도되"

VARCO(이건호의 회사 NC AI의 3D 생성 도구)로 만든 모델 다섯 개가 `Assets/Models/ColonyBuildings`에 들어왔다. `ColonyApartment`, `ColonyCivicHall`, `ColonyTower_Central`, `ColonyTower_Glass`, `ColonyTower_Twin`이다. `LoadColonyBuildingModels`가 이 폴더의 모델을 이름순으로 읽어 오고, 블록 주사위가 랜드마크를 뽑으면 `landmark` 함수가 그 필지에 맞는 모델을 고른다.

고르는 규칙은 이렇다. 이름에 "Tower"가 들어간 모델은 `hScale`이 0.35를 넘는 핵 근처에만, 나머지(아파트, 시민회관)는 그 바깥에만 쓴다. 한 모델은 최대 두 번(`ColonyLandmarkUsesPerModel = 2`)까지만 쓰고, 그중 가장 적게 쓴 것을 우선한다. 주석은 "Each model is used a few times at most (the head camera renders the scene six times a frame)"라고 제한 이유를 적었다. 고해상도 생성 모델은 절차적 상자보다 훨씬 무겁기 때문이다. 목표 높이는 중앙 타워 360m, 쌍둥이 타워 300m, 그 밖의 타워 260m에 $(0.8 + 0.3$

× hScale)을 곱하고, 아파트는 45~65m로 한 필지를 반으로 나눠 두 동을 세운다. 시민회관은 목표 높이를 1000으로 두어 사실상 필지 넓이에 맞춰지게 했다. 각 모델은 90° 단위로 무작위 회전한다.

흔적: 세로로 늘렸더니 깊이 방향으로 늘어났다

필지가 좁아 발자국(footprint) 기준으로 줄이면 타워가 너무 낮아진다. 그래서 타워는 세로 방향으로 최대 1.5배까지 더 늘릴 수 있게 했다($sY = \min(sHeight, sXZ \times 1.5)$). 그런데 주석이 경고하듯 "FBX root is rotated 270 deg about X", 즉 가져온 FBX의 루트가 X축으로 270° 돌아가 있어서, 모델 자체에 세로 배율을 주면 그 배율이 모델의 로컬 축을 따라 "깊이" 방향으로 들어간다. 해결은 빈 홀더 오브젝트를 하나 만들어 월드 축 기준의 배율을 홀더에 주고, 모델은 그 아래에서 가져온 회전을 그대로 유지하게 하는 것이다. "so a vertical stretch really goes up and not into the depth."

배치 후에는 렌더러 바운드를 다시 재서 필지 중앙, 낮은 쪽 땅보다 2m 아래에 바닥이 오도록 옮기고, 모델에 달려 온 콜라이더는 모두 지운 뒤(물리 콜라이더는 쓰지 않는다), 최종 바운드 상자를 충돌 목록에 넣는다.

"파공 주변 잔해나 항구 크레인 같은 외관 소품"

Models/ColonyProps 에는 다섯 모델이 들어왔다. PlaceColonyVarcoProps 가 이들을 배치하는데, 공통 함수 PlaceColonyProp 는 모델의 가장 긴 변이 지정 크기가 되게 배율을 맞추고, 바닥 중심을 기준점에 놓고, 위 방향을 지정 벡터에 맞춘 뒤 그 축으로 돌린다. 모델이 없으면 건너뛴다.

모델	개수·크기	위치	그룹
BreachWreck	6개, 80~140m	파공 테두리 바깥 55m/45m, 바깥으로 뒤틀려 끝판에서 튀어나옴	BreachWreckage(항상 보임)
Rubble	4개, 70~100m	파공 안쪽 120~330m의 무너진 건물 더미, 땅에 8% 묻힘	InteriorProps, 충돌 있음
PortModule	6개, 260m	가까운 스피들(반지름 281m) 둘레 60°마다, 축 방향 -280/-610m	Exterior
DockingCrane	6개, 460m	가까운 끝판 바깥면, 반지름 700/1120/1540m	Exterior
HullAntenna	6개, 300m	바깥 선체 땅 띠 위, 여러 z	Exterior

그룹 구분은 다음 절의 안/밖 렌더 토글과 맞물린다. Exterior 에 든 것은 콜로니 안에서 꺼지고, 파공 잔해는 안에서도 입구로 보이므로 늘 켜져 있다.

8. 공기와 비용: ColonyAtmosphere

12:11 무렵 생긴 ColonyAtmosphere 는 "디테일을 더 살려줘"에 대한 세 번째 답이자, 거대한 콜로니를 헤드셋에서 돌리기 위한 비용 관리 장치다. 클래스 주석은 두 역할을 "AIR"와 "COST"로 나눠 설명한다.

AIR: 안에 들어가면 하늘이 생긴다

뷰어(HeadCam)가 콜로니 안에 있으면 선형 거리 안개를 켜다. 색은 옅은 하늘색 (0.64, 0.72, 0.84), 시작 250m, 끝 7000m다. 그러면 머리 위로 휘어 올라간 땅, 12km 떨어진 먼 끝판, 먼 구역들이 실제 하늘처럼 아지랑이 속으로 흐려진다. 주변광은 평면(Flat) 모드의 (0.46, 0.5, 0.56)으로 올려 창을 통해 들어오는 부드러운 낮빛을 흉내 낸다. 밖으로 나가면 시작할 때 기억해 둔 장면의 원래 값(안개 거리·색, 주변광 모드·색)으로 되돌린다.

"안"의 판정에는 여유가 있다. `IsInside(p)` 이면서 동시에 끝판에서 60m(`insideMargin`) 이상 들어와야 안으로 친다. 주석은 "so the breach itself still shows the outside"라고 적었다. 파공 구멍에 걸쳐 있을 때 바로 안개가 끼고 외관이 꺼지면, 뒤를 돌아봤을 때 바깥 우주 풍경이 갑자기 사라지기 때문이다.

함정: 안개를 꺼 두면 빌드에서 안개 셰이더가 사라진다

밖에서는 안개가 없어야 하니 `RenderSettings.fog = false` 로 두고 안에 들어갈 때만 켜면 될 것 같지만, 그러면 Unity가 빌드할 때 "이 장면은 안개를 안 쓴다"고 보고 안개 셰이더 변형(variant)을 빼 버린다. 그러면 헤드셋에서 안개를 켜도 아무 일도 일어나지 않는다. 그래서 `BuildColony` 는 장면 안개를 **켄 상태로** 두되 시작 거리를 90000m, 끝 거리를 100000m로 도달할 수 없게 설정한다. "Scene fog stays ENABLED (unreachable start distance) so the fog shader variants are kept in the player build; ColonyAtmosphere moves the distances in and out." 런타임에는 거리만 옮긴다.

COST: 보이지 않는 것은 끈다

안에서는 선체 바깥이 보일 일이 없다(창 떠는 거울빛을 받은 유리로 표현했다). 그래서 `Exterior` 그룹, 즉 바깥 선체·골조·거울·조명·창 바깥면·항구 크레인·안테나·항구 모듈의 렌더러를 전부 끈다. 반대로 밖에서는 거리 디테일(차, 가로등, 도로 도색, 가로수)을 끈다. 그리고 안에서도 거리 디테일은 HeadCam의 레이어별 컬링 거리로 1400m(`detailCullDistance`)까지만 그린다. `Start` 에서 카메라의 `layerCullDistances` 배열 27번 칸에 이 값을 넣어 두면, 카메라가 알아서 그 거리 밖의 레이어 27 오브젝트를 건너뛴다.

이 클래스는 렌더러의 `enabled` 와 `RenderSettings` 만 건드린다. 충돌, 건담, 카메라 위치, XR 리그에는 손대지 않는다는 점을 주석이 따로 밝혀 두었다. 또 `OnDisable` 에서 안 상태였다면 밖 상태로 되돌려, 에디터에서 플레이를 멈췄을 때 안개 설정이 남지 않게 했다.

우주 먼지, 잔해, 별

콜로니가 생기자 기존 우주 효과들이 어색해졌다. 11:08 무렵 다시 쓰인 두 스크립트가 그 흔적이다.

`SpaceDebrisField` 에는 `colony` 필드가 생겼다. 툴팁은 "the wrecks are hidden while the viewer is inside this colony (they'd otherwise float inside the city)". 잔해장은 뷰어를 일정 비율로 따라다니므로, 콜로니 안에 들어가면 수백 미터짜리 우주 잔해가 도시 한복판에 떠 있게 된다. 안에 있는 동안 잔해의 렌더러를 모두 끈다. 또 잔해 여섯 덩어리 중 두 개의 방향을 옮겼다. 35°에 있던 것을 75°로("moved off the colony, straight ahead"), 330°에 있던 것을 300°로. 출발점에서 정면의 콜로니를 가리던 잔해를 비켜 놓은 것이다. 이 잔해장 자체는 전날

라는 요청으로 만들어졌는데, 그 문장 속의 "건물"이 이날 콜로니로 실현된 셈이다.

`SpaceDustField` 도 같은 식으로 콜로니 안에서는 가까운 먼지 타일을 끈다("no space dust floating in a city"). 그리고 먼 별에는 별도 재질 `farMaterial` 을 쓰게 했다. 이 재질은 렌더 큐 1000(Background)에 깊이 쓰기(`_ZWrite`)를 끈 것이다. 주석은 "so the stars never draw in front of anything (e.g. the kilometres-big colony)". 별 구(球)는 뷰어를 따라다니는 유한한 반지름의 메시라서, 수 km 크기의 콜로니가 그 반지름보다 멀리 있으면 별이 콜로니 앞에 그려지는 문제가 있었다. 가장 먼저, 깊이도 남기지 않고 그리면 그 뒤에 그려지는 모든 것이 별을 덮는다.

9. 중력과 충돌: "부딪히게", "콜로니 안에서는 중력이 있을거야"

물리 엔진을 쓰지 않는 충돌

`ColonyStructure` 의 클래스 주석에는 설계 원칙이 한 줄로 들어 있다. "This component holds only its collision description and answers 'push this body out of the structure' queries - no physics colliders, so weapons, XR grabs and lock-on are unaffected." 수천 개의 건물에 Unity 콜라이더를 붙이면 손 추적 잡기(XRI), 무기의 물리 레이캐스트, 락온 판정이 모두 그 콜라이더와 얽힌다. 그래서 콜로니는 축·반지름·길이·파공 크기·땅 떠 폭, 그리고 건물 상자 배열(`boxMin`, `boxMax`)이라는 순수한 데이터만 들고, "이 몸을 구조물 밖으로 밀어내라"는 질의에 답하는 수학 함수만 제공한다.

ResolveBody: 선체, 끝판, 땅, 건물

건담은 발 위치·높이·반지름을 가진 똑바로 선 원기둥으로 취급된다. `ResolveBody(feet, height, r, out touching, out grounded)` 는 네 단계로 이 원기둥을 밀어낸다.

첫째, **선체**. 발에서 높이의 10%, 50%, 90% 세 점을 보며, 안에 있으면 축에서의 거리가 $R - r$ 을 넘지 않게 안으로, 밖에 있으면(그리고 원통 길이 범위 안이면) $R + r$ 보다 가깝지 않게 밖으로 민다. 다만 안에서 땅 떠 위($|x| < 1550$, 축보다 아래)에 있는 점은 건너뛴다. 거기서는 선체가 아니라 땅이 바닥 역할을 하기 때문이다.

둘째, **끝판**. 가까운 끝판은 파공 안($|x| < \text{파공 반폭} - r$, 머리끝이 파공 위쪽 끝보다 아래)에 있지 않으면 벽이다. 안에서는 z 가 r 이상이 되게, 밖에서는 $-r$ 이하가 되게 민다. 먼 끝판은 언제나 벽이다. 주석의 표현대로 "the only way in or out is the open near end".

셋째, **땅**. 안에 있으면 `GroundY(feet.x)` 보다 발이 낮을 때 끌어올리고, 5cm 이내면 "서 있음(grounded)"으로 표시한다.

넷째, **건물**. `ResolveBoxes` 가 각 상자에 대해 발 위치를 상자 수평 단면에 클램프한 최근접점과의 거리를 재고, 반지름보다 가까우면 바깥으로 민다. 발이 상자 안 깊숙이 들어가 거리가 0이면 네 면 중 가장 가까운 쪽으로 뺀다. 그리고 발이 지붕 높이에서 6m(`roofStep`) 이내이면 옆으로 밀지 않고 지붕 위에 세운다. "you can land on rooftops", 즉 건담이 빌딩 옥상에 내려앉을 수 있다.

SuitCollision: 중력의 순서

실제로 건담을 미는 것은 `MobileSuitRoot` 에 붙은 `SuitCollision` 이다. 이 오브젝트는 이동 스크립트들 (`ShipMovementController`, T레버의 `VerticalThrustController`)이 움직이는 루트이고, 건담 몸체는 `ExternalGundamFollower`가 매 프레임 이 루트를 따라 옮긴다. `SuitCollision`은 `[DefaultExecutionOrder(-70)]`로 따라가기 스크립트(-60)보다 먼저, 이동 스크립트들보다는 나중인 `LateUpdate`에서 돈다. 이동이 끝난 위치를 고친 다음 몸체가 따라가야 한 프레임이라도 벽 속에 박힌 모습이 보이지 않기 때문이다.

```
// 콜로니 안: 중력. T레버로 상승 중이면 낙하를 상쇄
InColony = colony.IsInside(feet0 + Vector3.up * (bodyHeight * 0.5f));
if (InColony)
{
    float dt = Time.deltaTime;
    bool climbing = thrust != null && thrust.CurrentVerticalSpeed > 0.5f;
    if (climbing) _fallSpeed = Mathf.MoveTowards(_fallSpeed, 0f, 40f * dt);
    else _fallSpeed = Mathf.Max(_fallSpeed - gravity * dt, -maxFallSpeed);
    transform.position += Vector3.up * (_fallSpeed * dt);
}
else _fallSpeed = 0f; // 우주에서는 무중력
```

중력은 9.8m/s^2 , 최대 낙하 속도는 70m/s 다. 콜로니 안에 들어오면 건담은 떨어지기 시작하고, 땅이나 지붕에 서면(`grounded`) 낙하 속도가 0으로 리셋된다. T레버를 올려 상승 속도가 0.5m/s 를 넘으면 "추진기가 켜진" 것으로 보고 낙하 속도를 초당 40m/s 씩 0으로 되돌린다. 그러면 레버 자체의 상승이 건담을 들어 올리고, 레버를 놓으면 다시 떨어진다. 밖으로 나가면 낙하 속도를 0으로 두어 예전처럼 무중력이다. 연결은 빌드 코드에서 `gravityCollision.thrust = verticalThrust` 한 줄로 이루어졌다. 원통 콜로니에서 "아래"는 사실 축에서 바깥 방향이지만, 도시가 서 있는 바닥 땅 떠는 축 바로 아래이므로 월드 -Y를 중력 방향으로 써도 무리가 없다.

"건담이 상대를 통과하지않게 충돌을 넣어"

`SuitCollision`은 원래 이 요청에서 출발한 컴포넌트다. 클래스 주석의 첫 문단이 그것이고, 콜로니와 중력은 그 뒤에 덧붙었다. 1초마다(`rescanInterval`) 장면의 모든 `EnemyHealth`를 찾아 그 오브젝트의 `CapsuleCollider`(자쿠의 피격 판정 캡슐)를 목록에 담는다. 매 프레임 건담을 반지름 3.5m , 높이 18m 의 원기둥으로 보고, 살아 있는 적의 캡슐과 높이가 겹치는지(위로 넘어가거나 아래로 지나가면 접촉 없음), 수평 거리가 두 반지름 합 $+0.3\text{m}$ (`skin`)보다 가까운지를 본다. 가까우면 수평으로 그 차이만큼 루트를 밀어낸다. 두 번 반복(`pass`)해 여러 적에 동시에 걸렸을 때의 떨림을 줄였다.

결과는 주석 그대로다. "So you stop at the enemy and slide around it instead of flying through, with no physics bodies involved and nothing about the movement code changed." 이동 코드는 한 줄도 바뀌지 않았고, 사후에 위치만 보정한다. 죽은 적(`IsDead`)은 판정에서 뺀다.

10. 출격 위치 선택: SORTIE POINT

콜로니가 생기자 새 문제가 생겼다. 출발점은 콜로니 1.5km 앞의 우주인데, 매번 거기서 파공까지 날아가야 한다. 이진호의 요청은

요청 · 처음에 스폰 위치를 지정해야할듯 어디서 시작할지 우주 콜로니 외관 콜로니 안 이런식으로

였고, Claude가 내놓은 선택지 가운데

요청 · 게임 시작 시 선택 화면

이 골라졌다.

그 결과가 `SpawnSelectPanel` 이다. 보정(Cockpit Calibration)이 끝나면(보정이 없으면 곧바로) 보정 안내가 있던 자리, 콕핏 로컬 (0, 1.2, 0.34)에 화면 소품 없이 떠 있는 월드 스페이스 캔버스가 나타난다. 520×360 단위 캔버스를 0.0009배(1단위 = 0.9mm)로 줄였으니 약 47×32cm 크기다. 제목은 "SORTIE POINT / 출격 위치 선택"이고, 226×130 단위(약 20×12cm)의 큰 버튼 네 개가 2×2로 놓인다.

버튼	위치(발)
SPACE / 우주	그대로(콜로니 1.5km 앞 우주)
COLONY EXTERIOR / 콜로니 외관	파공 정면 520m 앞, 바닥보다 150m 위에 떠서
COLONY ENTRANCE / 콜로니 안 (입구)	파공 안쪽 260m 광장, 땅 + 0.3m
DOWNTOWN / 콜로니 안 (도심)	대로 한가운데 1900m 안쪽, 땅 + 0.3m, 자쿠 근처

버튼은 WEAPON 디스플레이와 똑같이 검지 끝으로 누른다. `XRHandTrackingEvents.jointsUpdated`에 자기 리스너만 붙여 `IndexTip` 관절 위치를 받아 트래킹 공간에서 월드로 변환하고, 손가락 끝이 패널 앞 2cm(`armDistance`) 밖에서 "장전(armed)"된 뒤 버튼 영역 안에서 패널 앞 1.2cm(`touchDepth`) 이내로 들어오면 눌림으로 친다. 패널 뒤로 3cm(`behindTolerance`)까지는 눌림으로 인정하고, 앞 5cm 이내에 들어오면 버튼을 밝게(hover) 칠한다. 버튼 판정 영역은 4mm(`hitMargin`) 넓혔다. 누르면 0.25초 동안 버튼이 연두빛 흰색으로 번쩍인 뒤 이동한다.

패널이 떠 있는 동안에는 `pauseUntilChosen`에 넣은 이동 컴포넌트(수평 이동 `ship`과 T레버 `verticalThrust`)의 `enabled`만 꺼 둔다. 선택 전에 스틱을 건드려 건담이 흘러가는 일을 막기 위해서다. 선택하면 `Apply`가 건담 몸체와 루트 사이의 오프셋을 계산해 `MobileSuitRoot`를 옮기고, 같은 프레임에 몸체도 직접 옮긴다("no one-frame lag before the follower runs"). 콜로니 지점은 런타임에 `ColonyStructure`에서 계산하므로 땅 높이가 정확히 반영되고, 콜로니가 없는 장면에서는 SPACE 버튼만 남는다. `OnDisable`에서 리스너를 떼어 낼 때도 "Only our own listener - the same events feed the other hand scripts"라고, 같은 손 이벤트를 쓰는 다른 스크립트를 건드리지 않게 주의했다.

11. "콕핏을 줄라 멀리 배치해"

도시가 생기자 전에 없던 문제가 터졌다. 이진호의 보고는 이랬다.

문제 보고 · 콕핏이 건담을 따라다니니까 콕핏에서 자꾸 밖에 사물이 들어와서 시야를 가려 콕핏을 줄라 멀리 배치해 건담을 안따라 다니게해

원인은 구조에 있었다. 콕핏 내부(`CockpitInterior`), 조종사가 앉은 XR 리그, 콕핏 조명(`Cockpit_FillLight`)은 모두 `MobileSuitRoot`의 자식이었다. 즉 콕핏은 실제로 건담과 함께 월드 속을 날아다녔다. 빈 우주에서는 문제가 없었지만, 도시의 거리와 건물과 잔해 속을 날자 그것들이 조종사의 실제 카메라에 "콕핏 안에" 보이기 시작했다. 조종사는 돔(HeadCam 큐브맵)을 통해 바깥을 보는데, 그 돔 안쪽으로 진짜 건물이 뚫고 들어온 것이다.

해결은 콕핏을 건담에서 떼어 내는 것이다. 장면 빌드 마지막에 `CockpitStation`이라는 빈 오브젝트를 만들고, 콕핏 내부·XR 리그·조명을 `MobileSuitRoot` 기준의 로컬 위치와 회전을 그대로 유지한 채 이 스테이션 아래로 옮긴다. 스테이션은 출발 위치에서 `CockpitStationOffset = (0, -4000, -8000)` 만큼 떨어진 곳에 고정되어 움직이지 않는다.

```
GameObject station = new GameObject("CockpitStation");
station.transform.SetPositionAndRotation(
    suitRoot.transform.position + CockpitStationOffset, suitRoot.transform.rotation);
foreach (Transform t in new[] { interior.transform, xrOrigin?.transform, fillLight?.transform })
{
    if (t == null) continue;
    // MobileSuitRoot 기준 자세를 그대로 옮겨 붙인다
    Vector3 lp = suitRoot.transform.InverseTransformPoint(t.position);
    Quaternion lr = Quaternion.Inverse(suitRoot.transform.rotation) * t.rotation;
    t.SetParent(station.transform, false);
    t.localPosition = lp;
    t.localRotation = lr;
}
```

(실제 코드는 `?`. 대신 삼항 연산자로 null을 거른다.) 이렇게 해도 아무것도 깨지지 않는 이유를 주석이 정리했다. `MobileSuitRoot`는 전과 똑같이 움직이고 건담도 그것을 따라간다. 콕핏의 "월드 위치"를 읽는 코드는 없고 "방향"만 읽으므로 조준, 시점, 스틱, 터치 화면이 그대로 동작한다. XR 리그는 부모만 바뀌었을 뿐 설정은 하나도 바뀌지 않았다. 조종사는 이제 오직 돔을 통해서만 세계를 본다.

거리 선택에도 이유가 있다. "~9 km from the battlefield - well past the pilot camera's 1 km far clip in every direction from the colony, wreckage and start area - yet close enough to the world origin that XR tracking keeps sub-millimetre float precision (much farther, e.g. 60 km, would make the cabin and hands visibly jitter)." 조종사 카메라의 원거리 클립(1km) 밖이어야 콜로니도 잔해도 보이지 않는다. 그렇다고 너무 멀면 float 정밀도가 떨어져 손과 실내가 떨린다. (0, -4000, -8000)은 원점에서 약 8.9km로, 그 사이의 타협점이다. 이 "먼 좌표"는 오후에 가운데 디스플레이 깜빡임의 또 다른 원인으로 다시 등장한다.

12. 콧손질 두 가지

"위아래로 이동하는 손잡이가 너무 뒤에 있어"

전날 만든 T자 상하 이동 레버에 대한 요청이다.

요청 · 위아래로 이동하는 손잡이가 너무 뒤에 있어 조금 앞으로 옮겨줘 근데 왼손 조종기를 방해하지 않게

. 레버 위치는 앉은 자세의 어깨에서 켄 상수로 정해져 있었다. 앞으로 뻗는 거리 `TLeverReachForward` 를 0.34에서 0.42m로 8cm, 바깥쪽 거리 `TLeverOutboard` 를 0.14에서 0.17m로 3cm 늘렸다.

"왼손 조종기를 방해하지 않게"라는 조건은 계산으로 확인했다. 주석에 따르면 새 손잡이는 콧손 기준 약 (-0.36, 1.04, 0.09)에 있고, 왼쪽 조이스틱의 손잡이 공은 (-0.22, ~0.89, 0.18)에 있다. 레버가 스틱보다 9cm 뒤, 14cm 바깥, 15cm 위에 있으므로 "a hand on either never touches the other", 어느 쪽을 잡은 손도 다른 쪽에 닿지 않는다. 앞으로만 옮기지 않고 바깥으로도 3cm 비켜 놓은 것이 그 때문이다.

빔샤벨: 락온이 없으면 시점, 락온되면 공격

빔샤벨 모드에서는 오른쪽 조이스틱이 샤벨 휘두르기 전용이 되면서 시점을 돌릴 수단이 없었다. 이견호의 요청은

요청 · 빔샤벨 상태에서 아무도 락온 안되어있을때는 화면조작 락온이 되면 그때 공격기능으로

였다.

`WeaponModeController` 에 `UpdateSaberOwner` 가 생겼다. 빔샤벨 모드일 때 매 프레임 락온 링 (`OrbitHUDTargetLock.CurrentTarget`)을 보고, 살아 있는 적이 잡혀 있으면 시점 컨트롤러에서 오른쪽 스틱을 떼어 내고(`SetViewStickDetached(true)`) 시점은 `SaberLookAssist` 가 적을 따라가게 하며, 스틱 입력을 샤벨에 준다. 락온이 없으면 샤벨을 즉시 방어 자세로 돌리고 스틱을 다시 시점에 돌려준다. 샤벨 쪽에는 `BeamSaberArmController.inputEnabled` 플래그가 생겨, 꺼져 있으면 스틱 기울기 대신 `Vector2.zero` (방어 자세)를 쓴다.

세심한 부분은 전환 순간이다. 시점을 돌리던 중에 락온이 걸리면, 기울어진 스틱이 그대로 샤벨 휘두르기로 바뀌어 원치 않는 베기가 나간다. 그래서 스틱 주인이 바뀌는 것은 스틱 기울기가 0.35(`saberRecenterThreshold`) 아래로 돌아온 뒤로 미뤘다. 락온이 풀릴 때도 샤벨은 즉시 방어로 돌아가지만, 시점은 스틱이 중앙으로 돌아온 다음에 다시 스틱을 받는다. 이전의 `SetConsumersDetached` 는 빔샤벨 모드에 들어가면 시점 스틱을 무조건 떼었는데, 이제는 소총 조준만 끄고 시점 스틱은 `UpdateSaberOwner` 가 주고받는다.

`SaberLookAssist` 에도 `followOnlyLocked = true` 가 생겼다. 예전에는 락온이 없어도 마지막으로 잡았던 적이나 가장 가까운 적을 따라 시점을 돌렸는데, 그러면 "락온 안 되어 있을 때는 화면 조작"이 성립하지 않는다. 이제는 링이 잡은 적만 따라간다(옛 동작은 플래그를 끄면 돌아온다).

"시야가 아래로 떨어짐"

같은 기능에 대한 문제 보고도 있었다.

문제 보고 · 빔샤벨을 가지고 상대에게 다가가면 시야가 아래로 떨어짐 락온한 상대에 머리쪽을 바라보게

. 원인은 `SaberLookAssist.AimPoint` 가 적의 렌더러 바운드 중심을 보게 되어 있었다는 데 있다. 18m짜리 자쿠의 몸 중심은 건담 머리 카메라보다 한참 아래이므로, 가까이 다가갈수록 시선이 점점 아래로 꺾였다. 이제는 적의 자식 중 이름이 "Head"인 뼈를 찾아(대상이 바뀔 때 한 번만 검색해 캐시) 그 위치를 보고, 머리 뼈가 없으면 바운드 중심에서 높이 반쪽의 75% 위를 본다.

13. 자쿠: 사격 자세와 콜로니 순찰

"자쿠가 총쏘는 자세로 자쿠머신건을 잡고 쏘는거야"

전날 자쿠는 머신건을 오른손 하나로 들고 쏘다. 이건호가 원한 것은 진짜 사격 자세였다.

요청 · 자쿠가 총쏘는 자세로 자쿠머신건을 잡고 쏘는거야

. `ZakuMachineGun` 의 클래스 주석이 새 자세를 세 가지로 정의한다. 오른 주먹은 권총 손잡이, 왼 주먹은 총열 아래 앞손잡이(두 팔 모두 실제 뼈에 2본 IK), 개머리판은 오른 어깨에 붙이고 총열은 어깨 높이로 건담의 가슴을 겨눔, 상체는 왼 어깨가 앞으로 나오게 비틀고 총 쪽으로 약간 기울인 "블레이드 스탠스".

먼저 총이 바뀌었다. 앞손잡이 위치 `ZakuGunModelForeGrip = (0, -0.090, 0.219)` (모델 좌표, 총열 아래의 무늬진 손잡이)를 새로 찍었고, 총 배율을 9에서 8로 줄였다("short enough for a two-hand hold"). 9m 총은 양손으로 잡기에 너무 길었던 것이다. 그에 맞춰 손잡이 점(-1.0, -1.45 → -0.9, -1.36)과 총구 점(4.5 → 4.0)도 바뀌었다.

조준 방식도 바뀌었다. 예전에는 어깨에서 조준 방향으로 팔 길이의 72%(`aimReach`) 나가고 12%(`aimDrop`) 내린 곳에 손을 뻗다. 이제는 실제 사격 자세의 치수를 미터로 준다. 오른손 손잡이는 오른 어깨에서 조준 방향으로 2.1m 앞(`gripForward`), 1.0m 아래(`gripDrop`), 몸 중심 쪽으로 0.9m 안(`gripInward`, 개머리판이 어깨 오목한 곳에 오게)이다. 상체(Spine2, 없으면 Spine1)는 위축으로 30°(`stanceTwist`) 돌리고 앞으로 6°(`stanceLean`) 숙이며, 머리는 반대로 30° 돌려 계속 목표를 보게 한다.

왼손 쥐는 법은 오른 주먹의 쥐는 점을 자쿠의 좌우 대칭면으로 거울 반사해 만든다(`Mirror`).

앞손잡이에서는 엄지가 총열을 따라, 손가락이 가로로 감싸는(손바닥이 위) 방향이다. 그리고 앞손잡이가 왼팔이 닿지 않는 곳에 있으면 총을 몸 쪽으로 미끄러뜨린다.


```
// 앞손잡이가 왼팔 길이의 96% 안에 들 때까지 총을 뒤로 민다 (최대 4회)
float maxReach = 0.96f * (_l11 + _l12);
for (int it = 0; it < 4; it++)
{
    Vector3 lWrist = gunPos + gunRot * supportPoint - lHandRot * _leftFist;
    float over = Vector3.Distance(leftUpperArm.position, lWrist) - maxReach;
    if (over <= 0f) break;
    gunPos -= _aimDir * (over + 0.05f);
}
}
```

"so the left hand never floats", 왼손이 허공에 뜨지 않게 하기 위한 장치다. 순서는 오른손 IK로 총을 손에 붙이고, 총이 실제로 놓인 위치의 앞손잡이에 왼손 IK를 거는 식이다. 2본 IK 함수 `SolveTwoBone` 은 오른팔 전용 필드를 쓰던 것을 빼와 길이를 인자로 받는 정적 함수로 바꿔 양팔에 같이 쓴다. 팔꿈치 방향(pole)도 조금 조정했다. 사거리(260m) 밖에서는 자세를 풀고 총을 오른손에 들고 다니며, 자세 진입·해제는 0.5초 동안 `SmoothStep`으로 섞인다.

콜로니를 지키는 자쿠

13:14 커밋에서 자쿠 세 기는 우주가 아니라 콜로니 도시에 배치되었다. `PlaceZakuEnemy` 에 콜로니가 넘어오면 `ColonyZakuSpawns()` 를 쓰는데, 처음에는 대로($x = 0$), $x = -180$, $x = 320$ 의 땅 위, 끝판에서 600·1000·1500m 들어간 곳이었다. 주석의 근거는 "거기를 전장으로 사용하게"와 "내가 가서 싸우면되니까"다.

`ZakuCombatAI` 에는 `arena` (콜로니) 필드와 순찰 동작이 생겼다. 건담이 450m(`aggroRange`) 밖이면 출발점 둘레 70m(`patrolRadius`) 안의 무작위 지점 사이를 초속 5m(`patrolSpeed`)로 6~14초마다 목적지를 바꿔 가며 거닌다. 이동 결과는 언제나 `ColonyStructure.ResolveWalker` 를 통과한다. 이 함수는 발을 땅 띠 폭 (± 1550 m)과 원통 길이 안으로 클램프하고, 땅 높이에 세우고, 반지름 4.5m(`walkerRadius`)로 건물 상자 밖으로 민다(지붕 착지는 허용하지 않는다). 그래서 자쿠는 건물을 돌아 걷고, 콜로니 밖 우주로 건담을 쫓아 나가지 않는다.

14. 14:15 커밋: 전장을 진짜 전장으로

"건물뒤에 보이지 않는 적은 락온되면 안됨"

이 요청은 이미 13:14 커밋에 첫 구현이 들어 있었다. `OrbitHUDTargetLock` 에 `colony` 필드(비어 있으면 자동 검색)와 `InSight` 가 생겨, HeadCam에서 적의 바운드 중심까지, 그게 막히면 머리 쪽(중심 + 반높이의 80%)까지 직선을 그어 둘 다 막히면 "보이지 않음"으로 친다. 지붕 위로 머리만 내민 적은 보이는 것으로 한다는 뜻이다. 보이지 않는 적은 새로 잡을 수 없고, 이미 잡은 적은 0.35초(`occludedGrace`)까지만 가려져도 락을 유지한다. 가는 기둥 하나 지나갔다고 락이 풀리지 않게 하기 위해서다.

시선 판정 함수 `ColonyStructure.SegmentBlocked(a, b)` 의 첫 판은 이랬다. 두 점의 안팎이 다르면 이분법 24회로 선분이 원통 피부를 지나는 점을 찾아, 그 점이 파공 안이 아니면 막힘. 둘 다 밖이면 선분을 32등분해

하나라도 원통 안에 들어가면 막힘. 둘 다 안이면 원통은 볼록하므로 선체에는 닿지 않으니 건물 상자만 검사 (SegmentHitsBoxes, 축 정렬 상자의 슬랩 교차).

14:15 커밋에서 이 부분이 다듬어졌다. 락온 링이 건물 모서리에서 잡혔다 풀렸다 깜빡이는 문제가 있었던 듯, 새 적은 0.15초(captureSightTime) 동안 계속 보여야 잡을 수 있게 했고("stops the ring flashing on/off at building edges"), 가림 유예는 0.35초에서 0.6초로 늘렸다. 대상별로 처음 보인 시각을 사전(_seenSince)에 기록하고, 가려지면 지운다.

"총알이 건물을 뚫으면 안됨"

락온은 막혔는데 총알은 건물을 뚫고 지나갔다. 이견호의 지적은 짧았다.

문제 보고 · 총알이 건물을 뚫으면 안됨

. 문제는 "막혔나?"만 답하는 SegmentBlocked로는 총알을 어디서 멈출지 알 수 없다는 점이였다. 그래서 Raycast(a, b, out hit)가 새로 생겼고, SegmentBlocked는 그 결과만 돌려주는 얇은 함수가 되었다.

Raycast는 선분을 0~1의 매개변수 t로 보고 가장 먼저 부딪히는 t를 찾는다. 건물은 SegmentFirstBoxHit로 (예전엔 하나라도 닿으면 바로 true를 돌려줬지만 이제는 모든 상자 중 가장 작은 진입 t), 선체·끝판은 안팎이 다를 때 이분법으로 피부 통과점의 t를, 둘 다 밖일 때는 32등분 표본으로 처음 안에 들어가는 구간을 찾은 뒤 다시 이분법 20회로 정확한 진입 t를 구한다. 파공을 통과하는 경우는 막힘이 아니다. 가장 작은 t의 점이 hit다. 주석의 설명도 "the hull (seen from outside)"에서 "the land, the end caps ... or the hull"로 바뀌었다.

총알들은 런타임에 생성되므로 장면 참조를 들고 있지 않다. 그래서 ColonyStructure.Active라는 정적 속성을 두어, 활성화될 때 자신을 등록하고 비활성화될 때 지우게 했다. 세 종류의 탄이 이것을 쓴다.

탄	처리
BeamRifleShot	이번 프레임 이동 구간을 먼저 벽에 쏘 보고, 맞으면 이동 거리를 벽까지로 줄여 구체 캐스트. 적에 먼저 맞지 않았으면 벽 지점에서 섬광 후 소멸
HeadVulcanBullet	예광탄 끝의 이동 구간을 벽까지로 잘라 RaycastAll. 적에 안 맞았으면 벽 지점에서 Impact
ZakuBullet	이동 끝점을 벽 지점으로 당기고 건담 캡슐 판정. 안 맞았으면 새 Spark로 0.06초짜리 불꽃(크기 70%) 후 소멸

자쿠의 총알도 건물에 막히므로, 건물 뒤에 숨는 것이 실제 엄폐가 된다.

격자 가속

총알은 수백 발이 날고, 락온은 적마다 매 프레임 두 줄을 굶고, 자쿠는 열 기가 매 프레임 걷는다. 그런데 건물 충돌 상자는 수천 개다. 모든 질의가 매번 전체 상자를 훑으면 감당할 수 없다. 13:20 무렵 다시 쓰인 ColonyStructure에 그 답이 들어 있다. "Uniform grid over the building boxes (x/z), so line-of-sight, bullet and walking queries only test the few boxes nearby."

```
const float GridCell = 128f; // x/z 128m 칸
Dictionary<long, List<int>> _grid; // 칸 → 그 칸에 걸친 상자 번호들
int[] _stamp; int _stampId; // 한 질의에서 같은 상자 중복 검사 방지
static int Cell(float v) => Mathf.FloorToInt(v / GridCell);
static long Key(int cx, int cz) => ((long)cx << 32) ^ (uint)cz;
```

EnsureGrid 는 상자 수가 바뀌었을 때만 격자를 다시 만든다. 상자 하나가 여러 칸에 걸치면 모든 칸에 등록한다. 질의 때는 선분(또는 몸 원기둥)의 x/z 범위에 걸친 칸만 보고, 질의마다 _stampId 를 올려 이미 본 상자는 건너뛴다. 선분이 4096칸을 넘게 걸치면(수 km짜리 선분) 격자가 오히려 손해이므로 예전 방식의 전수 검사(SegmentFirstBoxHitBrute)로 돌아간다. ResolveBoxes 도 발 둘레 r 범위의 칸에서 후보(_near)만 모아 검사한다.

"자쿠도 내가 건물뒤에 있으면 몰라야 하고"

락온도 총알도 건물에 막히는데 자쿠만 투시를 하면 공평하지 않다. 요청은

요청 · 자쿠도 내가 건물뒤에 있으면 몰라야 하고

였다. ZakuCombatAI 에 시야가 생겼다. 0.2초(sightInterval)마다 자쿠의 눈(발 위 16m, eyeHeight)에서 건담의 몸(발 위 9m)으로, 막히면 머리(발 위 17m)로 선을 그어 SegmentBlocked 로 확인한다. 거리 제한은 기존의 aggroRange (450m)를 그대로 쓴다. 보이면 CanSeeTarget 이 참이 되고 마지막 목격 위치와 시각을 기록한다.

13:14 판의 "450m 밖이면 순찰"이라는 거리 조건은 지워지고, 행동이 세 갈래로 바뀌었다. 보이면 기존 전투. 안 보이지만 기억이 12초(memoryTime) 안이면 Hunt . 기억도 사라지면 Patrol . Hunt 는 마지막 목격 지점으로 초속 9m(huntSpeed)로 가서, 20m 안에 도착하면 그 둘레 45m(searchRadius) 안의 무작위 지점을 3~6초마다 바꿔 가며 순찰 속도의 1.4배로 뒤편다. 머리 추적도 보이지 않으면 건담이 아니라 마지막 목격 지점을 향한다. ZakuMachineGun 도 ai.CanSeeTarget 이 거짓이면 사격 자세에 들어가지 않는다. 주석의 요약대로 "It never tracks, faces or shoots at a Gundam it can't see".

"자쿠가 더 있어서 콜로니 안에 더 배치 되어 있었으면 좋겠어"

마지막으로 적의 수다.

요청 · 자쿠가 더 있어서 콜로니 안에 더 배치 되어 있었으면 좋겠어

. ColonyZakuSpawns 가 세 곳에서 열 곳으로 늘었다. x는 대로(0)와 남북 도로 중심선(±180, ±320, ±460)위에서 고르고, 끝판에서의 깊이는 600m부터 5200m까지 퍼뜨렸다.

#	x	안쪽 깊이	#	x	안쪽 깊이
1	0	600	6	-460	1800
2	-180	1000	7	0	3200
3	320	1500	8	460	3800

4	180	2100	9	-180	4400
5	-320	2600	10	320	5200

"from just inside the breach to deep downtown instead of 3 near the entrance". 마지막 5200m 지점은 도심(4600m)을 지나 교외에 걸친다. 입구 광장으로 들어서면 가까운 자쿠부터 차례로 만나고, 도심 깊이 들어갈수록 더 많은 적이 기다리는 배치다.

15. 화질, 깜빡임, 그리고 2cm

"화질이 너무 양좋아"

콜로니는 수 km 떨어진 곳의 작은 창문까지 그려야 하는 장면이다. 이견호의 보고는

문제 보고 · 화질이 너무 양좋아

였다. 돔이 샘플링하는 360° 큐브맵의 면당 해상도 `HeadCam360CubemapResolution` 이 512에서 2048로 올랐다. 면 하나당 픽셀이 16배다. 이 값은 뒤의 최적화 절에서 다시 내려온다.

"가운데 디스플레이가 자주 번쩍거린"

더 까다로운 보고는 이것이었다.

문제 보고 · 가운데 디스플레이가 자주 번쩍거린

. 가운데 디스플레이에는 HeadCam의 평면 영상(HEAD CAM 인셋)과 HUD 글자가 있다. Claude는 이 깜빡임을 z-파이팅, 즉 깊이 버퍼 정밀도가 모자라 두 면이 앞뒤를 다투는 현상으로 진단했고, 원인이 두 군데에 있었다.

첫째는 HeadCam 쪽이다. 콜로니 때문에 원거리 클립이 16km가 되었는데, 근거리 클립은 여전히 5cm였다. 원근 투영의 깊이 정밀도는 근거리/원거리 비에 크게 좌우되는데, 0.05 대 16000이면 먼 곳에 거의 정밀도가 남지 않는다. 그래서 먼 콜로니 건물들이 서로 z-파이팅을 하며 HeadCam 영상 자체가 깜빡였다. 해결은 두 가지다. 근거리 클립을 0.05에서 0.5m로 올렸다. 주석은 "own body is culled, nothing is that close"라고, 자기 몸은 어차피 레이어로 가려져 50cm 안에 보일 것이 없다는 근거를 댔다. 그리고 HeadCam의 렌더 텍스처 두 개(평면 1024와 큐브맵)의 깊이 버퍼를 16비트에서 24비트로 바꿨다. 이미 만들어진 에셋도 깊이가 24가 아니면 해제하고 다시 만들게 조건을 고쳤다("16-bit depth z-fought -> flicker").

둘째는 디스플레이 자체다. 화면 상자와 그 앞의 UI 캔버스는 2mm 떨어져 있었다. 그런데 콕핏이 원점에서 약 9km 떨어진 `CockpitStation` 으로 옮겨지면서, 그 먼 월드 좌표에서는 2mm 간격이 깊이 정밀도 안에서 구분되지 않게 되었다. `CreateWorldCanvas` 의 앞면 간격을 2mm에서 5mm로 늘렸다. 주석은 "at the CockpitStation's far world coordinates the screen box and this canvas z-fought"라고 적었다. 오전에 시야 문제를 풀려고 한 원거리 배치가 오후의 깜빡임으로 돌아온 셈이다.

한 증상, 세 가지 수정

"가운데 디스플레이 깜빡임" 하나에 대해 17번 커밋은 세 군데를 고쳤다. HeadCam 근거리 클립 0.05 → 0.5m, HeadCam 렌더 텍스처 깊이 16 → 24비트, 디스플레이 캔버스 간격 2 → 5mm. 앞의 둘은 16km 원거리 클립(콜로니) 때문에, 셋째는 9km 떨어진 콕핏 스테이션 때문에 필요해졌다. 큰 월드를 만들면 깊이 정밀도라는 비용이 이렇게 여러 곳에서 나타난다.

"디스플레이를 진짜 아주 조금 아래로 내려줘"

마지막 콕핏 손질은 미세 조정이다.

요청 · 디스플레이를 진짜 아주 조금 아래로 내려줘 시선이 방해되서

. 디스플레이 묶음은 전날 "조종기 사이 공간에 들어올 사이즈로" 반으로 줄여 조이스틱 손잡이 공 위로 올려둔 상태였고, 옆 화면 아래 끝이 손잡이 위로 얼마나 떠 있을지를 `DisplayBottomAboveGrip` 가 정한다. 이 값을 0.08에서 0.058m로 줄여 약 2cm 내렸다. "진짜 아주 조금"이라는 말 그대로다. 주석은 여전히 "clear of the knuckles on the grip", 손잡이를 권 손가락 마디에는 닿지 않는다고 확인해 둔다.

16. 오후 늦게 · 최적화

14:15 커밋 뒤로 이날의 커밋은 없다. 그러나 14:39 무렵 `Shaders/HeadCamInsetCube.shader` 가, 14:42 무렵 `Models/Optimized` 폴더가 생겼고, 그 작업은 다음 날 커밋(10월 1일 12:10)에 섞여 들어갔다. 여기서는 최신 코드(`GundamCockpitSetup.cs` 의 `OptimizeHeavySceneMeshes`, `OptimizedTriangleTarget`, `GetOrCreateReducedMesh`, `WireHeadCamInsetToCubemap`, `GundamHeadCam360.cs` 의 `faceCulling`, `HeadCamInsetCube.shader`)를 근거로 그 내용을 정리한다. 계기가 된 요청은 코드 곳곳에 인용되어 있다.

문제 보고 · 랙이 조금 거림 화질은 별로 차이 없고 최적화좀해줘



자쿠 원본(왼쪽)과 감축본(오른쪽) 비교 렌더.



건담 원본과 14.5만 삼각형 감축본 비교.

어디가 무거웠나

최적화 절의 머리 주석이 진단을 담고 있다. 프레임 비용의 대부분은 장면의 다른 무엇보다 훨씬 조밀한 몇몇 수입 모델이었다. 자쿠 한 기가 50만 삼각형(스킨드 정점 38.3만)이고 머신건도 50만이다. 자쿠가 열 기가 되었으니 그것만으로 1000만 삼각형이고, 그것을 보는 큐브 면·뷰마다 다시 그리며, 380만 정점을 매 프레임 스키닝한다. 빔 라이프와 빔샤벨 자루도 각각 50만이다. 그에 비하면 콜로니 전체는 약 100만 삼각형이다. 절차적 저폴리 콜로니가 아니라, VARCO 등으로 만든 고밀도 무기·기체 모델이 범인이었다.

축소 메시: Unity의 Mesh LOD 단순화기로 한 단계만 골라 쓰기

방법은 무거운 모델마다 축소본을 만들어 바꿔 끼우는 것이다. 단순화에는 Unity 자체의 Mesh LOD 생성기 (MeshLodUtility.GenerateMeshLods)를 썼다. 이 단순화기는 삼각형만 버리고 원본 정점은 그대로 두므로, UV·법선·뼈 가중치·바인드 포즈가 정확히 보존되고 SkinnedMeshRenderer의 뼈와 애니메이션이 그대로 동작한다. 이게 핵심 선택 이유다. 자쿠의 걷기·사격 자세가 모두 뼈를 직접 돌리는 절차적 애니메이션이기 때문이다.

BuildReducedMesh의 절차는 이렇다. 원본을 복제해 LOD를 생성하고, 목표 삼각형 수 이하가 되는 첫 단계를 고른다. 그 단계의 서브메시별 인덱스를 읽어 오고(LOD 범위가 절대 인덱스인지 서브메시 상대 인덱스인지도 판별한다), 실제로 쓰인 정점만 새 번호로 압축한다. 그다음 정점·법선·탄젠트·색·UV 8채널(차원 2/3/4 모두)·정점별 뼈 가중치(개수 제한 없이 BoneWeight1)·바인드 포즈·블렌드 셰이프까지 원본에서 그 정점들만 골라 옮긴다. 정점이 65535개를 넘으면 32비트 인덱스를 쓴다. 결과는

Assets/Models/Optimized/{FBX이름}__{메시이름}__{목표}.asset으로 한 번 저장되어 다음 빌드부터 재사용되고, CPU 사본을 남기지 않도록 Read/Write를 끈다. 원본 FBX는 건드리지 않는다.

OptimizeHeavySceneMeshes는 장면을 다 지은 뒤 모든 렌더러를 훑어, 정점 10만 개 (HeavyMeshVertexThreshold) 이상이고 FBX에서 온 메시만 축소본으로 바꿔 끼운다. 같은 원본은 사전으로 한 번만 처리한다. 어떤 카메라도 그리지 않는 GundamBodyLayer(조종사 자신의 건담 몸)는 건너뛴다. 실패하면 경고만 남기고 원본을 유지한다.

목표 수치는 비교 렌더로 골랐다

```
// 단순화기 단계별 비교 렌더로 결정: 이 값들은 50만 원본과 똑같이 보이고,  
// 한 단계 더 내리면 표면이 울퉁불퉁해지기 시작한다.  
static int OptimizedTriangleTarget(string fbxPath, bool skinned)  
{  
    if (fbxPath.IndexOf("ZakuMachineGun", ...) >= 0) return 50000;  
    if (fbxPath.IndexOf("HeatHawk", ...) >= 0) return 40000;  
    if (fbxPath.IndexOf("ZakuBazooka", ...) >= 0) return 50000;  
    // 건담: ~140.7k 단계가 140k를 살짝 넘고, 그 아래 단계(76k)는 울퉁불퉁  
    if (skinned && fbxPath.IndexOf("Gundam", ...) >= 0) return 145000;  
    return skinned ? 140000 : 85000;  
}
```

대상	목표 삼각형	비교
자쿠(스킨드 기본값)	140,000	원본 50만

건담(스킨드)	145,000	LOD 단계가 약 140.7k라 140k 목표로는 76k까지 내려가 버리므로 145k
자쿠 머신건	50,000	원본 50만
히트호크	40,000	모델은 10월 1일에 들어옴(최신 코드에 함께 정리됨)
자쿠 바주카	50,000	모델은 10월 1일에 들어옴
기타 정적 메시(라이플, 사벨 자루 등)	85,000	원본 50만

이 숫자들은 추정이 아니라 비교로 정해졌다. 주석은 "Chosen from side-by-side renders of each simplifier level: the Zaku at ~135k triangles and the weapons at ~50-85k look the same as the 500k originals; one level lower the surfaces start to go lumpy"라고 적었다. 단순화기가 만드는 단계들을 나란히 렌더해 보고, 원본과 구별되지 않는 가장 낮은 단계를 골랐다는 것이다. 건담의 145k라는 어중간한 숫자도 그 결과다. 건담의 단계는 약 140.7k인데, 목표를 140k로 두면 "목표 이하인 첫 단계"가 그다음인 76k가 되어 표면이 울퉁불퉁해진다.

시행착오: 런타임 LOD를 붙였다가 뺐다

처음 판은 축소본에 다시 Unity의 런타임 Mesh LOD를 얹어, 멀리 있을 때 더 낮은 단계로 자동 전환되게 했다. 흔적이 코드에 남아 있다. `GetOrCreateReducedMesh` 는 기존 에셋이 있어도 `lodCount > 1` 이면 "Older version of this asset (built with runtime LODs)"으로 보고 새로 만들어 덮어쓴다. 이때

`EditorUtility.CopySerialized` 로 내용만 바꿔 에셋의 GUID, 즉 장면이 걸고 있는 참조를 유지한다.

`BuildReducedMesh` 끝의 주석이 뻥 이유다. "No runtime Mesh LODs on top: tested, Unity's automatic selection switched to the lumpy lower levels already at mid range." 자동 선택이 중간 거리에서 이미 울퉁불퉁한 낮은 단계로 넘어가 버렸다. 그래서 축소본은 고정 한 단계만 쓴다.

HEAD CAM 인셋을 큐브맵에서 뽑기

두 번째 큰 낭비는 렌더 횟수였다. 가운데 디스플레이의 HEAD CAM 인셋은 HeadCam이 1024×1024 평면 텍스처에 장면 전체를 매 프레임 한 번 더 그린 결과였다. 그런데 바로 그 카메라 위치에서

`GundamHeadCam360` 이 이미 큐브맵 여섯 면을 그리고 있다. 같은 위치의 같은 장면을 두 번 그리는 셈이다.

`HeadCamInsetCube.shader (Custom/HeadCamInsetCube)`는 이 두 번째 렌더를 없앤다. `uGUI RawImage`에 쓰는 셰이더로, 인셋의 각 픽셀이 HeadCam의 절두체를 통해 바라보는 방향을 계산해 라이브 큐브맵에서 그 방향을 샘플링한다.

```
half4 Frag(Varyings IN) : SV_Target
{
    float2 ndc = IN.uv * 2.0 - 1.0;    // 인셋 위 위치 → -1..1
    float3 dirCam = float3(ndc.x * _TanHalfFov * _Aspect, ndc.y * _TanHalfFov, 1.0);
    float3 dir = RotateByQuat(normalize(dirCam), _CamRotQ); // 카메라 회전 적용
    half3 col = SAMPLE_TEXTURECUBE(_CubeTex, sampler_CubeTex, dir).rgb;
    return half4(col * IN.color.rgb, IN.color.a);
}
```


큐브맵의 면은 월드 축에 정렬되어 있으므로 월드 방향만 있으면 된다. 카메라 회전(쿼터니언 `_CamRotQ`), 세로 반화각의 탄젠트(`_TanHalfFov`), 종횡비(`_Aspect`)는 `GundamHeadCam360` 이 매 프레임 넣어 준다. UI의 정점 색과 알파는 유지하고, `uGUI`가 늘 설정하는 `_MainTex` 는 받기만 하고 쓰지 않는다. 빌드 때 `WireHeadCamInsetToCubemap` 이 `HeadCamInset/Feed` 라는 `RawImage` 를 찾아 이 셰이더의 재질을 입히고 `cam360.insetImage` 에 연결한다. 런타임에 `GundamHeadCam360.Start` 는 인셋 재질 사본에 큐브맵을 넣고, 인셋의 텍스처를 비우고, `headCam.enabled = false` 로 카메라의 자동 평면 렌더를 끈다. `RenderToCubemap` 은 카메라 컴포넌트가 꺼져 있어도 동작하므로 큐브맵은 계속 갱신된다. 셰이더나 대상을 못 찾으면 예전 평면 렌더를 유지한다.

큐브면 컬링: 보이는 면만 매 프레임

세 번째는 큐브맵 여섯 면 자체다. 조종사는 한 번에 여섯 면을 다 보지 않는다. `GundamHeadCam360` 에 `faceCulling` (기본 켜짐)이 생겨, 매 프레임 조종사가 실제로 볼 수 있는 면과 HEAD CAM 인셋이 쓰는 면만 다시 그리고, 나머지는 돌아가며 갱신한다.

`VisibleFaceMask` 는 조종사의 XR 카메라 회전에 오른쪽 스틱 시점 회전(`CockpitViewController.LookRotation`, 돔 셰이더가 쓰는 것과 같은 회전)을 곱한 방향으로 절두체를 만들고, 그 절두체를 9×9 격자 방향으로 표본 추출해 각 방향이 속한 면(+X 0, -X 1, +Y 2, -Y 3, +Z 4, -Z 5)을 비트 마스크로 모은다. 화각은 헤드셋이 좁게 보고해도 최소 반화각 55° (`minHalfFovDeg`)로 잡고, 거기에 18° (`cullMarginDeg`) 여유를 더해(최대 80°) 한 프레임보다 빠른 고개 돌림에도 낡은 면이 보이지 않게 했다. 인셋용으로는 HeadCam 화각 + 6° 를 5×5 로 표본 추출한다. 그리고 2프레임마다(`hiddenFaceInterval`) 숨은 면 하나를 순서대로 골라 마스크에 더한다. 첫 프레임은 여섯 면을 모두 그린다. 그 마스크를 `headCam.RenderToCubemap(_cubemap, mask)` 에 넘기면 표시된 면만 렌더된다.

그 밖의 정리

같은 요청으로 두 가지가 더 바뀌었다. 큐브맵 면 해상도는 낮에 "화질이 너무 싫어"로 2048까지 올렸던 것을 1536으로 되돌렸다. 주석은 "512 -> 2048 (per '화질이 너무 싫어') -> 1536 ... 2048 cost ~1.8x the pixels of 1536 for no visible gain"이라고, 2048은 1536보다 픽셀이 약 1.8배인데 눈에 띄는 차이가 없었다고 적었다. 이견호의 요청 문장 "화질은 별로 차이 없고"가 이 판단을 뒷받침한다. 또 HeadCam은 한 프레임에 최대 여섯 면을 그리므로, 필요 없는 것을 모두 켜다. HDR 버퍼, URP 카메라 데이터의 깊이 텍스처·불투명 텍스처 복사, 그림자 패스, 후처리, 안티앨리어싱이다.

정리하면 이날 오후의 최적화는 세 갈래였다. 그리는 삼각형 수를 줄이고(무거운 모델을 50만에서 4만 ~14.5만으로), 그리는 횟수를 줄이고(인셋의 중복 렌더 제거, 보이지 않는 큐브면 생략), 한 번 그릴 때의 비용을 줄였다(해상도 1536, HeadCam 부가 패스 제거). 화질을 지키는 기준은 비교 렌더였고, 자동 LOD처럼 눈으로 확인한 결과 품질을 해치는 수단은 되돌렸다.

17. 이날의 정리

거대 구조물, 콜로니 안에 도시, 더 크게·자세하게, 부딪히게	지름 6.4km × 길이 12km 오닐 실린더, 도심 4.6km, 수백 동의 건물, 고가도로, 교외·농지 (BuildColony)
콜로니 안 중력	SuitCollision : 9.8m/s ² , 최대 70m/s, T레버 상승 시 상쇄, 지붕 착지
VARCO 건물 / 외관 소품	랜드마크 5종(모델당 최대 2회), 소품 5종(파공 잔해, 잔해 더미, 항구 모듈, 크레인, 안테나)
디테일을 더 살려줘	텍처 v3(256px, 창만 빛나는 발광 맵), 거리 디테일 레이어 27, 대기 안개·주변광, 안/밖 렌더 토글
상대 통과 금지	적 캡슐과 원기둥 판정, 수평으로 밀어내기
스폰 위치 선택	SpawnSelectPanel : 우주/외관/입구/도심 4곳, 검지 터치
콕핏을 멀리	CockpitStation (0, -4000, -8000)에 고정
T레버 앞으로	앞 0.34 → 0.42m, 바깥 0.14 → 0.17m
샤벨: 락은 없으면 시점, 있으면 공격 / 머리 쪽 보기	UpdateSaberOwner (재중앙 0.35), followOnlyLocked , Head 뼈 조준
자쿠 사격 자세	양손 IK, 블레이드 스탠스 30°/6°, 총 배율 9 → 8
건물 뒤 락온 금지 / 총알 벽 정지 / 자쿠 시야	Raycast · SegmentBlocked , 128m 격자, 포착 0.15초·유예 0.6초, 자쿠 추적·수색
자쿠 더 배치	3기 → 10기, 600~5200m
화질 / 깜빡임 / 디스플레이 2cm	큐브맵 512 → 2048(이후 1536), 근거리 0.5m, 깊이 24비트, 캔버스 5mm, 0.08 → 0.058m
랙 / 최적화	축소 메시(140k/145k/50k/40k/85k), 인셋 큐브맵 셰이더, 큐브면 컬링

빈 우주에서 싸우던 게임은 이날 하루 만에 도시 전장을 얻었다. 그리고 그 도시는 단지 배경이 아니라, 부딪히고, 떨어지고, 숨고, 숨은 상대를 찾는 규칙을 가진 공간이 되었다. 동시에 큰 월드가 가져오는 비용, 즉 깊이 정밀도와 렌더링 부하도 이날 처음으로 정면으로 마주했다. 다음 날의 작업은 이 전장 위에서 무기와 소리를 채우는 쪽으로 이어진다.



콜로니 도심 대로에 배치된 적 자쿠. 콜로니 안에는 자쿠 10기가 흩어져 순찰한다.

10월 1일은 이 프로젝트에서 하루 동안 가장 많은 것이 들어온 날이다. 오전 10시 무렵에는 지금까지 아무 소리도 나지 않던 전장에 처음으로 소리가 붙었고, 11시 무렵에는 「자쿠를 플레이할 수 있게」라는 큰 요청이 들어와 기체 선택 화면, 플레이어용 자쿠, 적 건담 여섯 기, 지온풍 콕핏 색이 한꺼번에 만들어졌다. 이어 두 기체가 걷는 애니메이션과 「우주에서는 걷지 않는다」는 규칙, 자쿠 바주카가 붙었고, 정오 무렵 Asset Store에서 내려받은 부스터 불꽃이 적 기체의 등과 발바닥에 붙었다. 이 모든 변경은 12시 10분 커밋(18번, c974825) 하나에 들어 있다. 같은 커밋에는 9월 30일 오후에 한 최적화 작업과 락온 링 축소도 섞여 있는데, 최적화는 다른 장에서 다룬다. 락온 링은 10월 1일 아침 이전에 이미 손본 상태였다는 것만 짚고 넘어간다. 「락온이 거리에 따라서 작아졌으면」과 「멀리있는 적을 락온하면 너무 멍쳐서 덩어리로 보임」이라는 요청에 따라 `OrbitHUDTargetLock`의 최소 락온 원 반지름이 0.08 m에서 0.035 m로 줄었고, 원이 작아지면 작은 눈금과 큰 눈금이 차례로 사라져 네 개의 게이트 괄호만 남게 되었다.

시각은 이견호의 PC에 남은 파일 시각으로 짐작할 수 있다. `Assets/Audio`의 소리 파일들은 9시 59분부터 10시 17분 사이에 생겼고, 히트 호크 모델 폴더는 10시 52~53분, 자쿠 바주카 모델은 11시 14분 무렵, `MechBoosterFlames.cs`는 12시 07분 무렵에 생겼다. 이 장은 이 순서를 따라 간다.

1. 전장에 처음 소리를 입히다

왜 그냥 3D 사운드로는 안 되는가

소리를 붙이기 전에 먼저 정리해야 할 구조가 있었다. 이 게임에서 이견호가 앉아 있는 콕핏은 건담 안에 있지 않다. 9월 30일에 「콕핏이 건담을 따라다니니까…」라는 문제 때문에 콕핏을 고정된 위치(`CockpitStation`)로 옮겼는데, 그 위치는 기동 전사 루트에서 `CockpitStationOffset = (0, -4000, -8000)`, 즉 전장에서 약 9 km 떨어진 곳이다. 이견호가 보는 바깥 풍경은 건담 머리에 달린 카메라(`HeadCam`)가 큐브맵으로 찍어 콕핏 돔에 비추는 것이다. 문제는 Unity의 `AudioListener`(귀)가 이견호의 XR 카메라, 즉 9 km 떨어진 콕핏에 있다는 점이다. 빔 라이플 총구에 일반적인 3D 사운드를 달면 그 소리는 9 km 밖에서 나는 셈이라 아무것도 들리지 않는다.

그래서 이날 만든 소리는 전부 2D 재생(`AudioSource.spatialBlend = 0`)으로 했다. 자기 기체가 내는 소리(빔 라이플, 빔 사벨, 헤드 발칸, 이동 바람 소리)는 그냥 콕핏 안에서 크게 들리면 되므로 2D로 충분하다. 적이 내는 소리(자쿠 머신건, 적의 근접 무기)는 거리와 방향이 느껴져야 하므로, Claude는 소리의 크기와 좌우 위치를 건담 머리 카메라를 기준으로 직접 계산해서 2D 소리에 넣었다. 즉 "귀는 콕핏에 있지만, 듣는 위치는 HeadCam"이라는 규칙이다.

이날 소리 요청의 순서

이건호는 소리 파일을 하나씩 내려받아 바로바로 붙여 달라고 했다. 순서는 다음과 같다. ①

요청 · 방금다운로드한 사운드를 빔라이플이 나갈때 나오게 해줘

② (Beam_shoot_1.wav)

요청 · 이거를 자쿠가 총을 쏘는 소리로 넣어줘

③ (Light_saber_3.wav)

요청 · 빔샤벨을 휘둘렀을때만 나오게해줘

④ (Wind_sound_1.wav)

요청 · 건담으로 움직일때 이소리가 나오게해줘

⑤

요청 · gundam 안에 있는 optimize havey mesh 이게 뭐임?

→

요청 · 그림 빼줘

⑥ (Machine_gun_2.wav)

요청 · 이거를 자쿠 총소리로 해주고 아까한거는 건담 헤드 발칸 소리로 하자

⑦

요청 · 그리고 그 사운드 이름들을 전부 지금 사용하는 곳에 맞게 이름을 바꿔줘

⑧

요청 · 건담에 속도를 조금 더 빠르게 하고 속도 단위는 km/h로 해줘

빔 라이플 발사음

첫 요청은

요청 · 방금다운로드한 사운드를 빔라이플이 나갈때 나오게 해줘

였다. 플레이어 건담의 빔 라이플은 9월 29일에 만든 `BeamRifleController`가 담당한다. Claude는 여기에 소리 항목을 추가했다. `fireSound` (클립), `fireVolume = 0.9`, 그리고 같은 소리가 연달아 나도 기계적으로 들리지 않도록 매 발마다 음높이를 ± 0.03 범위에서 흔드는 `firePitchJitter = 0.03`이다. `Awake()`에서 `AudioSource`를 하나 만들고 `spatialBlend = 0`으로 둔다. 실제 재생은 탄이 나가는 순간, 즉 `LastFireTime`을 기록하는 발사 함수 안에서 `PlayOneShot`으로 한다. `PlayOneShot`은 이전 발사음이 아직 끝나지 않았어도 겹쳐서 재생되므로 빠르게 쏘아도 소리가 끊기지 않는다.

```
if (_fireAudio != null && fireSound != null)
{
    // 매 발 음높이를 조금씩 다르게
    _fireAudio.pitch = firePitch + Random.Range(-firePitchJitter, firePitchJitter);
    _fireAudio.PlayOneShot(fireSound, fireVolume);
}
```

여기 보이는 `firePitch` (기본 1)는 처음부터 있던 것이 아니라 뒤에 바주카를 만들 때 추가된 기본 음높이 값이다. 빔 라이플은 1 그대로이고, 바주카는 같은 소리를 0.6으로 낮춰 쏜다. 씬을 만드는 에디터 스크립트 `GundamCockpitSetup`은 `Assets/Audio/Gundam_BeamRifle_Fire.wav`를 불러와 넣고, 파일이 없으면 「the rifle fires silently」라는 경고만 남기고 그대로 진행한다. 이후 모든 소리가 이 방식(없으면 경고, 소리 없이 동작)을 따른다.

자쿠의 총소리 — 첫 번째 버전

다음으로 이건호는 `Beam_shoot_1.wav`를 내려받아

요청 · 이거를 자쿠가 총을 쏘는 소리로 넣어줘

라고 했다. 적 자쿠의 머신건은 `ZakuMachineGun` 스크립트다. 앞에서 말한 거리 문제가 여기서 처음 실제로 드러난다. 자쿠는 전장에 있고 귀는 9 km 밖 콧속에 있으므로, Claude는 `PlayShotSound(muzzle)`라는 함수를 만들어 소리 크기와 좌우 위치를 직접 계산했다.

```
// 귀 = 건담 머리 카메라 "HeadCam"
Vector3 d = muzzle - ear.position;
float dist = d.magnitude;
if (dist > soundMaxRange) return; // 1500 m 밖은 안 들림
float vol = fireVolume * Mathf.Clamp01(soundFullVolumeRange / Mathf.Max(dist, 1f));
vol *= 1f - Mathf.SmoothStep(0f, 1f, Mathf.InverseLerp(soundMaxRange * 0.6f, soundMaxRange, dist));
float pan = Mathf.Clamp(Vector3.Dot(d / dist, ear.right), -1f, 1f) * 0.8f;
```

규칙은 이렇다. 80 m(`soundFullVolumeRange`) 안에서는 최대 음량, 그보다 멀면 1/거리로 줄어든다. 900 m(최대 거리 1500 m의 60%)부터 1500 m 사이에서는 부드럽게 0까지 내려가고, 1500 m 밖은 아예 재생하지 않는다. 좌우 위치(`panStereo`)는 총구 방향 벡터와 HeadCam의 오른쪽 벡터의 내적에 0.8을 곱한 값이다. 오른쪽 옆에서 쏘면 오른쪽 귀에서, 정면이면 가운데에서 들린다. 끝까지 한쪽으로 몰지 않으려고 0.8을 곱했다. 귀 역할을 하는 Transform은 씬에서 이름이 "HeadCam"인 오브젝트를 처음 한 번 찾아 정적 변수에 담아 두고, 열 대의 자쿠가 같이 쏜다.

빔 사벨은 「휘두를 때만」

세 번째 파일 Light_saber_3.wav에 대한 요청은

요청 · 빔사벨을 휘두를때만 나오게해줘

였다. 「만」이라는 한 글자가 설계의 핵심이다. 빔 사벨을 꺼내 들고 있을 때나 건담이 날아갈 때, 시점을 돌릴 때는 소리가 나면 안 되고, 팔로 칼을 휘두를 때에만 나와야 한다.

Claude는 9월 29일에 만든 칼날 판정 스크립트 BeamSaberBlade 에 UpdateSwingSound(tip) 을 넣었다. 매 프레임 칼끝 위치를 구한 뒤 **건담 루트의 좌표계로 옮겨서**(루트의 위치와 회전을 빼서) 속도를 낸다. 이렇게 하면 건담이 비행하거나 회전해서 칼끝이 움직인 만큼은 빠지고, 팔이 칼을 휘두른 움직임만 남는다. 속도 단위는 「초당 칼 길이」(칼끝 속도 ÷ (칼날 시작점 + 칼날 길이))다. 9 m짜리 빔 사벨이든 그보다 짧은 히트 호크든 같은 기준을 쓸 수 있게 하려는 것이다.

값	의미
swingStartSpeed = 3.5	칼끝이 초당 칼 길이 3.5배보다 빨라지는 순간 소리 시작(상승 에지)
swingRearmSpeed = 1.2	한 번 울린 뒤에는 칼끝이 이 속도 아래로 내려져야 다음 소리를 낼 수 있음
swingMinInterval = 0.3 s	두 휘두름 소리 사이의 최소 간격
drawQuietTime = 0.5 s	사벨을 꺼낸 직후 팔이 자세를 잡으려 움직이는 0.5초는 휘두름으로 치지 않음
swingSegments	(0~0.95 s), (1.6~2.95 s) — 클립 안의 두 번의 "부웅" 소리를 번갈아 재생
swingFadeOut = 0.06 s	구간 끝에서 딸깍 소리가 나지 않게 짧게 줄임

Light_saber_3.wav 안에는 휘두르는 소리가 두 번 들어 있었다. 그래서 휘두를 때마다 클립 전체를 틀지 않고, 두 구간 중 하나를 번갈아 잘라서 재생한다. AudioSource.time 을 구간 시작점으로 옮겨 Play() 하고, Update() 에서 구간 끝 시각(음높이로 나눈 길이)이 되면 0.06초 동안 음량을 줄인 뒤 멈춘다. 소리 크기는 휘두른 속도에 비례해 Clamp(속도 ÷ 7, 0.6, 1) 배이고, 음높이는 ±0.06 범위에서 흔들린다. 처음 감지할 때 위치를 기억해 두는 변수 _prevTipLocal 은 칼을 꺼낼 때(OnEnable) 초기화하므로, 칼을 꺼내는 첫 프레임의 "순간이동"도 휘두름으로 잘못 세지 않는다.

건담이 움직일 때의 바람 소리

네 번째 요청은 Wind_sound_1.wav와 함께

요청 · 건담으로 움직일때 이소리가 나오게해줘

였다. 새 스크립트 GundamMoveSound 를 기동 전사 루트(MobileSuitRoot)에 붙였다. 설계에서 중요한 결정은 **이동 스크립트를 건드리지 않는다**는 것이다. 왼쪽 조이스틱 이동(ship)과 T 레버 상승(VerticalThrustController)은 이미 복잡하게 얹혀 있으므로, 이 스크립트는 그 안의 속도 값을 읽지 않고 루트가 매 프레임 실제로 움직인 거리를 시간으로 나눠 속도를 낸다. 이렇게 하면 어떤 방식으로 움직이든

소리가 난다. 단, 한 프레임에 600 m/s(`teleportSpeed`)보다 빠르게 움직이면 출격 지점 순간이동으로 보고 무시한다.

어려웠던 점은 소리 파일 자체였다. `Wind_sound_1.wav`는 반복용 루프가 아니라 바람이 한 번 불어왔다가 잦아드는 "한 줄기 돌풍" 녹음이었다. 이것을 그냥 `loop = true` 로 돌리면 바람이 커졌다 꺼지기를 되풀이한다. Claude는 클립의 가운데 부분(1.0~4.0초, `segment`)만 쓰고, 두 개의 목소리(`AudioSource`)를 구간 길이의 절반만큼 어긋나게 동시에 틀었다. 각 목소리의 음량에는 구간 안의 위치 $u(0\sim1)$ 에 따라 $\sin(\pi u)$ 모양의 봉투를 씌운다. 두 목소리가 반 구간 어긋나 있으므로 한쪽이 $\sin$ 일 때 다른 쪽은 $\cos$ 가 되고, $\sin^2 + \cos^2 = 1$ 이라 두 소리를 합한 세기가 늘 일정하다. 한 목소리가 구간 끝(봉투가 0인 곳)에 닿으면 처음으로 되돌아가 다시 시작하므로 이음새가 들리지 않는다.

```
// 이 목소리의 구간 내 위치(0..1) → 사인 봉투
float u = Mathf.Clamp01((v.time - segment.x) / len);
v.volume = maxVolume * _level * Mathf.Sin(u * Mathf.PI);
```

속도에 따른 반응은 이렇다. 1.5 m/s(`minSpeed`) 아래에서는 무음이다. 그보다 빨라지면 음량이 최대의 25%에서 시작해 최고 속도(`fullSpeed`)에서 100%가 되도록 `SmoothStep` 곡선으로 커지고, 음높이도 0.85에서 1.15로 올라간다. 커질 때는 0.35초(`attackTime`), 작아질 때는 0.9초(`releaseTime`)에 걸쳐 바뀌므로 급정거해도 바람 소리가 뚝 끊기지 않고 잦아든다. 최대 음량(`maxVolume`)은 0.7이다. `fullSpeed` 의 기본값은 당시 최고 속도인 40이었지만, 에디터 스크립트는 여기에 `ShipTopSpeed` 를 넣으므로 뒤에서 속도가 55 m/s로 오르자 바람 소리도 함께 따라갔다.

「optimize havey mesh 이게 뭐임?」

소리 작업 중간에 이건호는 Unity 메뉴에서 낯선 항목을 발견했다.

요청 · gundam 안에 있는 optimize havey mesh 이게 뭐임?

전날 오후 최적화 작업 때 Claude가 Gundam 메뉴에 만들어 둔, 무거운 모델의 축소 메시를 미리 만들어 두는 명령이었다. 설명을 들은 이건호는

요청 · 그림 빼줘

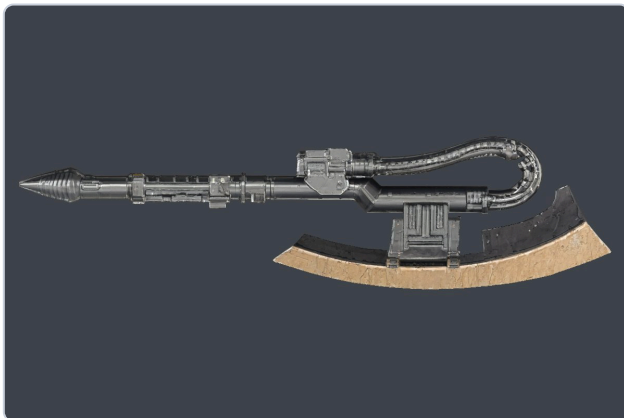
라고 했다. Claude는 메뉴 항목만 없었다. 축소 메시를 만드는 기능(`PrebuildOptimizedMeshes` , `OptimizeHeavySceneMeshes`)은 씬을 만드는 과정 안에서 그대로 자동으로 실행된다. 그래서 18번 커밋의 `GundamCockpitSetup.cs` 에 남은 `[MenuItem]` 은 「Gundam/Build Cockpit Scene」 하나뿐이다. 이 최적화 기능 자체는 다른 장에서 다룬다. 다만 이날 추가된 히트 호크(4만 삼각형 목표)와 자쿠 바주카(5만 삼각형 목표)도 이 축소 목록에 들어갔다.



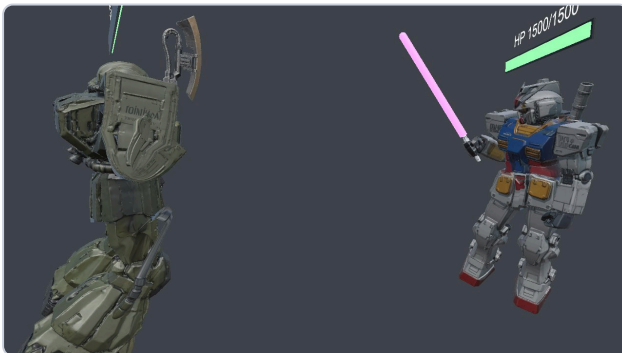
자쿠 바주카 측면 렌더. 초록 점이 오른손 권총 손잡이, 노랑 점이 왼손 앞 손잡이.



감축된 바주카 메시 사시도.



히트 호크 모델 측정 렌더(위에서 본 모습).



임시 씬 플레이 모드 테스트: 히트 호크를 든 자쿠와 빔 샤벨을 든 적 건담, 머리 위 HP 바.

머신건 소리 교체와 헤드 발칸 — 녹음된 연사를 루프로

여섯 번째로 이건호는 Machine_gun_2.wav를 내려받아

요청 · 이거를 자쿠 총소리로 해주고 아까한거는 건담 헤드 발칸 소리로 하자

라고 했다. 자쿠 총소리로 넣었던 Beam_shoot_1은 건담의 헤드 발칸으로 옮기고, 자쿠에는 새 머신건 소리를 넣는 것이다.

여기서 새 문제가 생겼다. 두 파일 모두 총알 한 발 소리가 아니라 **여러 발이 이어지는 연사 녹음**이었다. 자쿠 머신건은 분당 280발, 헤드 발칸은 0.15초 간격으로 쏘는데, 매 발마다 연사 녹음을 처음부터 다시 틀면 소리가 겹겹이 쌓여 소음 덩어리가 된다. Claude는 새 도우미 클래스 `LoopingFireSound`를 만들었다. 컴포넌트가 아니라 무기가 직접 들고 쓰는 일반 클래스다. 무기는 한 발 쏘 때마다 `Trigger()`를 부르고, 매 프레임 `Tick()`을 부른다. 루프 소리는 첫 `Trigger()`에서 시작되고, 마지막 `Trigger()` 뒤 유지 시간(hold)이 지나면 페이드아웃된다. 「계속 쏘는 동안은 녹음이 끊김 없이 이어지고, 손을 떼면 짧게 꼬리를 남기며 멈춘다」는 동작이다.

```
public void Trigger(float volume, float pan = 0f, float hold = -1f)
{
    _targetVolume = Mathf.Clamp01(volume);
    _src.panStereo = Mathf.Clamp(pan, -1f, 1f);
    _keepUntil = Time.time + (hold > 0f ? hold : holdTime);
    if (!_src.isPlaying) { _volume = 0f; _src.time = 0f; _src.Play(); }
}
```

기본값은 유지 0.25초, 페이드인 0.03초, 페이드아웃 0.12초다. 무기들은 유지 시간으로 「발사 간격 × 1.6」을 넘긴다. 헤드 발칸은 $0.15 \times 1.6 = 0.24$ 초, 자쿠 머신건은 $60/280 \times 1.6 \approx 0.34$ 초다. 다음 발이 조금 늦게 나가도 루프가 끊기지 않을 만큼 여유를 둔 것이다.

루프로 쓸 소리도 다듬어야 했다. 연사 녹음 전체를 루프하면 녹음의 시작(첫 발이 터지는 부분)과 끝(잡아드는 부분)이 반복될 때마다 된다. 그래서 Claude는 고르게 연사가 이어지는 부분만 잘라 루프 파일을 따로 만들었다. 헤드 발칸 루프는 Beam_shoot_1에서 0.27초 구간, 자쿠 머신건 루프는 Machine_gun_2에서 2.94초 구간이다. 그래서 최종 파일 이름에 「FireLoop」가 붙어 있다.

코드에서는 이렇게 연결된다. HeadVulcanController에 fireSound와 fireVolume = 0.8이 생겼고, Awake()에서 LoopingFireSound를 만들며, 발사할 때마다 Trigger(fireVolume, 0, fireRate * 1.6)을 부른다. 자기 기체의 소리이므로 좌우 위치는 0(가운데)이다. ZakuMachineGun에는 loopSound 스위치(기본 true)가 생겨서, 켜져 있으면 LoopingFireSound, 꺼져 있으면 한 발씩 PlayOneShot을 쏜다. 앞에서 만든 PlayShotSound는 계산한 음량과 좌우 위치를 그대로 Trigger(vol, pan, ...)에 넘긴다. 자쿠가 죽으면 루프를 바로 멈춘다(_fireLoop.Stop()). loopSound = false는 같은 날 뒤에 나오는 적 건담의 빔 라이플(한 발씩 쏘는 무기)을 위해 남겨 둔 길이다.

소리 파일 이름 정리

마지막으로 이번호는

요청 · 그리고 그 사운드 이름들을 전부 지금 사용하는 곳에 맞게 이름을 바꿔줘

라고 했다. 내려받은 파일 이름(Beam_shoot_1, Light_saber_3, Wind_sound_1, Machine_gun_2)은 실제 용도와 맞지 않았다. 특히 "Beam_shoot"가 헤드 발칸 소리가 되었으니 헷갈리기 쉬웠다. 최종 이름은 「기체_무기_용도」 규칙을 따른다. 코드의 모든 LoadAssetAtPath 경로도 새 이름으로 바뀌었다.

최종 파일	원본	쓰는 곳
Gundam_BeamRifle_Fire.wav	처음 받은 소리	건담 빔 라이플(한 발씩). 뒤에 적 건담의 빔 라이플, 자쿠 바주카(음높이 0.6)에도 사용
Gundam_BeamSaber_Swing.wav	Light_saber_3.wav	건담 빔 사벨. 뒤에 히트 호크와 적의 근접 무기에도 사용
Gundam_HeadVulcan_FireLoop.wav	Beam_shoot_1.wav의 0.27초 구간	헤드 발칸 루프
Gundam_Move_Wind.wav	Wind_sound_1.wav	이동 바람 소리(GundamMoveSound)

2. 더 빠르게, 그리고 km/h

소리 작업의 마지막 요청은

요청 · 건담에 속도를 조금 더 빠르게 하고 속도 단위는 km/h로 해줘

였다. 왼쪽 조이스틱을 끝까지 밀었을 때의 최고 속도 `ShipTopSpeed` 는 처음 8 m/s였고, 9월에 40 m/s(144 km/h)로 올라 있었다. 이번에 「조금 더」 라는 말에 맞춰 **55 m/s(약 198 km/h)**로 올렸다. T 레버의 최대 상승 속도(`verticalThrust.maxClimbSpeed = ship.maxMoveSpeed`)와 바람 소리의 `fullSpeed` 도 같은 값을 쓰므로 함께 올라갔다.

단위 변경은 `CockpitHUD` 에서 했다. 상수 `MsToKmh = 3.6` 을 두고, 가운데 화면의 「SPEED」 표시를 「SPEED 0.0 m/s」 에서 「SPEED 0 km/h」 형식(소수점 없음)으로 바꿨다. 이때 표시하는 속도의 의미도 바뀌었다. 전에는 조이스틱에 의한 수평 속도만 보여 주었지만, 이제는 수평 속도와 T 레버의 수직 속도를 합친 **기체 전체의 속력** $\sqrt{(\text{수평}^2 + \text{수직}^2)}$ 을 보여 준다. 위쪽 정보 줄의 「ALT」 옆 수직 속도도 km/h로 바뀌어 「ALT ...m + ...km/h」 형식이 되었다.

```
float planar = Mathf.Abs(ship.CurrentSpeed);
float climb = vertical != null ? vertical.CurrentVerticalSpeed : 0f;
speedText.text = $"SPEED {Mathf.Sqrt(planar * planar + climb * climb) * MsToKmh:0} km/h";
```

같은 파일에는 `mechName` (기본 "GUNDAM") 필드도 생겼다. 정보 줄 첫머리가 "GUNDAM COCKPIT"으로 고정되어 있던 것을 `mechName + " COCKPIT"` 으로 바꾼 것이다. 이 필드는 한 시간 뒤 자쿠 모드에서 "ZAKU II"로 바뀐다.

3. 히트 호크, 그리고 자쿠를 조종하다

요청과 방향 확인

10시 50분 무렵, 이건호는 새 모델 두 개를 내려받고 이날 가장 큰 요청을 했다.

요청 · 방금 자쿠가 사용할 도끼를 다운했어 텍스처도 다운했어 그거를 자쿠손에 잡고 휘두러서 공격하게해줘 그리고 자쿠를 플레이할수있게해줘야해 게임을 시작하고 자쿠로 할지 건담으로 할지 고르게 하는거야 그리고 자쿠로 고르면 무기창에 히트호크랑 머신건이 있는거야 그래서 골라서 싸울수있어

이 문장 하나에 사실상 세 가지 일이 들어 있다. 적 자쿠가 히트 호크(도끼)로 공격하는 것, 게임 시작 때 기체를 고르는 것, 그리고 자쿠를 직접 조종하며 무기 화면에서 히트 호크와 머신건을 고르는 것이다. 정해지지 않은 부분이 많아서 Claude는 먼저 질문으로 방향을 확인했다. 답은 이렇게 정리되었다.

질문	정해진 방향
자쿠로 플레이할 때 적은?	건담 여러 기
적 자쿠는 언제 도끼를 쓰나?	「지금 내 건담처럼 ai가 골라선 전투」 — 이건호가 무기 화면에서 고르듯, AI가 상황을 보고 머신건과 히트 호크 중에서 고른다
플레이어 히트 호크 조작은?	빔 사벨과 같은 방식(오른쪽 조이스틱으로 찌르기·휘두르기)
자쿠의 콕핏은?	「같은 콕핏, 색만 지온풍」

20 MB가 넘는 모델 가져오기

첫 장애물은 파일 크기였다. 내려받은 도끼 모델("Sci-Fi Battle Axe") FBX는 약 59 MB, 기본 색 텍스처 PNG는 약 25 MB였다. Claude가 이건호의 PC와 파일을 주고받는 전송 도구는 20 MB가 넘는 파일을 보낼 수 없었다. 그래서 Claude는 파일을 직접 옮기지 않고, Unity 에디터 안에서 C# 명령을 실행해 이건호의 다운로드 폴더에 있는 파일을 프로젝트의 `Assets/Models/HeatHawk/` 로 복사했다. 그 결과가 `HeatHawk.fbx` 와 `HeatHawk-baseColor.png` 다(파일 시각 10시 53분).

히트 호크 모델 측정 — 손잡이, 날, 쥐는 점

모델을 손에 쥐어 주려면 모델의 어느 축이 손잡이이고, 날이 어느 쪽을 향하며, 손이 어디를 쥐는지 알아야 한다. Claude는 모델을 옆과 위에서 찍은 렌더 이미지와 정점 좌표로 이것을 잴다. 결과는 코드 주석에 그대로 남아 있다. 모델은 메시 +X 방향으로 누워 있고(손잡이 끝의 뾰족한 부분이 -X, 도끼 머리가 +X), 흰 날은 메시 -Z 쪽으로 늘어져 있으며, 손잡이 축은 $z \approx 0.04$ 위를 지난다. 길이가 약 1 단위인 모델을 18 m 자쿠에 맞추어 7.5배(`HeatHawkScale`, 약 7.5 m)로 키웠다.

상수	값	의미
<code>HeatHawkModelGrip</code>	$(-0.32, 0, 0.04)$	쥐는 점: 손잡이 위, 끝 쪽
<code>HeatHawkBladeStart</code>	$(0.32 - 0.08) \times 7.5 = 1.8 \text{ m}$	도끼 머리는 메시 $x \approx -0.08$ 에서 시작
<code>HeatHawkBladeLength</code>	$0.56 \times 7.5 = 4.2 \text{ m}$	$x \approx +0.48$ 에서 끝남
<code>HeatHawkModelToRoot</code>	<code>LookRotation(back, right)</code>	메시 +X → 루트 +Y(손잡이 방향), 메시 -Z → 루트 +Z(날)

여기서 중요한 것은 **빔 사벨과 같은 좌표 약속**이다. `BuildHeatHawk` 가 만드는 루트는 +Y가 손잡이를 따라 머리 쪽, +Z가 날 쪽, 원점이 주먹 속 쥐는 점이다. 빔 사벨도 같은 약속을 따르므로, 9월 29일에 만든 빔 사벨용 팔 제어 `BeamSaberArmController` 와 칼날 판정 `BeamSaberBlade` 를 히트 호크에도 그대로 쓸 수 있다. 날(+Z)이 손가락 마디 방향과 같으므로 내려칠 때 날이 앞장선다. 모델의 원래 재질은 모두 텍스처를 입힌 새 재질 하나로 바꾸고, 그림자는 끄고, 콜라이더는 지웠다. 루트는 꺼진 상태로 만들어 쓸 때만 보이게 했다.

적 자쿠가 무기를 고르는 AI — `MeleeWeaponAI`

적 자쿠는 이미 머신건(`ZakuMachineGun`)과 이동 AI(`ZakuCombatAI`)를 갖고 있다. 여기에 오른손 히트 호크(`AddZakuHeatHawk`)와 새 스크립트 `MeleeWeaponAI` 가 더해졌다. 이 스크립트는 근접 무기를 꺼낼지 말지

스스로 결정하고, 휘두르는 팔 동작과 타격 판정까지 맡는다.

무기 고르기. 2.5초(`decideInterval`, 매번 0.7~1.3배로 흔들)마다 상황을 본다. 표적이 보이지 않거나 쓰러져 있으면 총, 수평 거리 45 m(`alwaysMeleeRange`) 안이면 무조건 근접, 120 m(`alwaysRangedRange`) 밖이면 무조건 총이다. 그 사이에서는 가까울수록 돌진할 확률이 높아진다. 확률은 `meleePreference(0.55) × k` 이고, k는 45 m에서 1, 120 m에서 0이다. 머신건이 재장전 중이면 확률이 최소 0.75로 올라간다. 탄창을 갈아 끼우는 동안 도끼를 들고 달려드는 것이다. 휘두르는 도중에는 결정을 바꾸지 않는다.

근접 모드. 근접을 고르면 `SetMelee(true)` 가 세 가지를 한다. 머신건을 집어넣고(`gun.holstered = true`), 이동 AI에게 싸움 거리를 덮어쓰게 하며(`ai.rangeOverride = approachDistance`, 적 자쿠는 14 m), 최소 접근 거리도 줄인다(`minDistanceOverride = max(4, 14 × 0.5) = 7 m`). 이를 위해 `ZakuCombatAI` 에 두 필드가 생겼고, 덮어쓰기가 켜져 있는 동안 접근 속도에 돌진 배율 `chargeSpeedMultiplier` (2배)를 쓴다. `ZakuMachineGun` 에는 `holstered` 필드가 생겨서, 켜져 있으면 총 모델을 숨기고 겨누지도 쏘지도 않으며 루프 소리도 멈춘다. 무기는 0.3초(`drawTime`)에 걸쳐 꺼내진다.

휘두르기. 수평 거리가 `strikeRange` (적 자쿠 24 m, 기본값) 안에 들어오고 쿨다운이 끝났으면 1초짜리 휘두르기를 시작한다. 진행도 `u(0~1)`를 세 구간으로 나눈다. 0~0.4는 어깨 위로 들어 올리기, 0.4~0.62는 몸을 가로질러 대각선으로 빠르게 내려치기, 0.62~1은 원래 자세로 돌아오기다. 각 구간은 「권 점의 위치 (어깨 기준, 팔 길이 단위)」와 「칼날 방향」 두 가지 키 포즈 사이를 `SmoothStep`으로 보간한다. 손 회전은 칼날 방향과 날 방향(표적 쪽으로 약간 아래)으로 정하고, 거기서 손목 위치를 거꾸로 계산한 뒤 2-본 IK(`SolveTwoBone`, 팔꿈치는 아래 바깥쪽을 향함)로 위팔과 아래팔 회전을 구한다. 실행 순서는 `[DefaultExecutionOrder(110)]` 다. 걷기(0)와 머신건(100)이 자세를 잡은 뒤 오른팔만 덮어쓴다.

```
// 키 포즈: 어깨에서 본 권 점(f=앞, up=위, r=오른쪽, 팔 길이 단위)
Vector3 readyGrip = f * 0.55f - up * 0.35f + r * 0.05f;
Vector3 raiseGrip = up * 0.5f - f * 0.05f + r * 0.2f;
Vector3 downGrip = f * 0.75f - up * 0.5f - r * 0.3f;
...
if (u < 0f) { g = readyGrip; b = readyBlade; }
else if (u < 0.4f) { /* 들어 올리기 */ }
else if (u < 0.62f) { /* 내려치기 */ }
else { /* 돌아오기 */ }
```

타격 판정. `u`가 0.38~0.72인 동안 칼날(무기 +Y 방향, `bladeStart` 부터 `bladeLength` 만큼)이 플레이어 기체의 캡슐(`PlayerHealth.CapsuleHit`)에 닿는지 본다. 한 프레임 사이에 칼이 크게 움직여 캡슐을 건너뛰는 일을 막으려고, 이전 프레임의 칼끝과 이번 칼끝을 잇는 선분, 칼날 가운데끼리 잇는 선분도 함께 검사한다. 맞으면 휘두르기 한 번에 한 번만 `TakeDamage` 를 준다. 적 자쿠의 히트 호크는 피해 250, 판정 반지름 1.6 m다.

소리. 휘두르기를 시작할 때 `Gundam_BeamSaber_Swing.wav`를 재생한다. 머신건과 같은 규칙으로 `HeadCam` 기준 거리와 방향을 계산한다. 80 m 안이면 최대 음량, 그보다 멀면 1/거리, 900 m 밖은 무음, 좌우 위치는 $\times 0.8$ 이고, 음높이는 0.92~1.05에서 무작위로 정한다.

플레이어 자쿠 만들기 — BuildPlayerZaku

이제 이진호가 탈 자쿠다. Claude가 택한 방식은 「아무것도 새로 만들지 않고 바꿔 끼우기」였다. 건담 모드는 지금까지의 게임 그대로 두고, 자쿠 세트는 썬에 미리 만들어 두되 꺼 둔다. 그리고 자쿠를 고르는 순간 참조만 바꾸고 오브젝트를 켜고 끈다. 건담을 고르면 아무것도 바뀌지 않으므로 기존 게임이 망가질 위험이 없다.

`BuildPlayerZaku` 는 자쿠 FBX를 건담과 같은 자리에 놓고, 측정한 키를 18 m(`MobileSuitTargetHeight`)에 맞추며, 몸 전체를 `GundamBodyLayer` (30번 레이어, 어떤 카메라도 그리지 않음)에 둔다. 조종사는 자기 몸을 보지 않기 때문이다. 이어 주요 본(Head, RightArm/ForeArm/Hand, LeftArm/ForeArm/Hand)을 찾고, 하나라도 없으면 자쿠를 지우고 건너뛴다.

머리 카메라 자리. 자쿠를 고르면 바깥 풍경을 찍는 HeadCam이 자쿠의 머리, 즉 모노아이로 옮겨 가야 한다. Claude는 건담의 HeadCam과 같은 규칙으로 자리를 잡았다. 스킨 메시의 경계 상자에서 **높이의 92%** 지점, 그리고 머리 형상 안에 묻히지 않도록 **높이의 4%만큼 앞으로** 내민 지점에 빈 오브젝트 "HeadCamMount"를 만들고 Head 본의 자식으로 붙였다.

```
mount.transform.position = new Vector3(b.center.x,
    b.min.y + b.size.y * 0.92f,           // 키의 92 %
    b.center.z + b.size.y * 0.04f);      // 4 % 앞으로
mount.transform.SetParent(head, true);
```

몸이 따라오게. 건담 몸에 붙어 있던 것들을 자쿠에도 똑같이 붙였다. 기동 전사 루트를 따라가며 시선 방향으로 몸을 돌리는 `ExternalGundamFollower` (회전 축은 Head 본), 체력 `PlayerHealth`, 그리고 뒤에서 설명할 걷기 `MechWalkAnimator` 다. 체력은 처음에 건담과 같은 방식으로 만들었다가, 바주카 작업 중 이진호가

요청 · 그리고 플레이어 자쿠 체력은 5000이야

라고 해서 `maxHealth = 5000` 이 되었다. 캡슐 위쪽 끝(`capsuleTop`)은 17 m다.

팔 화면. 몸은 보이지 않지만, 무기를 들 때 팔은 보여야 한다. 9월 29일 건담 빔 라이플 때 만든 `BuildGundamArmView` 로 오른팔(RightArm·ForeArm·Hand)과 왼팔만 잘라 낸 별도 메시를 만들어 `ZakuRightArm_View.asset`, `ZakuLeftArm_View.asset` 으로 저장했다. 오른팔은 히트 호크용과 머신건용으로 렌더러를 하나 더 복제했다("ZakuRightArm_GunView"). 무기마다 자기 팔 화면을 따로 켜고 끄게 하려는 것이다. 이 구조가 왜 필요했는지는 바주카 절에서 드러난다.

히트 호크(플레이어용). 오른손에 `BuildHeatHawk` 로 도끼를 쥐여 주고, 빔 사벨과 똑같이 `BeamSaberBlade` (피해 300, 판정 반지름 1.4 m, 주황색 불꽃 재질, 휘두름 소리)와 `BeamSaberArmController` (오른쪽 조이스틱으로 찌르기·휘두르기)를 붙였다. 쥐는 점은 이전에 적 자쿠의 머신건을 위해 쥔 주먹 위치 `ZakuFistGripUnscaled = (-0.005, 0.0045, 0.048)` 에 손 본의 스케일을 곱한 값이다.

머신건(플레이어용). 머신건은 건담 빔 라이플의 `BeamRifleController` 를 그대로 다시 썼다. 양손으로 들고, 락온한 뒤 오른손 엄지로 쏜다. 다른 점은 버튼을 누르고 있으면 연사된다는 것과 숫자들이다.

항목	플레이어 자쿠 머신건
피해 / 탄수 / 재장전	50 / 100발 / 5초
발사 간격	60/280초(분당 280발)
탄 속도 / 길이 / 굵기 / 판정	320 m/s / 5 m / 0.18 m / 0.6 m
총구 섬광	1.6
락온 없을 때 흠어짐	(1, 3)
락온 시간	1초
탄 색	주황(1, 0.75, 0.3), 발광 (3, 1.8, 0.5)
WEAPON 화면	「MACHINE GUN」, 「AMMO 100/100」, 「RELOADING」
소리	Zaku_MachineGun_FireLoop.wav, fireSoundLoop = true

이를 위해 `BeamRifleController` 에 WEAPON 화면용 이름 필드(`hudName`, `ammoLabel`, `rechargeLabel`, 기본값 "BEAM RIFLE"/"ENERGY"/"RECHARGING")가 생겼다. 탄수가 100 이상이면 「000」 세 자리로 표시한다. 연사 소리를 위한 `fireSoundLoop` 도 생겼다. 켜져 있으면 `LoopingFireSound` 를 쓰고, 유지 시간은 `fireCooldown × 1.6` 이다. 모델 위의 개머리, 총구, 손잡이, 앞손잡이 위치는 적 자쿠 때 쯤 값 (`ZakuGunModel*`)을 그대로 쓰고, 총 크기는 8 m다. 개머리(`ZakuGunModelStock = (0, -0.02, -0.47)`)만 이번에 새로 줬다.

모든 것을 만든 뒤 자쿠는 `zaku.SetActive(false)` 로 꺼 둔다. 콘솔에는 「Playable ZAKU built (switched off until chosen): machine gun True, heat hawk True, bazooka True, HP 5000.」 같은 줄이 남는다.

적 건담 여섯 기 — PlaceEnemyGundam

자쿠로 플레이할 때의 적은 건담이다. `SetupZakuMode` 는 콜로니 안의 자쿠 출현 지점(`ColonyZakuSpawns`) 열 곳 중 앞의 여섯 곳에 "EnemyGundam_1"부터 "EnemyGundam_6"까지 놓는다. 콜로니 대로와 거리 위의 지점으로, x는 0, -180, 320, 180, -320, -460, 깊이는 콜로니 입구에서 600, 1000, 1500, 2100, 2600, 1800 m다. 각 건담은 플레이어 쪽을 바라보게 놓이고, 18 m로 맞춰지며, 건담 텍스처를 입는다. 그리고 적 자쿠와 같은 부품을 받는다. `EnemyMarker` (락온 대상), 몸 크기 캡슐 콜라이더(반지름 = 키 × 0.22), `EnemyHealth`, `ZakuCombatAI` 다.

빔 라이플은 새로 만들지 않았다. 적 자쿠의 머신건 스크립트 `ZakuMachineGun` 에 건담 빔 라이플 모델을 쥐여 주고, 숫자를 「한 발씩 쏘는 빔」으로 바꿨다. 빔 사벨은 `BuildBeamSaber` 로 만든 뒤, 거기 붙어 있던 플레이어용 `BeamSaberBlade` (적의 `EnemyHealth` 를 때리는 판정)를 지우고 `MeleeWeaponAI` 를 붙였다. 이렇게 하면 칼날이 플레이어를 때린다. 적 자쿠와 마찬가지로 AI가 거리를 보고 둘 중 하나를 고른다.

항목	적 건담 값
체력 / 부활 대기	1500 / 6초
AI 거리 / 접근 / 옆걸음 속도	70 m / 12 m/s / 10 m/s

빔 라이플 피해 / 탄창 / 재장전	150 / 15발 / 5초
발사 속도 / 탄속 / 흩어짐	분당 45발 / 380 m/s / 0.8°
점사 / 휴식	1~2발 / 1.0~2.6초
교전 거리 / 조준 높이	320 m / $18 \times 0.6 = 10.8$ m
빔 색	분홍(1, 0.3, 0.65), 발광 (3.2, 0.7, 1.8), 길이 12 m, 굵기 0.7 m
빔 라이플 소리	Gundam_BeamRifle_Fire.wav, loopSound = false (한 발씩)
빔 사벨 피해 / 판정	300 / 0.9 m
빔 사벨 휘두르기 거리 / 접근 거리	22 m / 13 m
칼날	손잡이 길이 2.4 m의 절반(1.2 m)부터 9 m

적 건담들은 모두 꺼진 채로 만들어지고, 자쿠 모드에서만 켜진다. 「건담 모드의 적」은 그 시점에 켜져 있는 EnemyMarker 를 가진 오브젝트 전부, 즉 적 자쿠 열 기다.

기체 선택 패널 — MechSelectPanel

게임을 시작할 때 자쿠와 건담 중 하나를 고르게 하려고 새 패널 MechSelectPanel 을 만들었다. 모양과 위치는 출격 지점 선택 패널(SORTIE POINT)과 같은 방식이다. 콕핏 안 (0, 1.2, 0.34) 위치에 떠 있는 월드 공간 캔버스이고, 크기는 520×360 캔버스 단위에 배율 0.0009, 즉 약 47×32 cm다. 제목은 「MOBILE SUIT / 기체 선택」이고, 버튼은 「RX-78-2 / GUNDAM / 건담」과 「MS-06 / ZAKU II / 자쿠」 두 개다. 버튼 하나는 226×230 단위로 약 20×21 cm다.



기체 선택 패널 “MOBILE SUIT / 기체 선택”. 캘리브레이션이 끝나면 나타나고 손가락으로 누른다.

누르는 방식은 무기 화면, 출격 화면과 똑같다. XR Hands의 검지 끝 관절(`IndexTip`) 위치를 캔버스 좌표로 바꾼다. 손가락이 화면 앞 2 cm(`armDistance`) 밖에 있으면 「장전」 되고, 1.2 cm(`touchDepth`) 안으로 들어오면 눌린 것으로 친다(화면 뒤 3 cm까지 허용, 버튼 테두리 여유 4 mm). 5 cm 안에 들어오면 버튼이 밝아진다(hover). 누르면 버튼이 0.25초 동안 흰색으로 번쩍인 뒤에 선택이 적용된다. 패널은 보정(Calibration)이 끝난 뒤에 나타나고, 고를 때까지 이동 스크립트(`ship`, `verticalThrust`)를 꺼서 기체가 움직이지 않게 한다. 출격 지점 패널(`SpawnSelectPanel`)에는 `mechSelect` 필드가 생겼다. 기체를 고르기 전에는 출격 패널이 나타나지 않으므로, 순서는 「보정 → 기체 선택 → 출격 지점 선택」이 된다. 코드나 인스펙터에서 시험할 수 있도록 `Choose(mech)`는 public이다.

기체를 바꾸는 순서 — PilotMechSwitcher

선택 결과는 `PilotMechSwitcher.Apply(mech)`로 전달된다. 건담을 고르면 적 건담을 끄고 적 자쿠를 켜는 것이 전부다. 자쿠를 고르면 다음 여섯 단계가 이 순서대로 실행된다. 순서 자체에 이유가 있다.

단계	하는 일	순서의 이유
1	자쿠 몸을 루트 기준의 원래 상대 위치로 옮기고 켜다	자쿠의 무기 스크립트들이 쓰기 전에 휴식 자세에서 초기화(Awake)되도록 먼저 켜다
2	HeadCam을 자쿠의 HeadCamMount 자식으로 옮기고(위치·회전 0), <code>GundamHeadCam360.SetHead(자쿠 Head)</code> , <code>CockpitViewController.RecaptureCameraStart()</code>	바깥 풍경 전체가 모노아이 시점이 된다. 새 머리의 현재 자세를 고개 돌림의 기준으로 다시 잡는다
3	<code>WeaponModeController.SetLoadout(자쿠 머신건, 히트 호크, 발칸 없음, zaku=true, 시작=BeamRifle 칸, 바주카)</code> , <code>WeaponTouchPanel.ApplyMechLayout()</code> , 그다음 건담 몸을 끈다	건담의 무기를 내려놓는 처리(<code>SetActive(false)</code>)가 건담 몸이 아직 켜져 있을 때 일어나야 한다
4	<code>SuitCollision.SetBody(자쿠)</code> , 출격 패널의 <code>body</code> , HUD 이름 「ZAKU II」, HUD의 헤드 발칸 참조 해제와 잔탄 글자 지우기	충돌, 순간이동, 체력이 자쿠 몸을 따라가게 한다
5	적 자쿠 끄기, 적 건담 켜기	적 편이 바뀐다
6	<code>CockpitTheme.ApplyZeon(콕핏)</code> , 무기·출격 패널의 버튼 색 변환, 락온 링 재질 다시 읽기	지온풍 색

이를 위해 작은 메서드 몇 개가 기존 스크립트에 추가되었다. `GundamHeadCam360.SetHead()` (따라갈 머리와 본과 기준 회전 교체), `CockpitViewController.RecaptureCameraStart()` (HeadCam의 현재 로컬 회전을 새 휴식 자세로), `SuitCollision.SetBody()` (몸 교체 후 오프셋 다시 측정), `OrbitHUDTargetLock.RefreshMaterials()` (색이 바뀐 눈금 재질을 「락온 안 된 평소 모습」으로 다시 기억)다. 이 스크립트는 오브젝트를 지우거나 다시 만들지 않는다. 그래서 자쿠의 상대 위치는 `Awake()`에서 한 번 기억해 두고 `Apply`에서 그대로 되돌린다.

무기 화면의 변화 — WeaponModeController와 WeaponTouchPanel

요청에는

요청 · 자쿠로 고르면 무기창에 히트호크랑 머신건이 있는거야

라는 말이 있었다. Claude는 무기 칸(Mode)을 새로 만들지 않고 기존 칸의 의미를 기체에 따라 바꾸는 방법을 택했다. 자쿠 모드에서는 BeamRifle 칸이 MACHINE GUN, BeamSaber 칸이 HEAT HAWK이고 HeadVulcan 칸은 없다. 이미 만들어진 씬과 호환되도록 enum 값은 그대로 두고(BeamRifle = 0, BeamSaber = 1, HeadVulcan = 2), 뒤에 바주카용 Bazooka = 3 만 덧붙였다.

WeaponModeController 에 추가된 것은 다음과 같다. zakuMode 플래그, 기체에 따라 무기 이름을 돌려주는 NameOf(m), 「이 기체가 이 무기를 갖고 있는가」를 답하는 Has(m) (자쿠에는 발칸이 없고, 바주카는 자쿠 전용), 그리고 무기 세트 전체를 바꾸는 SetLoadout(...) 다. SetLoadout 은 대기 중인 전환을 취소하고, 조이스틱 차단을 풀고, 현재 무기를 모두 내려놓은 뒤 새 무기를 받아 시작 무기를 강제로 적용한다. SelectWeapon 은 첫 줄에서 Has(m) 을 확인하므로 없는 무기는 고를 수 없다. 자쿠 모드에서 히트 호크를 들면 WEAPON 화면에 「HEAT HAWK」, 「MELEE」, 주황색 막대(1, 0.55, 0.15), 「READY」가 표시된다.

```
public bool Has(Mode m) => m == Mode.Bazooka
    ? zakuMode && bazooka != null
    : !(zakuMode && m == Mode.HeadVulcan);
```

WeaponTouchPanel 에는 ApplyMechLayout() 이 생겼다. 현재 기체에 없는 무기 버튼은 숨기고, 남은 버튼들을 같은 줄에 126 단위 간격으로 가운데 정렬하며, 버튼 글자를 NameOf 로 다시 쓴다(공백은 줄바꿈으로 바꿔 「MACHINE GUN」처럼). 이 함수는 Start() 에서도 한 번 불린다. 그래서 건담 모드에서는 바주카 버튼(씬에는 x = 252 위치로 만들어지지만 꺼져 있음)이 숨겨진 채 원래 세 버튼(-126, 0, 126)이 그대로 남고, 자쿠 모드에서는 머신건·히트 호크·바주카 세 버튼이 같은 자리에 놓인다. 숨겨진 버튼은 손가락 판정과 색 갱신에서도 빠지고, 아래쪽 「SELECT:」 글자도 NameOf 를 쓴다.

지온풍 색 — CockpitTheme

요청 · 같은 콕핏, 색만 지온풍

을 위해 정적 클래스 CockpitTheme 을 만들었다. 규칙은 단순하다. 파랑·청록 계열의 빛만 지온의 모니터 녹색으로 바꾸고, 나머지는 그대로 둔다.

```
public static Color ToZeon(Color c)
{
    Color.RGBToHSV(c, out float h, out float s, out float v);
    if (s < 0.12f) return c; // 회색·흰색은 그대로
    if (h < 0.42f || h > 0.76f) return c; // 빨강·주황·노랑·초록·자홍은 그대로
    Color o = Color.HSVToRGB(ZeonHue, Mathf.Min(1f, s * 1.05f), v, true);
    o.a = c.a;
    return o;
}
```

색상환 값(Hue, 0~1)이 0.42~0.76 사이인 색, 즉 청록에서 파랑·남색까지를 Hue 0.31(`ZeonHue` , 초록)로 옮기고 채도는 5% 올린다. 밝기(v)는 그대로 두며, 마지막 인자 `true` 는 HDR을 허용해서 발광 색처럼 1보다 큰 값도 밝기를 잃지 않게 한다. 채도가 0.12 미만인 회색·흰색은 바꾸지 않으므로 경고용 빨강, 호박색, 회색 판넬은 원래 색으로 남는다. `ApplyZeon(root)` 는 콧핏 아래의 모든 렌더러 재질에서 `_BaseColor` , `_Color` , `_EmissionColor` 를 바꾸는데, **공유 에셋을 직접 고치지 않도록** 재질을 복사해 이름 끝에 " (Zeon)"을 붙여 쓴다(같은 재질은 한 번만 복사). UI 그래픽은 색만 바꾸고, 카메라 영상을 보여 주는 `RawImage` 는 건너뛰다. 텍스처는 건드리지 않는다. 이 변환으로 화면, HUD 글자, OrbitHUD 눈금, 조종간 강조색, 패널 버튼이 함께 초록으로 바뀐다.

임시 씬에서 근접 AI 시험

근접 AI는 VR 헤드셋에서 직접 확인하기 전에 에디터에서 먼저 시험했다. 에디터 스크립트에 에셋을 저장하지 않는 시험용 함수 `BuildEnemyMeleePreview(player, zakuPos, gundamPos)` 를 만들고, 임시 씬에서 `PlayerHealth` 를 가진 표적 하나와 적 자쿠 한 기, 적 건담 한 기를 놓고 플레이 모드를 돌렸다. 자쿠가 히트 호크를, 적 건담이 빔 사벨을 꺼내 휘두르고, 그때 플레이어의 HP가 줄어드는 것을 확인했다.

4. 걷기, 그리고 우주에서는 걷지 않기

자쿠 모드 작업에 이어 이건호는

요청 · 건담도 걷는 애니메이션을 만들어야해

라고 했고, 곧바로

요청 · 근데 자쿠랑 건담 둘다 우주로 나가면 걷는거를 하면안됨

이라고 덧붙였다.

적 자쿠는 9월부터 `ZakuCombatAI` 안의 절차적(procedural) 걷기 동작으로 걸어 다녔다. FBX 리그에는 애니메이션 클립이 없어서 다리 본을 코드로 직접 돌리는 방식이다. 플레이어의 기체(건담, 그리고 새로 생긴 자쿠)에는 그런 동작이 없었다. Claude는 같은 생각으로 새 컴포넌트 `MechWalkAnimator` 를 만들어 플레이어 건담 몸과 플레이어 자쿠 몸 둘 다에 붙였다.

언제 걷는가. 기준은 `SuitCollision.Grounded` , 즉 콜로니 바닥에 발이 닿아 있는지다. 바닥에 서서 움직이면 걷고, 그 밖의 모든 경우(넓은 우주, 콜로니 안이라도 T 레버로 떠 있거나 떨어지는 중)에는 걷지 않고 「떠 있는 자세」를 취한다. 두 상태는 0.35초(`blendTime`)에 걸쳐 섞인다. `SuitCollision` 이 없으면 아예 걷지 않는다.

걷기 동작. 속도는 몸이 실제로 움직인 거리로 잰다(초속 400 m가 넘는 순간이동은 무시). 보폭은 8 m(`strideLength`)이고, 위상(phase)은 실제 지면 속도에 맞춰 진행하므로 발이 미끄러지지 않는다. 허벅지는 이동 방향에 수직인 축으로 $\pm 26^\circ$ (`legSwingDegrees`), 무릎은 앞으로 나가는 다리만 최대 38° (`kneeBendDegrees`) 굽힌다. 12 m/s(`fullStrideSpeed`)에서 동작이 최대가 된다.

떠 있는 자세. 무릎을 24°(오른쪽은 1.3배) 부드럽게 굽히고, 비행 속도가 40 m/s(`floatFullSpeed`)에 가까울수록 다리를 최대 22°(`floatTrailDegrees`)까지 뒤로 젖힌다. 사인파로 $\pm 3^\circ$ 정도 천천히 흔들며 다리가 살짝 엇갈리며 떠다니게 했다.

건드리지 않는 것. 이 컴포넌트는 다리 본(`LeftUpLeg`, `LeftLeg`, `RightUpLeg`, `RightLeg`)만 돌린다. 엉덩이, 척추, 팔은 건드리지 않는다. 엉덩이나 척추가 움직이면 머리에 달린 `HeadCam`도 함께 출렁이고, 이진호가 보는 바깥 풍경 전체가 흔들린다. 팔은 무기 IK가 맡고 있다. 실행 순서는 `[DefaultExecutionOrder(-50)]` 로, 몸을 따라가게 하는 `ExternalGundamFollower` (-60) 뒤, 무기 IK(0) 앞이다.

적 기체 쪽. 「둘 다 우주로 나가면」이라는 말은 적에게도 해당한다. `ZakuCombatAI` 의 절차적 걷기에 `floatInSpace` (기본 켜짐)를 추가했다. 전장이 되는 콜로니(`arena`)가 지정되지 않은 경우, 즉 콜로니 없이 우주에 놓인 경우에는 걷는 대신 새 함수 `FloatPose` 를 쓴다. 다리는 속도에 따라 뒤로 젖히고 무릎은 22° (`floatKneeDegrees`) 굽힌다. 팔꿈치는 느슨하게 두고, 몸통은 비행 방향으로 최대 14° (`floatLeanDegrees`) 기울이며, 엉덩이는 천천히 위아래로 0.25 m쯤 출렁인다. 머리는 걸을 때처럼 계속 표적을 따라 돈다. 적 건담도 같은 `ZakuCombatAI` 를 쓰므로 같은 규칙을 따른다.

5. 자쿠 바주카

요청

11시 14분 무렵, 이진호는 새 모델을 내려받았다.

요청 · 자쿠 바주카를 다운했고 그거를 자쿠에게 적용해줘 조작법은 빔라이플이랑 같아

. 이어

요청 · 데미지는 1000이야

, 앞에서 인용한 플레이어 자쿠 체력 5000, 그리고 작업 도중

요청 · 야 바주카 텍스처도 다운받아줬어

라는 말이 차례로 왔다. 내려받은 원본의 이름은 "Military Rocket Launcher"였다. 프로젝트에는 `Assets/Models/ZakuBazooka/ZakuBazooka.fbx` 와 `ZakuBazooka-baseColor.png` 로 들어갔다.

모델 측정과 감축

원본은 정점이 약 31만 개였다. 앞에서 말한 최적화 목록에 바주카를 넣고 목표를 5만 삼각형으로 잡았더니, Unity 메시 LOD 단계 중 목표 아래의 첫 단계인 약 3.9만 삼각형으로 줄었다. 옆에서 찍은 렌더로 모델을 재 보니 길이는 메시 Y축을 따라 약 1 단위였다. 어깨 받침(뒤쪽)이 +Y, 나팔 모양 포구가 -Y, 조준기가 위(+Z)에 있었다. 이것은 빔 라이플 모델과 같은 배치라서, 빔 라이플용 회전 `RifleModelToRoot` 을 그대로 쓰면 「+Z

= 포구, +Y = 위」인 라이플 루트 좌표계에 맞는다. 18 m 자쿠에 맞춰 **9 m**(`ZakuBazookaScale = 9`)로 키웠다.

메시 좌표 점	값	용도
<code>ZakuBazookaModelStock</code>	(0, 0.42, 0.016)	어깨 받침(포신 축 위)
<code>ZakuBazookaModelMuzzle</code>	(0, -0.48, 0.016)	포구 앞
<code>ZakuBazookaModelGrip</code>	(0, 0.03, -0.07)	권총 손잡이(오른손)
<code>ZakuBazookaModelForeGrip</code>	(0, -0.18, -0.06)	앞 손잡이(왼손)

권총 손잡이가 살짝 기울어 있어서 오른손 엄지 방향은 정확히 위가 아니라 (0, 1, 0.25)를 정규화한 방향으로 잡았다.

「조작법은 빔라이플이랑 같아」 — 두 번째 `BeamRifleController`

조작이 빔 라이플과 같으므로 바주카도 `BeamRifleController` 를 하나 더 붙여 만들었다. 양손으로 들고, 락온한 뒤 오른손 엄지로 쏜다. 숫자는 무거운 포를 느끼게 하도록 정했다.

항목	자쿠 바주카
피해	1000(「데미지는 1000이야」)
탄수 / 재장전 / 발사 간격	5발 / 6초 / 1.2초
포탄 속도 / 길이 / 굵기 / 판정	170 m/s / 3.5 m / 0.55 m / 2 m
락온 시간	1.5초
락온 없을 때 흩어짐	(1.5, 4)
총구 섬광 / 총돌 섬광	4.5 / 16
포탄 색	주황(1, 0.5, 0.15), 발광 (3.5, 1.3, 0.3)
WEAPON 화면	「BAZOOKA」, 「AMMO 05/05」, 「RELOADING」
소리	Gundam_BeamRifle_Fire.wav, 음높이 0.6, 음량 1

총돌 섬광을 키우려고 `BeamRifleController` 에 `impactFlashSize` (기본 6) 필드를 새로 만들고, 탄이 맞을 때 생기는 섬광의 크기로 넘기게 했다. 빔 라이플은 6 그대로, 바주카는 16이다. 바주카 전용 소리는 아직 없었다. 그래서 Claude는 빔 라이플 소리를 **0.6배 음높이**로 낮춰 「낮고 무겁게」 재생하도록 했다. 앞에서 본 `firePitch` 필드가 이때 생겼다.

무기 화면에는 네 번째 버튼 「Btn_Bazooka」가 생겼다. 앞서 설명한 대로 `Has(Mode.Bazooka)` 는 자쿠 모드이고 바주카가 있을 때만 참이므로, 이 버튼은 자쿠 모드에서만 보인다. 무기를 바꿀 때는

`WeaponModeController.ApplyMode` 가 쓰지 않는 **라이플형 무기를 먼저 내려놓은 뒤** 새 무기를 꺼낸다.

라이플형 무기는 꺼낼 때 락온 시간을 자기 값으로 바꾸고 내려놓을 때 되돌리므로, 순서가 뒤집히면 되돌린 값이 새 무기의 값을 덮어쓴다. 헤드 발칸이 켜지는 조건도 「빔 라이플도 바주카도 아닐 때」로 바뀌었다.

내려놓은 무기가 왼팔을 풀어 버리던 문제

두 번째 `BeamRifleController` 를 붙이자 문제가 생겼다. 머신건과 바주카가 같은 팔 본(오른팔·왼팔)을 쓰는 같은 스크립트이기 때문이다. 원래 `BeamRifleController` 는 매 프레임 `LateUpdate` 첫머리에서 왼팔을 휴식 자세로 되돌린 뒤 무기를 들고 있으면 왼손을 앞 손잡이로 가져갔다. 「왼팔은 이 스크립트만 움직인다」는 가정 때문이었다. 컨트롤러가 하나뿐이던 건담에서는 문제가 없었다. 그런데 자쿠에는 두 개가 있다. 바주카를 들고 있으면, 내려놓은 머신건 컨트롤러가 매 프레임 왼팔을 휴식 자세로 되돌려 바주카의 왼손 그림을 풀어 버렸다. 실행 순서에 따라 결과가 달라지는 종류의 버그다.

숨기기에도 같은 문제가 있었다. 원래 코드는 무기가 내려가 있으면 매 프레임 `SetVisuals(false)` 를 불러 총 모델과 팔 화면을 숨겼다. 그러면 내려놓은 쪽이 다른 무기가 켜 둔 팔 화면까지 매 프레임 꺼 버린다. Claude는 세 가지를 고쳤다.

```
// ① 들고 있거나, 바로 이번 프레임에 내려놓았을 때만 왼팔을 만진다
if (_left.ok && (_blend > 0f || prevBlend > 0f)) { /* 휴식 자세로 */ }
...
// ② 숨기기는 한 번만(매 프레임이 아니라)
if (!Active && _visualsOn) SetVisuals(false);
```

세 번째는 에디터 쪽 수정이다. 숨기기를 한 번만 하더라도 두 무기가 같은 팔 화면 렌더러를 공유하면, 한쪽이 끌 때 다른 쪽 팔도 꺼진다. 그래서 `CloneArmView` 라는 함수를 만들어, 바주카가 쓸 오른팔·왼팔 화면을 같은 메시·같은 본을 쓰는 별도의 `SkinnedMeshRenderer` 로 복제했다 ("ZakuRightArm_BazookaView", "ZakuLeftArm_BazookaView"). 이제 히트 호크, 머신건, 바주카가 각자 자기 팔 화면을 갖고, 자기 것만 켜고 끈다.

바주카가 생기면서 `PilotMechSwitcher` 에 `zakuBazooka` 필드가 추가되었고, `SetLoadout` 의 마지막 인자로 바주카를 넘기게 되었다. 콘솔 메시지도 「Pilot is now in the ZAKU II (machine gun + heat hawk + bazooka); enemies: GUNDAMs.」로 바뀌었다.

6. 부스터 불꽃

요청과 에셋 가져오기

정오 무렵의 마지막 기능 요청은 이것이었다.

요청 · 유니티 에셋에 부스터 다운했거든 자쿠랑 건담 등뒤에 부스터에서 효과가 나와야하고 발바닥에서도 나와야하는데 발바닥은 우주에서만

이건호가 받은 것은 Unity Asset Store의 "2D Booster Flame Shader Controller"(RockingProjects)였다. Claude는 패키지를 통째로 가져오지 않았다. 패키지 안에는 `Packages/manifest.json`, 데모 씬, 다른 URP 렌더 파이프라인 에셋이 함께 들어 있었는데, 이것들을 가져오면 프로젝트의 패키지 목록과 렌더 설정을 덮어쓸 수 있다. 열흘 동안 맞춰 놓은 OpenXR·URP 설정이 바뀌면 VR 빌드가 깨질 수 있다. 그래서 필요한 것만 골라 원래의 GUID(.meta 파일)와 함께 가져왔다. 셰이더 그래프(`BoosterFlameShader.shadergraph`),

머티리얼(`BoosterFlameMaterialBlue.mat` , `BoosterFlameMaterialRed.mat`), 마스크 텍스처 (`flameMask.png` , `flameMask2.png`), 스크립트(`BoosterFlameController.cs` 등), 설명서다. `.meta`를 함께 가져와야 머티리얼이 셰이더와 텍스처를 GUID로 제대로 찾는다. 모두 `Assets/RockingProjects/2DBoosterFlame/` 아래에 있다.

Claude는 질문으로 두 가지를 확인했다. 첫째, 불꽃은 **적 기체에만** 붙인다. 플레이어의 몸은 어차피 보이지 않는 레이어에 숨겨져 있기 때문이다. 둘째, 불꽃은 **움직일 때** 세게 나온다.

2D 불꽃을 입체로 — 엇갈린 판 세 장

이 에셋은 이름처럼 2D 게임용이라 불꽃이 평평한 판 하나에 그려진다. 3D 기체에 판 하나를 붙이면 옆에서 볼 때 선 하나로 보이거나 사라진다. Claude는 새 컴포넌트 `MechBoosterFlames` 에서 노즐마다 **60°씩 엇갈린 양면 판 세 장**으로 된 메시를 만들었다. 셋이 별 모양으로 겹치므로 어느 방향에서 보아도 불꽃이 입체적인 분사로 보인다.

```
for (int k = 0; k < 3; k++)
{
    Quaternion r = Quaternion.AngleAxis(k * 60f, Vector3.forward);
    Vector3 side = r * Vector3.right * 0.5f;
    // z 0 = 노즐(UV v=1, 텍스처의 넓은 쪽) → z 1 = 끝(v=0)
    v.Add(-side); v.Add(side); v.Add(side + Vector3.forward); v.Add(-side + Vector3.forward);
    tri.AddRange(new[] { b, b + 1, b + 2, b, b + 2, b + 3 }); // 앞면
    tri.AddRange(new[] { b, b + 2, b + 1, b, b + 3, b + 2 }); // 뒷면
}
```

이 단위 메시는 +Z로 길이 1, 폭 1이고, 정점 색은 흰색이다(셰이더가 정점 색을 곱하기 때문). 모든 노즐이 한 메시를 공유한다. 노즐은 에디터가 만든 빈 Transform으로, +Z가 분사 방향, 원점이 노즐 입구다. 노즐은 기체의 뼈(등은 `Spine2`, 발은 `LeftFoot/RightFoot`)에 붙어 있으므로 걸거나 떠다니는 자세를 따라 움직인다.

설계에서 눈여겨볼 점이 하나 있다. 불꽃 오브젝트 자체는 기체의 자식이 아니다. 조준점과 락온 링은 적 기체의 크기를 렌더러들의 경계 상자로 켜다. 발밑에 4 m짜리 불꽃이 자식으로 붙어 있으면 락온 링이 아래로 늘어나고 조준점이 내려간다. 그래서 불꽃은 별도의 루트("이름_BoosterFlames") 아래에 두고, 매 프레임 노즐의 위치와 회전으로 옮긴다. 실행 순서는 `[DefaultExecutionOrder(200)]` 으로, 걸기·AI·무기 자세가 모두 정해진 뒤다.

노즐 위치 측정

노즐의 위치는 모델마다 다르다. Claude는 건담과 자쿠를 **정사영(직교) 카메라**로 뒷면, 옆면, 밑면에서 렌더하고, 그 위에 좌표 격자를 겹쳐 등 쪽 추진기 구멍의 좌표를 읽었다. 좌표는 기체 기준 미터(18 m 기체, 발바닥 $y = 0$, 정면 +Z)다.

기체	등 노즐(좌, 우)	분사 방향	등 불꽃 길이 / 폭
건담	(-0.78, 12.25, -2.04), (0.78, 12.25, -2.04)	(0, -1, -0.3): 아래로, 조금 뒤로	6.5 m / 3.0 m
자쿠	(-0.83, 13.10, -2.36), (0.57, 13.10, -2.36)	(0, -0.7, -0.7): 뒤 아래로	6 m / 3.4 m

자쿠의 두 노즐은 좌우 대칭이 아니다(-0.83과 0.57). 렌더에서 읽은 값을 그대로 쓴 것이다. 발바닥 노즐은 측정 대신 뼈에서 계산한다. 발(Foot)과 발끝(ToeBase) 본 사이 40% 지점을 기체의 바닥 평면에 투영한 뒤 0.15 m 올리고, 분사 방향은 기체의 아래쪽(-up)이다. 발바닥 불꽃은 길이 4.5 m, 폭 2.8 m다.

`MakeNozzle` 은 분사 방향이 위쪽 기준 벡터와 거의 평행하면(내적 0.95 초과) 기준 벡터를 정면으로 바꿔서 `LookRotation` 이 뒤틀리지 않게 한다.

처음 렌더해 보니 불꽃이 너무 가늘었다. 원인은 불꽃 텍스처가 판 폭의 가운데 절반 정도만 채운다는 데 있었다. 판을 3 m로 만들어도 보이는 불꽃은 1.5 m 남짓이다. 그래서 폭을 약 2배로 키워 지금의 3.0~3.4 m가 되었다. 이 사정은 `backLength` 툴팁의 「The flame texture fills about the middle half of its width」라는 문장으로 남아 있다.

세기와 「우주에서만」

`LateUpdate` 는 기체가 실제로 움직인 속도를 잰다(600 m/s가 넘는 순간이동과 부활은 무시). 등 불꽃은 늘려져 있는데, 서 있을 때는 35%(`idleLevel`) 크기이고, 30 m/s(`fullSpeed`)에서 100%가 된다. 커질 때는 0.15초, 작아질 때는 0.5초에 걸쳐 바뀐다. 길이는 세기에 비례하고, 폭은 0.6~1배로만 변한다. 셰이더 밝기 `_intensity` 는 0.55~1.25 사이를 오간다. 셰이더 자체의 일렁임 위에 두 개의 사인파(31 rad/s에서 7%, 53 rad/s에서 5%)로 길이를 조금씩 흔들어 불꽃이 떨리게 했다. 노즐마다 위상이 무작위라 두 불꽃이 똑같이 떨리지 않는다.

「발바닥은 우주에서만」은 콜로니 판정으로 구현했다. 기체 몸통 가운데(발바닥에서 키의 절반 위)가 `ColonyStructure.IsInside` 로 콜로니 안이면 발바닥 불꽃은 0이다. 콜로니 밖이거나 콜로니가 아예 없으면 우주로 보고, 발바닥 불꽃을 최소 50%(`footIdleLevel`)로 유지한다. 「발바닥 불꽃이 기체를 받치고 있다」는 느낌을 주려는 것이다. 발바닥 불꽃은 커질 때 등 불꽃의 두 배 시간(0.3초)에 걸쳐 커진다. 기체가 죽으면(`EnemyHealth.IsDead`) 모든 불꽃이 꺼지고, 컴포넌트나 기체가 꺼지면 불꽃도 숨는다.

에디터 쪽에서는 `AddBoosterFlames(suit, gundam, colony)` 가 이 모든 것을 연결한다. 적 자쿠는 `PlaceZakuEnemy` 의 끝에서 머신건, 히트 호크 다음에 이 함수를 부르고, 적 건담은 `PlaceEnemyGundam` 에서 부른다. 렌더 검사용으로 정해진 세기를 바로 보여 주는 `PreviewNow(level, feet)` 도 만들었다.

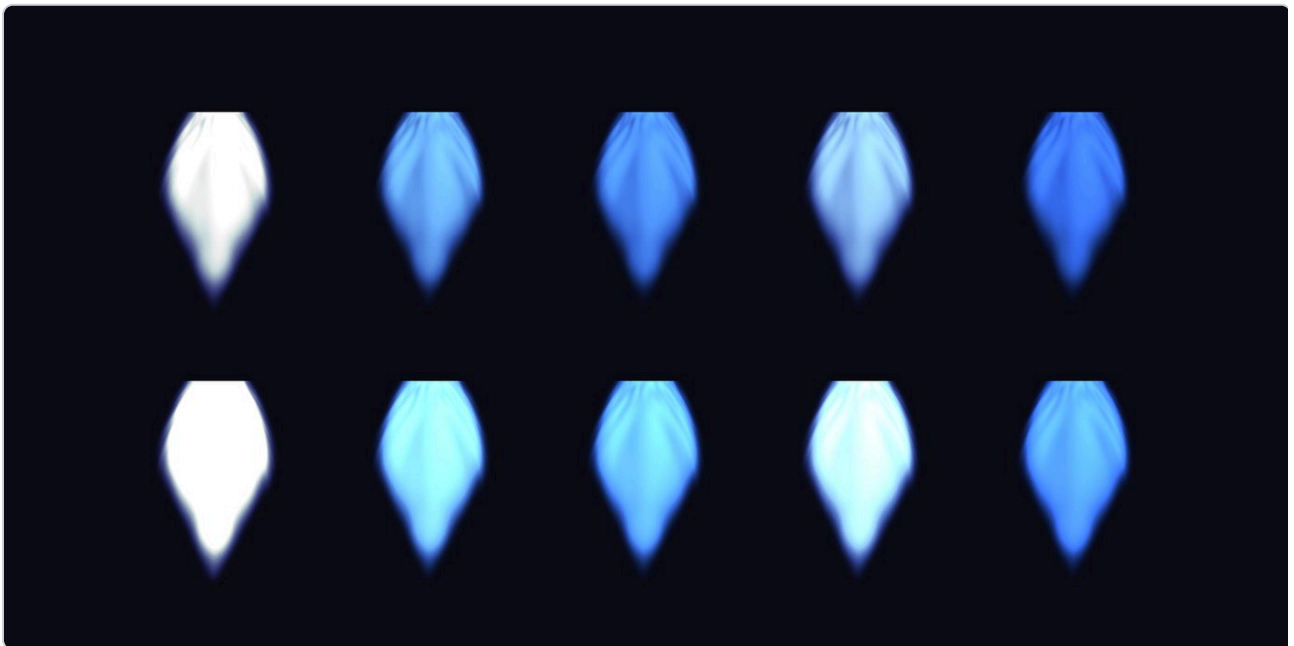
「파랑색」 — HDR 색이 흰색으로 보이던 문제

처음에는 에셋에 들어 있던 파란 머티리얼 `BoosterFlameMaterialBlue.mat` 을 그대로 썼다. 그런데 이 머티리얼의 `_Color` 는 (1.16, 1.44, 7.87)이라는 HDR 값이다. 블룸(bloom) 같은 HDR 후처리가 있는 화면을 전제로 한 값이라, 이 게임의 화면에서는 세 채널이 모두 1을 넘어 포화되었고 불꽃이 흰색으로 보였다. 이견호의 요청은 한 단어였다.

요청 · 파랑색



최종 파란 부스터. 왼쪽: 멈춘 자쿠(작은 등 불꽃, 콜로니 안이라 발바닥 없음), 가운데·오른쪽: 우주에서 최고 속도인 자쿠와 건담.



색 후보 비교(위: 최소 출력, 아래: 최대 출력). 맨 왼쪽이 에셋 원래 색, 세 번째 (0.08, 0.3, 1.6)이 선택되었다.

Claude는 후보 다섯 가지 색을 같은 조건으로 나란히 렌더해서 비교했고, **(0.08, 0.3, 1.6)**을 골랐다. 빨강과 초록을 크게 낮추고 파랑만 1을 조금 넘게 두어, 포화되어도 흰색이 아니라 선명한 파랑으로 남는 값이다. 이 색은 공유 머티리얼 파일을 고치지 않고 `MaterialPropertyBlock` 으로 렌더러마다 지정한다 (`_mpb.SetColor("_Color", flameColor)`). 에셋의 원래 머티리얼은 그대로 남고, 색을 바꾸고 싶으면 인스펙터의 `flameColor` (HDR 색 선택기) 하나만 고치면 된다.

```
[ColorUsage(false, true)] public Color flameColor = new Color(0.08f, 0.3f, 1.6f);
...
_mpb.SetFloat(IntensityId, Mathf.Lerp(intensityRange.x, intensityRange.y, lvl));
_mpb.SetColor(ColorId, flameColor);
f.renderer.SetPropertyBlock(_mpb);
```

7. 12시 10분 커밋, 그리고 「책으로」

여기까지가 12시 10분 커밋(18번)에 들어간 10월 1일 작업이다. 이날 새로 생긴 스크립트는 LoopingFireSound, GundamMoveSound, MeleeWeaponAI, MechWalkAnimator, MechSelectPanel, PilotMechSwitcher, CockpitTheme, MechBoosterFlames 여덟 개다. 기존 스크립트 중에는 BeamRifleController, BeamSaberBlade, HeadVulcanController, ZakuMachineGun, ZakuCombatAI, WeaponModeController, WeaponTouchPanel, CockpitHUD, CockpitViewController, GundamHeadCam360, SuitCollision, SpawnSelectPanel, OrbitHUDTargetLock 이 바뀌었다. 썬을 만드는 GundamCockpitSetup.cs 는 이 커밋 하나에서 1,000줄 넘게 늘었다(9월 30일 최적화 코드 포함).

시각(파일 기준)	요청	결과
09:59~10:17	빔 라이플·자쿠 총·빔 사벨·이동 소리, 소리 교체, 이름 정리	2D 재생 + HeadCam 기준 거리 감쇠·좌우 위치, 연사 루프, 휘두름 감지, 크로스페이드 바람 소리
그 직후	속도 증가, km/h	40 → 55 m/s(≈198 km/h), 수평+수직 합산 속도 표시
10:52~	히트 호크, 자쿠 플레이	기체 선택 패널, 플레이어 자쿠(HP 5000), 적 건담 6기, AI 무기 선택, 지온 색
이어서	건기, 우주에서는 걷지 않기	MechWalkAnimator, ZakuCombatAI.FloatPose
11:14~	자쿠 바주카	9 m, 1000 피해, 5발, 원팔 충돌 버그 수정
~12:07	부스터, 파랑색	엇갈린 판 3장, 정사영 측정 노즐, 발바닥은 우주에서만, (0.08, 0.3, 1.6)
12:10	커밋 18(c974825)	

그리고 이건호는 다음 요청을 했다.

요청 · 지금까지 작업한거를 책으로 만들거야

9월 21일 빈 프로젝트에서 시작해 손가락으로 잡는 조종간, 360° 머리 카메라, 스페이스 콜로니, 그리고 이날의 소리와 자쿠까지 이어진 기록이 지금 읽고 있는 이 책이다.



첫 부스터 렌더(에셋 원래 색). 폭을 넓힌 뒤의 모습으로, 원래 “파란” 머티리얼이 흰색으로 포화되어 보인다.

PART 3

시스템별 해설

같은 11일을 이번에는 기능 단위로 다시 읽는다. 각 장은 “무엇을 원했나 → 어떻게 동작하나 → 핵심 코드 → 숫자”의 순서로 정리했다.

콕핏 — 구형 돔과 360도 머리 카메라

조종사는 바깥 세상을 직접 보지 않는다. 불투명한 구 안에 앉아, 그 구의 안쪽 면에 비치는 “건담의 눈”을 본다.

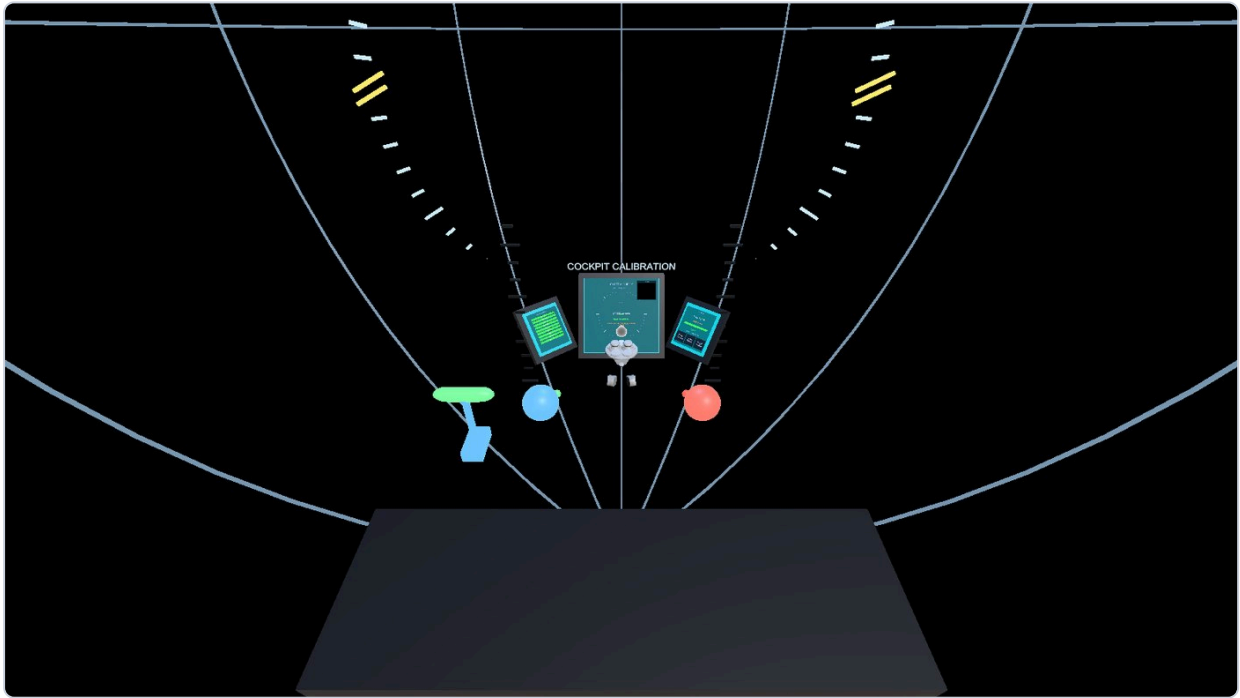


그림 3-1 · 좌석 뒤에서 본 콕핏. 경선·위선이 그어진 구형 돔, 위쪽의 궤도 눈금 링, 가운데의 디스플레이 3개와 그 아래 조종간 2개(파랑·빨강 구), 초록색 T자 레버. 에디터에서 찍은 사진이라 돔에 바깥 영상은 비치지 않는다.

구조

구성 요소	역할
Cockpit_Dome	조종석을 감싸는 구. 머티리얼의 셰이더 Custom/CockpitDomeCubemap 이 큐브맵을 표면 법선 방향으로 샘플링한다. 얇게 빛나는 테두리 선(“콕핏 테두리선을 더얇게하고 아주 조금 빛나게해줘”).
HeadCam	바깥 건담의 Head 뼈대에 달린 카메라. 매 프레임 큐브맵 렌더 텍스처를 찍는다 (GundamHeadCam360). 해상도 1536, near 0.5m.
머리 따라 돌리기	조종사가 고개를 돌리면 건담의 Head 뼈대도 같은 방향으로 돈다.
CockpitStation	콕핏 전체는 전장에서 멀리(y -4000, z -8000) 떨어져 있다. 바깥 물체가 콕핏 안으로 들어와 시야를 가리지 않게 하기 위해서다.
레이어	30 GundamBody(플레이어 자기 몸, 어떤 카메라에도 안 그림), 29 CockpitInterior, 28 SpaceBackdrop, 27 ColonyDetail

디스플레이 3개



그림 3-2 · 가운데 전술 HUD(SYSTEM CHECK — 속도 km/h, 상태 NORMAL, 조준점)와 오른쪽 무기 디스플레이(HEAD VULCAN / BEAM RIFLE / BEAM SABER 터치 버튼).

화면	내용	스크립트
왼쪽 STATUS	HP, SHIELD, ENERGY, 머리·몸통·양팔·양다리 9개 게이지	CockpitStatusHUD
가운데 전술	속도(km/h), 스로틀·방향, 조준점, NORMAL/WARNING/CRITICAL 배지, 머리 카메라 작은 화면	CockpitHUD
오른쪽 WEAPON	현재 무기 이름, 탄수, 재장전 상태, 손가락 터치 버튼	CockpitWeaponHUD, WeaponTouchPanel

체력 데이터는 화면과 분리된 `CockpitHUDManager (GundamVitals)`에 있다. 아직 부위별 데미지가 없으므로 모든 부위 게이지는 전체 HP 비율을 따라가지만, 나중에 `SetPartFraction(part, value)` 만 호출하면 화면 코드는 바꿀 필요가 없다.

디스플레이 위치의 역사

처음에는 조종간과 겹쳐 손가락으로 누를 수 없었다 → 절반 크기로 줄여 조종간 사이·위로 → 시선을 가린다는 말에 약 2cm 아래로 → 가운데 화면 깜빡임(겹친 UI의 z-파이팅)은 캔버스를 5mm 앞으로 띄워 해결.

손으로 잡는 조종간

컨트롤러 없이 맨손으로. 이 프로젝트에서 가장 많은 시간이 들어간 부분이다.

조종간(JoystickLever)

왼쪽(파랑)·오른쪽(빨강) 구형 조종간은 손을 주먹 쥐듯 감싸거나 엄지-검지로 꼬집으면 잡힌다. 잡힌 판정은 두 경로가 있다 — XR Interaction Toolkit의 Near-Far Interactor(공식 판정), 그리고 손바닥 근접 + 왼 정도로 직접 판정하는 예비 경로. 어느 쪽이든 실제 위치 계산은 JoystickLever 혼자 한다.

```
// 잡는 순간 "튀지 않게" — 손과 손잡이의 처음 위치를 기억하고, 이후 손의 이동량만 더한다.
// (T자 레버 VerticalTLever도 같은 규칙)
handleZ = Clamp(grabStartHandleZ + (handZ - grabStartHandZ), -travel, +travel);
```

세 가지 규칙

- ① 손잡이를 손 위치로 스냅하지 않는다. ② 각 조종간은 자기 손의 상호작용기만 받는다 (HandExclusiveGrabAdapter). ③ 한 프레임에 비정상적으로 큰 손 이동은 추적 오류로 보고 무시한다.

손가락 버튼(JoystickFingerButtons)

각 조종간에는 손가락마다 하나씩 버튼 5개가 있다. 조종간을 쥐고 잠시(settleTime) 지나면 각 손가락 끝마디(원위 관절과 끝 사이)가 닿는 공 표면으로 버튼이 옮겨진다. 지금은 오른손 엄지 버튼이 사격에 쓰이고, 나머지는 나중 기능을 위해 이벤트(onPressed/onReleased)가 준비되어 있다.

```
// 누름 점수 = 손바닥 쪽으로 밀린 거리 점수 + 손가락이 더 굽은 각도 점수
float pushPts = push / (pressDepth / sensitivity);
float curlPts = curlPush / (pressCurlDegrees / sensitivity);
float score = Max(0, pushPts) + Max(0, curlPts);
// pressScore 이상이 pressHoldSeconds 동안 유지되면 누름.
// 단, 손가락이 펴지는 신호가 하나라도 있으면 누름을 거부한다 ("엄지를 펴도 발칸이 나갈때가 있어").
```

모든 값은 손 자체를 기준으로 잴다. 그래서 조종간을 움직이거나 손 전체를 들어도 판정이 흔들리지 않는다. 한 샘플만 튀는 관절 값은 무시하고, 쉬는 자세 기준값은 확실히 손을 떼 동안에만 천천히 갱신한다.

T자 수직 레버(VerticalTLever)

왼손 조종간 옆의 T자 손잡이. 앞으로 밀면 상승, 당기면 하강, 놓으면 중앙으로 돌아간다. 이동 범위는 $\pm 3.5\text{cm}$. 같은 손이 왼쪽 조종간을 잡고 있는 동안은 잡히지 않는다. 위치는 앞의 자세의 어깨 높이

기준이며, “위아래로 이동하는 손잡이가 너무 뒤에 있어 조금 앞으로 옮겨줘 근데 왼손 조종기를 방해하지 않게”에 따라 앞으로 옮겨졌다.

조종간 위치 설정(CalibrationManager)

1. “지금부터 조종간 위치를 설정합니다 — 왼손으로 왼쪽 조종간을 잡아 원하는 위치로 옮긴 뒤 놓으세요”
2. “왼쪽 조종간 위치가 괜찮으면 손을 쥐거나 꼬집어 확인하세요”
3. 오른쪽도 같은 순서로 진행한다.
4. 둘 다 확인되면 기체 선택 → 출격 위치 선택으로 넘어간다.

저장 기능은 일부러 없었다. 예전에 잘못 저장된 낮은 위치가 매번 다시 불러 와 “조종간이 바닥으로 내려가 안 올라오는” 문제가 생겼기 때문이다. 이제 앱을 켤 때마다 새로 맞춘다.

이동과 시점

세 개의 입력

입력	동작	스크립트
왼손 조종간	앞뒤 = 전진·후진, 좌우 = 옆으로 이동. 방향은 지금 보고 있는 시점 기준	ShipMovementController
T자 레버	상승·하강(수평과 같은 최고 속도)	VerticalThrustController
오른손 조종간	시점 회전(좌우 360°, 위아래 $\pm 60^\circ$). 기울인 만큼 계속 돌고, 놓으면 그 자리에 멈춤	CockpitViewController
고개	건담의 머리가 따라 돈다	GundamHeadCam360

```
// ShipMovementController - "앞으로 밀면 지금 보는 방향으로"
Quaternion viewHeading = Quaternion.AngleAxis(viewYaw, Vector3.up);
Vector3 suitForward = viewHeading * transform.forward;
Vector3 suitRight = viewHeading * transform.right;
Vector2 moveInput = t.sqrMagnitude > 1f ? t.normalized : t; // 대각선이 더 빨라지지 않게
Vector3 planarVelocity = (suitForward * moveInput.y + suitRight * moveInput.x) * maxMoveSpeed;
transform.position += planarVelocity * Time.deltaTime;
```

멀미를 피한 설계

초기 버전은 왼손 조종간을 돌리면 기체 루트를 회전시켰는데, 조종사(XR Origin)가 그 아래에 있어서 머리 시야까지 같이 돌았다. VR에서 머리를 움직이지 않았는데 시야가 도는 것은 강한 멀미 원인이다. 이후 루트는 절대 회전시키지 않고, 시점은 돔 세이더의 회전으로만 처리한다.

바깥 건담이 따라오기

조종사가 움직이는 것은 MobileSuitRoot이고, 실제 건담 몸체(ExternalGundam)는

ExternalGundamFollower가 매 프레임 그 위치로 데려온다. 건담의 몸은 항상 내가 바라보는 방향으로 돈다 (“건담이 내가 바라보는 방향으로 항상 몸이 돌아야함”).

속도

처음 8m/s였던 최고 속도는 “어느정도 미는정도에 따라서” 빨라지도록 바뀌었고, 10월 1일 55m/s로 올라갔다. HUD는 km/h로 표시한다(55m/s $\approx$ 198km/h, 수평 속도와 상승 속도를 합성).

충돌과 중력(SuitCollision)

물리 엔진 없이 직접 계산한다. 몸을 서 있는 캡슐로 보고, 살아 있는 적의 캡슐과 겹치면 밀어낸다. 콜로니 안에서는 9.8m/s^2 로 떨어지고(최대 70m/s), T자 레버로 올라가는 동안은 중력이 꺼진다. 건물 지붕에도 착지할 수 있다.

걷기와 떠 있기(MechWalkAnimator)

FBX에 애니메이션 클립이 없어서 다리 뼈대를 코드로 움직인다. 콜로니 바닥에 서 있으면 실제 이동 속도에 맞춰 걷고(보폭 8m), 우주나 공중에서는 무릎을 살짝 굽힌 채 다리가 비행 방향 뒤로 젖혀진다. 엉덩이·척추·팔은 건드리지 않아 머리 카메라가 흔들리지 않는다.

락온 시스템

콕핏 위쪽의 궤도 눈금 링(OrbitHUD)이 적을 포착하면 적 크기로 모여 줄어든다.

동작

1. 적(EnemyMarker)이 링 안에 보이면 포착을 시작한다. 0.15초 이상 보여야 하고, 잠깐 가려져도 0.6초는 유지한다.
2. 모든 눈금이 적의 화면 위치로 미끄러져 모이고, 원은 적의 겉보기 크기로 줄어든다.
3. 완전히 닫히면 빨간색(locked)이 된다. 기본 락온 시간 0.5초, 빔 라이플을 들면 2초, 머신건 1초, 바주카 1.5초.
4. 적이 링 밖으로 나가거나 사라지면 원래 모양으로 퍼진다.

조종사가 “보는” 위치 계산

조종사는 세상을 직접 보지 않으므로, 적의 월드 방향을 머리 카메라 기준 → 시점 회전 → 돔 표면의 점 → 조종사 눈에서 링 평면까지의 교점으로 차례로 변환해 링의 위치를 정한다. 크기도 적의 경계 상자 모서리를 같은 변환에 넣어 구한다.

거리에 따라 작아지는 링

요청 · 락온이 거리에 따라서 작아졌으면 좋겠어 멀리있는 적을 락온하면 너무 뭉쳐서 덩어리로 보임

```
// OrbitHUDTargetLock - 최소 반지름을 0.08 → 0.035로 낮추고, 눈금 크기를 반지름에 비례시킴
_radius = Mathf.Lerp(_radius, Mathf.Clamp(bestRadius, minLockRadius, ringRadius), f);
float lockedScale = Mathf.Clamp(_radius * tickScalePerRadius, minLockedTickScale,
lockedTickScale);
// 링이 아주 작아지면 바깥쪽 눈금 층은 서서히 사라진다.
```

가림 판정

콜로니 안에서는 건물 뒤에 숨은 적이 락온되면 안 된다. `ColonyStructure.Raycast / SegmentBlocked`가 건물 상자·땅·외벽·끝판을 격자 가속으로 빠르게 검사한다. 같은 함수를 자쿠의 시야와 모든 탄환이 함께 쓴다.

건담의 무기

오른쪽 무기 디스플레이를 손가락으로 눌러 바꾼다. 건담은 헤드 발칸·빔 라이플·빔 샤벨 세 가지.

헤드 발칸(HeadVulcanController)

- 조준: 헤드셋이 바라보는 방향. 발사: 오른손 조종간의 엄지 버튼.
- 머리 양쪽 60발씩 총 120발(화면에는 60발로 표시), 0.15초 간격, 재장전 2.5초. 재장전 중에는 발사되지 않는다.
- 탄 한 발 데미지 1. 날아가는 탄이 보여야 한다는 요청에 따라 작은 예광탄을 그린다.

빔 샤벨(BeamSaberArmController + BeamSaberBlade)



그림 7-1 · 빔 샤벨 손잡이 모델의 방향 맞추기 렌더.

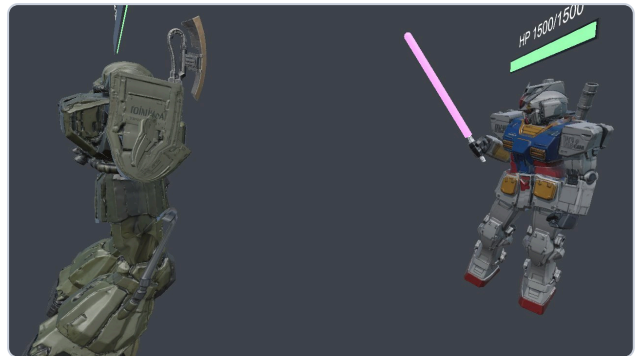


그림 7-2 · 근접 AI 테스트. 히트 호크를 든 자쿠와 빔 샤벨을 든 적 건담, 머리 위 HP 바.

- 오른손 조종간을 앞뒤로 밀고 당기면 찌르기, 옆으로 당기면 휘두르기. 건담 모델의 진짜 오른팔 뼈대를 2-본 IK로 움직인다.
- 락온이 없을 때는 오른손 조종간이 시점 조작, 락온되면 공격으로 바뀐다. 바뀌는 순간 조종간이 중앙 근처로 돌아와야 해서, 돌던 동작이 갑자기 베기로 바뀌지 않는다.
- 공격 중에는 시점이 락온한 적의 머리 쪽을 따라간다(SaberLookAssist).
- 데미지 200. 휘두를 때만 소리가 난다.

빔 라이플(BeamRifleController + BeamRifleShot)



그림 7-3 · 우주에서 빔 라이플 사격 테스트. 분홍색 빔이 락온한 적을 향한다.

- 양손으로 든다(오른손 권총 손잡이, 왼손 앞 손잡이, 2-본 IK). 들고 있는 동안 헤드 발칸은 꺼진다.
- 락온한 적에게 쏜다. 락온 전에 쏘면 일부러 빗나가고, 락온 시간이 2초로 늘어난다.
- 데미지 500, 15발 충전식, 다 쓰면 5초 충전(처음 요청은 1분 → 5초로 변경).
- 탄은 매 프레임 구간을 구 형태로 검사해(sphere cast) 빠른 속도로도 적을 뚫고 지나가지 않고, 콜로니 벽에서 멈춘다.

```
// BeamRifleShot - 벽에 먼저 닿으면 거기까지만 검사하고, 섬광 후 소멸
Vector3 wallPoint = default;
bool wall = colony != null && colony.Raycast(from, from + direction * dist, out wallPoint);
if (wall) dist = Vector3.Distance(from, wallPoint);
int n = Physics.SphereCastNonAlloc(from, hitRadius, direction, _hits, dist, ~0,
QueryTriggerInteraction.Collide);
```

무기 디스플레이(WeaponModeController + WeaponTouchPanel)

버튼은 손가락 끝을 디스플레이 앞 2cm 밖에서 한 번 뺐다가 1cm 안으로 넣어야 눌린다(같은 손가락으로 연속 오작동 방지). 기체에 없는 무기 버튼은 숨기고 나머지를 같은 줄에 다시 배치한다.

```
// 기체별 무기 이름과 보유 여부
public string NameOf(Mode m) => !zakuMode ? DisplayName(m)
    : m == Mode.BeamSaber ? "HEAT HAWK" : m == Mode.BeamRifle ? "MACHINE GUN"
    : m == Mode.Bazooka ? "BAZOOKA" : "-";
public bool Has(Mode m) => m == Mode.Bazooka ? zakuMode && bazooka != null
    : !(zakuMode && m == Mode.HeadVulcan);
```

적 — 자쿠와 적 건담



그림 8-1 · 콜로니 도심 대로에 배치된 적 자쿠. 콜로니에는 자쿠 10기가 곳곳에 흩어져 있다.

전투 기동(ZakuCombatAI)

- 매 기동마다 거리를 바꾼다(가까이 돌진, 중거리, 멀리 이탈) — “자쿠가 너무 거리유지함”.
- 좌우 옆걸음을 끊어서 하고, 가끔 멈추거나 대시한다. 몇 초마다 패턴을 다시 굴러 예측하기 어렵다.
- 속도는 부드럽게 가감속하고, 자쿠마다 다른 느린 노이즈로 움직임이 겹치지 않는다. 여러 기가 뭉치지 않게 서로 거리를 둔다.
- 콜로니에서는 자기 구역을 천천히 순찰하다 플레이어가 450m 안에 오면 싸운다. 건물을 돌아서 걷고 콜로니 밖으로는 나가지 않는다.
- 시야: 눈에서 플레이어 몸이나 머리까지 막힘 없이 보일 때만 위치를 안다. 건물 뒤에 숨으면 마지막으로 본 곳에 가서 잠시 찾다가 순찰로 돌아간다.
- 걷기: 다리 스윙, 무릎 굽힘, 팔 반대 스윙, 상체 기울임, 엉덩이 흔들림을 모두 코드로 만든다.

체력과 폭발(EnemyHealth)

자쿠 체력 1000. 0이 되면 폭발 이펙트가 나오고 렌더러·콜라이더가 꺼진 뒤 5초 후 원래 자리에서 다시 나타난다. 머리 위에는 숫자가 붙은 HP 바가 뜬다. HP 바는 기체의 자식이 아니라서 락온 원 크기 계산에 끼어들지 않는다.

무기와 판단



그림 8-2 · 머신건을 사격 자세로 든 자쿠(“자쿠가 총쏘는 자세로 자쿠머신건을 잡고 쏘는거야”).

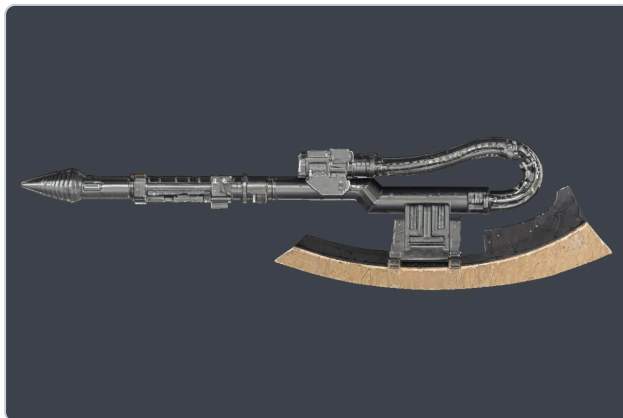


그림 8-3 · 히트 호크 모델. 손잡이 축, 날 방향, 쥐는 점을 측정해 손에 맞췄다.

적	원거리	근거리	체력
자쿠 ×10	머신건 — 50 데미지, 탄창 100발, 분당 280발, 재장전 5초, 연사 소리 루프	히트 호크 — 250 데미지	1000
적 건담 ×6 (자쿠 모드)	빔 라이플 — 150 데미지, 15발, 분당 45발, 단발 소리	빔 샷벨 — 300 데미지	1500

MeleeWeaponAI 는 거리를 보고 원거리 무기와 근접 무기 중 하나를 고른다. 가까워지면 총을 내리고 (holstered) 근접 무기를 꺼내 키프레임 스윙 + 팔 IK로 휘두른다. 보이지 않는 플레이어는 쏘지 않는다.

전장 — 우주와 콜로니

우주

- **SpaceDustField** — 가까운 먼지 입자를 $3 \times 3 \times 3$ 타일로 깔아 지나갈 때 시차가 느껴지게 하고(타일은 월드에 고정, 보는 위치 칸으로 재배치), 먼 곳에는 보는 위치를 따라다니는 별 껍질을 둔다.
- **SpaceDebrisField** — VARCO 3D로 만든 전함 잔해·콜로니 파편·정거장 모듈을 450~750m 밖에 140~300m 크기로 띄우고 천천히 회전시킨다. 전투를 방해하지 않게 멀리, 살짝 어둡게.

스페이스 콜로니(ColonyStructure)

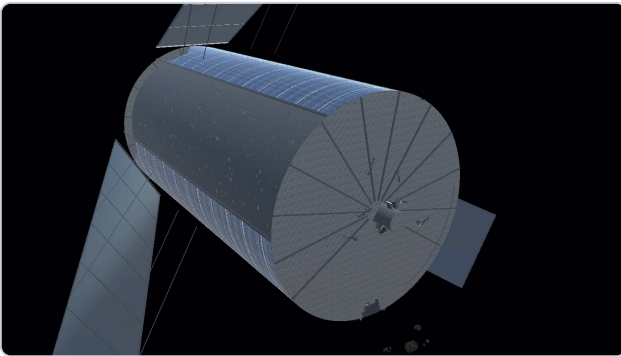


그림 9-1 · 우주에서 본 콜로니. 원통 몸체와 거울 판, 앞쪽 끝판.

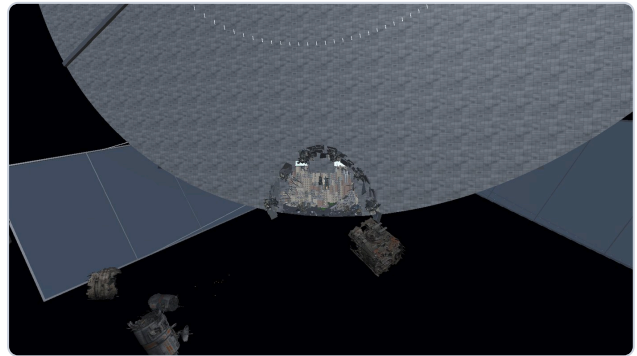


그림 9-2 · 끝판에 뚫린 파공(입구)과 주변 잔해.

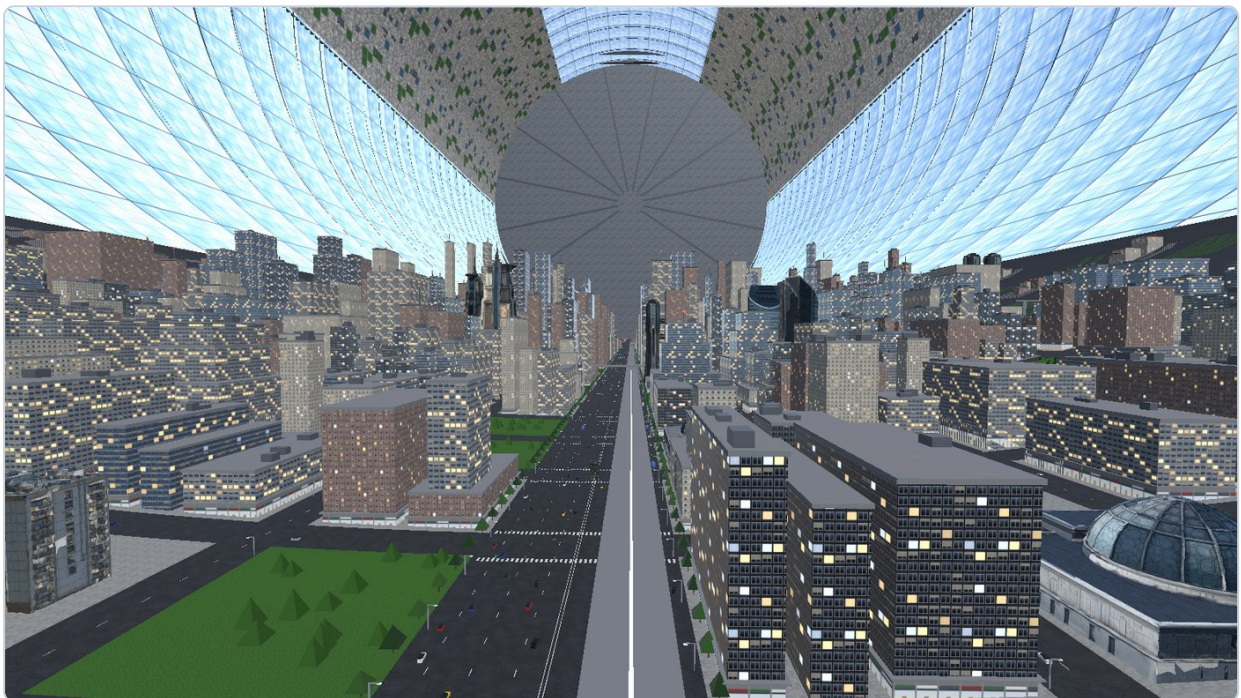


그림 9-3 · 콜로니 내부 도심. 양옆 땅이 위로 말려 올라가 머리 위에서 만나고, 창문 띠로 빛이 들어온다. 멀리 반대쪽 끝판이 보인다.

항목	값 / 내용
형태	오닐 “아일랜드 3” 원통. 축은 +Z 방향, 앞 끝판 중심 (0, 3200, 1500)
크기	반지름 3,200m, 길이 12,000m
입구	앞 끝판 지면 높이에 뚫린 전쟁 피해 파공. 반대쪽 끝판은 막혀 있다.
도시	바닥 땅 띠 위의 블록형 도시, 대로, 차량, 가로등, 가로수, 차선. 건물은 축 정렬 상자로 충돌 처리
중력	안에서는 아래(-Y) 방향 9.8m/s ² . 밖은 무중력
대기	안에 있으면 하늘색 거리 안개와 밝은 주변광(ColonyAtmosphere). 안에서는 외벽·거울 등 바깥 장식을 끄고, 밖에서는 거리 소품을 끈다

출격 위치(SpawnSelectPanel)

버튼	시작 위치
SPACE 우주	콜로니 앞 1.5km 우주
COLONY EXTERIOR 콜로니 외관	파공 앞에 떠 있는 위치
COLONY ENTRANCE 콜로니 안(입구)	파공 바로 안쪽 광장 바닥
DOWNTOWN 콜로니 안(도심)	자쿠들 근처 대로 한가운데

패널이 떠 있는 동안 이동 스크립트는 잠시 꺼져(enabled 플래그만) 기체가 미끄러지지 않는다.

자쿠 플레이 모드



그림 10-1 · 기체 선택 패널 “MOBILE SUIT / 기체 선택” — RX-78-2 GUNDAM 건담 / MS-06 ZAKU II 자쿠. 손가락으로 누른다.

정해진 방향(질문으로 확인)

질문	선택
자쿠 모드의 적은?	건담 여러 기
적 자쿠의 무기 사용은?	지금 내 건담처럼 시가 골라서 전투
히트 호크 조작은?	빔 사벨과 같은 방식
콕핏은?	같은 콕핏, 색만 지오폴

기체 교체 순서(PilotMechSwitcher.Apply)

1. 자쿠 몸체를 건담이 있던 자리에 켜다(무기 스크립트가 쉬는 자세에서 초기화되도록 먼저 켜).
2. 머리 카메라를 자쿠의 모노아이 위치로 옮기고, 머리 추적·시점 회전 기준을 다시 잡는다.
3. 무기를 MACHINE GUN / HEAT HAWK / BAZOOKA로 바꾸고 디스플레이 버튼을 다시 배치한다(헤드 발칸 없음).
4. 건담 몸체를 끈다. 충돌·출격 위치·HUD가 자쿠 몸체를 따르게 한다(HUD 이름 “ZAKU II”).
5. 적을 바꾼다: 자쿠 10기 끄고 적 건담 6기 켜기.

6. 콕핏을 지은 색으로 칠한다(CockpitTheme: 파랑·청록 계열 색만 초록 색상으로 옮기고, 빨강·노랑·회색은 유지. 공유 머티리얼은 복사해서 바꾼다).

아무것도 지우거나 새로 만들지 않고 참조만 바꾸고 켜고 끄므로, 건담을 고르면 게임은 예전과 똑같이 동작한다.

플레이어 자쿠의 무기



그림 10-2 · 자쿠 바주카(9m). 오른쪽이 포구, 위쪽이 조준경. 초록 점은 오른손 권총 손잡이, 노랑 점은 왼손 앞 손잡이.



그림 10-3 · 바주카 사시도. 50만 폴리곤 원본을 약 3.9만으로 줄인 메시.

무기	조작	수치
MACHINE GUN	빔 라이플과 같음(락온 + 엄지, 누르고 있으면 연사)	50 데미지, 100발, 분당 280발, 재장전 5초
HEAT HAWK	빔 샷벨과 같음(오른손 조종간 찌르기·휘두르기)	300 데미지
BAZOOKA	빔 라이플과 같음	1000 데미지, 5발, 재장전 6초, 발사 간격 1.2초, 락온 1.5초, 주향 포탄, 큰 폭발 섬광

플레이어 자쿠 체력은 5000(건담은 3000). 바주카 전용 소리는 아직 없어 빔 라이플 소리를 낮은 음(0.6배)으로 재생한다.

한 팔에 총 두 자루

머신건과 바주카가 같은 공용 사격 스크립트(BeamRifleController)를 쓰기 때문에, 처음 구조 그대로면 내려놓은 무기가 매 프레임 왼팔을 쉬는 자세로 되돌리고 공용 팔 화면을 숨겨 버린다. 그래서 각 무기는 자신이 들려 있을 때(또는 막 내려놓는 프레임)에만 왼팔을 만지고, 숨기기도 한 번만 하며, 무기마다 팔 화면을 따로 갖게 했다.

소리와 부스터 불꽃

사운드

파일	쓰임	재생 방식
Gundam_BeamRifle_Fire.wav	빔 라이플 발사, 적 건담 빔, 바주카(낮은 음)	한 발마다 1회, 음 높이 약간 무작위
Gundam_BeamSaber_Swing.wav	빔 사벨·히트 호크 휘두르기	휘두를 때만, 구간 재생
Gundam_HeadVulcan_FireLoop.wav	헤드 발칸 연사	0.27초 루프 구간 (LoopingFireSound)
Gundam_Move_Wind.wav	이동 바람 소리	속도에 따라 크기·음 높이, 두 목 소리 교차 페이드로 끊김 없는 루프
Zaku_MachineGun_FireLoop.wav	자쿠 머신건 연사(적·플레이어)	2.94초 루프 구간

콕핏(과 AudioListener)은 전장에서 수 km 떨어져 있어서 3D 소리를 쓰면 들리지 않는다. 그래서 모두 2D로 재생하고, 거리에 따른 감쇠와 좌우 위치를 머리 카메라 기준으로 직접 계산한다.

부스터 불꽃(MechBoosterFlames)



그림 11-1 · 적 기체의 부스터 불꽃. 왼쪽: 멈춰 있는 자쿠(작은 등 불꽃, 콜로니 안이라 발바닥 불꽃 없음). 가운데·오른쪽: 우주에서 최고 속도인 자쿠와 건담(등 + 발바닥).

- **에셋:** Asset Store “2D Booster Flame Shader Controller”. 패키지 안의 Packages/manifest.json과 데모 씬·렌더 파이프라인 에셋은 프로젝트 설정을 덮어쓸 수 있어 빼고, 셰이더·머티리얼·텍스처·스크립트만 가져왔다.
- **입체화:** 2D 불꽃을 60°씩 엇갈린 양면 판 3장에 그려 어느 방향에서 봐도 입체로 보이게 했다.
- **노즐 위치:** 정사영(직교) 뒷면·옆면·밑면 렌더에 좌표 격자를 겹쳐 측정했다. 건담은 백팩 아래 2개, 자쿠는 백팩 아래 2개, 발바닥은 발 뼈대와 발가락 뼈대 사이.
- **규칙:** 등 불꽃은 항상(멈추면 35% 크기), 발바닥은 콜로니 밖 우주에서만. 속도 30m/s에서 최대. 죽어 있으면 꺼짐.
- **대상:** 적 기체만. 플레이어 몸은 원래 숨겨져 있으므로 불꽃도 그리지 않는다.
- **락온과 무관:** 불꽃은 기체의 자식이 아닌 별도 오브젝트라, 4m짜리 발 불꽃이 조준점·락온 원 크기에 끼어 들지 않는다.

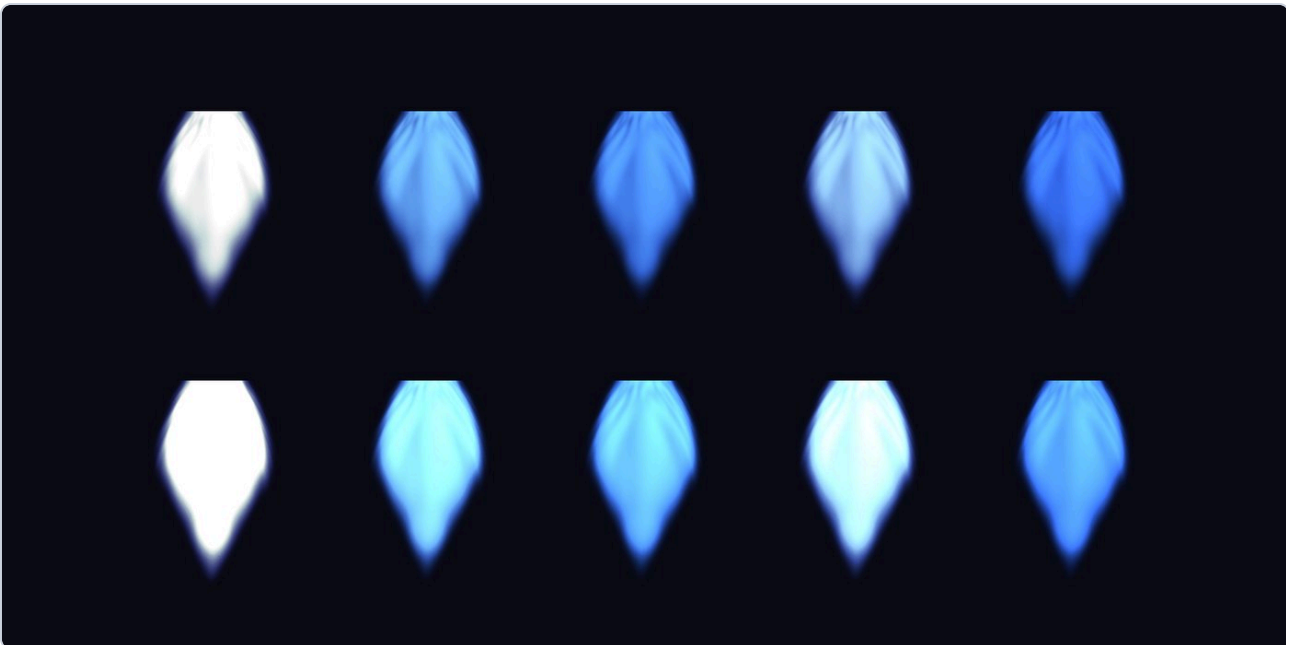


그림 11-2 · 색 후보 비교(위: 최소 출력, 아래: 최대 출력). 맨 왼쪽이 에셋 원래 파란 머티리얼 — 밝기가 넘쳐 흰색으로 보인다. 세 번째 색 (0.08, 0.3, 1.6)이 “파랑색”으로 선택되었다.

```
// MechBoosterFlames — 우주에서만 발바닥 불꽃
ColonyStructure c = colony != null ? colony : ColonyStructure.Active;
bool inSpace = c == null || !c.IsInside(p + transform.up * (bodyHeight * 0.5f));
float footWant = dead || !inSpace ? 0f : Mathf.Max(footIdleLevel, want);
```

최적화와 화질

요청 · 랙이 조금 거림 화질은 별로 차이 없고 최적화좀해줘

머리 카메라는 360도 화면을 위해 매 프레임 장면을 6번 찍는다. 그래서 장면에 있는 모든 폴리곤이 6배로 비싸다. 최적화는 세 갈래로 진행되었다.

1. 무거운 메시 줄이기



그림 12-1 · 자쿠 원본(왼쪽)과 14만 폴리곤 감축본(오른쪽) 비교.



그림 12-2 · 건담 원본과 14.5만 감축본 비교.

Unity의 MeshLodUtility 로 단계별 감축본을 만든 뒤 나란히 렌더해, 겉모습이 원본과 같아 보이는 가장 낮은 단계를 골랐다. 한 단계 더 내리면 표면이 울퉁불퉁해졌다(자쿠 6.2만, 건담 7.6만에서 확인). 실행 중에 단계를 바꾸는 런타임 LOD도 시도했지만 울퉁불퉁해 보여 빼고, 고른 단계 하나만 압축해

Assets/Models/Optimized 에 저장한다.

모델	원본	감축본
자쿠(스킨드)	약 50만	14만
건담(스킨드)	약 50만	14.5만
자쿠 머신건 / 바주카	약 50만	5만 / 약 3.9만
히트 호크	약 50만	4만
그 외 정적 메시	10만 정점 이상	8.5만

플레이어 자기 몸(레이어 30)은 어떤 카메라에도 그려지지 않으므로 감축 대상에서 뺀다.

2. 머리 카메라 작은 화면을 큐브맵에서

가운데 디스플레이 구석의 머리 카메라 화면은 따로 한 번 더 찍지 않고, 이미 찍은 큐브맵을 셰이더 (HeadCamInsetCube)로 잘라 보여 준다. 머리 카메라 렌더 텍스처는 HDR·깊이 텍스처·그림자를 끄고,

조종사가 지금 보고 있지 않은 큐브 면은 그 프레임의 렌더를 건너뛰다(시야각 + 여유 18° 밖의 면을 생략하는 face culling).

3. 보이지 않는 것은 그리지 않기

콜로니 안에서는 외벽·거울·항구 크레인 같은 바깥 장식을 끄고, 밖에서는 차량·가로등·차선 같은 거리 소품을 끈다. 거리 소품은 카메라의 레이어별 컬링 거리로 일정 거리 밖에서 그리지 않는다.

화질

문제 보고 · 건담시야로 보는 밖이 화질이 깨짐 / 화질이 너무 안좋아

큐브맵 해상도를 1536으로 올리고, 가까운 클리핑 거리를 0.5m로 조정해 깊이 정밀도를 확보했다. 줄인 비용을 해상도에 다시 썼다는 것이 이 장의 요점이다.

PART 4

문제 해결 노트

테스트하다 막혔던 문제들. 무엇이 보였는지, 진짜 원인이 무엇이었는지, 어떻게 고쳤는지를 한 줄씩 정리했다. 비슷한 VR 프로젝트를 할 때 가장 쓸모 있을 부분이다.

막혔던 문제와 해결 방법

손과 조종간

증상	원인	해결
조종간을 잡는 순간 몸 쪽으로 끌려온다	잡는 순간 손잡이를 손 위치로 옮겼다(스냅)	잡은 순간의 손-손잡이 차이를 기억하고 이후 이동량만 반영
양손으로 잡으면 두 조종간이 서로의 손에 반응	각 조종간이 어느 손이 잡았는지 확실히 고정하지 않음	허용된 손의 상호작용기만 받고, 잡은 동안은 그 상호작용기의 Transform만 읽음
가끔 기체가 영동한 방향으로 날아감	손 추적값이 한 프레임 텀	프레임당 최대 이동량을 넘는 값은 무시, 데드존과 저역 필터
조종간이 앞으로 안 밀리고 지정 위치에 고정 안 됨	XRI·캘리브레이션·조종간 스크립트 셋이 같은 Transform을 동시에 움직임	XRI는 잡기 신호만, 위치는 JoystickLever만 계산
조종간이 바닥으로 내려가 안 올라옴	예전 캘리브레이션 때 잘못 저장된 낮은 위치를 계속 불러옴	저장 제거, 매 실행마다 새로 설정
엄지를 펴도 발칸이 나감	굽힘 각도와 거리 신호 중 하나만 커져도 누름으로 판정	두 신호를 점수로 합산 + 일정 시간 유지 + 펴는 신호가 있으면 거부
디스플레이를 눌러도 안 눌림	손 관절 좌표의 기준이 XR Origin이 아니라 1.36m 높은 Camera Offset	XRHandSpace로 올바른 추적 공간을 찾아 변환
디스플레이가 조종간과 겹쳐 누를 수 없음	디스플레이 묶음이 너무 크고 낮음	절반 크기, 조종간 사이 위로 이동, 이후 2cm 하향

시점과 이동

증상	원인	해결
왼손 조종간을 돌리면 내 머리 시야까지 돈다	XR Origin이 자식으로 붙은 기체 루트를 회전 시킴	루트는 회전시키지 않고, 시점은 돔 셰이더 회전으로만 처리
오른손 조종간으로 돌려도 돔이 안 돈다	RenderToCubemap은 카메라 회전을 무시하고 월드 축으로 찍음	같은 회전을 돔 셰이더의 _ViewRotQ로 넘겨 샘플링 방향을 회전
앞으로 밀면 옆으로 간다(처음에만 정상)	이동 방향은 기체 기준, 시점은 따로 회전	이동 벡터를 시점의 yaw로 돌림 — 보는 방향이 앞
옆으로 밀어도 앞으로 가야 옆으로 감	좌우 입력이 회전만 하고 이동을 안 함	좌우 입력에 옆 이동(strafe)을 더함
콕핏 안으로 바깥 물체가 들어와 시야를 가림	콕핏이 기체를 따라 전장 안을 다님	콕핏을 전장에서 멀리 고정 배치

빔 샷벨로 다가가면 시야가 아래로 떨어짐	적의 중심(허리)을 바라봄	락온한 적의 머리 쪽을 바라보게
------------------------	----------------	-------------------

화면과 렌더링

증상	원인	해결
돔이 검정이거나 그냥 유니티 공간만 보임	평면 카메라 화면을 돔에 붙임 / 스카이박스 연결 누락	큐브맵 렌더 텍스처 + 전용 돔 셰이더
자쿠 색이 건담에도 들어감	두 모델이 같은 머티리얼 공유	모델마다 머티리얼 생성
밖에 자쿠가 보이지 않음	시야각 60°짜리 평면 영상 한 장을 구 전체에 눌러 붙여서, 정면 원뿔 밖의 자쿠는 영상에 아예 없었음	큐브맵을 실제 방향으로 샘플링하는 돔 셰이더 (진짜 360°)
디스플레이가 파랗기만 하고 글자가 안 보임	UI 캔버스를 크기가 늘려진 화면 상자 안쪽(앞면보다 뒤)에 자식으로 붙여, 글자가 불투명한 상자 속에 묻히고 비율도 찌그러짐	캔버스를 상자 밖, 화면 앞면 바로 앞에 독립 배치
가운데 디스플레이 깜빡임	겹친 UI의 z-파이팅	캔버스를 화면 앞 5mm로 띄움
건담 시야 화질이 깨짐	큐브맵 해상도와 깊이 정밀도 부족	해상도 1536, near 0.5m, 최적화로 비용 상쇄
최적화 후 모델이 울퉁불퉁	감축 단계가 너무 낮음 / 런타임 LOD	비교 렌더로 단계를 고르고 런타임 LOD 제거 (자쿠 14만, 건담 14.5만)
먼 적을 락온하면 링이 덩어리로 뭉침	최소 반지름이 크고 눈금 크기 고정	최소 반지름 축소, 눈금 크기를 반지름에 비례, 바깥 눈금 층 페이드
부스터 “파란” 불꽃이 흰색으로 보임	에셋 머티리얼이 HDR 값이라 HDR/블룸이 없으면 하얗게 포화	채도 높은 파랑 (0.08, 0.3, 1.6)을 MaterialPropertyBlock으로 지정

게임플레이

증상	원인	해결
헤드 발칸이 나가는지 안 보임	탄이 너무 작고 빠름	보이는 예광탄과 남은 탄수 표시
재장전 중에도 발칸이 나감	발사 조건에 재장전 상태 누락	재장전 중 발사 금지
건담과 자쿠 키가 머리 하나 차이	패딩된 렌더러 경계로 크기 측정	실제 정점 높이로 측정해 둘 다 18m
자쿠가 거리만 유지	목표 거리가 고정	기동마다 거리 무작위 변경, 돌진·이탈
건물 뒤 적이 락온되고, 총알이 건물을 뚫음	가림 판정 없음	콜로니 전용 고속 레이캐스트로 락온·시야·탄환 모두 차단
머신건과 바주카를 바꾸면 왼손이 풀림	내려놓은 무기도 매 프레임 같은 왼팔을 초기화	들고 있는(또는 막 내려놓는) 무기만 팔을 만짐, 팔 화면은 무기별로

작업 과정에서 배운 것

- **저장은 확인까지가 저장이다.** 파일을 사용자 PC로 보낼 때 가끔 이전 버전이 덮어써졌다. 매번 다시 읽어와 바이트 비교를 하고, 다르면 다시 보냈다.
- **20MB가 넘는 FBX는 브리지로 보낼 수 없었다.** Unity 안에서 다운로드 폴더의 파일을 프로젝트로 직접 복사하는 방법을 썼다.
- **에셋 패키지는 통째로 가져오지 않는다.** 부스터 패키지에는 Packages/manifest.json과 다른 렌더 파이프라인 에셋이 들어 있었다. 필요한 파일만 원래 GUID(.meta)와 함께 골라 넣었다.
- **사용자 씬을 지킨다.** 렌더 테스트는 추가로 연 임시 씬에서 하고 저장하지 않고 닫았다. (책을 만들며 찍은 렌더 하나는 사용자가 이미 같은 씬을 열어 둔 상태에서 실행되어 임시 카메라가 남았고, 바로 제거했다.)
- **측정해서 맞춘다.** 무기를 쥐는 위치, 노즐 위치, 기체 키는 모두 정사영 렌더에 좌표 격자를 겹치거나 실제 정점을 재서 정했다. 눈대중 숫자는 VR에서 바로 티가 난다.

APPENDIX

부록

수치표, 스크립트 지도, 에셋 목록.

기체·무기 수치표

기체

기체	역할	체력	무기
RX-78-2 건담	플레이어	3000 (0이 되면 5초 후 수리)	헤드 발칸, 빔 라이플, 빔 샷벨
MS-06 자쿠 II	플레이어(선택 시)	5000	머신건, 히트 호크, 바주카
자쿠 ×10	적(건담 모드)	1000 (5초 후 재등장)	머신건, 히트 호크
적 건담 ×6	적(자쿠 모드)	1500 (6초 후 재등장)	빔 라이플, 빔 샷벨

모든 모빌슈트 키 18m. 최고 속도 55m/s(약 198km/h), 상승·하강도 같은 최고 속도. 콜로니 안 중력 9.8m/s².

플레이어 무기

무기	데미지	탄 / 재장전	발사 / 조작	락온 시간
헤드 발칸	1	120발(60×2) / 2.5초	0.15초 간격, 시선 조준 + 오른손 엄지	-
빔 라이플	500	15발 충전 / 5초	락온 + 엄지, 양손	2초
빔 샷벨	200	-	오른손 조종간 찌르기·휘두르기	락온 시 공격
머신건(자쿠)	50	100발 / 5초	분당 280발, 누르고 있으면 연사	1초
히트 호크(자쿠)	300	-	빔 샷벨과 같음	락온 시 공격
바주카(자쿠)	1000	5발 / 6초	발사 간격 1.2초, 포탄 170m/s	1.5초

적 무기

무기	데미지	탄 / 재장전	발사
자쿠 머신건	50	100발 / 5초	분당 280발, 점사, 보이는 대상만
자쿠 히트 호크	250	-	근거리에서 AI가 선택
적 건담 빔 라이플	150	15발 / 5초	분당 45발, 1-2발 점사
적 건담 빔 샷벨	300	-	타격 거리 22m

스크립트 지도

분야	스크립트 (Assets/Scripts/Cockpit)
씬 생성	Editor/GundamCockpitSetup.cs (메뉴 Gundam > Build Cockpit Scene)
손과 조종간	HandJointTracker, HandExclusiveGrabAdapter, JoystickLever, JoystickFingerButtons, VerticalTLever, CalibrationManager, XRHandSpace, HandTrackingStatusLogger, HandSubsystemBootDiagnostics
이동	ShipMovementController, VerticalThrustController, SuitCollision, ExternalGundamFollower, MechWalkAnimator
시점·콕핏	GundamHeadCam360, CockpitViewController, HeadCamManualLook, CockpitTheme (셰이더: CockpitDomeCubemap, HeadCamInsetCube)
HUD	CockpitHUDManager, CockpitHUD, CockpitStatusHUD, CockpitWeaponHUD, OrbitHUDTargetLock, TargetingHUD
무기	WeaponModeController, WeaponTouchPanel, HeadVulcanController, HeadVulcanBullet, BeamRifleController, BeamRifleShot, BeamSaberArmController, BeamSaberBlade, BeamSaberControlStick, SaberLookAssist, WeaponAimFireController(초기 포탑, 보존)
적	EnemyMarker, EnemyHealth, ZakuCombatAI, ZakuMachineGun, ZakuBullet, MeleeWeaponAI, HitTarget
플레이어	PlayerHealth, PilotMechSwitcher, MechSelectPanel, SpawnSelectPanel
전장	ColonyStructure, ColonyAtmosphere, SpaceDustField, SpaceDebrisField
소리·이펙트	LoopingFireSound, GundamMoveSound, MechBoosterFlames

에셋과 사운드 목록

경로	내용	들어온 날
Models/Gundam	건담 FBX, 베이스 컬러, 머리 카메라 렌더 텍스처, 팔 화면 메시	9/22
Models/Zaku	자쿠 FBX, 텍스처, 팔 화면 메시	9/22
Models/Debris	전함 잔해, 콜로니 파편, 정거장 모듈(VARCO 3D)	9/29
Models/BeamSaber	빔 샤벨 손잡이	9/29
Models/BeamRifle	빔 라이플	9/29
Models/ZakuMachineGun	자쿠 머신건	9/29
Models/Colony, ColonyBuildings, ColonyProps	콜로니 메시 조각, 창문·아스팔트 텍스처, VARCO 건물, 항구 크레인 등 소품	9/30
Models/Optimized	감축된 메시(자동 생성)	9/30
Models/HeatHawk	히트 호크	10/1
Models/ZakuBazooka	자쿠 바주카(원본 “Military Rocket Launcher”)	10/1
Audio	Gundam_BeamRifle_Fire, Gundam_BeamSaber_Swing, Gundam_HeadVulcan_FireLoop, Gundam_Move_Wind, Zaku_MachineGun_FireLoop	10/1
RockingProjects/2DBoosterFlame	부스터 불꽃 셰이더 그래프, 머티리얼, 마스크 텍스처, 스크립트	10/1

정리해도 되는 파일

작업 중 테스트로 생긴 파일들로, 지워도 게임에 영향이 없다: `Assets/Scenes/_MeleeTest.unity`, `Assets/Models/Optimized` 안의 `__36000`, `__65000`, `__18000` 버전과 건담 `__140000` 버전, `Assets/_Recovery/0.unity` (9월 29일 에디터가 만든 복구용 백업).

AFTERWORD

맺음말

11일 동안 콕핏은 평평한 판에서 구가 되었고, 조이스틱은 손으로 쥐는 공이 되었고, 빈 우주는 길이 12km의 콜로니 도시가 되었다.

돌아보면 가장 좋은 설계 결정들은 대부분 짧은 피드백에서 나왔다. “콕피트에 외관은 전부 디스플레이어야해 그래야 로봇안에 있는거지”라는 한 문장이 콕핏 전체 구조를 정했고, “움직이는거를 콕핏에서의 시선을 기준으로 해야할거 같아”가 이동 체계를 정했고, “삭제하지 말고 기존 코드는 보존한다”가 프로젝트의 습관을 정했다.

아직 남은 일도 있다. 부위별 데미지 시스템(데이터 구조는 준비됨), 나머지 손가락 버튼 9개의 기능, 바주카 전용 소리, 그리고 적이 우주로 나와 발바닥 불꽃을 보여 줄 전투. 다음 장은 다시 헤드셋을 쓰고 테스트한 뒤 짧은 한 문장으로 시작될 것이다.

2026년 10월 1일

이건호 · Claude